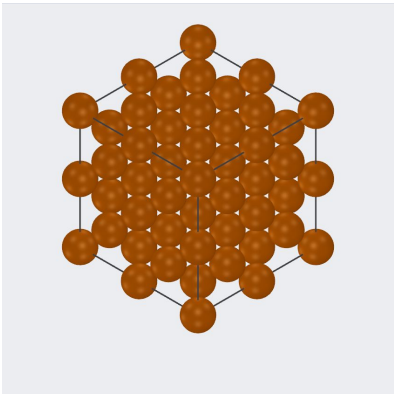
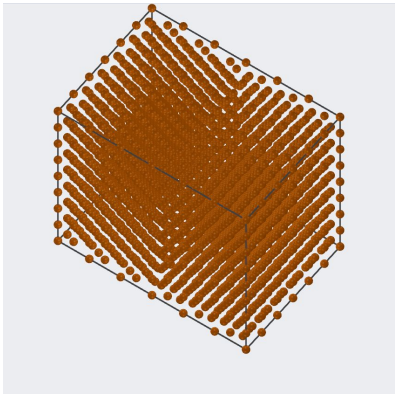
# AtomForge

## Detailed user and reference manual

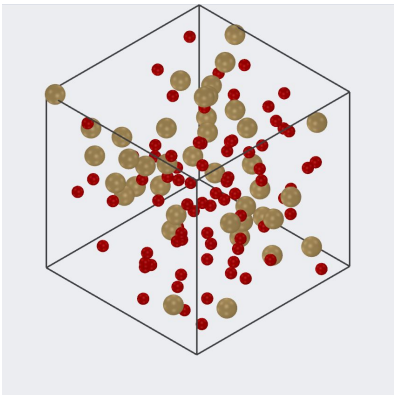
Building structures, exploring atomic geometry, and analysing electronic fields



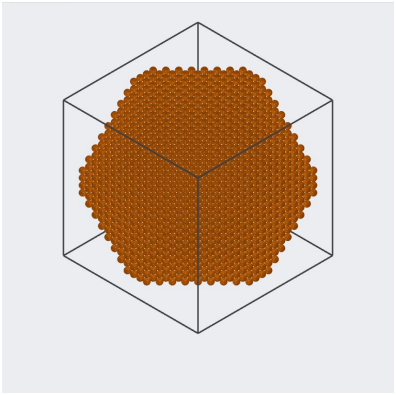
**Bulk crystal**



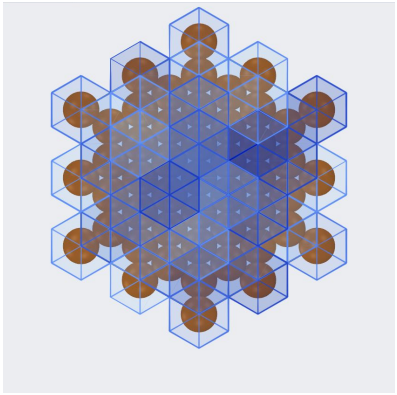
**Grain boundary**



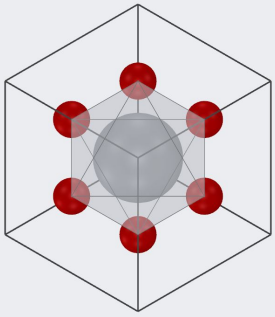
**Amorphous packing**



**Wulff particle**



**Voronoi cells**



**Coordination polyhedron**

Rendered in AtomForge with default atomic radii.

Version 0.3.0

Documentation edition: 11 September 2026

# How to use this manual

Start with Chapter 1 to open a file, construct a crystal, and save a result. Chapters 2–4 explain the structural workflows. Chapters 5–7 cover electronic loading, visualisation, charge redistribution, and the complete electronic operation catalogue. Chapter 8 provides scripting recipes; the appendices cover installation, reproducibility, and troubleshooting.

Menu names and controls describe the documented application features and the Windows executable used to prepare the figures. Features controlled by build options are marked explicitly. A source file does not by itself mean a feature is accessible in the desktop menu. Common-neighbour analysis, angular-distribution analysis, the interface builder, and stacking-fault generation are enabled in the current source build. The scientific integrations and project workflows are described in Chapter 9; these additions require a rebuilt application and the matching Python source package. The detailed builder, edit, and display control references supplement the worked examples. Appendix C maps the feature families back to source and explains the documentation coverage check.

GUI screenshots, including all six cover views, are captured from AtomForge. The cover keeps the application's default atomic radii; camera framing varies between examples. The CHGCAR plots and shape-cut diagrams are separately rendered scientific figures calculated from real AtomForge outputs. Example structures are starting configurations, not validated equilibrium structures. The supplied CHGCAR is used to demonstrate the software; its physical system and calculation settings were not independently established.

All paths beginning with `examples/` are relative to this manual's folder. Commands use `AtomForge` for readability; on Windows substitute the path to `AtomForge.exe`. Long command listings may wrap visually; each displayed command is entered as one shell command. Python examples assume the package and native electronic library are available as described in Chapter 8.

Chapter 10 contains 60 illustrated quick tutorials. Each gives a starting input, a short procedure, a result check, and options to try. Use the contents or PDF bookmarks to jump directly to a builder, edit tool, or display option. Seven native nanocrystal examples accompany the shape-cut exercises.

# Contents

**How to use this manual** i

**1 Getting started** 1

1.1 What AtomForge does . . . 1
1.2 First structure: a copper crystal . . . 1
1.3 Opening, saving, and browsing . . . 2
1.4 File types and what survives export . . . 2
1.5 Drag and drop . . . 2
1.6 The main window . . . 3
1.7 Orienting and inspecting the main view . . . 3
1.8 A productive project layout . . . 3

**2 Building atomic structures** 4

2.1 Common builder workflow . . . 4
2.2 Bulk Crystal . . . 4
2.3 Substitutional Solid Solution . . . 5
2.4 CSL Grain Boundary . . . 6
2.5 Nanocrystal: shape cuts . . . 6
2.6 Nanocrystal: Wulff construction . . . 7
2.7 Custom Structure from OBJ/STL . . . 7
2.8 Polycrystal . . . 8
2.9 Amorphous Structure . . . 8
2.10 Defect, lattice and nanostructure builders (command line) . . . 9
2.11 Optional and unavailable builder menus . . . 9
2.12 Worked study: compare bulk and finite Cu environments . . . 10
2.13 Builder control reference . . . 10
2.13.1 Bulk Crystal: symmetry, lattice, and site editor . . . 11
2.13.2 Substitutional Solid Solution: every composition control . . . 11
2.13.3 CSL Grain Boundary: search, geometry, and result . . . 12
2.13.4 Nanocrystal: shape parameters and shared output controls . . . 12
2.13.5 Custom Structure: reference, mesh, and orientation . . . 13
2.13.6 Polycrystal: dimensions, randomisation, and individual grains . . . 14

2.13.7 Amorphous Structure: composition and packing constraints . . . . . 14
2.13.8 Optional Stacking Faults: sequence and export controls . . . . . 15

**3 Editing, assembly, and defects 16**
3.1 Selection and edit history . . . . . 16
3.2 Edit Structure: atoms and cell . . . . . 16
3.3 Transform Structure . . . . . 17
3.4 Merge Structures . . . . . 17
3.5 Cell Sculptor: supercells and cutting slabs . . . . . 17
3.6 Insert Dislocation . . . . . 18
3.7 Add Interstitial Atoms and detect voids . . . . . 18
3.8 Structure pipeline: non-destructive editing . . . . . 19
3.9 Checking an edited model . . . . . 23
3.10 Detailed edit controls and application points . . . . . 23
3.10.1 Atom selection and the right-click menu . . . . . 23
3.10.2 Edit Structure: exact effects of the table . . . . . 23
3.10.3 Transform Structure: matrix entry and validation . . . . . 24
3.10.4 Merge Structures: component transforms and output cell . . . . . 24
3.10.5 Cell Sculptor: closing a region and defining each slab . . . . . 25
3.10.6 Insert Dislocation: geometry, elastic controls, and commit . . . . . 25
3.10.7 Add Interstitial Atoms: finding, filtering, and occupying voids . . . . . 26

**4 Structural analysis and presentation 28**
4.1 Structure Info and Atom Info . . . . . 28
4.2 Distance and angle measurements . . . . . 28
4.3 Radial Distribution Function . . . . . 28
4.4 Short-Range Order (SRO) . . . . . 29
4.5 Lattice planes and Miller directions . . . . . 29
4.6 Voronoi and coordination polyhedra . . . . . 30
4.7 Atomic sizes, colours, and lighting . . . . . 30
4.8 Large datasets: billions of atoms . . . . . 30
4.9 Exporting images . . . . . 31
4.10 Dislocation Post-Processing . . . . . 32
4.11 Features that are not current analysis-menu commands . . . . . 32
4.12 Workspace, display, and analysis control reference . . . . . 32
4.12.1 Screen resolution and interface size . . . . . 32
4.12.2 Structure tabs and file navigation . . . . . 33
4.12.3 Main view, representation, and information . . . . . 33
4.12.4 Camera workflow: orientation, projection, and framing . . . . . 34
4.12.5 Every View menu option: dependencies and interpretation . . . . . 34

4.12.6 Voronoi volume: a geometric overlay . . . 36
4.12.7 Settings workflow: choose the control that changes the desired property . 36
4.12.8 Lattice Planes: every row and visibility control . . . 37
4.12.9 Miller Directions: vectors, length, and colour . . . 37
4.12.10 Polyhedral Settings: selecting centres and ligands . . . 37
4.12.11 Atomic Sizes and Display Settings tabs . . . 37
4.12.12 RDF and SRO: run state and all numerical inputs . . . 38
4.12.13 Image export settings . . . 38
4.12.14 Electronic display controls that are separate from calculations . . . 38

**5 Electronic files and interactive views 40**
5.1 Open a scalar field . . . 40
5.2 VASP, Cube, and XSF conventions . . . 40
5.3 Units, sampling, and periodic endpoints . . . 41
5.4 What does Reference mean? . . . 41
5.5 3D volume and isosurface views . . . 42
5.6 Choosing an initial level and understanding warnings . . . 43
5.7 2D lattice-plane sections . . . 43
5.8 Arbitrary planes and the 3D guide . . . 43
5.9 Worked CHGCAR walkthrough . . . 44
5.10 Exporting the right result . . . 45

**6 Charge redistribution and Boolean masks 46**
6.1 Define the study before subtracting . . . 46
6.2 Grid compatibility . . . 46
6.3 Weighted density difference: complete GUI recipe . . . 46
6.4 Accumulation and depletion fields . . . 47
6.5 Charge summary . . . 47
6.6 Threshold mask . . . 47
6.7 Boolean masks and inversion . . . 48
6.8 Apply mask and quantify a region . . . 48
6.9 Cumulative charge profile . . . 48
6.10 Recommended record for a charge study . . . 49

**7 Electronic operation reference 50**
7.1 How to run an operation . . . 50
7.2 Total integral . . . 50
7.3 Add and subtract reference . . . 50
7.4 Multiply and divide reference . . . 50
7.5 Scale . . . 51
7.6 Gaussian smoothing . . . 51

7.7 Cartesian gradient . . . 51
7.8 Laplacian . . . 51
7.9 Energy-density conversion . . . 51
7.10 Line profile . . . 52
7.11 Planar average . . . 52
7.12 Macroscopic average . . . 52
7.13 Numerical plane section . . . 52
7.14 Contour segments . . . 53
7.15 Peak search . . . 53
7.16 Voronoi integration . . . 53
7.17 Sphere integration . . . 53
7.18 Structure factors from a grid . . . 54
7.19 Fourier synthesis . . . 54
7.20 Patterson map . . . 54
7.21 Ewald point-charge electrostatics . . . 54
7.22 Isosurface extraction and reference colouring . . . 55
7.23 Resample to reference grid . . . 55
7.24 Verify periodic endpoint planes . . . 55

**8 Python and command-line integration 56**
8.1 Install the Python package . . . 56
8.2 Structure objects and file I/O . . . 56
8.3 Python package 0.2.0: native builders . . . 57
8.4 Structure pipelines . . . 58
8.5 Atom clouds . . . 58
8.6 Desktop and notebook viewing . . . 58
8.7 Loading electronic volumes . . . 58
8.8 A complete scripted charge workflow . . . 59
8.9 Profiles, sections, derivatives, and integration . . . 59
8.10 Fourier and scattering models . . . 60
8.11 Point charges, surfaces, and explicit geometry . . . 60
8.12 Electronic CLI . . . 61
8.13 Native builder CLI . . . 61
8.14 Headless dislocation construction . . . 62

**9 Projects and scientific workflows 63**
9.1 Save a project and recover a session . . . 63
9.2 Electronic history, cancellation and export . . . 63
9.3 Structural analysis and automation . . . 64
9.4 Recipes and batch electronic processing . . . 64

9.5 Trajectories and animation . . . 65
9.6 Install scientific integrations . . . 66
9.7 Bands, DOS, projected data and orbitals . . . 66
9.8 Reciprocal-space surfaces . . . 66
9.9 Charge partitioning . . . 67
9.10 Diffraction, relaxation, dynamics and phonons . . . 67
9.11 Condensed-matter tools: desktop, Python and command line . . . 68
9.11.1 Trajectory and local-structure analyses (priorities 1–9) . . . 70
9.11.2 Electronic and thermomechanical analyses (priorities 10–16) . . . 71
9.11.3 Simulation and reciprocal-space workflows (priorities 17–20) . . . 72
9.11.4 Defect identification, clusters and diffraction . . . 73
9.11.5 Electronic-structure output and DFT inputs . . . 74
9.11.6 Symmetry, mechanical properties and defect energies . . . 75
9.11.7 Trajectory-averaged structure . . . 76
9.11.8 Batch runs, LAMMPS export and project persistence . . . 76
9.11.9 Native tools from Python . . . 77
9.11.10 Repeatable commands and Python examples . . . 77

**10 Illustrated quick tutorials** **80**
10.1 Build: source, symmetry, and composition . . . 81
10.1.1 Choose a builder . . . 81
10.1.2 Build FCC copper . . . 82
10.1.3 Change crystal system and lattice constraints . . . 83
10.1.4 Choose an element for a site or species row . . . 84
10.1.5 Create a reproducible Cu–Ni alloy . . . 85
10.1.6 Find and build a Sigma-5 boundary . . . 86
10.1.7 Compare grain-boundary cell options . . . 87
10.2 Build: finite shapes and meshes . . . 88
10.2.1 Choose a nanocrystal construction . . . 88
10.2.2 Shape Cut: sphere . . . 90
10.2.3 Shape Cut: ellipsoid . . . 91
10.2.4 Shape Cut: box . . . 92
10.2.5 Shape Cut: cylinder . . . 93
10.2.6 Shape Cut: octahedron . . . 94
10.2.7 Shape Cut: truncated octahedron . . . 95
10.2.8 Shape Cut: cuboctahedron . . . 96
10.2.9 Build a Wulff particle from facet families . . . 97
10.2.10 Fill an OBJ mesh with copper sites . . . 100
10.2.11 Orient the lattice inside a mesh . . . 102
10.2.12 Align a crystallographic direction to the mesh . . . 104

10.3 Build: grains and random packing . . . 106
10.3.1 Build a four-grain copper sample . . . 106
10.3.2 Specify all or selected grain orientations . . . 108
10.3.3 Pack a Si–O starting configuration . . . 109
10.3.4 Control packing distances and repeatability . . . 110
10.3.5 Use an explicit packing box . . . 111
10.4 Edit: selection, coordinates, and assembly . . . 112
10.4.1 Select atoms and recover an edit . . . 112
10.4.2 Use atom context actions . . . 113
10.4.3 Check coordinates and nearest neighbours . . . 114
10.4.4 Move, add, delete, or relabel an atom . . . 115
10.4.5 Make a periodic supercell . . . 116
10.4.6 Assemble two separated particles . . . 117
10.4.7 Preview a crystallographic slab . . . 118
10.4.8 Cut with explicit slab distances . . . 119
10.5 Edit: dislocations and interstitials . . . 120
10.5.1 Configure a dislocation preview . . . 120
10.5.2 Restrict the displaced region . . . 121
10.5.3 Review and commit a defect . . . 122
10.5.4 Find candidate interstitial sites . . . 123
10.5.5 Inspect and filter candidate environments . . . 124
10.5.6 Insert a small random population . . . 125
10.5.7 Insert only at selected centres . . . 126
10.5.8 Confine insertion to an OBJ/STL region . . . 127
10.6 File, View, and Settings options . . . 128
10.6.1 Open, save, convert, and manage tabs . . . 128
10.6.2 Orient and frame the structure . . . 129
10.6.3 Choose structure colouring . . . 130
10.6.4 Compare atom representations and overlays . . . 131
10.6.5 Add and manage lattice-plane guides . . . 132
10.6.6 Draw crystallographic directions . . . 133
10.6.7 Limit and style coordination polyhedra . . . 134
10.6.8 Adjust an element radius . . . 135
10.6.9 Tune scene illumination . . . 136
10.6.10 Control highlights on atoms and bonds . . . 137
10.6.11 Change a species display colour . . . 138
10.6.12 Select a light or dark desktop theme . . . 139
10.6.13 Verify composition, cell, and symmetry . . . 140
10.6.14 Export a structure illustration . . . 141

10.6.15 Inspect geometric Voronoi cells . . . 142
10.6.16 Adapt the interface to the screen . . . 143
10.7 Analysis and electronic display . . . 144
10.7.1 Run an RDF and inspect its shell window . . . 144
10.7.2 Run SRO and validate the shell diagnostics . . . 145
10.7.3 Read the SRO result tabs . . . 146
10.7.4 Compare electronic volume and surface views . . . 147
10.7.5 Choose a 2D intersection and its guide . . . 148
10.7.6 Run charge-transfer and general operations . . . 149

**A Installation, builds, and troubleshooting 150**
A.1 Running a packaged desktop build . . . 150
A.2 Build requirements . . . 150
A.3 Automatic build resource protection . . . 151
A.4 Linux build . . . 151
A.5 Windows build with MSYS2 UCRT64 . . . 151
A.6 Portable packaging and CI . . . 152
A.7 Reporting a reproducible problem . . . 152
A.8 Troubleshooting by symptom . . . 152

**B Examples, validation, and maintaining this manual 154**
B.1 Included example inventory . . . 154
B.2 Nanocrystal shape examples . . . 154
B.3 CHGCAR provenance . . . 155
B.4 Regenerate examples and figures . . . 155
B.5 Compile the LaTeX manual . . . 155
B.6 Documentation coverage and limitations . . . 156
B.7 Source map for maintainers . . . 156
B.8 Glossary . . . 157

**C Feature coverage and source-only interfaces 158**
C.1 Feature checklist . . . 158
C.2 Common-neighbour analysis: desktop and API . . . 159
C.3 Angular-distribution analysis: desktop and API . . . 159
C.4 Interface matching: desktop and CLI workflow . . . 160
C.5 Keeping the documentation complete as code changes . . . 160

**D Complete Python signatures and native CLI options 161**
D.1 Using this reference . . . 161
D.2 Atom and Structure . . . 161
D.2.1 Atom . . . 161

D.2.2 Structure . . . 161
D.3 Structure file functions . . . 163
D.4 Desktop viewer function . . . 163
D.5 Native Python builders . . . 163
D.5.1 BuildResult . . . 163
D.6 Electronic data types and operations . . . 164
D.6.1 Grid . . . 164
D.6.2 BaderPartition . . . 168
D.6.3 Volume . . . 169
D.6.4 Surface . . . 169
D.7 Electronic model helpers . . . 169
D.8 Structural analyses and sculpting . . . 170
D.9 Electronic recipes and batch jobs . . . 172
D.10 Relaxation, dynamics and phonons . . . 172
D.10.1 RelaxationResult . . . 173
D.11 Bands and density of states . . . 174
D.12 Population analysis and orbital reconstruction . . . 174
D.13 Trajectory conversion and animation . . . 175
D.14 Diffraction . . . 175
D.15 Reciprocal-space band grids . . . 176
D.15.1 BandGrid . . . 176
D.16 Trajectory statistics and diffusion . . . 176
D.17 Local strain, disorder and defects . . . 177
D.18 Band edges, effective masses and work functions . . . 178
D.19 Elasticity, equation of state and harmonic thermodynamics . . . 178
D.20 NEB, NVT, NPT and symmetry paths . . . 179
D.21 Shared scientific workflow interface . . . 179
D.22 Native scientific tools (no dependencies) . . . 180
D.22.1 NativeResult . . . 180
D.23 Structure pipelines . . . 181
D.23.1 PipelineError . . . 181
D.23.2 Step . . . 181
D.23.3 Pipeline . . . 181
D.23.4 PipelineResult . . . 182
D.24 Atom clouds (billions of atoms) . . . 183
D.24.1 CloudInfo . . . 183
D.25 Native builder options . . . 183
D.25.1 bulk . . . 184
D.25.2 gb . . . 185

D.25.3 poly . . . 186
D.25.4 nano . . . 187
D.25.5 amorphous . . . 188
D.25.6 sss . . . 189
D.25.7 dislocation . . . 190
D.25.8 custom . . . 192
D.25.9 interface . . . 193
D.25.10 stacking-fault . . . 194
D.25.11 vacancy . . . 195
D.25.12 strain . . . 196
D.25.13 primitive . . . 197
D.25.14 surface . . . 198
D.25.15 sqs . . . 199
D.25.16 nanowire . . . 200
D.25.17 core-shell . . . 201

# Chapter 1

# Getting started

## 1.1 What AtomForge does

AtomForge is a desktop structure builder and viewer with a native electronic post-processing engine. It constructs atomistic starting configurations, edits positions and cells, renders structures, measures geometric quantities, and processes scalar fields from electronic-structure calculations. It does not run a self-consistent electronic calculation, relax a structure, or predict a formation energy merely by building a model.

The main structure workspace and the electronic dialog maintain different data. Opening a CIF in the main workspace supplies atom positions and a cell. Opening CHGCAR in **Analysis > Electronic structure > Electronic Post-processing** supplies a sampled scalar field and imported sites. A structure alone does not contain a charge density; a rendered mesh alone does not contain an electronic wavefunction.

## 1.2 First structure: a copper crystal

1. Start the executable. Choose **Build > Bulk Crystal**. Select the cubic crystal system and space group 225.
2. Enter $a = 3.61$ Å. Use a Cu site at fractional coordinates $(0, 0, 0)$ in the asymmetric-unit list. Remove unrelated default sites if present; do not add the three symmetry-equivalent FCC positions yourself.
3. Build the crystal. Inspect the generated structure and its cell. The supplied example `examples/cu_fcc.cif` has four Cu atoms in the conventional cubic cell.
4. Close the builder when finished. Use **View > Structure Info** to inspect counts and lattice information.
5. Choose **File > Save As**, browse to your work folder, enter a new filename such as `cu_first.cif`, select CIF, and save. Reopen the saved file to check that the cell and species are retained.

If symmetry support is unavailable in a custom build, install/build with spglib or open the supplied expanded CIF instead. The lattice parameter is an example input; select a parameter appropriate to your intended temperature, composition, and calculation method.

## 1.3 Opening, saving, and browsing

**File > Open** reads a structure into the workspace; **Open Recent** returns to previously used paths. The browser supports directory navigation and file selection. Double-click a directory to enter it, select a file, then use the dialog's opening action. The location field is useful for jumping to a directory; it is not necessary to type the entire file path.

**Save** writes the active structure to its current destination. Use **Save As** to preserve the original and choose a new filename/format. A structure save records supported structural data, not every display setting or electronic intermediate. Save derived electronic grids separately from the electronic dialog. **Close** closes the current structure; **Quit** exits the program. Save valuable edits before closing.

| Task | Control or shortcut |
|---|---|
| Open a structure | **File > Open**, Ctrl+O. |
| Save active structure | **File > Save**, Ctrl+S. |
| Save a new copy | **File > Save As**, Ctrl+Shift+S. |
| Export rendered image | **File > Export Image**, Ctrl+Alt+S. |
| Close active structure | **File > Close**, Ctrl+W. |
| Undo / redo | **Edit > Undo** and **Edit > Redo**; availability follows the edit history. |
| Electronic analysis | **Analysis > Electronic structure > Electronic Post-processing**; available without a structure in the main scene. |
| Appearance controls | **Settings > Atomic Sizes**, **Display Settings**, **Polyhedral Settings**. |

## 1.4 File types and what survives export

The desktop uses Open Babel for structure I/O. Its advertised structure extensions include XYZ, CIF, PDB, SDF, MOL, VASP, MOL2, PWI, and GJF. Actual conversion depends on the installed Open Babel plugins. Select a format for the information you need: CIF or VASP for periodic cells, XYZ for simple coordinate interchange, and a molecular format for molecular connectivity where supported.

Plain XYZ does not reliably communicate a periodic lattice to every downstream reader. PDB has format-specific field limits. Format conversions may not preserve custom atom colours, grain identifiers, overlays, or all calculation metadata. Check the exported file in the program that will consume it, especially after transforming a non-orthogonal cell.

Electronic inputs are a separate family: VASP charge/potential/ELF grids, Gaussian Cube, and XSF datagrids. Electronic outputs include XSF, Cube, and VASP grids; CSV tables; and OBJ/PLY isosurface meshes. A CSV profile is a numerical table, while an OBJ surface is geometry. Neither is a replacement for saving the full scalar grid.

## 1.5 Drag and drop

With the electronic dialog open, drop a supported volumetric file onto the application to load it there. Enable **Drop files as reference** before dropping an operand density. Files arriving during a calculation are queued; wait for the active calculation to finish before interpreting the new field selection. A reference-file drop supplies the second input; it does not subtract anything until you choose and calculate an operation.

Several structure builders accept dropped structures or meshes in their own preview/source regions. Use the visible drop guidance in the active dialog. For an unambiguous workflow, open only the builder you intend to supply and verify the displayed source filename and atom count immediately afterwards.

## 1.6 The main window

With nothing open, the window shows a welcome panel: **Open a structure**, **Build a crystal**, **Open a project**, **Read the manual** and the recently opened files. Files can also be dropped anywhere on the window.

The toolbar under the menus groups the common actions: **Open** and **Save**; undo and redo; **View** (look along x, y, z or the lattice vectors a, b, c, and reset the view); **Rotate** (an angle and the axis to rotate the crystal about); **Measure** (**Distance** and **Angle** between picked atoms). On the right, **Pipeline** opens the structure pipeline, the projection button switches between perspective and orthographic projection, and the theme button switches between the light and dark themes. Every button explains itself in a tooltip, with its keyboard shortcut. In a narrow window the toolbar wraps onto more rows.

The status bar along the bottom shows the file, the number of atoms, how many are selected, the cell lengths, the projection and the colouring.

## 1.7 Orienting and inspecting the main view

Use the toolbar's X, Y, Z buttons for Cartesian-axis views and a, b, c for lattice-direction views when a cell is available. The rotation angle and the adjacent axis buttons apply a repeatable rotation. **View > Reset View** (key R, or the toolbar's home button) restores framing; it does not undo atom edits. **View > Projection** chooses **Perspective** or **Orthographic**. Use orthographic views when comparing projected distances or alignments.

**View > Show** gathers everything that can be shown or hidden: **Atoms**, **Element Labels**, **Bounding Box**, **Lattice Planes**, **Miller Directions**, **Voronoi Volumes** and **Dislocation Lines**. **View > Colour Atoms By** switches colouring between elements, crystal orientation, grain boundaries and a per-atom property. Orientation/boundary colouring requires corresponding structure metadata; a plain XYZ file may not contain it.

**View > Atom Style** offers balls, ball-and-stick, space-filling, and polyhedral representations when supported by the current state. Bond-like lines and coordination polyhedra are visual/geometric constructs based on neighbourhood criteria; do not infer a quantum-mechanical bond order from their appearance.

## 1.8 A productive project layout

Create separate folders for input calculations, generated structures, derived fields, tables, and figures. Keep original electronic outputs unchanged. Name derived fields by operation, for example `AB_minus_A_minus_B.xsf`, and record the reference weights, grid geometry, selected channel, and units beside numerical results. For publication, retain the actual density difference and quantitative tables as well as the screenshot.

The folder containing this manual follows that pattern on a smaller scale: `chapters/` holds editable LaTeX, `figures/` holds published illustrations, `examples/` holds reusable structures and tables, `scripts/` holds regeneration tools, and `build/` holds disposable LaTeX/render intermediates.

# Chapter 2

# Building atomic structures

## 2.1 Common builder workflow

Choose the appropriate command under **Build**, supply a periodic reference where required, set parameters, inspect the preview, and run the build action. Use Load reference in Nanocrystal, CSL, or Polycrystal; Load structure or model in Custom Structure; and Add structure in Merge. These buttons open the shared file picker, while file drops remain available. Read status messages before saving the result. Source previews and generated previews serve different purposes: changing a parameter may update an illustrative preview while the atomic result still needs rebuilding.

Successful Build-menu results open in a new structure tab through the main application's builder callback, preserving the source tab. Merge uses this same new-tab route. Select the generated tab before saving or running another tool. Direct atom edits, Cell Sculptor, Dislocation replacement, and Interstitial insertion update the current structure instead. In the detailed controls below, "updates the main scene" refers to the displayed result; it does not mean that every builder overwrites the original tab.

Use modest cells first. Replicating a structure multiplies its atom count; a $10 \times 10 \times 10$ replication of a four-atom cell contains 4,000 atoms before cuts. Increasing all three dimensions by two multiplies the workload by eight. Once the geometry is correct, increase size and inspect boundary contacts. A builder does not automatically relax strained bonds or remove every physically undesirable local environment.

## 2.2 Bulk Crystal

**Purpose.** Generate a symmetry-expanded periodic crystal from lattice parameters and asymmetric-unit sites. Inputs are the crystal system, space group, cell dimensions/angles, and element/fractional-coordinate rows.

**Procedure.** Select one of the seven crystal systems and a compatible space group (1–230). Set independent lattice parameters; check constraints implied by the selected system. Add each crystallographically independent site using an element and direct coordinates. Use the row deletion or clear controls to remove unwanted defaults. Build, inspect the expanded sites, and save a periodic format.

Fractional coordinates are coefficients in $\mathbf{r} = f_a\mathbf{a} + f_b\mathbf{b} + f_c\mathbf{c}$. They are not distances in Å. Equivalent positions are generated by symmetry; entering a full conventional basis as though every site were independent can duplicate sites. Inspect the final count and short contacts.

**Worked examples.** FCC Cu with group 225, $a = 3.61$ Å, and Cu at $(0, 0, 0)$ gives four atoms. BCC Fe with group 229, $a = 2.87$ Å, and Fe at $(0, 0, 0)$ gives two. These examples exercise symmetry

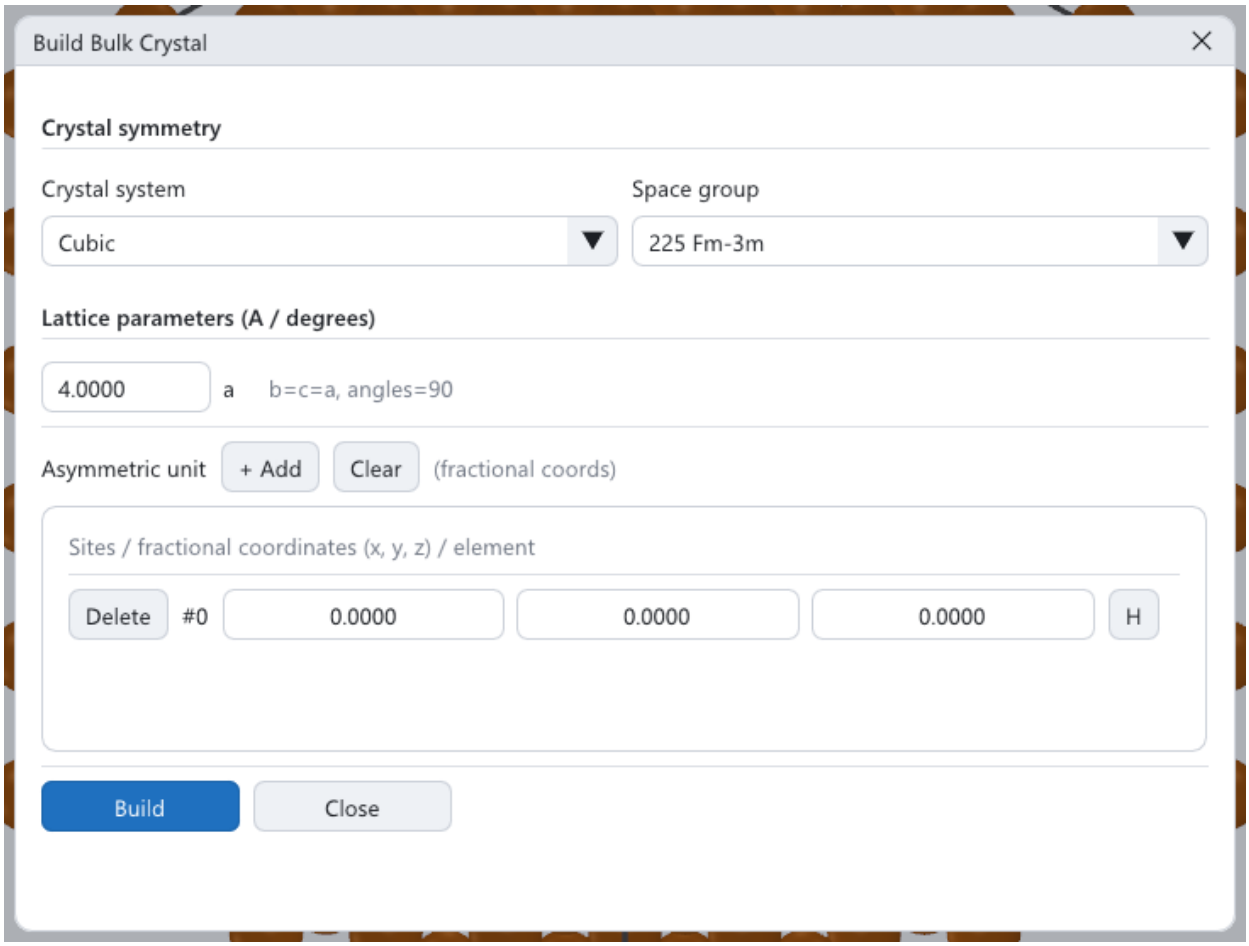



Figure 2.1: The Bulk Crystal dialog at its initial settings. Change the default H site to Cu and the default lattice parameter to 3.61 Angstrom for the FCC Cu walkthrough. The element button opens a periodic-table picker.

expansion; they do not determine a thermodynamically optimal lattice parameter.

```
AtomForge --build bulk --system cubic --spacegroup 225 --a 3.61 --atom "Cu 0 0 0" --
    output cu_fcc.cif
AtomForge --build bulk --system cubic --spacegroup 229 --a 2.87 --atom "Fe 0 0 0" --
    output fe_bcc.cif
```

For lower-symmetry cells, set `--b`, `--c`, and angles `--alpha`, `--beta`, `--gamma`. Repeat `--atom` for independent sites. See `--help` for accepted system names.

## 2.3 Substitutional Solid Solution

**Purpose.** Assign species to existing lattice sites with requested fractions. This changes chemical labels on a host structure; it does not search for a special quasirandom structure or optimise chemical short-range order.

**Procedure.** Open a host with enough sites to represent the composition, then choose **Build > Substitutional Solid Solution**. Select the source and species percentages. The GUI uses at%, with the final row automatically balancing the total to 100%; the CLI uses fractions summing to one. Choose a fixed seed for reproducibility. Generate the alloy and check the resulting species counts before export. For small hosts, integer atom counts limit the achievable composition.

The example host is a $5 \times 5 \times 5$ repetition of conventional FCC Cu: 500 sites. The 70% Cu / 30% Ni assignment uses seed 42. Keep the same host atom ordering and seed when reproducing an assignment; changing the host can change which particular sites are substituted.

```
AtomForge --build sss --input cu_host.cif --frac "Cu=0.7,Ni=0.3" --seed 42 --output
    cu_ni_alloy.cif
```

The menu is controlled by `ATOMFORGE_ENABLE_SSS_BUILDER`, enabled by default. A seed of zero in the CLI requests a time-dependent seed. After substitution, an external relaxation may be needed to account for atomic size mismatch.

## 2.4 CSL Grain Boundary

**Purpose.** Construct a coincidence-site-lattice bicrystal from a compatible cubic periodic reference. A coincidence index $\Sigma$ describes lattice coincidence, not a calculated grain-boundary energy.

**Procedure.** Load the reference cell in the grain-boundary dialog. Enter a rotation axis $[u\ v\ w]$ and select a supported $\Sigma$ and candidate boundary plane. Set the thickness/repetition of each grain, any gap and vacuum, and an overlap-removal distance. Build and inspect both the interface and periodic cell boundaries. Save the bicrystal and relax it externally before comparing energies.

**Controls.** The axis is a crystallographic direction. The plane selector chooses from generated candidates. The CLI's `--plane` is a zero-based candidate index, not a Miller index. `--uca` and `--ucb` control stacking repetitions for the two grains. `--gap` separates the grains; `--vacuum` adds vacuum on each side. `--overlap` removes close contacts using a distance in Å; an excessive cutoff can delete desirable interfacial atoms.

```
AtomForge --build gb --input cu_fcc.cif --axis "0 0 1" --sigma 5 --plane 0 --uca 3 --
    ucb 3 --overlap 1.5 --output cu_sigma5.cif
```

This manual's generated example contains 60 atoms. The small thickness makes it a geometry demonstration, not a converged boundary-energy supercell. If `--sigma` is omitted, candidate search uses `--sigmamax`; specify the exact value when reproducibility matters. `--conventional` skips the primitive-cell conversion. Check both lattice compatibility and the selected candidate before interpreting a failed build as a file-format problem.

## 2.5 Nanocrystal: shape cuts

**Purpose.** Fill a finite shape with atoms from a periodic crystal. Load a periodic reference, select **Shape Cut**, choose a shape, set dimensions and centre, and use automatic replication unless you need explicit repetition counts. Build the nanocrystal and inspect the output bounding cell.

| Shape | Main dimensions |
|---|---|
| Sphere | Radius. |
| Ellipsoid | Radii along the three local axes. |
| Box | Half-lengths along the three axes; full side lengths are twice these values. |
| Cylinder | Radius, full height, and cylinder axis. |
| Octahedron | Size parameter for the octahedral cut. |
| Truncated octahedron | Size and truncation parameters. |
| Cuboctahedron | Size parameter for the cuboctahedral cut. |

**Auto-center from atoms** derives a centre from the reference. Manual centring changes which lattice sites intersect the boundary and can change surface termination and atom count. **Auto-replicate** chooses enough source repetitions for the cut; manual repetitions that are too small truncate the intended object. **Rectangular cell** and vacuum settings provide an enclosing output cell; this does not make a nanoparticle a bulk material.

```
AtomForge --build nano --input cu_fcc.cif --shape sphere --radius 10 --vacuum 5 --
    output cu_sphere.xyz
```

The example contains 336 atoms. Use `--rx/--ry/--rz` for ellipsoid radii, `--hx/--hy/--hz` for box half-lengths, and `--cylradius/--cylheight/--cylaxis` for the cylinder. Explicit source replication uses `--repa`, `--repb`, and `--repc`; zero selects automatic sizing in the CLI.

## 2.6 Nanocrystal: Wulff construction

Select **Wulff Construction** in the nanocrystal dialog. Add facet families, each with nonzero $(hkl)$ indices and a positive relative surface energy. AtomForge expands families using the reference symmetry, previews the bounding planes, and cuts the periodic crystal by the resulting shape.

1. Load the periodic reference and check its orientation and symmetry.
2. Add, for example, (100) and (111) families with illustrative energies 1.0 and 0.9. These are demonstration ratios, not material data.
3. Set **Max radius (A)**. The implementation scales plane distances by energy ratios, placing the highest-energy family at this maximum distance. This control is not necessarily the circumscribed vertex radius of the final polyhedron.
4. Inspect the **Wulff Plane View**; family colours help identify the contribution of each family. Remove redundant families or add missing ones as required.
5. Build the atomic nanocrystal. Inspect discrete surface termination separately from the continuous plane preview, then export.

The construction requires a bounded intersection of planes. A family may disappear from the final shape when other planes enclose it. Energy ratios must come from comparable surface calculations if a physical equilibrium-shape interpretation is intended. Atomic discretisation can break an ideal continuous shape's apparent symmetry in a very small particle. The same construction is available through Python `af.wulff` and native `--buildnano--shapewulff`; use repeated `--facet"hklenergy"` arguments for the families.

## 2.7 Custom Structure from OBJ/STL

**Purpose.** Fill a closed mesh with lattice sites. The mesh is a spatial boundary, not a collection of atoms. Use a watertight OBJ or STL, a periodic atomic reference, a physical scale, and an orientation.

**Procedure.** Open **Build > Custom Structure**. Load/drop the reference and mesh into their corresponding inputs. Set the mesh scale in Å per model unit, then adjust rotation or the Miller-direction alignment. Select enough replication to cover the mesh, set vacuum, and build. Inspect thin features and the atom count; a feature narrower than the lattice spacing may contain no sites.

```
AtomForge --build custom --input cu_fcc.cif --mesh octahedron.obj --scale 12 --vacuum 5
    --output cu_mesh.xyz
```

The included octahedron has six vertices at the positive and negative unit axes and eight triangular faces. At scale 12 it produces 224 Cu atoms in this example. No Blender installation is required. To use Blender, create a closed mesh, apply the intended object transforms before export, export OBJ/STL, and verify the scale in AtomForge. Inspect for holes, self-intersections, and reversed or inconsistent faces when the fill is missing or unexpected.

The CLI provides `--rotx`, `--roty`, and `--rotz` in degrees. `--miller"hkl"` aligns a crystallographic direction with $+Z$ and overrides Euler orientation. A crystallographic direction $[hkl]$ and a plane normal $(hkl)$ differ in a general non-cubic cell; choose the alignment convention deliberately.

## 2.8 Polycrystal

**Purpose.** Populate a box with grains cut by a Voronoi partition. Inputs include a periodic reference, box dimensions, grain count, random seed, and optional orientations. The construction creates geometric grain boundaries; it does not optimise their energies.

**Procedure.** Load a reference, enter box lengths and grain count, set the seed, and choose orientations or random assignment. Generate, inspect the grain sizes and contacts, and save. Use orientation colouring when the output retains orientation metadata. Reopening a format that lacks grain metadata can preserve atoms while losing those colours.

```
AtomForge --build poly --input cu_fcc.cif --sizex 25 --sizey 25 --sizez 25 --grains 4
    --seed 7 --output cu_polycrystal.cif
```

This example contains 1,329 atoms. The CLI accepts repeated `--euler"phi1Phiphi2"` triplets in Bunge convention, one per grain; unspecified orientations are random. Use the same seed, dimensions, reference, and ordering to reproduce a sample. Increase the box relative to grain count when grains become too small to have a meaningful crystalline interior.

## 2.9 Amorphous Structure

**Purpose.** Produce a random packing subject to distance constraints. Specify elements and counts, a mass density or explicit box, pair-distance constraints, and a seed. This is an initial configuration generator; random packing alone does not establish a realistic glass network.

**Procedure.** Add the required species/count rows. Set density in $\mathrm{g\,cm^{-3}}$ or explicit box dimensions. Check the minimum separation settings; overly strict cutoffs at a high density can make placement impossible. Set a fixed seed, generate, and inspect the reported inserted count and any placement failure. Relax or equilibrate the configuration with a suitable external method before structural or energetic claims.

```
AtomForge --build amorphous --element "Si 40" --element "O 80" --density 1.5 --seed 7
    --output sio2_pack.xyz
```

The example has 120 atoms with a 1:2 Si:O ratio. The deliberately low packing density is useful for demonstrating construction and is not a claim about silica's equilibrium density. `--boxa/--boxb/--boxc` override density-derived dimensions. `--mindist"Z1Z2distance"` adds a pair constraint using atomic numbers; `--covtol` controls the covalent-radius fallback; `--attempts`

bounds placement attempts. If placement fails, examine the geometry and constraints before merely raising the attempt limit.

## 2.10 Defect, lattice and nanostructure builders (command line)

Seven further builders run from `AtomForge--buildMODE--inputFILE--outputFILE` and from `atomforge.build(mode,...)` in Python. `AtomForge--helpMODE` lists every option.

- **vacancy** removes randomly chosen atoms, optionally of one `--element`, by `--percent` or exact `--count`. `--min-separation` keeps vacancies at least that far apart, measured through periodic images when the input has a cell. A fixed `--seed` reproduces the same vacancies. Requesting more vacancies than eligible atoms is an error, not a silent cap. The remaining atoms stay on their unrelaxed lattice sites; relax the result before computing formation energies.
- **strain** applies a homogeneous deformation gradient to the cell while keeping fractional coordinates fixed. Give engineering components (`--exx` ... `--eyz`; the symmetric shear terms enter $F = I+\varepsilon$) or a full `--matrix`. The volume scales by $\det F$. Internal relaxation is not performed.
- **primitive** reduces a periodic structure to its spglib-standardized primitive cell. The result may be rotated into the standard setting.
- **surface** cleaves a slab parallel to the Miller plane $(hkl)$ of the input cell, with `--layers` repeats of the shortest out-of-plane lattice translation and `--vacuum` padding. The slab is oriented with its surface normal along the third cell vector. By default the input is first reduced to its primitive cell (fewer atoms per layer); Miller indices are converted to that basis so they always refer to the input cell. Layers are stacked along the true lattice translation, so close-packed stacking (for example fcc ABC) is preserved. For fcc, the exposed atoms of (111), (100) and (110) slabs have 9, 8 and 7 nearest neighbours.
- **sqs** rearranges species on the fixed input lattice to a target composition (`--element"SYMBOLFRACTION"`, repeated) by simulated annealing that drives Warren–Cowley parameters of the first `--shells` shells toward zero. It is a local optimizer of short-range order, not a full cluster-expansion correlation match. Check the reported objective and the SRO table, and use enough sites and steps for the target composition.
- **nanowire** cuts a wire periodic along cell vector `--axis` with a circular (`--sides` below 3) or regular-polygon cross-section of apothem `--radius`. The square cross-section box has side 2(radius + vacuum), so neighbouring images are separated by about twice the vacuum. `--axis-repeats` lengthens the periodic cell.
- **core-shell** relabels a finite structure: atoms closer than `--core-radius` to the centroid (or `--center`) become `--core-element`, the rest `--shell-element`. Positions are unchanged; relax mismatched core/shell lattices before analysis.

## 2.11 Optional and unavailable builder menus

The stacking-fault builder is compiled into the desktop menu only with `ATOMFORGE_ENABLE_SFE_BUILDER=ON`; the default is ON. Its purpose is to construct faulted configurations, not to calculate a stacking-fault energy. In an enabled build, open **Build > Stacking Faults**, supply a reference with **Use Current Scene** or the source workflow, and click **Detect Structure**. Select a supported Plane, choose **Smallest Unit Cell** or **Orthogonal Cell**, and set Layers (2–96). Click **Generate Sequence**, use the Structure slider to inspect its members, and choose **Use Selected Structure** to adopt one. **Export All Structures** writes the sequence to the specified Folder/Subfolder; **Use**

**Source Folder** supplies a convenient base when available. Validate the stacking geometry and calculate energies externally. The current source build enables this menu; the CLI can export a complete sequence.

The interface-builder dialog has a drawing call but no current main-menu entry to open it. Its source/API capabilities are described in Appendix C. Use the exposed merge/transform workflows for manual assembly in the current executable.

## 2.12 Worked study: compare bulk and finite Cu environments

This exercise connects building, file inspection, and structural analysis. It uses only generated structures and does not require an electronic calculation.

1. Open `examples/cu_fcc.cif`. Inspect its four sites and cubic cell. The cell edge is 3.61 Å. Save a working copy before changing it.
2. Open the supplied `cu_host.cif`, or create it by repeating the conventional cell $5 \times 5 \times 5$ through Python. It contains 500 atoms. Use this larger periodic host for bulk neighbour statistics.
3. Open `cu_sphere.xyz` separately. It contains 336 atoms from the same reference lattice, cut at radius 10 Å. Rotate it and inspect surface sites. The smaller count is caused by the finite shape, not by a different crystal symmetry.
4. Run RDF on the periodic host with Cu as both species, PBC enabled, and no smoothing. Check the first-neighbour peak near 2.553 Å. Begin with a cutoff that is comfortably smaller than the box dimensions.
5. Run RDF on the finite sphere with PBC disabled. Compare raw counts and cumulative coordination before comparing normalised curves: the particle has surface atoms with incomplete neighbour shells and its enclosing vacuum does not define a bulk atomic density.
6. Open `cu_ni_alloy.cif`. Verify 350 Cu and 150 Ni atoms, then inspect Cu–Cu and Cu–Ni partial distributions and the SRO table. The unrelaxed substitution changes labels while retaining ideal host positions; it cannot demonstrate size-mismatch relaxation.

Keep the source filenames, cutoffs, species choices, PBC setting, bin count, and smoothing with each plot. If the bulk nearest-neighbour peak moves unexpectedly, first check lattice scaling and units. If coordination falls at the surface of the sphere, inspect its geometric boundary before treating the change as disorder. If the alloy has nonzero measured SRO, compare seeds and finite-size effects before interpreting it as evidence of an energetic ordering preference.

This comparison illustrates why a common visual appearance does not guarantee comparable statistics. The bulk, particle, and alloy answer different questions even though they originate from the same Cu cell. Choose analysis settings according to the geometry and composition of each saved structure.

## 2.13 Builder control reference

The following reference supplements the worked examples with the controls that are easy to miss in a scrolled dialog. Open the appropriate Build command, then scroll its settings panel independently of its preview. Drag numeric fields for exploration; Ctrl+click an ImGui drag field to enter a precise value. Check the result after rebuilding: changing a parameter does not necessarily regenerate the atomic structure.

### 2.13.1 Bulk Crystal: symmetry, lattice, and site editor

**Crystal system.** Select the system first. This restricts the space-group list and determines which lattice parameters can be entered. Changing system can change the valid group selection, so recheck the group before building.

**Space group.** Choose the numbered group in the filtered list. The asymmetric-unit sites are expanded using this symmetry. A site on a special position has fewer distinct equivalents than a general site.

**Lattice parameters.** Lengths are in Å; angles are degrees. Triclinic exposes all six values. Monoclinic exposes $a, b, c, \beta$ with $\alpha = \gamma = 90°$. Orthorhombic exposes $a, b, c$ with right angles. Tetragonal exposes $a, c$ with $b = a$. Trigonal uses the hexagonal setting, as does Hexagonal: $b = a$, $\alpha = \beta = 90°$, $\gamma = 120°$. Cubic exposes $a$, with $b = c = a$ and right angles. Do not enter rhombohedral primitive-cell parameters into the trigonal hexagonal-setting controls.

**+ Add**, **Clear**, **Delete**. Add creates an independent site row. Clear empties the asymmetric-unit list; Delete removes only its row. A nonempty valid site list is required for a useful build.

**Site coordinates.** The three values are direct coordinates, not Cartesian distances. Enter one representative of each independent site. Negative or out-of-cell input should be checked in the symmetry-expanded result rather than treated as an extra supercell site.

**Element button and Apply Element.** Click the row's element button, choose an element in the periodic table, and apply it to that row. This selects chemical identity; it does not set oxidation state, occupancy, magnetic moment, or a pseudopotential.

**Build and Close.** Build generates the expanded crystal and updates the main scene when successful. Read the status and inspect atom count. Close dismisses the dialog; save the generated scene with File > Save As.

**Diagnostic example.** For NaCl, use group 225 and independent Na $(0, 0, 0)$ and Cl $(1/2, 1/2, 1/2)$ sites. The conventional cell should contain four of each species. Use an appropriate lattice parameter for the intended study. If the count is wrong, check group, coordinates, and duplicated independent rows before expanding the cell.

### 2.13.2 Substitutional Solid Solution: every composition control

**Source / Use Current Scene.** Load a host or copy the current scene into the source preview. Composition controls become useful after a nonempty host is present. The host supplies the sites to relabel.

**Element rows.** Click an element symbol to select a species. **+ Add Element** adds another species; the row's X removes it when more than one row exists. Avoid duplicate species rows when specifying a composition.

**at%.** The GUI uses atomic percentages from 0 to 100. The *last row is a read-only remainder*: its percentage is 100 minus the preceding rows. To make Cu–Ni 70:30, put Cu first and Ni last, then enter 70 for Cu. To make a ternary 50:30:20, enter 50 and 30 in the first two rows. The last row becomes 20. Recheck the whole table after adding or removing a row.

**Atom count.** The displayed count is a consequence of the percentage and host size, not an independent composition input. Integer site allocation limits resolution to approximately $100/N$ percentage points for $N$ sites.

**RNG seed.** A positive fixed integer makes the site assignment reproducible for the same input ordering. Zero selects a random/time-dependent assignment; negative input is clamped to zero.

**Build Solid Solution.** Apply the assignment, then verify counts in Structure Info. The CLI equivalent uses fractions such as `Cu=0.7,Ni=0.3`; do not paste GUI percentages into `--frac`.

**If a percentage cannot be edited:** first check whether it is the last row. This is the balancing row, not a disabled builder. Change the preceding rows or their ordering/composition instead.

### 2.13.3 CSL Grain Boundary: search, geometry, and result

**Rotation axis** $[uvw]$**.** Enter a nonzero integer direction for the cubic reference. Changing the axis requires a new candidate search.

**Maximum Sigma and Find.** The maximum bounds the candidate search; Find populates the Sigma selector. A larger bound expands the search, not the physical sample size.

**Sigma and GB plane.** Select a coincidence candidate, then a boundary plane from the generated choices. The plane selector contains candidate planes; the CLI index 0, 1, or 2 selects one of these choices. It is not the value of $h$, $k$, or $l$.

**Grain A (uc), Grain B (uc).** Set repetitions used to thicken each grain along the stacking construction. Use both controls to study symmetric or unequal thicknesses. These are integer cell repetitions, not distances in Å.

**Vacuum (A), Gap (A).** Vacuum adds the specified outside spacing in the construction; Gap separates the stacked grains. Both alter geometry. Neither is a relaxation or a search over rigid translations.

**Overlap distance (A).** Remove interfacial close contacts within the chosen cutoff. Inspect the removed-atom count and resulting stoichiometry. Set a conservative cutoff and compare before raising it.

**Conventional cell.** Retain the conventional construction when enabled; otherwise the builder attempts its primitive grain-boundary cell reduction. This can change the final cell representation and atom count.

**Build and result summary.** Build updates the main structure. Record Sigma, misorientation angle, boundary plane/type, axis, CSL matrix, grain repetitions, gap, vacuum, input/generated counts, and removed overlaps. These identify the geometry more precisely than a filename containing only Sigma.

**Troubleshooting.** If Find produces no appropriate candidate, verify the axis and cubic reference first. If Build produces too few atoms, compare overlap removal and primitive reduction before changing the reference. This dialog has no general rigid-translation energy-search control.

### 2.13.4 Nanocrystal: shape parameters and shared output controls

All shape dimensions below refer to the local shape coordinates about the chosen centre. For relative coordinates $(x, y, z)$, the implemented cuts are:

| Shape | Parameters and membership rule |
|---|---|
| Sphere | Radius $R$: $x^2 + y^2 + z^2 \leq R^2$. |
| Ellipsoid | Semi-axis $a, b, c$: $(x/a)^2 + (y/b)^2 + (z/c)^2 \leq 1$. |
| Box | Half-size X/Y/Z: each absolute coordinate is bounded by its half-size. |
| Cylinder | Radius and full Height, with Axis X/Y/Z selecting the longitudinal direction. |

| Shape | Parameters and membership rule |
|---|---|
| Octahedron | Octahedron radius $R$: $\|x\| + \|y\| + \|z\| \leq R$. |
| Truncated octahedron | Octahedron R and Truncation R: both the octahedron bound and $\max(\|x\|, \|y\|, \|z\|) \leq R_t$ apply. |
| Cuboctahedron | The currently named cut uses the three bounds $\|x\| + \|y\| \leq R$, $\|y\| + \|z\| \leq R$, $\|x\| + \|z\| \leq R$. Use this implemented definition when reproducing its geometry. |

**Auto-center from atoms / Center X,Y,Z.** Automatic centring follows the reference atoms. Disable it to specify a Cartesian centre in Å. A shift smaller than a lattice spacing can still change the surface termination.

**Auto-replicate / Rep a,b,c.** Automatic mode sizes the source coverage. Manual mode uses the three integer repetitions; increase all necessary directions if the shape is clipped by insufficient source coverage.

**Rectangular cell / Vacuum.** Choose a rectangular enclosing output cell where needed and add the requested surrounding vacuum in Å. Inspect the saved cell in a periodic format; a plain XYZ consumer may ignore the cell.

**Wulff family editor.** Each row has $h, k, l$, a positive relative energy, a display colour, and Remove. Add Family adds a row. The last remaining family cannot simply be removed with the row control. Add enough symmetry-related constraints to bound the shape; zero indices or nonpositive energies are not meaningful input.

**Max radius and plane preview.** Max radius sets the distance of the highest-energy family, with others proportional to energy. Family colour changes the plane preview, not surface energy. Build the atoms again after changing family energies or geometry.

### 2.13.5 Custom Structure: reference, mesh, and orientation

**Reference source, Clear, Use Current Scene.** Load a periodic reference or copy the current scene. Clear removes that source. The separate mesh Clear control clears the mesh, not the atomic reference.

**OBJ/STL source and Scale.** Load the enclosing mesh and enter Å per model unit. A unit-radius mesh at scale 12 represents a 12 Å radius along the corresponding model axes. Verify the preview's physical extent before building.

**Auto-center / Center X,Y,Z.** Automatic centring derives placement from the reference; manual values specify the centre. Inspect reference and mesh together to avoid filling an unintended region.

**Apply crystal orientation.** Enables the orientation controls. The current radio labels **1** and **2** select Euler-angle and Miller-direction modes respectively. In mode 1 enter Rot X/Y/Z in degrees. In mode 2 enter $h, k, l$ for direction alignment. This rotates the crystal filling relative to the mesh; orbiting the preview camera does not substitute for it.

**Auto-replicate / Rep a,b,c.** Ensure enough periodic sites to cover the whole mesh. Manual undercoverage can look like a broken mesh cut.

**Output enclosure.** Set rectangular output cell and Vacuum configure the enclosure and surrounding spacing. These do not repair holes in the mesh or close a non-watertight boundary.

**Build Fill / Close.** Build Fill generates the atomic product from both inputs. Verify the count and thin sections before saving. Close dismisses the builder.

### 2.13.6 Polycrystal: dimensions, randomisation, and individual grains

**Reference and X/Y/Z.** Supply a periodic reference and the three target box lengths in Å. The dimensions define the total specimen, not the size of every grain.

**Number of grains.** Sets the number of Voronoi seeds/grains. Increasing it at fixed volume reduces typical grain size. Check that grains contain sufficient interior sites for the intended analysis.

**Seed.** Keep a fixed seed to reproduce the stochastic construction along with all other inputs.

**All Random.** Generate all orientations randomly.

**All Specified.** Edit the Bunge Euler triplet $(\phi_1, \Phi, \phi_2)$ in degrees for every grain. The table grows with the grain count and may require scrolling.

**Partial (specify some, rest random).** Enter **Grain #** using the GUI's one-based numbering, then **Add Grain Orientation**. Duplicate grain entries are not added. Edit that grain's three angles; X removes the explicit override so the grain returns to random assignment. Unspecified grains remain random.

**Build / Close.** Build the sample and inspect orientation colouring and boundary contacts. Preserve the generated `.atomforge-ipf` sidecar beside an associated exported structure when retaining orientation metadata. Close leaves the generated scene available for saving.

### 2.13.7 Amorphous Structure: composition and packing constraints

**Element and Count rows.** Choose each species with its element button, enter its integer atom count, and use X or Add Element to change the list. Check the total and formula before choosing density.

**Auto from target density.** Enter mass density in $\mathrm{g\,cm^{-3}}$. The builder estimates a cubic box using the total atomic mass. The displayed side length includes the Cell scale factor.

**Manual dimensions.** Enter $a, b, c$ in Å to set box lengths explicitly. This mode replaces density-derived sizing.

**Cell scale factor.** Multiplies box lengths before packing. Scaling lengths by $s$ multiplies volume by $s^3$, so a density-derived box has effective density $\rho/s^3$. For example, $s = 1.1$ yields approximately $0.751\rho$. The target-density label alone therefore does not describe the final density when scaling is enabled.

**Periodic boundary conditions (attach unit cell).** Controls whether the output carries the unit cell. The current packing implementation checks minimum-image separations across the box even when this output-cell option is off. Retain a cell-capable format for subsequent periodic analysis and inspect cross-boundary contacts.

**Advanced Settings.** Expand this section to reveal placement controls and the element-pair table. These are part of the builder, not separate menu commands.

**Covalent radius tolerance.** The fallback minimum separation is $(r_i^{\mathrm{cov}} + r_j^{\mathrm{cov}})t$. Smaller $t$ permits closer contacts; it does not calculate bonding.

**RNG seed (0 = time).** Use a nonzero integer to repeat the random placement. Zero requests a time-dependent seed.

**Max attempts per atom.** Bounds trials for each requested atom. Raising it increases work but cannot guarantee a feasible packing.

**Element-pair minimum distances.** Check a pair's row to enable its explicit minimum distance in Å. An unchecked row uses the displayed covalent-radius fallback. Apply different overrides only when their chemical/geometrical interpretation is intentional.

**Build and status.** Compare requested and inserted counts. If atoms are skipped or placement fails, increase box size, reduce a conflicting minimum distance, or revise composition before simply increasing attempts. Save the final structure and record the actual density.

### 2.13.8 Optional Stacking Faults: sequence and export controls

In a build with the SFE option enabled, **Detect Structure** classifies the reference and populates supported plane choices. Select the Plane and the Smallest Unit Cell or Orthogonal Cell mode, then set Layers. **Interval** controls the increment of the displacement sequence; **Max Displacement** controls its maximum displacement factor. These are sequence-construction parameters, not energies in eV. Smaller intervals create more configurations and more export files.

Choose **Generate Sequence** after changing these inputs. The **Structure** slider selects a zero-based member of the generated sequence for preview. **Use Selected Structure** replaces the main scene with that member. For batch output, enter the base **Folder** and **Subfolder**, or use **Use Source Folder** where available, then **Export All Structures**. Unlike the current general file picker, this optional source implementation still contains text folder fields. Record each member's displacement alongside its externally calculated energy before constructing a stacking-fault-energy curve. This optional dialog was audited in source, not enabled for the manual screenshots.

# Chapter 3

# Editing, assembly, and defects

## 3.1 Selection and edit history

Click an atom in the main scene to inspect/select it. Left-drag orbits the camera; right-drag normally pans. When **Edit > Box Select** is active, right-drag draws a selection rectangle instead. **Lasso Select** uses a right-drag path for a more irregular projected region. Selection is based on the current projection, so inspect the result from another direction before acting on a dense or overlapping set of atoms. Selected atoms are outlined by an amber ring with a soft glow; where a selected atom is hidden behind others, its ring shows faintly through them, so selections inside a structure remain visible.

Open **Edit > Edit Structure** to work with the atom table. Use **Undo** and **Redo** where enabled, but retain saved checkpoints before substantial assembly or defect-generation operations. A camera reset changes only the view and does not restore a previous structure.

Ctrl+left-click adds/removes an atom from the selection. Ctrl+A selects all; Ctrl+D or Escape clears selection. Delete removes selected atoms, and right-click opens the selection context menu. Ctrl+Z undoes; Ctrl+Y or Ctrl+Shift+Z redoes. Use these shortcuts in the main workspace with no text-entry field focused.

## 3.2 Edit Structure: atoms and cell

The editor exposes cell vectors when a cell is present and an atom table with element and coordinate editing. **Position Mode** switches between **Cartesian** and **Direct**. Cartesian coordinates are in Å; direct coordinates are fractional lattice coefficients. Direct editing is disabled without a unit cell.

1. Open the source structure and save a working copy.
2. Choose **Edit > Edit Structure**. Confirm the coordinate mode before typing a position.
3. Select a row to edit the desired atom. Use the element selection and **Apply Element** controls for species changes.
4. Use **Add Atom** to create a new row, then immediately set its element and position. The initial atom is H at the origin; leaving it there may create an overlap.
5. Inspect changed positions and atom counts in the viewport and information dialog. Save the revised structure under a descriptive name.

Cell-vector edits define the simulation cell. Do not assume that typing a new box automatically performs the same operation as a homogeneous strain or a supercell construction. Check the

resulting atom coordinates and boundary relationships. Use the integer transformation tool for deliberate periodic supercells.

## 3.3 Transform Structure

**Edit > Transform Structure** accepts three rows of integer coefficients. It is available for a structure with a unit cell. Begin with a diagonal transformation to establish the convention: $(2, 0, 0)$, $(0, 2, 0)$, $(0, 0, 2)$ creates a $2 \times 2 \times 2$ periodic enlargement. For conventional FCC Cu this should contain 32 atoms when the reference contains four.

Enter the matrix, choose **Apply**, and inspect the resulting lattice vectors, volume, and atom count. **Cancel** discards the pending matrix. Non-diagonal matrices define combinations of lattice directions and are useful for rotated supercells. The matrix must be nonsingular; a zero determinant cannot define a three-dimensional cell. For valid integer supercells, the determinant magnitude gives the ideal volume/atom-count multiplier before additional cuts or deduplication. Verify this count rather than assuming any arbitrary matrix represents the intended orientation.

## 3.4 Merge Structures

**Purpose.** Assemble several structures in an interactive preview. Open **Edit > Merge Structures**, load/drop the components, and select one component at a time. Its transform and gizmo controls translate or rotate that component; camera movement changes the view of the entire assembly.

1. Prepare and save each input independently with consistent length units.
2. Add the inputs to the merge dialog. Check each component's identity before moving it.
3. Select a component and use the visible gizmo handles to position/orient it. Inspect from at least two directions; two objects that appear separated in one projection can intersect in three dimensions.
4. Toggle **Show Bounding Box** to inspect the enclosure. Use **Delete Selected** to remove an unwanted component from the assembly, or **Clear All** to start the assembly again.
5. Choose **Merge Structures**, inspect the resulting scene, and save a new structure.

Merging is geometric assembly. It does not find a low-strain epitaxial match, remove all close contacts, or determine a chemically correct interface termination. Inspect separations, stoichiometry, cell conventions, and periodic contacts before using the merged structure in a simulation.

## 3.5 Cell Sculptor: supercells and cutting slabs

**Edit > Cell Sculptor** combines a periodic source, supercell repetitions, and cutting planes/slabs. Use **Use Scene** to obtain the current structure or supply a source through the dialog's load/drop workflow. The source and result previews help distinguish the uncut periodic model from the generated product.

**Procedure.** Set the supercell repetitions first. Add cutting slabs, enter nonzero Miller indices, choose periodic or distance-based limits, and inspect the plane preview. At least three independent slab-normal directions are needed to close the region; Generate stays disabled for an unbounded region. Set the outside-atom opacity to make excluded atoms less prominent, then choose **Generate** when the closed region contains the intended atoms. Save the resulting atomic structure only after checking which side of each boundary was retained.

In periodic slab mode, **Start** identifies the starting plane and **N** controls the number of structural repeats/layers along the slab normal. The displayed spacing derives from the lattice. A crystallographic repeat interval can include several chemically distinct atomic layers; do not equate **N** blindly with the number of visible atomic sheets. Distance-based limits use Å and are useful for a finite cut without an integer repeat interpretation.

For a lattice with reciprocal vectors defined without $2\pi$,

$$\mathbf{a}^* \cdot \mathbf{a} = 1, \quad \mathbf{a}^* \cdot \mathbf{b} = \mathbf{a}^* \cdot \mathbf{c} = 0, \qquad d_{hkl} = \frac{1}{|h\mathbf{a}^* + k\mathbf{b}^* + l\mathbf{c}^*|}.$$

The normal to a plane is a reciprocal-space combination. For a skew cell it is generally not the same Cartesian direction as $h\mathbf{a} + k\mathbf{b} + l\mathbf{c}$. Use the preview to confirm the orientation before generating a slab. Geometric cutting can create polar surfaces or nonstoichiometric terminations; choose terminations according to the intended study.

## 3.6 Insert Dislocation

**Purpose.** Apply a continuum-style displacement construction to a structure. Open **Edit > Insert Dislocation**, use the current scene or load a reference, and use **Detect Lattice** to inspect the lattice classification before choosing directions.

**Character and region.** Choose Edge, Screw, or Mixed character. Available region shapes are Half-plane, Cylinder, Sphere, Ellipsoid, and Freeform 2D. These control the construction region. They do not by themselves certify a particular relaxed dislocation-core structure or a stable physical loop.

1. Choose a sufficiently large reference and a fixed cell geometry.
2. Enable **Auto lattice vectors** for inferred directions, or enter the slip-plane indices, Burgers direction, and line direction manually. Use crystallographically compatible, nonzero directions.
3. Locate the line using a fractional point when convenient, or the Cartesian position/offset controls. Confirm its position in the preview.
4. Set Burgers-vector scale, Poisson ratio, core radius, cutoff, and line extent. For Mixed character, set the mixed angle. Supply the selected shape's dimensions or the Freeform 2D points.
5. Choose **Generate Dislocation**. Inspect RMS and maximum displacement and the displaced structure.
6. Use **Replace Main Scene** to adopt the generated result, then save a new file.

The core radius regularises the near-core field; it is a modelling parameter, not an experimentally measured core width inferred by AtomForge. The continuum approximation is least reliable near the core. Periodic boundary conditions can require compatible dislocation arrangements; an isolated displacement construction in a periodic box may create artificial long-range strain. Relax and validate with an appropriate external model.

## 3.7 Add Interstitial Atoms and detect voids

**Purpose.** Find geometric void candidates and place chosen atoms in selected subsets. Open **Edit > Add Interstitial Atoms**, select **Use Current Scene** or provide a host, and run **Detect Voids**.

**Detection controls. Grid** controls the search resolution (8–30 in the current UI); **Max Voids** limits candidates. Higher resolution can find different candidates at higher cost. Use the void-type filters to include the desired coordination geometries. **Select All Types** and **Clear All Types** manage those filters. **Show Void Polyhedra in Scene** helps relate candidate centres to their coordinating atoms.

**Placement modes.**

**Random distribution** Choose a percentage or atom count and a seed. The percentage refers to eligible candidate sites, not automatically to the final atomic composition.

**Inside OBJ/STL mesh region** Load a mesh region and transform it with Translate (T), Rotate (R), and Scale (S). Inspect the mesh relative to the host; **Reset Transform** restores its transform. Choose a target percentage/count among eligible sites in that region.

**Manual (selected voids only)** Select candidate centres in the preview and check the displayed selected-centre count. The selected subset determines placement.

Use **Select Element** to choose the inserted species, then **Apply Interstitials**. Inspect the atom-count change and nearest-neighbour separations before saving. A void is a geometric candidate, not a calculated formation-energy minimum. Site preference, charge state, local relaxation, and solubility require separate physical calculations.

## 3.8 Structure pipeline: non-destructive editing

The other editing tools change the structure directly. **Edit > Structure pipeline** edits it *non-destructively*, in the spirit of OVITO: the structure becomes the *input* of a pipeline, an ordered list of modifiers that each transform the result of the one before, like commands in a shell pipe. The input is kept unchanged. Changing, bypassing, reordering or deleting any modifier recomputes the output, which the view shows, so an edit made early in a long sequence never has to be redone by hand.

**Working with the panel.** Press **Start from the active structure**, or simply add a modifier: the first one starts the pipeline on the active structure. **Add modifier** lists the modifiers by category and inserts the chosen one after the selected step (or at the end). The list shows the input, each step and the output on a connecting line, with the number of atoms, the number of selected atoms and the time after every step; a step that fails shows its message in red and stops the pipeline there, so the output is the data before it. Click a step to edit its parameters below the list; the output follows every change. Untick a step to bypass it without losing its settings. Every step can be moved, replaced and removed. *Drag* a step by its grip (the six dots at its left) or by its name: a coloured line marks the gap between steps where it will land, the step can be dropped above the first or below the last step, and the list scrolls when the step is held near its top or bottom edge (Esc cancels). *Right-click* a step for **Edit in dialog**; **Replace with**, which swaps it for another modifier in the same place, keeping whether it is enabled, its name and the settings both modifiers share (for example the combine mode of selection steps); **Insert above** and **Insert below**; **Move up**, **Move down**, **Duplicate**, **Enable**/**Disable** and **Delete**. Right-clicking empty space in the list offers **Add modifier**. The up and down arrows and **X** on each row, **Move up**, **Move down**, **Duplicate** and **Remove** above the parameters, and the keys Alt+Up, Alt+Down and Delete for the selected step do the same. Bypassed steps stay draggable and editable. A step may be given a **Name**, for example “Upper half”. Untick **Update automatically** to batch several edits on large structures and press **Update**. **Colour atoms by selection** colours the output by the current selection (1 selected, 0 not), which shows what a selection step picked. The panel is an ordinary window: move, resize or close it; the pipeline keeps working while it is closed.

Edits made with other tools while a pipeline is active change only its current output and would be replaced at the next update; the panel then warns and offers **Use it as the pipeline input** or **Restore the pipeline output**. **Use the active structure as input** restarts the pipeline from the structure in the view. **Bake** makes the output the structure (one undoable edit) and ends the pipeline; **Discard** ends it and shows the input again. Pipeline updates themselves are not entered into undo history. Projects save the pipeline, its options and its unmodified input (project format 3); **Save pipeline** and **Load pipeline** exchange the modifier list alone as a JSON or text file, to apply the same steps to other structures.

**Selection.** As in OVITO, many modifiers act on the *selection*: selection modifiers choose atoms, and the modifiers after them (delete, assign element, displace, ...) act on those atoms. Every selection modifier has a **Combine** mode: *replace* the selection, *add* to it, *subtract* from it or *intersect* with it, so complex selections are built from simple steps.

**Modifiers.**

**Selection** `select-element` (element list); `select-expression` (a condition, below); `select-slab` between two coordinates along x, y or z (Å) or a, b or c (fractional, wrapped into $[0, 1)$); `select-sphere` within a radius of a point (minimum-image distance in periodic cells); `select-property` in a range of the per-atom property; `select-random`, exactly round($fN$) atoms, reproducible for a seed; `expand-selection` adds the neighbours within a cutoff, repeated for a number of steps; `invert-selection`; `clear-selection`.

**Modification** `delete-selected`; `assign-element` to the selected (or all) atoms, e.g. for substitutions; `displace` by a vector; `random-displacement`, Gaussian with the given standard deviation per component, reproducible for a seed; `slice` with the plane $\hat{n} \cdot \mathbf{r} = d$: it removes the half-space above the plane, or with a width the slab $|\hat{n} \cdot \mathbf{r} - d| \leq w/2$ (**Invert** keeps that region instead, **action select** only selects it).

**Cell** `replicate` along a, b and c; `supercell`, the cell of **Edit > Transform Structure**: new cell vectors that are integer combinations of a, b and c (nine integers, row by row, e.g. `supercell"110-110001"`), filled with the atoms; `transform`, the affine map $\mathbf{x}' = \mathbf{x}M$ of atoms and, for all atoms, the cell (row vectors); `strain`, homogeneous normal strains and engineering shears (twice the tensor components) of atoms and cell; `wrap` atoms into the cell; `center` the atoms' extent in the cell; `add-vacuum`, lengthening one cell vector, optionally centring the atoms, to make slabs.

**Simulation** The Analysis-menu tools whose result is a structure, with the parameters of their windows: `relax` (**Structure relaxation**), `nvt-dynamics` and `npt-dynamics` (the final frame of **NVT dynamics** and **NPT dynamics**) and `standardize-cell` (the primitive, conventional or symmetrized cell of **Symmetry and Wyckoff positions**). JSON-valued inputs such as the potential are given as their JSON text, e.g. `relaxcalculator='\{"potential":"EMT"\}'fmax=0.01`. Relaxation and dynamics keep the atoms' selection and metadata.

**Analysis** `compute-property`: the Ackland–Jones structure type, centrosymmetry, coordination within a cutoff or Steinhardt $q_6$ (§9.11.4), as the per-atom property for colouring and `select-property`. For example `compute-propertycentrosymmetry|select-property0.5100|delete-selected` removes defect-core atoms. The Ackland–Jones method tolerates one missing neighbour, so atoms next to a single vacancy remain fcc; select those by coordination instead.

**Edit** `add-atom` at a Cartesian or fractional position; `set-cell`, new lattice vectors with or without scaling the atoms; `merge`, the atoms of another structure file at an offset; `insert-interstitials`, filling tetrahedral, octahedral or irregular voids (found as by **Add Interstitial Atoms**) largest first or at random — an fcc cell has one octahedral and two tetrahedral sites per atom.

**Build and Build & edit** Every builder and editing tool of the Build and Edit menus is a step too, running the same code: `build-bulk`, `build-custom`, `build-gb`, `build-poly`, `build-nano`, `build-amorphous`, `build-interface` and `build-nanowire` make a structure; `build-surface`, `build-vacancy`, `build-sss`, `build-sqs`, `build-strain`, `build-primitive`, `build-dislocation`, `build-stacking-fault`, `build-core-shell` and `build-sculpt` (the Cell Sculptor) change the structure from the steps above. Their **Options** are the flags of `AtomForge--buildMODE` (Chapter D lists them; the step's help shows them too), for example `build-vacancy--count4--seed2` or `build-sss--frac"Cu=0.75,Ni=0.25"`. Steps that take a structure receive the current one; for grain boundaries, polycrystals, nanocrystals, custom structures and dislocations **Use the pipeline's structure as input** chooses whether it seeds them, and for interfaces it is layer A. The structure is passed in CIF, so coordinates are kept to about $10^{-5}$ in fractional units, and the selection is cleared after these steps.

Grain colours, region ids and per-atom properties follow the atoms through deletions and replication.

**Selection expressions.** `select-expression` selects atoms for which a condition is nonzero, for example `fz>0.5&&element==O` or `abs(x-10)<2orZ==29`. Variables: `xyz` (Å), `fxfyfz` (fractional, 0 without a cell), `index` (from 0), `count`, `element` (also `type`), `Z`, `property` (the per-atom property, NaN when absent) and `selected`. Operators: `+-*/\let\begingroup\let\relax\relax\endgroup\unhbox\voidb@x\bgroup\let\unhbox\voidb@x\setbox\@tempboxa\hbox{\global\mathchardef\accent@spacefactor\spacefactor}\let\begingroup\let\typeout\protect\begingroup\def\MessageBreak{˙(Font)}\let\protect\immediate\write\m@ne{LaTeXFontInfo:\def{}oninputline102.}\endgroup\endgroup\relax\let\ignorespaces\relax\accent2\egroup\spacefactor\accent@spacefactor\let\begingroup\let\relax\relax\endgroup\begingroup\let\relax\relax\endgroup` (power), `<<=>>===!=`, `&&||!` (also `andornot`) and parentheses; functions `abssqrtexplogsincostanfloorceilminmax`; `pi`. Element names may be written with or without quotes; they can only be compared with `==` and `!=`.

**Pipelines as commands.** A pipeline is also a line of text: modifier ids separated by `|`, each followed by its parameters in order or as `name=value`. Vectors take three numbers (or `a,b,c`), and an expression or element list takes the rest of its step; `||` inside an expression is the logical operator, not a pipe. **Copy as command** copies the pipeline as an `AtomForge--pipe` command, and **Edit as text** edits it in this form. The command line runs the same pipeline without the desktop:

```
AtomForge --pipe "replicate 4 4 4 | select-random 0.02 seed=7 | delete-selected" --
    input cu.cif --output cu_vacancies.xyz
AtomForge --pipe --list
AtomForge --pipeline steps.json --input POSCAR --output out.cif --save-pipeline copy.
    json
```

`--input-` reads extended XYZ from standard input and `--output-` writes it (with the selection and the per-atom property as extra columns) to standard output, so AtomForge runs combine with each other and with other programs through real shell pipes; the per-step report goes to standard error (`--quiet` silences it):

```
AtomForge --pipe "replicate 3 3 3 | select-expression 'fz > 0.5' | assign-element Ni"
    --input cu.cif --output - \
  | AtomForge --pipe "select-element Ni | expand-selection 2.6 | invert-selection |
    delete-selected" --input - --output alloy.xyz
```

A failing step stops the command with exit status 1 and names the step. `--reportFILE` writes the per-step results as JSON, and `--pipe--list-json` lists every step with its parameters and its place in the menus.

**The same features in the menus.** Every pipeline step is also a menu option, and every structure-changing Build and Edit feature is a pipeline step; the step's tooltip in the menu names it. Features with their own dialogs keep them (the builders of the Build menu, **Edit Structure**, **Insert Dislocation**, **Add Interstitial Atoms**, **Transform Structure**, **Merge Structures**, **Cell Sculptor** and the Analysis tools above). The others open a one-step operation window with the step's parameters: **Build > Vacancies, Strained Cell, Primitive Cell, Surface Slab, Special Quasirandom Structure, Nanowire, Core-Shell Particle**; **Edit > Structure Operations** (delete atoms, assign element, displace, random displacement, slice, replicate, affine transformation, strain, wrap, centre, add vacuum, compute per-atom property); and **Edit > Select Atoms** (by element, expression, slab, sphere, property range, random fraction, expand, invert, clear), which sets the selection in the view and combines with it by its mode. The Build operations show one field per option of the matching command-line builder (empty fields keep the default shown in grey; repeatable options such as the SQS composition take entries separated by semicolons) and a field for any other flags. Operations that act on atoms apply to all atoms, to the atoms selected in the view, or to the atoms where a condition holds. **Apply** changes the structure as one undoable edit; **Add to pipeline** appends the same step (with the condition as a select-expression step) to the structure pipeline instead.

**Editing a step in its dialog.** Double-click a step, press **Edit in dialog** above its parameters or choose it from the step's right-click menu to open the same dialog as its menu entry, filled with the step's settings: the builder of a Build step, **Edit Structure** for `add-atom` and `set-cell`, **Transform Structure** for `supercell`, the tool window for the Simulation steps, or the operation window. Dialogs that work on a structure show the one entering the step. The main button reads **Update step**: it writes the dialog's settings into the step (for Build steps, as their command-line options) instead of building, and the pipeline updates its output. Loading another reference structure from a file in the dialog makes the step read that file.

**From Python.** `atomforge.pipeline` builds, edits and runs the same pipelines:

```
import atomforge as af
from atomforge.pipeline import Pipeline
s = af.load("cu.cif")
p = Pipeline().replicate(4, 4, 4).select_expression("fz > 0.5").assign_element("Ni")
p.add("build-vacancy", "--count 3 --seed 1")      # Build and Edit steps too
result = p.run(s)                                 # s is unchanged
print(result.structure, result.selected_count, [st["atoms"] for st in result.stages])
p.move(3, 0).disable(1).remove(2).set(0, counts=(2, 2, 2))
Pipeline.parse("replicate 2 2 2 | select-random 0.1 | delete-selected").save("steps.
    json")
s.pipe("add-vacuum c 15 | center")
```

Steps are methods named after the modifier (hyphens as underscores), `add`/`insert` with positional values in parameter order or keywords, or pipe text; `|` joins pipelines and text. `run` raises `PipelineError` naming the failing step (or with `check=False` returns the data before it), takes per-atom values for `select-property` with `property=`, and returns the structure, the selection, the per-atom property and the per-step results. `to_text`, `to_json`, `save` and `load` use the formats of the desktop and the command line. Core steps run in the native library; Build and Edit steps run through the AtomForge executable (`ATOMFORGE_PATH`).

## 3.9 Checking an edited model

Before exporting a simulation input, check species counts, cell vectors, boundary contacts, minimum interatomic distances, and the intended periodicity. For nanoparticles and slabs, check vacuum in every relevant direction. For substitutions and defects, compare composition before and after the edit. Save both the starting model and the exact edited model used for later calculations.

## 3.10 Detailed edit controls and application points

The main distinction between edit tools is when and where they change structure data. Table edits change the current structure immediately; Dislocation has a separate preview-generation and replacement step; Merge opens its committed result in a new tab; Interstitials commits to the current structure; Cell Sculptor commits to the current structure and closes on a valid Generate. Save an input checkpoint before a sequence of operations.

### 3.10.1 Atom selection and the right-click menu

**Box Select Mode / Lasso Select Mode.** The two Edit menu modes are mutually exclusive. Their projected selection includes atoms along the viewing direction. Change the view to check selection depth before deletion or substitution.

**Substitute Atom.** Select the intended atoms, open the context menu, choose this action and an element, then apply. It changes the species of the selected underlying atoms. A displayed periodic image maps back to its base atom; selecting an image does not create an independent species assignment for that image.

**Insert Atom at Midpoint.** Requires at least two selected atoms. Choose the new element and apply. With two selections, the new position is their displayed Cartesian midpoint. With more than two, the implementation uses the centroid of all selected displayed positions, $\mathbf{r}_{\mathrm{new}} = N^{-1} \sum_i \mathbf{r}_i$. It is not a periodic minimum-image midpoint. Select the appropriate images if atoms straddle a boundary.

**Measure Distance / Measure Angle / Atom Info.** Context actions require exactly two, three, or one selected atom respectively. Choose the middle/vertex atom deliberately for angle measurements and inspect the overlay.

**Delete / Deselect.** Delete removes selected base atoms from the structure; Deselect only clears selection. The context menu can be unavailable when large-scene CPU picking caches are disabled. A visible atom rendering alone does not guarantee that all interactive per-atom actions are enabled.

**Example.** Select two neighbouring Cu sites and insert H at their midpoint, then measure H–Cu distances and inspect for overlaps. This creates a geometric trial position, not a validated interstitial site. For systematic void-based insertion, use Add Interstitial Atoms below.

### 3.10.2 Edit Structure: exact effects of the table

**Cell vectors a, b, c.** Each row contains three Cartesian components in Å. Editing a vector changes the cell and display buffers, but does not rescale atom positions. To double a periodic crystal with replicated sites, use Transform Structure instead of doubling a vector here.

**Position Mode: Cartesian / Direct.** Cartesian fields edit positions in Å. Direct fields convert fractional coefficients through the current cell. Direct mode is unavailable without a cell. Changing mode changes coordinate representation; it does not itself move the atom.

**Atom position rows.** Edit the three coordinates in the chosen mode. Table edits take effect in the active scene; there is no separate global Apply for all coordinate changes.

**Element button / Apply Element.** Choose a species in the periodic-table popup and apply it to the row being edited. This is a row-specific edit; use the context-menu substitution for a selected group.

**Add Atom / Delete.** Add Atom starts an H atom at the origin. Set its species and coordinates immediately. Delete removes the corresponding row. Atom numbering can change after row removal, so recheck any saved index-based filter or selection.

**Cell-edit check.** Record one atom's Cartesian position, change only a lattice-vector component, then read the atom position again. Its Cartesian coordinates remain unchanged while its fractional coordinates generally change. This is why direct cell editing must not be mistaken for applying a homogeneous strain.

### 3.10.3 Transform Structure: matrix entry and validation

Each of the three displayed rows is an integer vector. Begin with the identity matrix to understand the entry layout, then set the requested transformation and Apply. A diagonal $(2, 1, 1)$ enlargement means diagonal entries 2, 1, 1 and all off-diagonal entries zero. It should double an ideal periodic atom count and volume. Entering three copies of the vector $(2, 1, 1)$ instead would make the matrix singular.

For a non-diagonal transform, inspect the resulting cell vectors rather than inferring orientation solely from the matrix on screen. Cancel discards the pending transform. The command is disabled without a unit cell. A negative determinant changes handedness; zero determinant is invalid. Save the transformed result separately before performing a finite cut.

### 3.10.4 Merge Structures: component transforms and output cell

**Load/drop components.** Add each source to the component list. Select its row or preview object before changing its transform. The component label identifies which input the numeric controls affect.

**Pos (A).** The three values translate the selected component in Cartesian Å. Use these fields for reproducible offsets after coarse gizmo placement.

**Rot (deg).** Three rotation values orient the selected component. The preview reflects its component transform; orbiting the camera changes the view of all components together.

**Gizmo handles.** Drag translation/rotation handles of the selected component. Inspect from another camera direction to distinguish a transform from projection overlap.

**Reset and removal actions.** Reset restores the selected component's transform. Remove and Delete Selected remove a component from the assembly. Clear All clears the assembly inputs. These actions do not delete the input files on disk.

**Show Bounding Box.** Displays the assembly enclosure for inspection. This overlay is not a new periodic lattice definition.

**Merge Structures.** Commits transformed atoms to the main scene. The current implementation explicitly clears the unit cell and grain metadata. The result is a nonperiodic assembly, even if both inputs were periodic. Cell-dependent tools will consequently be unavailable until a valid cell is supplied through an appropriate workflow. Save a coordinate format when no periodic cell is intended.

**Example.** Add two copies of the supplied Cu particle, translate one along X by more than the particle diameter, merge, and inspect the atom count. Reduce the offset only after checking the nearest interparticle contacts. Merge does not automatically remove overlaps.

### 3.10.5 Cell Sculptor: closing a region and defining each slab

**Use Scene / source / Clear.** Supply a periodic atomic source. Clear removes it from the tool. Set the source before defining crystallographic cuts.

**Supercell $n_x, n_y, n_z$.** Set positive repetition counts (GUI range 1–30) to supply enough source sites. These counts describe available atomic coverage; the slabs determine retained geometry.

**Slab $h, k, l$.** Each slab is the intersection of two parallel half-spaces with reciprocal-lattice normal $h\mathbf{a}^* + k\mathbf{b}^* + l\mathbf{c}^*$. The zero triplet is invalid. Edit the existing row or use the new-slab controls and Add Slab.

**Periodic / Start / N.** Periodic mode uses a starting plane and an integer number of repeat intervals. Start can be negative; N is positive. Inspect the displayed spacing and retained sites because one crystallographic interval need not equal one chemically identical atomic layer.

**Distance limits $d_1, d_2$.** With periodic mode off, enter the two slab limits in Å. Ensure they enclose the desired interval. Add complementary slabs to bound other directions.

**Remove.** Deletes only the chosen slab constraint. Removing a slab can reopen a previously bounded region and disable Generate.

**Outside ghost alpha.** Sets visibility of excluded atoms in the guide preview. It does not change which atoms are retained.

**Status / Generate / Close.** Generate is disabled until the slab normals bound a closed three-dimensional region. At least three independent normal directions are needed; three parallel slabs cannot close a volume. A successful nonempty Generate replaces the main scene and closes the dialog. Close alone dismisses the tool.

**Worked finite box.** Load FCC Cu, increase source repetitions, and add (100), (010), and (001) slabs with finite intervals that overlap inside the source. Inspect the closed-region status and retained atom count, then Generate. To create a thin specimen, narrow the (001) interval while retaining the two lateral bounds. A single slab describes an unbounded mathematical region and is insufficient for this tool's Generate condition.

### 3.10.6 Insert Dislocation: geometry, elastic controls, and commit

**Use Current Scene / Detect Lattice.** Load the reference or copy the scene, then inspect the detected lattice family. Detection is a guide to construction directions, not proof that every atom belongs to an ideal lattice.

**Character.** Edge, Screw, and Mixed select the displacement construction. Mixed reveals Mixed angle in degrees (0–90). Check the resulting Burgers vector and line geometry before interpreting the character physically.

**Auto lattice vectors.** Derives the construction directions. Disable it to enter integer Plane $(hkl)$, Burgers $[uvw]$, and Line $[uvw]$ triplets. A direction and a plane normal are different objects in a general lattice.

**Line point in fractional coordinates.** Enables Line point frac, a position relative to the cell. Line point offset adds a Cartesian offset in Å. Use the preview to verify the final location rather than treating a fractional coefficient as a distance.

**Burgers scale.** Multiplies the selected Burgers magnitude. A continuously adjustable value can produce a geometric displacement that is not a crystallographically allowed perfect dislocation.

**Poisson ratio.** Elastic input to the displacement model. The GUI restricts it to 0.05–0.49. This is not fitted automatically from the structure.

**Core radius.** Regularisation length in Å near the line. Inspect core contacts and displacement maxima when changing it.

**Cutoff radius / Line half length.** Cutoff controls radial attenuation in Å; zero turns attenuation off. Line half length limits the extent along the dislocation line. Use the preview and result summary to assess how much of the specimen is affected.

**Shape.** Half-plane uses the plane-based construction. Cylinder and Sphere expose their respective radius. Ellipsoid exposes three radii. Freeform 2D exposes polygon points in local $(x, y)$ about the line: Add Point appends a vertex; Remove Last removes the final vertex; edit each coordinate pair to trace the intended region. Use a sensible non-self-intersecting polygon.

**Generate Dislocation.** Calculates a result preview. Inspect Shifted atoms, $|b|$, RMS displacement, maximum displacement, minimum interatomic distance, and lattice family before/after.

**Replace Main Scene.** Commits the generated structure and its line-overlay information. Generate alone is not this replacement action. Close dismisses the dialog; save after replacement and before external relaxation.

**Use anisotropic elasticity.** Replaces the isotropic Poisson-ratio displacement field with the Stroh sextic formalism for a fully anisotropic elastic medium, given the crystal's elastic constants ($C_{11}$, $C_{12}$, $C_{44}$, and $C_{13}/C_{33}$ for hexagonal symmetry). A small noise amplitude breaks the sextic solve's degeneracy at isotropic or high-symmetry line/plane orientations.

**Insert as dipole.** Superposes a second dislocation of opposite Burgers vector at a specified offset, giving a net-zero-Burgers-vector configuration that is periodicity-compatible, unlike a single dislocation in a periodic cell.

This tool and its post-processing counterpart (Section 4.10) are native reimplementations of the physical models and algorithms used by the BABEL dislocation toolkit (Douin, Clouet et al.); see that section for the full literature list.

### 3.10.7 Add Interstitial Atoms: finding, filtering, and occupying voids

This workflow has two stages: detect candidate voids in the host, then choose where to insert a selected element. Re-run detection after a substantial host change.

**Source / Use Current Scene.** Supply the host and inspect it in the preview. The host used by this tool must be the intended edited structure, not an earlier file with the same name.

**Grid / Max Voids.** Grid controls the search sampling (8–30 in the GUI); Max Voids caps the retained candidates. Increasing sampling can reveal sites missed by a coarse search and increases work.

**Min Clearance / Min Separation.** These are distinct detection controls in Å: clearance filters available space around a candidate; separation prevents retaining nearby duplicate candidates. They are not the requested number of inserted atoms.

**Detect Voids.** Run after changing detection inputs. Inspect the resulting void count and types before attempting insertion.

**Void type filters.** Select All Types and Clear All Types toggle the type filters together. Individual type checkboxes restrict eligible candidates. Show Void Polyhedra displays their local geometric environments; it does not insert atoms.

**Select Element.** Choose the chemical identity of the atoms to add. This does not assign a charge state or relax the host.

**Random placement.** Select from the eligible voids using the seed and target. The target can be By percentage or By atom count. Percentage is of the filtered candidate pool, not atomic percent of the final structure. The desired count is rounded and capped to the pool size. Inspect actual added count after applying because constraints can reduce it.

**Inside OBJ/STL.** Load/drop a closed region mesh into the preview. The eligible candidates must also lie in that region. Use Translate (T), Rotate (R), or Scale (S) gizmo modes; shortcuts act when a text field is not being edited. Numeric Scale, Translate, and Rotate (deg) provide precise transforms. Reset Transform restores unit scale, zero translation, and zero rotation.

**Manual placement.** Select the desired void centres in the preview. This mode inserts at the selected centres rather than using the percentage/count target. Inspect the displayed Selected centers count.

**Seed / Min Insert Distance.** Seed controls stochastic selection; zero is a deterministic seed in this tool, unlike builders where zero explicitly means time. Min Insert Distance checks separation from host atoms and already inserted atoms, using minimum-image distances when a usable cell is present. It is separate from void-search clearance and candidate deduplication distance. The implementation considers the requested number of shuffled candidates; rejected candidates are not necessarily replaced from the rest of the pool.

**Apply Interstitials.** Requires a source and detected voids; mesh mode additionally requires a loaded mesh, and manual mode requires a selection. A successful action updates the main scene and this tool's source, with the number of added atoms reported. Inspect the composition and contacts before applying again.

**Troubleshooting.** If Apply is disabled, check the mode-specific prerequisites rather than only the visible host. If the result adds fewer atoms than requested, compare type filtering, mesh placement, selected candidates, and minimum insertion distance. A 100% target is not a guarantee that every detected site can be occupied simultaneously without violating constraints.

# Chapter 4

# Structural analysis and presentation

## 4.1 Structure Info and Atom Info

**View > Structure Info** provides a compact check of the active structure's composition and lattice information. Use it after import, supercell construction, cutting, or merging. A disabled cell-dependent command usually means the current structure does not carry a usable cell, even when the coordinates visually resemble a crystal.

**View > Atom Info** enables atom inspection. Select the desired atom in the viewport and verify its identity and position. Displayed atom ordering follows the current structure; rebuilding or exporting can change order. Record stable identifiers in your own workflow if you need to track particular atoms across multiple files.

## 4.2 Distance and angle measurements

Use **View > Measure Distance** or the toolbar's **Distance** button, then select two atoms. For an angle, choose **Measure Angle** or **Angle**, then select three atoms in the intended order, with the middle atom as the vertex. Read the overlay and inspect the selected atoms rather than assuming the nearest projected point was selected.

Measurements are geometric. Distinguish a displayed Cartesian separation from a periodic minimum-image separation in your analysis; inspect the cell and selected image when atoms straddle a periodic boundary. For dense structures, hide unrelated species or change the camera before selection. Clear/leave the measurement mode when finished so subsequent clicks have the intended selection behaviour.

## 4.3 Radial Distribution Function

**Purpose.** Measure neighbour distributions as a function of distance. Open **Analysis > Structure and defects > Radial Distribution Function** with a structure loaded. Select reference and target species; using different species gives a partial pair distribution. Set the distance range, bin count, and periodic-boundary option, then choose **Run RDF**.

**Controls. Use PBC when unit cell is available** includes periodic geometry. **Normalize to g(r)** changes from counts to the program's density-normalised distribution. **Bins** controls resolution; the current slider spans 32–2,000. **Smoothing passes** (0–8) changes the plotted/profile smoothing. **Overlay raw counts** helps distinguish sparse sampling from a smooth-looking curve. **Overlay cumulative coordination** shows the integrated neighbour count trend.

For a homogeneous isotropic system, the usual interpretation of a normalised pair distribution is

$$dN(r) \simeq 4\pi r^2 n_{\text{target}} g(r)\, dr.$$

Finite boxes, partial species, surfaces, and chosen normalisation affect that interpretation. A nanoparticle surrounded by vacuum does not have the same homogeneous reference density as a bulk periodic crystal. Report the species pair, bin width, cutoff, PBC choice, and smoothing when comparing curves.

**Worked check.** Open `examples/cu_host.cif`, select Cu as both species, enable PBC, and start with no smoothing. The nearest-neighbour distance of the unstrained FCC example is $a/\sqrt{2} \approx 2.553$ Å. Check that the first peak is near that distance. This is a geometric checkpoint, not a fitted thermal distribution. A very small cell or an excessive distance range can make periodic-shell interpretation misleading.

**Distortion analysis.** Enable **Enable distortion analysis** to analyse a shell window. **Auto shell window from first-shell minima** estimates its limits; otherwise specify the visible manual bounds. Inspect the chosen window against the RDF. Overlapping shells, disorder, or low counts can make automatic minima unreliable. A broad peak can reflect multiple chemical distances, strain, temperature, or finite sampling; the plot alone does not separate those mechanisms.

## 4.4 Short-Range Order (SRO)

**Purpose.** Characterise chemical neighbourhood preferences in a multi-species structure. Open **Analysis > Short-Range Order (SRO)**, choose the number of neighbour shells and the shell-distance tolerance, and run the calculation. Inspect the species-resolved shell table.

The Warren–Cowley interpretation is

$$\alpha_{ij}^{(n)} = 1 - \frac{P(j \mid i, n)}{c_j},$$

where $P(j \mid i, n)$ is the probability of a species-$j$ neighbour in shell $n$ around species $i$, and $c_j$ is the bulk fraction of $j$. Negative values indicate an excess of that neighbour relative to the reference composition; positive values indicate a deficit. Zero is the ideal random reference, subject to finite-size and composition constraints.

Use `examples/cu_ni_alloy.cif` as a starting example. The random assignment need not give exactly zero SRO in every shell. Compare several seeds or larger cells to assess sampling fluctuations. Shell tolerance groups nearby distances; too small a tolerance splits one distorted shell, while too large a tolerance merges different shells. The current shell-count control supports 1–8 shells. SRO analysis measures the existing configuration; it does not optimise an alloy toward a requested SRO value.

## 4.5 Lattice planes and Miller directions

For a periodic structure, choose **View > Lattice Planes Settings** to configure crystallographic plane overlays, and toggle **View > Show > Lattice Planes** for visibility. Enter the desired nonzero Miller indices and adjust the visible plane settings. Planes are guides for orientation and geometry; displaying one does not cut the structure. Use Cell Sculptor to create an actual atomic cut.

Choose **View > Miller Directions Settings** to configure direction overlays and **View > Show > Miller Directions** to toggle them. A direct-lattice direction $[uvw]$ is parallel to $u\mathbf{a} + v\mathbf{b} + w\mathbf{c}$. A plane $(hkl)$ is normal to $h\mathbf{a}^* + k\mathbf{b}^* + l\mathbf{c}^*$. In cubic cells the same integer triplet can appear parallel in both roles; this shortcut fails for a general lattice.

## 4.6 Voronoi and coordination polyhedra

**View > Show > Voronoi Volumes** displays a geometric nearest-site partition. Use it to inspect local space filling and site environments. It is separate from electronic Voronoi integration, which assigns scalar-grid quadrature contributions to imported sites. Neither operation is a Bader partition of electron density.

**View > Polyhedral Viewer** and the polyhedral atom representation show coordination geometry. Use **Settings > Polyhedral Settings** to configure the visible coordination construction. Check neighbour/element choices and distance criteria against known bonds in a simple reference first. A change in cutoffs can change a displayed polyhedron without any change in the underlying atoms.

**View > Show > Dislocation Lines** controls line overlays where corresponding line information exists. An arbitrary imported XYZ file does not acquire a validated dislocation network merely by enabling the display checkbox.

## 4.7 Atomic sizes, colours, and lighting

**Settings > Atomic Sizes** lets you change display radii and restore literature defaults. Use the selected-element radius controls for a particular species or reset defaults for a consistent baseline. Display radii do not displace atom centres and should not be confused with a measured bond length.

**Settings > Display Settings** exposes lighting, material, and element-colour controls. Lighting controls change illumination; material controls change shading response. Use **Reset Lighting Defaults**, **Reset Material Defaults**, and colour reset actions to recover from an overly dark or saturated view. Avoid using visual brightness as a numerical observable.

**View > Theme > Light/Dark** (also in **Settings** and on the toolbar) changes the desktop theme. Electronic scalar colours and transparency are configured separately in the electronic dialog's **Appearance** and 3D controls. Use a consistent palette and numerical range when comparing several electronic fields.

## 4.8 Large datasets: billions of atoms

Structures far larger than memory — molecular-dynamics snapshots of hundreds of millions or billions of atoms — open with a separate large-data renderer. **File > Open**, dropping a file or naming it on the command line opens an atom cloud (`.afcloud`) directly; a LAMMPS text dump (`.dump`, `.lammpstrj`, or any file starting with `ITEM:TIMESTEP`) and an XYZ file of more than five million atoms are first converted into one, once, with a progress window (the cloud is kept next to the file, or in the user's cache folder when that folder is read-only, and reused while it is newer than the file). Large datasets are for viewing: editing, analysis and pipelines work on ordinary structures.

**How it scales.** An atom cloud stores the atoms by space in chunks of about half a million atoms, 8 bytes per atom (positions to 1/65535 of the chunk, about $10^{-3}$ Å, and a species), in random order inside each chunk, so that any first part of a chunk is a uniform random sample of it. Each frame the renderer

- skips chunks outside the view and chunks hidden behind atoms already drawn (occlusion queries, checked one frame later so that they never stall the display);

- gives every other chunk as many atoms as its size on screen needs: enough to cover it where atoms are smaller than a pixel, rising to every atom as atoms grow from 0.75 to 1.5 pixels in radius, where their arrangement becomes visible; distant parts are drawn as subsamples with slightly larger points;
- keeps within the **Atoms per frame** budget (the nearest chunks get full detail first) and streams the atoms it lacks from the file on a background thread, within the **GPU memory** budget (least recently drawn chunks are released first);
- draws atoms as lit sphere impostors with exact depth, front to back, so that hidden atoms cost little.

Memory therefore depends on the view, not on the dataset: the file can be as large as the disk. On a laptop GPU (NVIDIA RTX 2000 Ada, 8 GB) a 1.0-billion-atom crystal (8 GB file) and a 2.15-billion-atom crystal (17 GB) display at 60–400 frames per second, from the whole crystal to a close-up of individual atoms, each view streaming in within a few seconds.

**The Large dataset panel.** It shows the atoms in the dataset and drawn in the last frame, the visible and hidden chunks, the GPU time and memory, and whether streaming is complete. **Atoms per frame** trades detail for speed; **GPU memory** bounds the atoms kept on the GPU; **Level of detail** off draws every atom of the visible chunks (within the budgets); **Detail** sets how densely subsampled regions are filled; **Atom size** scales the radii; **Colour** colours by species or by height along x, y or z; **Show box** and **Occlusion culling** toggle the outline and the hidden-chunk test. **Export Image** renders the cloud at the export resolution and waits for its atoms to stream in.

**Making atom clouds.** `AtomForge--cloud` converts and generates them without the desktop; conversion streams its input, so memory does not grow with the file (it needs about 16 bytes per atom of free disk space for intermediate files, see `--temp`):

```
AtomForge --cloud build --input run.dump --types "Cu,Ni" --output run.afcloud
AtomForge --cloud build --input big.extxyz --frame 10 --output big.afcloud
AtomForge --cloud generate --lattice fcc --a 3.615 --cells 630 630 630 --element Cu --
    output cu1b.afcloud
AtomForge --cloud info run.afcloud
```

LAMMPS dumps may give positions as `xyz`, scaled `xsyszs` or unwrapped `xuyuzu`, in orthogonal or triclinic boxes, with an `element` or a `type` column (`--types` names the types); XYZ files may be extended XYZ (`Properties=`, `Lattice=`). Other formats are loaded whole and then converted. A dump of 10 million atoms converts in about 1.5 s; a billion-atom crystal is generated in about 80 s. In Python, `atomforge.cloud` offers the same (Chapter 8).

## 4.9 Exporting images

Choose **File > Export Image**, select PNG, JPEG, or SVG, configure the available export settings, browse to a destination, and export. Inspect the output file at its intended presentation size. PNG is appropriate for lossless raster figures; JPEG can introduce compression artefacts around small labels; SVG is useful when a downstream tool supports the exported representation.

An image records a view, not a reproducible calculation. Accompany it with the saved structure or density, camera orientation, visible species, radii, scalar palette/range, and isovalue where relevant. For quantitative electronic profiles, export CSV and generate a plot with explicit axes and units. For the electronic dialog itself, an operating-system screenshot records both the controls and current views, as illustrated in this manual.

## 4.10 Dislocation Post-Processing

**Purpose.** Compare a deformed structure against its undeformed reference to characterise dislocation content: Nye tensor, Vitek differential-displacement map, crystallographic pattern matching, drag/migration-barrier preparation, and dislocation position/Burgers-vector fitting. Open **Analysis > Structure and defects > Dislocation Post-Processing**, load a reference (undeformed) and a deformed structure with the same atom count and ordering, then run the desired analysis. Results with a numeric field export to CSV; the drag-preparation interpolated structure saves directly as a structure file.

**Origin.** Each analysis is a native reimplementation of the corresponding program in the BABEL dislocation toolkit (Douin, Clouet et al.), following the same published methods rather than porting BABEL's Fortran source:

- **Nye tensor** (**nyeTensor.f90**): J. F. Nye, *Acta Metall.* **1**, 153 (1953); Hartley and Mishin, *Acta Mater.* **53**, 1313 (2005) lattice-correspondence convention, including the index-order correction documented in BABEL's own source (attributed there to David Olmsted).
- **Vitek differential-displacement map** (**vitek.f90**): V. Vitek, *Philos. Mag.* **18**, 773 (1968).
- **Pattern matching** (**patternInit/patternDetect**): same two-stage neighbour-environment comparison as BABEL, generalised here to build the reference pattern from any supplied perfect structure rather than a hardcoded bcc/fcc/hcp table.
- **Drag/migration-barrier preparation** (**mainPrepareDrag.f90**): linear interpolation between reference and deformed configurations at a reaction coordinate $\zeta$, for use as the input to an external constrained-minimisation calculation; AtomForge does not itself run that minimisation.
- **Dislocation fit**: a non-iterative weighted centroid/integral estimate of dislocation line position and Burgers vector from the Nye tensor field (Nye, 1953, above), serving the same purpose as BABEL's **displacementFit/vitekFit** without their nonlinear least-squares fit against a fixed elastic solution.

The anisotropic-elasticity option in Insert Dislocation (previous section) uses the Stroh sextic formalism: A. N. Stroh, *Philos. Mag.* **3**, 625 (1958); *J. Math. Phys.* **41**, 77 (1962); D. M. Barnett and J. Lothe, *Phys. Norv.* **7**, 13 (1973); J. P. Hirth and J. Lothe, *Theory of Dislocations*, 2nd ed. (1982), ch. 13 — the same model BABEL's own anisotropic-dislocation code uses.

## 4.11 Features that are not current analysis-menu commands

Common-neighbour analysis (CNA) and angular-distribution analysis have source implementations but their menu entries are disabled in the documented baseline. They cannot be reached as ordinary Analysis commands in this executable. Their presence in source or earlier broad feature lists should not be interpreted as a tested desktop workflow. Use RDF, SRO, measurements, and the electronic tools described here for the exposed analysis features.

## 4.12 Workspace, display, and analysis control reference

### 4.12.1 Screen resolution and interface size

Fonts, widget dimensions, and spacing follow the display's content scale. Choose **Settings > Interface size** to use Automatic (display scaling), Compact (85%), Larger (125%), Large (150%), or Extra large (200%). These percentages multiply automatic display scaling and apply to the

current session. Automatic restores the display-derived size; restarting also restores Automatic. A change takes effect on the next frame without closing the current structure.

Dialogs stay inside the application work area, including when its window is resized. Large builder and analysis panels stack vertically in narrow layouts. Scroll the outer dialog to reach a lower panel, and scroll inside a panel to reach its remaining controls. Wide tables and the periodic table retain horizontal scrolling so labels and cells remain readable. The view toolbar uses multiple rows when space is limited. Tab and Shift+Tab navigate supported controls; Enter or Space activates the focused control.

The illustrated tutorials combine updated populated builder captures with earlier control-reference captures. Control meanings and scientific recipes remain the same, but placement, dialog size, and panel arrangement can differ with screen size. Any dimensions mentioned elsewhere describe the original design scale; runtime dialog bounds take precedence on smaller screens.

### 4.12.2 Structure tabs and file navigation

When several structures are open, select the desired structure tab before saving, editing, or analysing. Tabs can be reordered, and switching restores that tab's camera. The tab's close button closes that structure; File > Close is the active-structure command. Keep a saved copy of edits before closing. Two tabs can share a display title, so confirm the file and structure information rather than relying on the title alone.

In the shared path picker, Back and Forward traverse navigation history, Up opens the parent folder, and the folder field jumps to a typed directory when Enter is pressed. Locations offers Home, Desktop, Documents, Downloads, Working folder, and available drive roots. Select a file then Open, or double-click it. The electronic picker includes extensionless files such as CHGCAR. In save mode, choose a folder and filename, then Save. Replace confirms overwriting an existing destination; Cancel in that confirmation returns without replacing it. The main structure browser also has a Format selector for save conversion. A filename extension should agree with the selected format.

Builder source buttons and drop targets feed the currently open tool, not necessarily the main workspace. Read the resulting source label. An electronic Reference load fills the operand source; it does not become the active structure tab. Use Reload volume to reread a changed electronic input, and Clear to reset electronic data when starting a different study.

### 4.12.3 Main view, representation, and information

**Cartesian X/Y/Z and lattice a/b/c views.** Align the main camera to a Cartesian axis or input lattice direction. Lattice-direction buttons require a cell. The Rotate angle and adjacent X/Y/Z buttons apply a repeatable camera/view rotation; they do not edit atom coordinates despite the internal "Rotate Crystal" naming.

**Isometric / Orthographic / Reset Default View.** Choose the intended projection/view preset and restore framing with Reset. This does not undo structural edits. Orbiting changes viewing direction; panning changes framing; zoom changes scale.

**Show Element / Show Atoms / Show Bounding Box.** Toggle labels, atom rendering, and the cell enclosure separately. Hiding atoms does not delete them or exclude them from numerical analysis.

**View Structure By.** Element Type uses chemical colours; Crystal Orientation uses available grain/orientation metadata; Grain Boundary uses boundary metadata; Dislocation Lines toggles a separate line overlay without changing the selected colour mode. Missing metadata cannot be recovered just by changing the colour mode.

**View Atoms By.** Balls emphasises sites, Ball and Stick adds geometric connections, Space Filling emphasises occupied visual space, and Polyhedral uses coordination geometry when available. Representation does not alter the atom list.

**Structure Info.** Read formula, species counts, total atoms, lattice lengths and angles, and cell volume. Symmetry information includes space-group number, international symbol, Hall symbol, and point group when the calculation succeeds. Wait while symmetry is computing; an error or unavailable cell is not a symmetry assignment. The positions table includes element, Cartesian coordinates, and direct coordinates where available.

**Atom Info and measurements.** Select the intended atom(s), read the overlay, and clear selection when done. Measurements follow the selected displayed geometry. Save the selection/order and cell convention when a value is used quantitatively.

**Help > Keyboard Shortcuts.** Lists every key and mouse binding of the main view (files, editing, selection and view); F1 opens it too.

**Help > Manual / About.** Manual opens the packaged PDF in the system's default PDF viewer. About provides application information and a Manual button that opens the same PDF. If the PDF is missing, restore AtomForge-manual.pdf from the application package; if no viewer can open it, configure a default PDF application. A source-only feature is not made available by appearing in an older help description.

### 4.12.4 Camera workflow: orientation, projection, and framing

1. Open `examples/cu_mesh.xyz` or a generated periodic crystal. Start with **View > Reset Default View**. Left-drag the scene to orbit; right-drag to pan; use the wheel or middle-button vertical drag to zoom. Disable box/lasso selection before using right-drag for panning.
2. Choose **View > Orthographic View** to remove perspective size variation, then click toolbar **Z** for an XY projection. The atoms still form a 3D structure: atoms at different depths can overlap in this projection. It is not a 2D section or an electronic slice.
3. Choose **Isometric View** for the alternative view preset, and use **X/Y/Z** or **a/b/c** to compare directions. In a skew cell, a lattice vector need not coincide with a Cartesian axis. Orbiting remains available in either projection mode.
4. Enter a Rotate angle, then press the adjacent axis button for a repeatable viewing increment. This rotates the view, not the atomic coordinates. Use Edit tools for a structural rotation.
5. Reset Default View restores framing after panning or zooming. Export an image only after checking the whole object fits and required overlays are visible. Main-view camera settings do not set the electronic dialog's independent 2D/3D cameras.

### 4.12.5 Every View menu option: dependencies and interpretation

| Option | How to use it and what to expect |
|---|---|
| Show Element | Adds chemical-symbol labels. Use a small structure or zoom into a region to avoid overlapping labels. Toggle again to hide. |
| View Structure By: Element Type | Colours by species using Settings > Display Settings > Colors. This is the useful starting point for composition comparisons. |

| Option | How to use it and what to expect |
|---|---|
| Crystal Orientation | Uses available orientation/IPF information; a uniform single crystal may have one colour. Load the accompanying orientation sidecar when supplied with a generated polycrystal. A colour is not a strain or energy measurement. |
| Grain Boundary | Emphasises the boundary classification carried by the structure. Verify the underlying geometry and metadata; switching modes alone is not a new grain-boundary calculation. |
| Dislocation Lines / Show Dislocation Lines | Both menu locations control the same line-visibility flag. Lines require explicit dislocation geometry from a defect builder. The viewer does not infer lines from surface coordination changes. Building a new nanocrystal clears any old defect outline. Enabling the flag does not insert a dislocation or infer Burgers vectors from an arbitrary image. |
| View Atoms By: Balls | Shows atomic sites without bond sticks. Useful for dense crystals and point-like site comparisons. |
| Ball and Stick | Shows sites plus geometric connections. A displayed stick is not a calculated bond order or proof of chemical bonding. |
| Space Filling | Uses larger visual spheres to emphasise the envelope. Interior atoms can become hidden; return to Balls or reduce display radii when inspecting defects. |
| Polyhedral | Displays coordination polyhedra configured under Settings. Large structures are supported with a configurable centre limit. Hulls fill progressively and camera movement reuses their geometry. See the centre/ligand workflow below. |
| Show Lattice Planes / Lattice Planes | The first toggles the collection; the second edits its rows. A unit cell is required. Both collection and per-row visibility matter. |
| Show Miller Directions / Miller Directions | Toggle the collection or open its vector editor. A unit cell is required; row lengths and colours are independent. |
| Isometric View / Orthographic View | Select the view/projection preset and reset framing. These do not cut the structure into a slice. |
| Show Voronoi Volume | Draws geometric nearest-site cells, coloured by relative cell volume. See the limitations below before quantitative use. |
| Polyhedral Viewer | Enables the coordination overlay independently of the selected atom representation. Thus the viewer can remain enabled after selecting Balls; disable this toggle as well to remove the overlay. |
| Show Atoms | Hides or shows atom geometry without deleting sites. Analysis and export of structure coordinates still use the underlying atoms. |
| Show Bounding Box | Hides or shows the available unit-cell enclosure. It requires a cell and is independent of atom visibility. |
| Structure Info | Opens composition, cell, symmetry, and atom-position information for the active tab. Check direct coordinates only when a valid cell exists. |
| Measure Distance / Measure Angle | Select exactly two atoms for distance or three for angle, then open the corresponding command. The second selected atom is the angle vertex. If the selection count is wrong, close the message, correct the selection, and request the measurement again. |
| Atom Info | Displays information for the selected atom; select a visible site first. Recheck indices after editing or rebuilding. |

| Option | How to use it and what to expect |
|---|---|
| Reset Default View | Restores camera framing; it does not undo edits or reset lighting, colours, and radii. |
| Select Theme: Dark / Light | Changes interface/scene presentation. It does not change element identity or electronic data. Check text and geometry contrast after switching. |

### 4.12.6 Voronoi volume: a geometric overlay

Open a small periodic crystal and enable **View > Show Voronoi Volume**. Each cell encloses positions closer to its atom than to neighbouring sites. The overlay maps relative cell volumes to colours and highlights selected cells. Orbit to inspect faces, then temporarily hide atoms to expose the cells. Disable the toggle to return to the ordinary atom view.

This construction partitions geometric space; it is not Bader charge analysis and does not integrate a charge-density grid. With a valid cell, periodic neighbour images are used. Without periodicity, exterior cells are bounded by an artificial padded box, so their displayed volumes depend on that enclosure. The periodic implementation considers neighbouring images within one translation in each direction; inspect highly skew cells carefully. Equal colours in an ideal crystal can simply reflect equal local volumes. For publication, report the cell, boundary convention, and numerical method instead of treating a screenshot colour as a measured value.

### 4.12.7 Settings workflow: choose the control that changes the desired property

1. For small text or crowded buttons, open **Settings > Interface size**. Start with Automatic, increase to 125% or 150% for readability, and use 200% when needed. Compact reduces interface size. These are session settings and multiply display scaling. Resize the window and scroll stacked panels to reach lower controls.
2. For larger/smaller atomic spheres, choose **Atomic Sizes**, select a species, and edit its radius in Å. The selected species changes immediately. Apply Literature Radius To Selected Element restores that species; Reset to Literature Defaults restores all radii. Positions and lattice constants are unchanged.
3. For darker shadows or clearer depth cues, open **Display Settings > Lighting**. Adjust one slider at a time while keeping the same camera. Ambient brightens shadowed areas; Saturation affects colour vividness; Contrast changes tonal separation; Shadow Strength controls shadow darkness. Reset Lighting Defaults provides a reproducible starting point.
4. For broad/diffuse versus sharp highlights, use **Material**. Specular Intensity changes brightness, Shininess Scale changes the per-element highlight response, and Shininess Floor prevents very diffuse highlights. Reset Material Defaults restores only this tab's three controls.
5. For a different species palette, use **Colors**, select the element, and edit its colour. Reset Color restores the selected species; Reset All Colors restores the palette. Return to Element Type colouring to see chemical colours when an orientation mode is active.
6. For coordination faces, use **Polyhedral Settings**. Choose a few centre IDs, centre and ligand species, neighbour count, and opacity, then enable Polyhedral Viewer. Reset Defaults restores those overlay controls; it does not reset lighting or atomic radii.

Display Settings changes are live; closing the dialog retains them for the current application state. Keep camera, representation, palette, radii, illumination, and image-export scale consistent across comparison figures. These settings do not replace external relaxation or an electronic calculation.

### 4.12.8 Lattice Planes: every row and visibility control

Open View > Lattice Planes on a structure with a cell. The global Show control enables the collection. Enter nonzero H/K/L, Offset n, Opacity, and colour in the new-plane controls, then Add Plane. The offset specifies the member of the parallel plane family; it is not the electronic arbitrary-plane point. In the list, each plane has its own Visible checkbox, editable indices and offset, opacity/colour, and Delete. Individual visibility and global visibility must both allow a plane to be seen.

For an initial cubic example, add (100) at a convenient offset, then add (110) in a contrasting colour. Orbit to inspect their normals. Adjust opacity to see atoms behind the guides. These overlays do not retain/delete atoms and have no effect on a saved charge density. Use Cell Sculptor for an atomic cut or the electronic 2D plane controls for a density intersection.

### 4.12.9 Miller Directions: vectors, length, and colour

Open View > Miller Directions. Enter nonzero U/V/W, Length in Å, and colour, then Add Direction. Each row has its own Visible flag, editable direction, Length, colour, and Delete. The global Show Miller Directions toggle controls the collection. Length changes the drawn vector's size, not the cell length or the magnitude of a physical displacement. Compare [100] and [110] in a cubic reference, then repeat in a skew cell to distinguish direct-lattice directions from reciprocal plane normals.

### 4.12.10 Polyhedral Settings: selecting centres and ligands

**Max displayed centers.** Limits displayed coordination centres (1–2,000). A limit below the eligible count intentionally leaves some polyhedra undisplayed.

**Nearest neighbors.** Sets the neighbour candidate count (4–64). This changes the geometry considered for the visual construction; it is not a fitted coordination number.

**Specify center atom indices.** Enables Center atom IDs. Use one-based IDs, comma-separated, with ranges such as `1,4,10-20`. Inspect Parsed centers after editing. Recheck IDs after structural edits reorder atoms.

**Show polyhedral edges / Face opacity / Edge opacity.** Toggle edges and set face/edge alpha independently from 0 to 1. Zero hides that visual component; one is opaque.

**Filter center elements / Centers.** Restrict centres using comma-separated symbols such as `Ti,Zr,Fe`. A disabled filter does not restrict species.

**Filter ligand elements / Ligands.** Restrict neighbours using symbols such as `O,N,S,F`. Centre and ligand filters serve different roles and can both be active.

**Reset Defaults / Close.** Reset restores settings and clears index/species filter input. Close dismisses the dialog. Enable the corresponding View representation/overlay to inspect the configured result.

**Example.** For a metal oxide, filter centres to the metal and ligands to O. Start with a few specified centre IDs to inspect geometry, then broaden the selection. A blank view can result from hidden atoms/overlays, restrictive filters, insufficient neighbours, or zero opacity.

### 4.12.11 Atomic Sizes and Display Settings tabs

Atomic Sizes uses an element selector and Atomic Radius in Å. Apply Literature Radius to Selected restores that species; Reset to Literature Defaults restores the set. Radius changes visual sphere

size without moving centres. Record altered radii when comparing images or using tools that consume radius information.

Display Settings has separate tabs. In **Lighting**, Ambient sets shadow-side brightness (0–0.60, default 0.18); Saturation controls vividness (0.50–3.00, default 1.55); Contrast stretches dark-to-light response (0.50–2.50, default 1.25); Shadow Strength controls darkness (0–1, default 0.75). Reset Lighting Defaults restores all four.

In **Material**, Specular Intensity sets highlight brightness (0–2, default 0.65), Shininess Scale multiplies the per-element response (0.10–5, default 1.50), and Shininess Floor limits the minimum specular exponent (4–256, default 32). Reset Material Defaults restores these values. In **Colors**, select an element in the periodic table, edit its colour, use Reset Color for that species, or Reset All Colors for the entire palette. Settings apply to rendering; they are not charge-density values or chemical substitutions.

Light/Dark in View > Select Theme changes the desktop theme. Electronic Appearance has its own palette, range, and surface lighting controls. Keep these separate when trying to reproduce a figure.

### 4.12.12 RDF and SRO: run state and all numerical inputs

RDF Reference species and Target species determine the pair selection. Radius range provides both lower and upper distances in Å; the GUI allows 0–50. Bins subdivides this interval, so nominal bin width is $(r_{\max} - r_{\min})/N_{\text{bins}}$. Choose a range with $r_{\max} > r_{\min}$. PBC requires an available unit cell. Normalize to g(r), raw-count overlay, cumulative coordination overlay, and Smoothing passes have distinct purposes: normalisation changes interpretation, overlays add curves, and smoothing changes the profile presentation.

Enable distortion analysis reveals shell-window controls. Auto shell window from first-shell minima estimates bounds; disabling it exposes Manual shell window in Å. Confirm the selected interval contains the intended shell and inspect the reported distortion quantities. Run RDF starts the calculation; wait for completion before reading results. Changing controls requires a fresh run for a reproducible comparison. The plot is not an electronic CSV export; use the data interfaces actually offered by the selected analysis.

SRO Number of shells is 1–8. Shell tolerance has GUI range 0.001–0.20 and groups distances into shells; it is not an atomic composition tolerance. Run Analysis computes the species/shell results. Read the counts and shell distances alongside Warren–Cowley values, particularly for dilute species or distorted shells. Close dismisses the analysis dialog. Neither RDF nor SRO changes the underlying structure.

### 4.12.13 Image export settings

In File > Export Image, choose Filename, Format (PNG/JPEG/SVG), Include background, Include orientation gizmo, and Resolution scale (1–8), then Export. Scale increases raster dimensions and memory requirements. Background and gizmo are independent choices; a transparent-background format is useful for composition in a document. JPEG cannot preserve an alpha channel like PNG. SVG support does not imply that every downstream editor reproduces all rendering details identically. Inspect the exported file. Cancel dismisses the export dialog without exporting.

### 4.12.14 Electronic display controls that are separate from calculations

The layout selector offers 3D and 2D, Only 3D, and Only 2D. In 3D, Render mode chooses Volume (interior density) or Isosurface (boundary). Set the 3D level, use Estimate level for a data-derived starting point, and Update surface when a mesh needs regeneration. Transparency is zero for

opaque rendering and one for full transparency. Fit view and double-click reset camera framing; left drag orbits, right drag pans, and the wheel zooms. Show slice plane toggles the translucent 2D intersection guide without changing the section itself.

The 2D selector offers bc/ac/ab planes normal to $a^*/b^*/c^*$, with Slice position as a fraction of the cell. Arbitrary plane instead takes a Point in fractional $a, b, c$ coordinates and a Normal in Cartesian $x, y, z$ components. A zero normal or a plane outside the cell cannot define a useful section. Show contour line toggles the numerical contour; 2D isovalue and Estimate 2D level operate independently of the 3D level. Reset 2D restores its view; the angle slider and left drag rotate within the plane, right drag pans, and the wheel zooms. Double-click also resets it.

The 2D preview samples at a bounded resolution (up to 129 samples per direction). Use numerical section/contour operations at the requested resolution for quantitative export. The warning for a 2D level compares against that slice's values, while the 3D level is checked against the selected volume. An isovalue inside the full-volume range may still be outside a particular slice's range.

Appearance offers Spectrum, Blue-white-red, and Sequential blue. Automatic range estimates colour limits; disabling it reveals Minimum and Maximum. Lighting reveals Specular and Shininess for surface shading. These surface appearance controls do not all apply to volume ray rendering: the volume colour bar uses the volume display bounds. Always read the displayed legend and unit. Results table shows a preview of up to 200 rows; CSV export retains the calculation table, not just those preview rows.

# Chapter 5

# Electronic files and interactive views

## 5.1 Open a scalar field

Choose **Analysis > Electronic structure > Electronic Post-processing**. The left column contains tool selection, input files, field/operand selection, calculation parameters, export, and appearance. The right side contains the 3D and 2D views. On a smaller window, scroll the left column to reach **Calculate** and **Export**; the tool buttons remain near the top.

1. Expand **Import options** if you need to specify the physical quantity or Cube coordinate convention. Do this before loading, or use **Reload volume** after changing an import setting.
2. Click **Open volume**. Navigate with the location shortcuts, directory list, or folder bar. Select the file and click Open, or double-click the file.
3. Check the filename, selected **Field**, dimensions, units, and displayed value range. Multi-channel files supply multiple selectable fields.
4. Choose **Charge transfer** for the eight direct charge-operation buttons or **General analysis** for the other operations.
5. Select a field and parameters, click **Calculate**, and wait for completion. Grid-producing operations append derived fields; select the intended field before the next calculation or export.

**Clear** clears the electronic session state; save useful results first. **Reload volume** rereads the source. The initial appearance estimates are based on values in the selected data, so two different files need not start with the same threshold.

## 5.2 VASP, Cube, and XSF conventions

| Input | Interpretation and checks |
|---|---|
| VASP CHGCAR/CHG | Charge-density values are normalised by the cell volume to $e/\text{Å}^3$. The grid is periodic without duplicate endpoint planes. Check the integrated electron count against the originating calculation. |
| VASP LOCPOT | A potential field in eV. Its integral is not an electron count. Potential references and zero offsets matter when comparing calculations. |
| VASP ELFCAR | Dimensionless ELF. Choose an ELF-appropriate range; charge-only operations are not applicable. |

| Input | Interpretation and checks |
|---|---|
| Gaussian Cube | Explicit coordinate convention: Bohr by default, or Angstrom. Quantity **Auto** leaves scalar values raw. Explicit density assumes atomic-unit density; explicit potential assumes Hartree and converts to eV. Coordinate and scalar-unit choices are distinct. |
| XSF datagrid | Reads the cell/grid geometry and scalar values. Auto is conservative about scalar meaning. Specify density/potential/ELF only when the values have that meaning and the expected units. Treat finite endpoint geometry deliberately. |

VASP auto-detection uses conventional filenames such as CHGCAR, LOCPOT, and ELFCAR. A renamed generic file may need an explicit quantity. For Cube, negative dimension conventions vary among writers; use the explicit coordinate option rather than assuming the sign alone establishes units. Assigning a physical label to arbitrary numbers does not make them that physical quantity.

## 5.3 Units, sampling, and periodic endpoints

AtomForge's electronic geometry uses Cartesian Å. A periodic grid with $N_i$ samples along cell vector $\mathbf{a}_i$ has step $\mathbf{a}_i/N_i$ and excludes the duplicate final plane. A finite endpoint grid uses $\mathbf{a}_i/(N_i - 1)$. Its quadrature uses endpoint weights. These conventions affect coordinates and integration even when two files display similar shapes.

For a periodic density grid with $N = N_a N_b N_c$ and cell volume $V$,

$$Q = \int_V \rho(\mathbf{r})\, dV \simeq \frac{V}{N} \sum_j \rho_j.$$

The output is an electron count when $\rho$ is in $e/\text{Å}^3$. A finite grid uses trapezoidal boundary weights. Do not sum raw values without the geometry-dependent weights.

Cube/XSF covering a complete periodic cell may include duplicate endpoint planes. **Verify periodic endpoint planes** checks opposite values and removes those duplicates when compatible. Matching endpoint values is a numerical prerequisite, not proof that the physical system is periodic. A finite molecular box with nearly zero density at its faces should not automatically be declared a periodic material.

## 5.4 What does Reference mean?

**Reference is the second input to the selected operation.** It is not a literature citation, a standard electron density supplied by AtomForge, or an automatically calculated neutral-atom density.

For subtraction, the selected **Field** is $A$ and the reference/operand is $B$, giving $A - B$. For weighted density difference the result is $A - wB$. For resampling, the reference supplies the target grid geometry. For coloured isosurfaces, the reference supplies the colour values on the surface generated from $A$. For Boolean operations, the operand is the second binary mask.

Click **Reference** to load another file. Then select its intended channel in the **Reference** or **Operand** selector. If no external reference is loaded, the selector can refer to a field already in the main volume. When an external reference exists, **Use loaded/result field as reference** switches the operand source back to the main loaded/derived field list. Always verify the operand after loading a new file or changing tools.

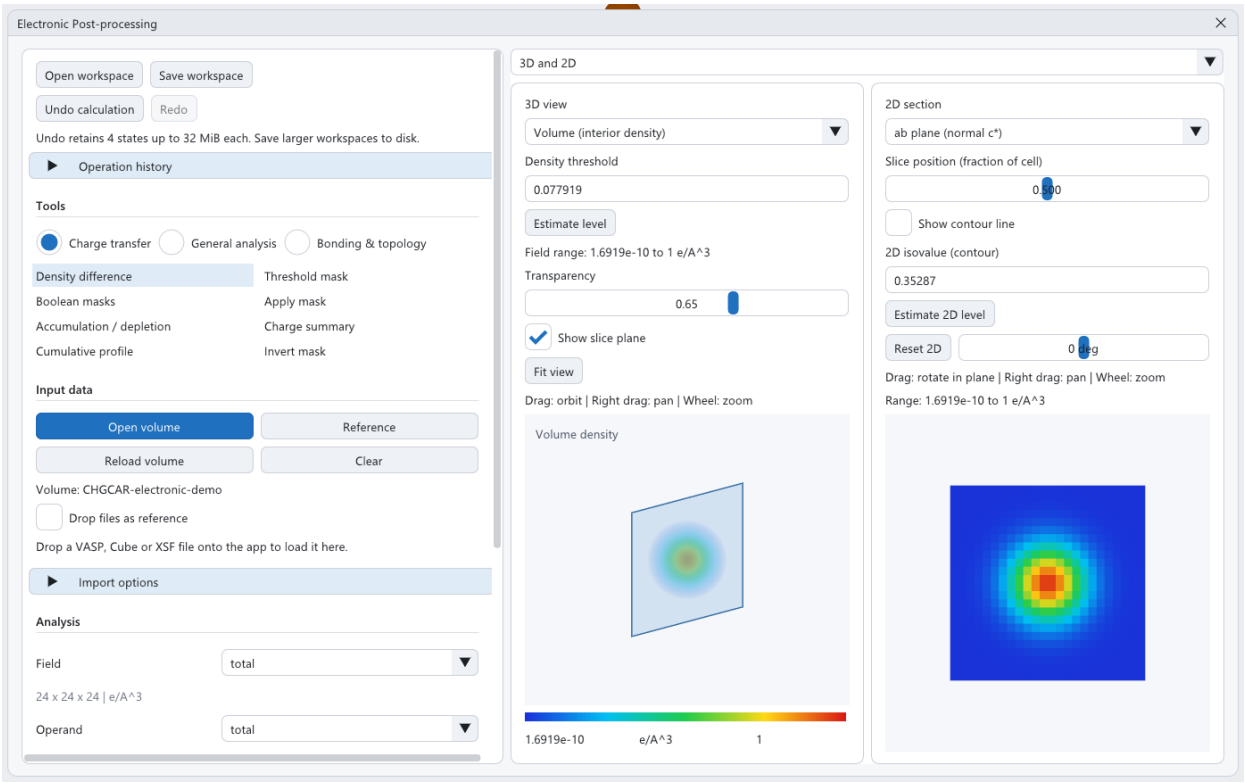



Figure 5.1: Actual AtomForge window with the supplied CHGCAR loaded. Charge-transfer tools are at the upper left. The 3D density threshold, transparency, 2D slice, and optional contour have independent controls.

**Avoid accidental self-subtraction**

Immediately after loading a single-channel density, both Field and Operand can display the same field. Calculating density difference then gives zero. Load the intended independent reference, or deliberately choose another derived field, before interpreting a result as charge transfer. Self-subtraction is useful as a software sanity check only.

## 5.5 3D volume and isosurface views

Choose the layout **3D and 2D**, **Only 3D**, or **Only 2D** at the top. In the 3D panel, **Volume (interior density)** ray-integrates the sampled field above the density threshold. Lowering the threshold admits more of the field; raising it emphasises high-valued regions. This is a visual selection, not an atom/fragment partition.

**Isosurface (boundary)** extracts a surface at the chosen scalar value. After entering the level, use **Update surface**. An isosurface is a boundary of constant value; it cannot by itself show the interior as a filled volume. Choose Volume when the interior density is the intended visualisation.

**Transparency** ranges from 0 (fully opaque contributing volume/surface) to 1 (fully transparent). Even at zero transparency, regions excluded by the threshold remain empty, and a separately enabled slice guide has its own translucent appearance. Opaque does not mean that the whole cell becomes a solid cuboid.

**Camera.** Left-drag inside the 3D canvas to orbit, right-drag to pan, and use the wheel to zoom. **Fit view** or a double-click resets framing. Camera actions do not rotate the numerical grid, change its cell, or alter an exported charge integral.

**Colour and lighting.** Expand **Appearance** to choose Spectrum, Blue-white-red, or Sequential blue. Automatic range uses the data range; manual minimum/maximum supports consistent comparison. Lighting controls change shading and specular response. Colour is mapped from values, while lighting modifies their rendered appearance; use the colour scale for numerical interpretation. For signed differences, use a diverging palette with symmetric limits $[-m, m]$ when comparing positive and negative regions.

## 5.6 Choosing an initial level and understanding warnings

The initial level is a display heuristic estimated from the file distribution: approximately the 90th percentile for a nonnegative field, zero for a signed field, with a low-range fallback for sparse fields and special handling for constant data. **Estimate level** restores the suggested 3D value. It is not an automatically determined bonding threshold or Bader dividing surface.

A 3D level outside the selected field's minimum–maximum range produces a warning and cannot generate a meaningful boundary. A 2D level is checked against the *current slice*, which can have a narrower range. A valid level in the full volume may therefore be out of range on one plane. Constant fields also cannot produce an ordinary internal contour. Enter finite numbers and check the displayed units.

Volume preview sampling is bounded at up to 96 points per axis and 256 ray steps. The interactive 2D preview uses up to 129 samples per direction. These limits keep navigation responsive; the calculation and export grids retain their own data. Use numerical sections and exported meshes/tables for analysis, and test resolution when interpreting very thin features.

## 5.7 2D lattice-plane sections

Select **bc plane (normal a*)**, **ac plane (normal b*)**, or **ab plane (normal c*)**. The **Slice position** is a fraction of the corresponding cell vector. In a skew cell these normals are reciprocal directions, not necessarily Cartesian X, Y, and Z.

The filled colour map shows the sampled values. **Show contour line** is off by default; enable it to display the line at **2D isovalue (contour)**. **Estimate 2D level** chooses a suggestion from that section. Changing this contour value does not change the 3D threshold or discard the rest of the 2D colour map.

Left-drag rotates the plane's image within its own plane, right-drag pans, and the wheel zooms. The angle slider gives an explicit in-plane rotation. **Reset 2D** or a double-click resets the 2D camera. These operations leave the physical intersection plane unchanged; moving the slice position changes which data are sampled.

## 5.8 Arbitrary planes and the 3D guide

Choose **Arbitrary plane**. Enter a point as fractional $(f_a, f_b, f_c)$ and a normal as Cartesian $(n_x, n_y, n_z)$. The point's Cartesian location is

$$\mathbf{p} = \mathbf{o} + f_a\mathbf{a} + f_b\mathbf{b} + f_c\mathbf{c}, \qquad \mathbf{n} \cdot (\mathbf{r} - \mathbf{p}) = 0.$$

The normal must be finite and nonzero; its length does not set a thickness. For a central oblique section, use point $(0.5, 0.5, 0.5)$ and normal $(1, 1, 1)$. The intersection is clipped to the actual cell. A plane that misses the cell produces a warning instead of a valid section.

Enable **Show slice plane** in the 3D controls to show the same intersection as a translucent guide. Disable it for an unobstructed density view. Rotating the 3D camera helps verify the plane

orientation; rotating the 2D image only changes its presentation. To specify a Miller-index plane in scripts, use `miller_section`, which converts the reciprocal normal explicitly.

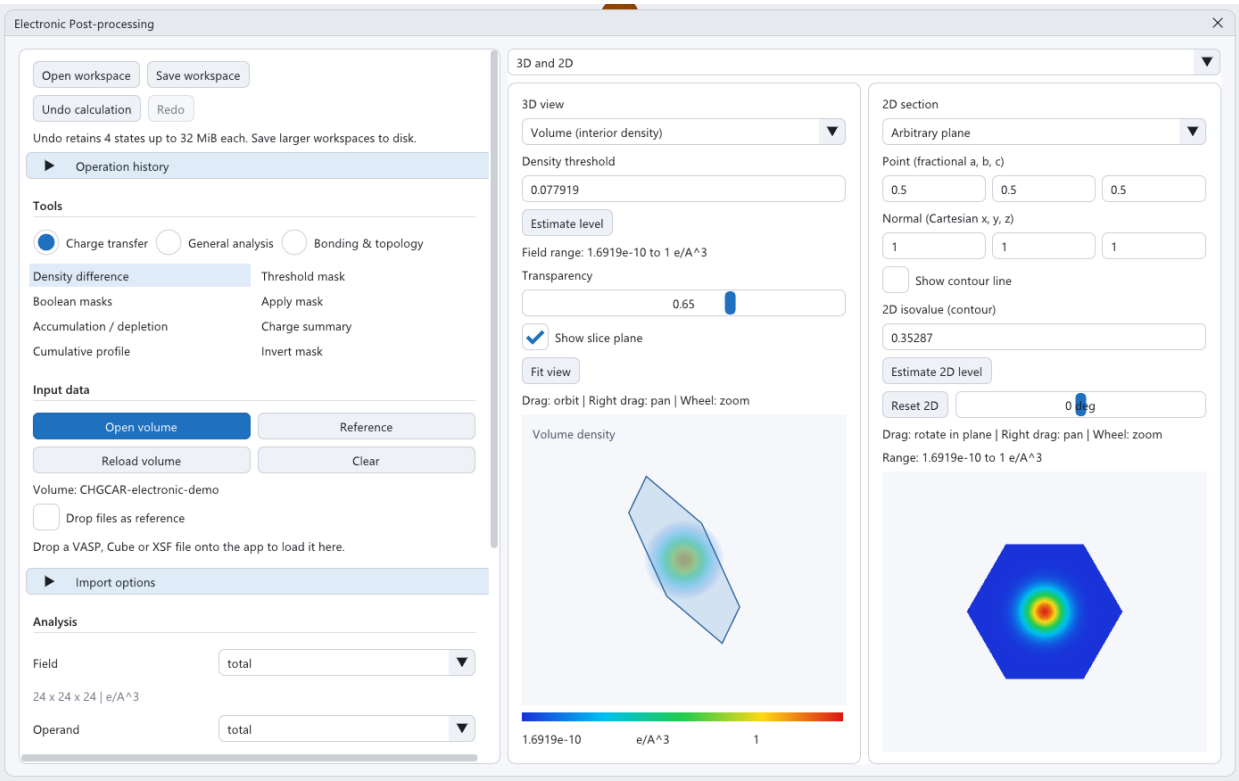



Figure 5.2: An arbitrary section of the supplied CHGCAR with a fractional point and a Cartesian normal. The translucent guide locates the section in the 3D volume.

## 5.9 Worked CHGCAR walkthrough

The documented file was supplied as `C:/Users/a.linda/Downloads/CHGCAR`. It is not copied into this repository. The first field has $80^3 = 512{,}000$ samples, 16 imported sites, periodic geometry, and unit $e/\text{Å}^3$. Its cell vectors in Å are

$$\begin{pmatrix} 5.943480 & -0.002166 & -0.002166 \\ -0.002166 & 5.943480 & -0.002166 \\ -0.002166 & -0.002166 & 5.943480 \end{pmatrix}.$$

1. Open this file with Auto quantity and select the first field, **total**. Check the dimensions and unit.
2. In General analysis, calculate the total integral. Expect approximately 183.99999985 electrons for this file.
3. Inspect the ab section at position 0.5. The full field range is approximately 0.151228–19.347671 $e/\text{Å}^3$; the section can contain a different subset of that range.
4. Calculate a planar average along $c^*$ and export CSV. Then choose Charge transfer, Cumulative profile, normal $c^*$, Calculate, and export that table separately.
5. Check the last cumulative value against the full integral. Both use the same grid quadrature; the generated example verifies their agreement.

6. Try an arbitrary central plane, then return to the lattice plane. Save images with their thresholds and colour ranges recorded.

The approximately 184-electron total is a numerical property of the supplied first field. Without the original calculation settings and valence definitions it should not be used to infer oxidation states. The input hash and precise values are recorded in `examples/chgcar-summary.json`.

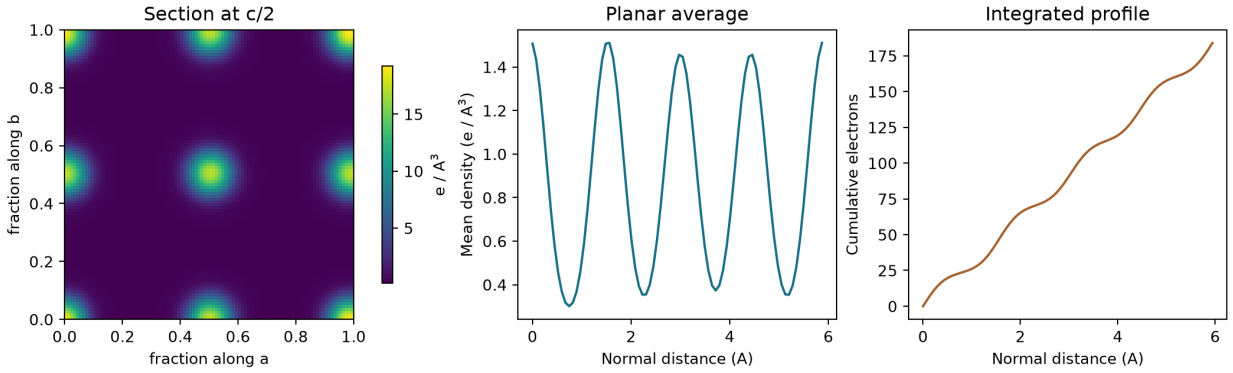



Figure 5.3: Numerical section, planar average, and cumulative integral generated through AtomForge's Python electronic API from the supplied CHGCAR. The left panel uses fractional in-plane coordinates; the profiles use distance along the cell-face normal.

## 5.10 Exporting the right result

Expand **Export**, choose Grid: XSF/Cube/VASP, Table: CSV, or Surface: OBJ/PLY, then click **Save as**. Browse to a directory and choose the output filename. Grid export uses the selected field; table export uses the calculated table; surface export requires a generated surface. Selecting a density field does not turn an older table into a table of that field: calculate the desired table again before exporting it.

When reopening exported grids, supply the correct quantity and endpoint convention. Generic filenames and interchange formats may not preserve enough metadata to infer physical units automatically. Cube export uses atomic-unit density/potential values; read it with the corresponding explicit quantity. Keep a record of whether duplicate periodic endpoints were written and verify round-trip geometry/integrals before combining exported data with the originals.

# Chapter 6

# Charge redistribution and Boolean masks

## 6.1 Define the study before subtracting

A common redistribution field for a combined system AB is

$$\Delta\rho(\mathbf{r}) = \rho_{AB}(\mathbf{r}) - \rho_A(\mathbf{r}) - \rho_B(\mathbf{r}).$$

Positive values mean electron accumulation relative to the chosen references; negative values mean depletion. This is an electron-density convention. The corresponding signed electric charge has the opposite sign after multiplying by the elementary charge.

References must answer the intended scientific question. For an interaction-induced difference, fragment calculations usually use positions consistent with the combined geometry, the same cell, and compatible electronic settings. Subtracting independently relaxed fragments mixes interaction and deformation effects. AtomForge does not generate fragment calculations or infer which reference is physically appropriate.

## 6.2 Grid compatibility

Before subtraction or masks, compare cell vectors, origin, dimensions, periodic/finite convention, and units. Matching atom counts or filenames is insufficient. Arithmetic requires aligned samples. A translated density with the same cell can produce a misleading difference if it is not registered consistently.

Use **Resample to reference grid** only as an explicit interpolation step when geometries are suitable. Resampling does not fix a physically inconsistent reference, and interpolation need not preserve the integral exactly. Compare integrals before and after and refine the grid when needed. Do not resample merely to silence an error without understanding what differs.

## 6.3 Weighted density difference: complete GUI recipe

1. Open `CHGCAR-AB` as the main volume. Select its total density field.
2. Choose **Charge transfer > Density difference**. Click **Reference** and load `CHGCAR-A`.
3. Check the **Operand** channel and set **Reference weight** to 1. Click **Calculate**. The appended field represents $\rho_{AB} - \rho_A$.

4. Select this derived field. Load `CHGCAR-B` through Reference, verify that the external reference is being used, and calculate again with weight 1.
5. Select the new result $\Delta\rho$. Export it as a grid and use a diverging palette with an explicit symmetric colour range.
6. Calculate **Charge summary** and **Cumulative profile** from this difference field, not from the original positive density.

For more fragments, repeat the subtraction using the latest result. A weight multiplies the reference pointwise; it is useful for explicit linear combinations and must be recorded. A fractional weight does not automatically represent a physically meaningful fractional occupation calculation.

The supplied single CHGCAR cannot demonstrate a physical AB–A–B charge transfer by itself. The executable example instead checks that subtracting the file from itself produces exactly zero. Real transfer studies require the independently calculated, compatible reference densities.

## 6.4 Accumulation and depletion fields

**Accumulation / depletion** splits a density-like field into

$$\rho_+(\mathbf{r}) = \max(\Delta\rho, 0), \qquad \rho_-(\mathbf{r}) = \max(-\Delta\rho, 0).$$

Both outputs are nonnegative. Depletion is stored as a positive magnitude, not as the original negative part. Select each result separately for display/export. Their difference recovers $\Delta\rho$; their sum is its absolute magnitude.

A 3D positive threshold on a signed difference highlights accumulation. To see depletion with the same positive threshold, select the depletion-magnitude field. Alternatively, generate a negative isosurface on the signed difference. Record the sign, level, and colour meaning in every figure caption.

## 6.5 Charge summary

**Charge summary** reports accumulation, depletion, and net electrons:

$$Q_+ = \int \rho_+ \, dV, \qquad Q_- = \int \rho_- \, dV, \qquad Q_{\mathrm{net}} = Q_+ - Q_-.$$

For references with matching total electron counts, a small net value is a useful check. A zero net value does not imply no redistribution: positive and negative regions can be substantial and cancel. Conversely, a nonzero net value can reflect electron-number choices, inconsistent normalisation, incomplete regions, or interpolation error.

These totals describe redistribution. They are not automatically the number of electrons transferred from atom A to atom B. To assign a transfer to a fragment or region, define a partition and integrate over it. Report the partition and its sensitivity alongside the result.

## 6.6 Threshold mask

**Threshold mask** creates a dimensionless binary field,

$$M(\mathbf{r}) = \begin{cases} 1, & L \le f(\mathbf{r}) \le U, \\ 0, & \text{otherwise.} \end{cases}$$

Bounds are inclusive, finite, and ordered. Select the source field, enter Lower bound and Upper bound in its units, and Calculate. The result has mask units and values 0 or 1. A threshold-selected region is a definition you choose, not a unique chemical boundary.

Example: a density threshold $0.01 \leq \rho \leq \rho_{\max}$ can identify a density-supported region in a fragment field. If the lower bound lies below the entire field's range, the mask may fill the complete cell. Inspect it and its volume before interpreting a regional integral. On the supplied CHGCAR, 0.01 $e/\text{Å}^3$ is below the minimum and would select the whole grid with an adequate upper bound.

## 6.7 Boolean masks and inversion

Choose two aligned binary masks as Field and Operand, select **Boolean masks**, then choose an operation:

| Operation | Retained samples |
|---|---|
| Union | In A or B, including their overlap. |
| Intersection | In both A and B. |
| Difference | In A and not in B. Order matters. |
| XOR | In exactly one mask; excludes overlap. |
| Invert mask | In the current grid domain and not in A; a single-input operation. |

To combine intermediate masks, enable **Use loaded/result field as reference** if an external reference is loaded, then select the desired derived mask as Operand. A raw density is not a binary mask and should be thresholded first. Inversion is limited to the grid domain; it does not construct the infinite space outside a molecule.

## 6.8 Apply mask and quantify a region

Select the density or difference as Field and a binary mask as Operand. Choose **Apply mask** and Calculate. Values inside the mask are retained; values outside become zero. The output retains the field's units. Select this output and calculate the total integral under General analysis to obtain the regional value.

**Example sequence.** Create fragment masks $M_A$ and $M_B$, intersect them to form an overlap region, and apply this to $\Delta\rho$. Its integral describes net redistribution inside that particular threshold-defined overlap. Repeat with several sensible bounds to estimate partition sensitivity. The result should not be labelled a Bader charge or a unique atomic charge.

Integrating a binary mask itself gives a selected geometric volume in $\text{Å}^3$ under the grid quadrature. This is useful for checking whether a mask selects everything, nothing, or an unexpectedly tiny region before applying it to density.

## 6.9 Cumulative charge profile

Choose **Cumulative profile** and a plane normal $a^*$, $b^*$, or $c^*$. The profile accumulates charge from the lower face of the cell to a position $s$ along that normal:

$$Q(s) = \int_0^s \int_{\text{face}} \Delta\rho \, dA \, ds'.$$

The last row includes the entire cell and agrees with the total integral under the same quadrature. Periodic profiles include a final full-cell endpoint; finite grids use endpoint-aware integration. A cumulative electron profile is distinct from a planar-average density, whose units remain $e/\text{Å}^3$.

For a slab/interface, select the actual interface normal. The location of the lower face, and therefore the cumulative curve, depends on the chosen cell origin. Moving a periodic cell boundary changes where accumulation begins. Use the same origin across comparisons and identify the interface/partition position used to quote a transfer value. A plateau can be informative, but its physical interpretation still depends on the reference and partition.

## 6.10 Recommended record for a charge study

Retain input filenames and checksums, selected channels, cell vectors/origin/grid shape, units and conversion options, all reference weights, any interpolation, and output integrals. Record mask definitions or partition geometry, profile direction and origin, 3D/2D levels, and colour scale. Verify that the integrated difference equals the corresponding weighted difference of input integrals to numerical accuracy. This check catches many bookkeeping errors before a visually convincing plot is interpreted.

# Chapter 7

# Electronic operation reference

## 7.1 How to run an operation

Load a volume, select **General analysis**, choose an operation, set the visible parameters, and Calculate. The selected Field is always the primary input. Operations producing a grid append one or more fields; operations producing a table replace the current table output. Export after verifying the selected field or table heading. The eight charge-transfer operations are detailed in Chapter 6; this chapter covers the remaining 25 operation choices.

## 7.2 Total integral

**Input:** any scalar grid. **Output:** one number, with units of field unit times $Å^3$. For density this is electrons; for a mask it is selected volume; for potential it is an integrated potential, not charge. Select the field, Calculate, and export CSV if needed. Use the integral to check imports and arithmetic. The full-cell result depends on periodic/finite endpoint conventions and the determinant of the cell, not just the raw sample sum.

## 7.3 Add and subtract reference

Select A as Field and B as Reference; calculate $A + B$ or $A - B$. Grid alignment and compatible units are required for meaningful addition/subtraction. These are general scalar operations. For charge studies, the dedicated weighted density-difference tool also enforces density-specific intent. Check operand order and the derived-field selection before repeating an operation.

## 7.4 Multiply and divide reference

These operations combine aligned values pointwise, $A_j B_j$ or $A_j / B_j$. They are not convolution or matrix operations. Division is unsuitable where the denominator is zero or numerically close to zero; inspect the field range and error output. Dimensional meaning follows the operands, so a multiplied density does not retain the interpretation of a density integral. For region selection, use a binary mask and Apply mask, which explicitly preserves the original field unit.

## 7.5 Scale

Enter a finite **Scale factor** and Calculate. Every value is multiplied by that number. A factor of $-1$ reverses signs; a factor of zero creates a zero field. Scaling is useful for explicitly defined linear combinations, but it does not infer the unit conversion needed by an arbitrary producer. Record any conversion factor and the original unit metadata.

## 7.6 Gaussian smoothing

Set **Sigma (A)** and **Radius (steps)**. Sigma describes the physical Gaussian width, while radius bounds the neighbourhood in grid steps. Both matter: an extremely small radius can truncate a broad Gaussian severely. The operation smooths sampled data and creates a new field; it does not increase the original physical resolution.

Start with a small kernel, inspect peaks and the integral before and after smoothing, and compare against the unsmoothed field. Finite boundaries and discrete normalisation can affect behaviour. Do not use smoothing to remove a physically meaningful oscillation without documenting it. Derivatives of a noisy field are often sensitive to smoothing and grid spacing, so report both settings.

## 7.7 Cartesian gradient

Calculate to obtain three component fields representing $\partial f/\partial x$, $\partial f/\partial y$, and $\partial f/\partial z$. The derivatives account for the actual cell geometry; they are Cartesian components even when the sampling lattice is skew. Their units are field units per Å.

Select each output independently and inspect signs/ranges. A potential gradient can be related to a field only after applying the appropriate physical sign/convention. Differentiation amplifies high-frequency noise; compare resolutions and boundary behaviour. A colour map of one component does not represent the vector magnitude unless you explicitly construct that magnitude.

## 7.8 Laplacian

The Laplacian is $\nabla^2 f$, with units of field units per Å$^2$. Select the field and Calculate to create a derivative grid. Positive and negative regions measure local curvature in the scalar field under the numerical discretisation.

Check a simple smooth input before interpreting fine structure. Grid resolution, smoothing, and finite boundaries can change extrema. Applying the operation to electron density does not by itself perform a full topological bonding analysis; applying it to potential does not automatically establish a charge density without the physical constants and electrostatic convention.

## 7.9 Energy-density conversion

This operation produces approximate kinetic, potential-energy, and total-energy density fields through the implemented gradient-expansion kinetic and local-virial expressions. The required input is a nonnegative electron density in $e/\text{Å}^3$; outputs are in eV/Å$^3$. Samples at or below $10^{-12}$ $e/\text{Å}^3$ are masked to zero to avoid singular low-density expressions.

Select an actual density, Calculate, and inspect the three appended fields. A signed difference density is not an acceptable substitute for the required nonnegative density. The output is an approximation derived from the available scalar density. It is not the exact kinetic energy from

occupied orbitals or the complete DFT energy decomposition of the originating code. Check resolution and the effect of low-density masking before integrating or comparing these quantities.

## 7.10 Line profile

Enter **Origin (A)**, **End (A)**, and **Sample count**. The coordinates are Cartesian. Calculate to sample along the segment and obtain distance/value rows. Export CSV and plot value against distance with the selected field's units.

For a bond-axis example, obtain two atom positions and use them as endpoints, or extend slightly beyond them to inspect both sides. Finite-grid endpoints must lie within the supported domain; periodic sampling follows the periodic convention. Increasing sample count interpolates the existing grid and does not create information finer than the original calculation. Check the actual geometric length against the final profile coordinate.

## 7.11 Planar average

Choose the plane normal $a^*$, $b^*$, or $c^*$. The operation averages over planes parallel to the other two cell vectors and returns normal distance and mean scalar value. A density average remains in $e/\text{Å}^3$; it is not a plane-integrated electron count.

For the $c^*$ direction, let $A = |\mathbf{a} \times \mathbf{b}|$ and $L = V/A$. Then the relationship is $Q = A \int_0^L \bar{\rho}(s)\, ds$ with the corresponding numerical quadrature. This relationship is useful for interpreting the cumulative profile. In a skew cell, $L$ is the cell height normal to the face, not necessarily $|\mathbf{c}|$.

## 7.12 Macroscopic average

Choose a plane normal and an odd **Window**. The operation averages the planar profile over a moving window of samples to suppress short-range oscillations. A window of one leaves the planar profile unchanged; a larger odd window spans more planes.

Relate the sample count to a physical width using the normal grid spacing. When comparing calculations with different grid sizes, the same integer window does not necessarily mean the same physical smoothing length. Inspect the unsmoothed planar average alongside the macroscopic average; broad smoothing can erase interface or surface features.

## 7.13 Numerical plane section

Enter a Cartesian origin and two non-collinear span vectors **Plane span u (A)** and **Plane span v (A)**. The sampled rectangle/parallelogram is $\mathbf{r}(s,t) = \mathbf{o} + s\mathbf{u} + t\mathbf{v}$, with $s, t$ spanning the section. The GUI's sample count applies to both directions; Python accepts an explicit pair.

This is a numerical operation that creates a section field, distinct from merely changing the interactive 2D preview plane. The internal result uses two identical layers so it can be represented by the grid container. Use its section values or contours for two-dimensional analysis; integrating its artificial three-dimensional volume has no physical section-charge meaning. Finite inputs must cover the requested section, while periodic data can wrap.

## 7.14 Contour segments

Enter an isovalue and Calculate. The output is a table of Cartesian endpoints, with successive pairs defining contour segments on the source grid's first section layer. Generate/select a numerical section first when you need a particular plane. Applying contours directly to a full volume does not extract a 3D isosurface.

Choose a level within the section's range. Export CSV and connect rows in pairs rather than connecting every row as one continuous polyline. Equal-valued cells, grid resolution, and interpolation can influence topology. The interactive **Show contour line** checkbox is a display convenience; the numerical contour tool supplies coordinates for downstream work.

## 7.15 Peak search

Calculate local maxima using a 26-neighbour grid comparison. The output rows are $x, y, z,$ value. Adjacent equal values use a deterministic tie break, so a plateau does not yield every sample as an independent peak. Python's optional limit can restrict the number returned.

Peak coordinates are based on the sampled grid; they are not sub-grid fitted critical points. Refine the grid to check location stability. Density maxima often occur near nuclei, but assigning a peak to an atom or a bond requires additional geometric and physical interpretation.

## 7.16 Voronoi integration

The GUI uses imported atomic sites and assigns sample weights to the nearest-site Voronoi regions. The result contains integral and region volume for each site in imported atom order. Python can accept explicit Cartesian partition sites. Boundary samples share weight where appropriate, and periodic geometry uses the periodic distance convention.

Check that imported sites are present and correspond to the intended structure. Compare the sum of region integrals with the full integral. The partition is geometric and sensitive to site positions. These are not zero-flux Bader basins and must not be labelled Bader charges. A region electron count also needs an explicit reference valence/nuclear convention before being converted into a net ionic charge.

## 7.17 Sphere integration

Enter a Cartesian centre in **Origin (A)** and a positive **Sphere radius (A)**. Calculate the integral over the selected spherical region according to the grid sampling and boundary convention. The radius is in Å, not fractional coordinates.

Compare several nearby radii and grid resolutions. A sphere cutting through a rapidly varying density can produce a strongly radius-dependent electron count. Check whether the sphere crosses a cell boundary and whether the finite/periodic convention matches your intended region. This tool defines a spherical partition; it does not locate an atom's natural electronic boundary.

## 7.18 Structure factors from a grid

Enter one integer `hkl` row per requested reflection. The output gives $h, k, l, \Re F, \Im F$. The implemented forward phase convention is positive:

$$F(\mathbf{h}) = \int_V f(\mathbf{r}) \exp(+2\pi i\, \mathbf{h} \cdot \mathbf{f})\, dV,$$

where $\mathbf{f}$ denotes the fractional coordinate of the Cartesian position relative to the global zero, including the grid-origin phase. Use $F(0, 0, 0)$ as an integral/normalisation check. Nonzero reflections depend on origin and phase convention. Do not compare complex values from another program until those conventions agree. These Fourier tools require periodic endpoint-excluded grids and reject indices outside the unique sampled Nyquist range.

These are Fourier coefficients of the supplied field. They are not a full experimental diffraction intensity prediction including instrument response, occupancies, thermal motion, anomalous scattering, and other corrections unless those effects have been supplied by an appropriate model.

## 7.19 Fourier synthesis

Enter rows `hklrealimag`. Include conjugate pairs for a real field, $F(-\mathbf{h}) = F(\mathbf{h})^*$. Calculate to synthesise a field on the selected grid geometry. The inverse uses the matching phase convention and cell normalisation.

An incomplete reflection set produces a band-limited approximation and can generate termination ripples or negative values. Include the zero reflection when the mean value matters. The real-space grid must be fine enough for the highest frequencies; adding large Miller indices beyond a grid's sampling capacity does not produce a trustworthy high-resolution field.

## 7.20 Patterson map

Calculate the Patterson/autocorrelation field of a periodic input. It describes displacement-vector correlations and discards the phase information needed to uniquely recover the original field. A strong origin peak is expected for an autocorrelation and should not be mistaken for an extra atom.

Use a modest grid first and inspect runtime; reciprocal/direct transforms can be expensive. Interpret maxima as correlations of separations, not ordinary density maxima at original atom positions. Finite, incomplete, or differently normalised inputs change the meaning of the result. Preserve the original field and its units/normalisation with any exported Patterson map.

## 7.21 Ewald point-charge electrostatics

This tool requires periodic geometry, imported sites, and an explicit charge for each site in file order. Supply charges in electron-charge units. The cell must be neutral; the implementation uses a conducting boundary convention. Do not enter atomic numbers unless bare nuclear point charges are truly the intended model, and do not infer charges automatically from element names.

Set **Ewald alpha (1/A)**, **Real cutoff (A)**, and **Reciprocal cutoff (1/A, includes 2*pi)**. Calculate. The table contains the cell energy in eV followed by the site potentials in volts. Alpha partitions the calculation between real and reciprocal space; an adequately converged result should be insensitive to reasonable alpha changes when both cutoffs are converged.

Increase both cutoffs and compare the energy and potentials before accepting a result. Ewald point-charge energy is not a self-consistent electronic total energy and does not include every energy

term of a force field or DFT calculation. Record the charge model, boundary convention, cell, alpha, and cutoffs.

## 7.22 Isosurface extraction and reference colouring

Set the 3D level and choose the Isosurface operation or **Update surface** in boundary mode. **Color from reference** samples an aligned reference field on the extracted surface. For example, a density can determine geometry while a compatible potential determines colour. Confirm both the geometric field and colour-field units.

**Append surface level** retains an existing surface and adds another level; appended surfaces must use compatible colouring units. This supports multiple boundaries but can visually obscure one another. Set transparency deliberately and export the mesh when required. OBJ stores scalar comments and PLY stores a scalar vertex property; downstream tools may need an explicit colour mapping and may not automatically reproduce AtomForge's palette. Surface vertices are emitted as triangles without welding.

## 7.23 Resample to reference grid

Select the source as Field and the desired target geometry as Reference. Calculate an interpolated source field sampled on that reference geometry. The reference's scalar values are not subtracted or blended by this operation. Check that the target points lie in the valid source domain for a finite source.

Compare output shape, cell, origin, and periodicity with the target, and compare integrals and extrema with the source. This is interpolation, not a conservative charge-remapping guarantee. Resampling is useful for explicit alignment of compatible grids but cannot repair inconsistent physical calculations.

## 7.24 Verify periodic endpoint planes

Use this only for finite Cube/XSF-style grids that span a full periodic cell with duplicated endpoints. The operation checks opposite planes and removes the duplicates, giving periodic sampling. A mismatch is a reason to inspect the input rather than force conversion. Python exposes the tolerance, default $10^{-8}$.

After conversion, check the shape (one fewer sample along each duplicated direction), cell, periodic flag, and integral. Re-exported periodic data can again acquire explicit endpoint planes in a finite-grid format. Record the convention at every conversion step to avoid an extra plane being counted as independent data.

# Chapter 8

# Python and command-line integration

## 8.1 Install the Python package

From the repository root, install the local package into the Python environment you intend to use:

```
python -m pip install ./python
```

The structure API uses Python implementations of its supported file formats. Electronic operations additionally require the compiled native electronic library. Set `ATOMFORGE_PATH` to the viewer executable when it is not discoverable. If necessary, set `ATOMFORGE_ELECTRONIC_LIBRARY` to the absolute native library path before importing/using electronic operations. Keep required runtime DLLs beside the Windows library/executable or in the appropriate loader search path.

```
import os
os.environ["ATOMFORGE_PATH"] = r"C:\path\to\AtomForge.exe"
# Only needed when automatic native-library discovery is insufficient:
# os.environ["ATOMFORGE_ELECTRONIC_LIBRARY"] = r"C:\path\to\atomforge_electronic.dll"
import atomforge as af
```

Inspect the actual library filename produced by your platform/build instead of assuming every platform uses the Windows prefix and suffix. A Python package installation alone does not compile or bundle every native dependency.

## 8.2 Structure objects and file I/O

`af.load(path)` returns a Structure. `structure.atoms` contains Atom objects with element symbols and Cartesian coordinates. `structure.cell` contains the three lattice vectors when available. `len(structure)` returns the atom count. Save with `structure.save(path)` or the package-level save function.

```
from pathlib import Path
import atomforge as af

examples = Path("docs/manual/examples")  # run from repository root
cu = af.load(str(examples / "cu_fcc.cif"))
assert len(cu) == 4
host = cu.repeat(5, 5, 5)
assert len(host) == 500
host.save("cu_host.cif")
```

```
edited = host.copy()
edited.translate(0.1, 0.0, 0.0)  # in place; cell is unchanged
edited.scale(1.01)             # in place; atoms and cell scale
edited.save("cu_shifted_scaled.cif")
```

`repeat(nx,ny,nz)` returns a new structure and requires positive integers and a cell. `copy()` makes a separate structure. `filter_species("Cu","Ni")` returns a filtered copy. `add_atom(symbol,x,y,z)` appends a Cartesian site; `remove_atom(index)` removes one by index. `set_cell(a,b,c,alpha,beta,gamma)` defines a lattice from lengths and angles. Check indexing after removals because later indices shift.

```
s = af.Structure()
s.set_cell(8.0, 8.0, 8.0, 90.0, 90.0, 90.0)
s.add_atom("Fe", 2.0, 4.0, 4.0)
s.add_atom("C", 3.8, 4.0, 4.0)
s.save("illustrative_pair.xyz")
# This is an illustrative geometry, not a relaxed Fe-C molecule.
```

The Python structure codecs cover XYZ and extended XYZ, VASP files, PDB, CIF with explicit P1 sites, and LAMMPS data. This is not identical to the desktop's Open Babel coverage. The Python CIF path does not replace the bulk builder's space-group expansion: use already expanded sites when importing a symmetry-rich CIF through this API. LAMMPS restricted-triclinic conversion can rotate geometry and shift origins; verify the resulting cell in the consuming simulator.

## 8.3 Python package 0.2.0: native builders

Install `atomforge-py==0.2.0` from PyPI and import it as `atomforge`. The wheel contains Python code; install the current AtomForge executable and electronic shared library separately for native features. Set `ATOMFORGE_PATH` to the executable, or place it on PATH. The electronic loader searches alongside that executable; `ATOMFORGE_ELECTRONIC_LIBRARY` remains an explicit override. Basic structure editing and codecs need neither binary.

```
import atomforge as af
cu = af.build("bulk", ["--a", "3.61", "--atom", "Cu 0 0 0"]).structure
alloy = af.build("sss", ["--frac", "Cu=0.7,Ni=0.3", "--seed", "42"],
                 source=cu.repeat(5, 5, 5), output="alloy.cif")
print(alloy.stdout, alloy.stderr)
particle = af.wulff(cu, [(1, 0, 0, 1), (1, 1, 1, 1)],
                    radius=20, vacuum=5, output="wulff.cif")
assert len(particle.structure) == 3552
print(af.builder_help("nano"))
```

`af.build` accepts bulk, gb, poly, nano, amorphous, sss, dislocation, and custom. Pass an argument list, with each vector or site specification in one string; repeated flags are preserved. Pass the reference as `source`, a Structure or a file path. The result exposes `structure`, `stdout`, `stderr`, and `output`. Inspect diagnostics for scientific warnings. An explicit output is written/replaced and retains native orientation sidecars; without it, temporary files are removed after loading. Python Structure does not import grain/IPF sidecar metadata. Supported result suffixes are CIF, XYZ/extXYZ, VASP, PDB, and LAMMPS variants listed by the API.

Native errors raise `subprocess.CalledProcessError` with diagnostic streams. The default timeout is 300 seconds; a time limit raises `subprocess.TimeoutExpired`. No command shell or GUI is opened. Use `builder_help(mode)` to check the installed executable's supported flags. Wulff scripting requires the updated executable with `--shapewulff`; repeated `--facet"hklenergy"`

entries specify nonzero Miller indices and positive relative energies. Radius is the maximum plane distance, not the farthest vertex distance. GUI camera, transparency, lighting, interface scaling, and file-dialog controls are presentation settings and do not alter these numerical APIs.

## 8.4 Structure pipelines

`atomforge.pipeline.Pipeline` is the non-destructive structure pipeline of the desktop (**Edit > Structure pipeline**) and `AtomForge--pipe`: add, insert, move, enable, disable, change and remove steps, then `run(structure)`; `Structure.pipe(text)` applies one directly. §3.8 describes every step and gives an example.

## 8.5 Atom clouds

`atomforge.cloud` writes and reads the desktop's large datasets (§4.8): `generate(path,"fcc", 3.615,(630,630,630),"Cu")`, `build("run.dump","run.afcloud",types=["Cu","Ni"])` and `from_structure(structure,path)` run the AtomForge executable; `info(path)` reads the header (atoms, box, species, chunks) and `read(path,chunk,count)` the positions and species of a chunk (NumPy arrays when NumPy is installed), both without the executable. A prefix of a chunk is a uniform random sample of it.

## 8.6 Desktop and notebook viewing

`structure.view()` opens the structure in AtomForge without blocking the Python workflow. `af.view(structure)` provides the package-level viewer. `structure.view_notebook(width=620,height=460)` requests an inline notebook view; if IPython support is absent it falls back to the desktop viewer. The notebook representation is a visual aid and does not run an electronic calculation.

```
host.view()
# In a Jupyter notebook:
host.view_notebook(width=800, height=600)
```

## 8.7 Loading electronic volumes

```
from atomforge.electronic import load_volume

volume = load_volume("CHGCAR")
rho = volume.fields[0]
print(rho.shape, rho.cell, rho.origin, rho.periodic, rho.unit)
print(len(rho.sites), rho.integrate())

# Cube coordinates and scalar quantity are separate choices:
cube = load_volume("density.cube", quantity="density",
                   cube_coordinates="bohr").fields[0]
xsf = load_volume("density.xsf", quantity="density").fields[0]
```

`Volume.fields` is a collection of scalar channels. `Grid.values` is a flat sequence with x varying fastest, then y, then z: index $x + N_x(y + N_y z)$. If using NumPy, reshape to `(nz,ny,nx)` for conventional z/y/x array indexing. `Grid.sites` contains rows of atomic number and Cartesian

$x, y, z$. Derived grids generally return new objects; preserve names in your script so that the intended intermediate is explicit.

## 8.8 A complete scripted charge workflow

The following recipe requires three independently calculated, aligned density files. It is not executed using fabricated fragment densities in this manual.

```
import csv
from atomforge.electronic import load_volume

total = load_volume("CHGCAR-AB").fields[0]
a = load_volume("CHGCAR-A").fields[0]
b = load_volume("CHGCAR-B").fields[0]
delta = total.density_difference(a, b, weights=[1.0, 1.0])
delta.save("AB_minus_A_minus_B.xsf")

summary = delta.charge_summary()
print(summary["accumulation"], summary["depletion"], summary["net"])
accumulation, depletion = delta.split_density()
accumulation.save("accumulation.xsf")
depletion.save("depletion_magnitude.xsf")

mask_a = a.threshold_mask(0.01, max(a.values))
mask_b = b.threshold_mask(0.01, max(b.values))
overlap = mask_a.boolean(mask_b, "intersection")
regional = delta.apply_mask(overlap)
print("Overlap redistribution, electrons:", regional.integrate())

with open("cumulative.csv", "w", newline="") as stream:
    writer = csv.writer(stream)
    writer.writerow(["distance_A", "cumulative_e"])
    writer.writerows(delta.cumulative_charge(axis=2))
```

Use `boolean(...,"union")`, `"difference"`, or `"xor"` for the other mask combinations. `invert_mask()` complements a binary mask. `density_difference` accepts multiple references and one optional weight per reference. Array operators support aligned arithmetic; scalar multiplication/division support explicit scaling. Check units and division denominators.

## 8.9 Profiles, sections, derivatives, and integration

```
gx, gy, gz = rho.gradient()
lap = rho.laplacian()
smoothed = rho.smooth(sigma=0.2, radius=2)
kinetic, potential, total_energy = rho.energy_density()

profile = rho.line_profile((0, 0, 0), (1, 1, 1), count=100)
planar = rho.planar_average(axis=2)
macro = rho.macroscopic_average(axis=2, window=5)
q_sphere = rho.integrate_sphere(center=(1, 1, 1), radius=0.5)
site_rows = rho.voronoi_integrate()  # imported sites, file order
peaks = rho.peaks(limit=10)

origin = tuple(rho.origin[i] + 0.5*rho.cell[2][i] for i in range(3))
section = rho.section(origin, rho.cell[0], rho.cell[1], (100, 100))
```

```
values_2d = section.section_values
segments = section.contours(level=1.0)
```

Coordinate examples above assume those locations lie in the input domain. The returned section is a two-layer container; use `section_values` for 2D data. Returned profile and integration tables are Python row sequences suitable for CSV or plotting. Choose a contour level inside the section range rather than blindly reusing 1.0 for every file.

`miller_section(grid,hkl,offset,width,height,shape)` constructs an orthonormal section with a reciprocal-space normal. Offset defines the fractional equation $hf_a + kf_b + lf_c = \text{offset}$; width and height are in Å. The in-plane orientation is selected deterministically. For finite data the entire requested rectangular section must lie inside the supported field.

## 8.10 Fourier and scattering models

`structure_factors(indices)` returns reflection rows. `fourier_synthesis(reflections)` creates a field from explicit complex reflection rows. Both require periodic endpoint-excluded geometry, unique sampled Miller indices, and consistent phase conventions. `patterson()` returns the periodic autocorrelation field.

```
reflections = rho.structure_factors([(0, 0, 0), (1, 0, 0), (-1, 0, 0)])
low_frequency = rho.fourier_synthesis(reflections)
low_frequency.save("three_reflections.xsf")
```

This deliberately incomplete synthesis retains only three Fourier terms and is not a reconstruction of the full original field. The zero reflection should equal the integral. Include conjugate pairs explicitly; invalid duplicates, out-of-range reflections, and a complex-valued result are rejected.

For atom-based scattering calculations, `atomic_structure_factors` accepts an explicit complex scattering value for every reflection/site, plus optional occupancies and isotropic B factors in $Å^2$. Occupancies are 0–1 and B factors nonnegative. No oxidation states or element coefficient tables are inferred.

`scattering_factor(s,a,b,c)` evaluates $\sum_i a_i \exp(-b_i s^2) + c$, with $s = \sin\theta/\lambda$. Supply a valid coefficient source and respect its applicable range. `model_density` requires exactly one mapping keyed by atomic number: X-ray coefficient tuples or neutron scattering lengths. It constructs a finite Fourier model on periodic geometry. Even-grid Nyquist planes are omitted to retain a real conjugate-symmetric expansion. These free-atom/scattering models are not self-consistent DFT charge densities and should not be substituted silently for fragment reference calculations.

## 8.11 Point charges, surfaces, and explicit geometry

`rho.ewald(charges,alpha,real_cutoff,reciprocal_cutoff)` returns a dictionary with `energy` in eV and `potentials` in volts. Charges must match the imported sites and sum to a neutral cell. Verify convergence using both cutoffs.

`rho.isosurface(level,color=other_grid)` returns a Surface. Its `vertices` property provides triangle/vertex data; `save("surface.obj")` or `save("surface.ply")` writes mesh output. Reference colouring requires compatible geometry and meaningful units. A scalar property in an exported mesh may require colour-map configuration in the next application.

```
from atomforge.electronic import Grid
geometry = Grid((8, 8, 8), ((4, 0, 0), (0, 4, 0), (0, 0, 4)),
                [0.0] * (8*8*8), periodic=True, unit="e/A^3")
```

```
# Attach atomic-number / Cartesian-coordinate sites explicitly:
geometry = geometry.with_sites([(1, 0, 0, 0), (1, 2, 2, 2)])
result = geometry.ewald([1.0, -1.0], alpha=0.8,
                        real_cutoff=12.0, reciprocal_cutoff=8.0)
print(result)
```

This is an illustrative neutral point-charge model. The two H labels supply site metadata; the explicitly supplied $+1, -1$ charges define the electrostatic model. It is not a prediction for a real hydrogen compound.

## 8.12 Electronic CLI

The Python module offers conversion and selected numerical operations without opening the GUI:

```
python -m atomforge.electronic CHGCAR --operation info
python -m atomforge.electronic CHGCAR --operation integrate
python -m atomforge.electronic CHGCAR --operation planar --axis 2 --output planar.csv
python -m atomforge.electronic CHGCAR --operation isosurface --level 1.8 --output
    density.ply
python -m atomforge.electronic density.cube --quantity density --operation convert --
    format xsf --output density.xsf
```

Choose channels with `--channel` (zero-based). Use `--quantityauto|raw|density|potentia l|elf` and `--cube-coordinatesbohr|angstrom`. `--periodic` verifies and removes duplicate endpoints; it is not a generic instruction to make arbitrary data periodic. Operations other than info/integrate require an output. Additional choices include laplacian, patterson, kinetic, potential-energy, total-energy, and peaks. Charge-mask combinations are available through Python scripts and the GUI, not every one as a dedicated CLI choice.

## 8.13 Native builder CLI

`AtomForge--helpMODE` prints the mode's accepted options. Available modes are bulk, sss, gb, nano, poly, amorphous, dislocation, and custom. Each returns an output structure without requiring a GUI interaction. Appendix D lists every mode's options and defaults. Use a subprocess argument list from Python to preserve spaces and avoid shell quoting problems:

```
import os
import subprocess
subprocess.run([
    os.environ.get("ATOMFORGE_PATH", "AtomForge"), "--build", "bulk",
    "--system", "cubic", "--spacegroup", "225", "--a", "3.61",
    "--atom", "Cu 0 0 0", "--output", "cu_fcc.cif"
], check=True)
```

Check the return code and read error output on failure. Keep the exact input arguments and seed with generated files. The manual's example generator uses this pattern and verifies every produced structure can be loaded with a nonzero atom count.

## 8.14 Headless dislocation construction

The dislocation CLI applies the same displacement builder without opening its dialog. Supply a periodic structure, character, region, and elastic parameters. Automatic direction presets are used unless `--manual-vectors` is supplied. Manual mode takes `--plane`, `--burgers`, and `--line`; these represent a slip plane, Burgers direction, and line direction, respectively. `--bmag` explicitly overrides automatic Burgers magnitude and is a CLI option beyond the GUI scale control. `--line-frac` positions the line in fractional coordinates; `--line-offset` adds a Cartesian offset. Cutoff zero disables radial attenuation. The freeform region accepts semicolon-separated local coordinate pairs through `--poly2d`.

```
AtomForge --build dislocation --input cu_host.cif --character edge --shape cylinder --
    cyl-radius 5 --core 1.2 --cutoff 8 --output cu_dislocation.cif
```

The added example preserves all 500 host atoms and reports 136 shifted atoms with $|b| = 2.55265$ Å. It also reports a lattice-family change warning (FCC-like to HCP-like); retain that warning with the example rather than treating it as a validated core. Check displacement and minimum-distance diagnostics and inspect the saved result. This is a starting geometry for further validation, not a relaxed core. Refer to the complete CLI listing for all shape and elastic options.

# Chapter 9

# Projects and scientific workflows

This chapter describes additions in the current source build. Rebuild the application and install the matching Python source package before using them; previously downloaded release binaries do not acquire new features automatically.

## 9.1 Save a project and recover a session

Use the File menu's project commands to save all open tabs in one `.afproject` file. A project stores atomic structures, grain colours and region IDs, camera framing, basic display switches, electronic primary and reference fields, the current mesh and result table, slice settings, and electronic operation history. It also stores the per-atom property shown with the *Atom Property* colour mode (with its range and hide filters) and the scientific tools window: the selected tool, every input, and the last result with its plots, generated files and result structures, so a reopened project shows the same result without recomputing it. Inputs that refer to files store the path only; keep those files beside the project. Projects written by this version use format 2; format 1 projects from earlier releases still open. Opening a project creates tabs without replacing unrelated open work. Projects store the numerical fields themselves, so the original CHGCAR or Cube file is not required to restore the saved calculation.

Use the electronic workspace's Open/Save workspace buttons to exchange a single analysis independently. A project is not a copy of every application preference: per-element material settings, every builder's transient inputs, and running calculations are not serialized. Save important builder parameters alongside the structure. A calculation must finish or be cancelled before a project can be saved. File writes replace the destination only after successful serialization; a failed write preserves the previous project.

Enable autosave in the File menu to write a recovery project approximately once per minute while calculations are idle. The recovery command loads the latest completed snapshot. On Windows the recovery file is below `LOCALAPPDATA/AtomForge`; on Linux it is below `XDG_STATE_HOME/AtomForge`, falling back to `HOME/.local/state/AtomForge`. Autosave is a recovery aid, not a versioned backup system. Work after the latest successful snapshot may need to be repeated.

## 9.2 Electronic history, cancellation and export

Electronic Undo and Redo restore the previous numerical workspace and views. Interactive history keeps at most four snapshots and clears the undo stack for a state larger than 32 MiB; save a project before expensive operations on large grids. The history records operation selections, numeric parameters and input filenames. It does not independently establish the provenance or physical validity of an external file.

Cancel calculation requests cooperative cancellation. The current stage's progress is shown where the algorithm can measure it. The completed result is discarded if cancellation was requested. Loading and long grid loops check cancellation, although external blocking I/O and uninstrumented work may delay the response.

The Appearance quality selector offers four resolutions. It changes the displayed volume texture, ray-sampling budget and 2D sampling grid; the underlying numerical field and numerical exports retain their original resolution. Use lower quality for interaction and higher quality for a final view. Rendering quality cannot recover detail absent from the input file.

The export selector includes a 3D PNG and a 2D SVG. The PNG is rendered at 1600 by 1200 pixels using the current camera, palette and opacity. It exports the data viewport rather than the whole application dialog. The SVG preserves the current section, pan, zoom, rotation and contour display. For a rotatable downstream model export OBJ or PLY instead. Save a project to retain all loaded and derived fields together.

## 9.3 Structural analysis and automation

Common-neighbour and angular-distribution analysis are available from Analysis. Interface Builder and Stacking Faults are available from Build. Their C++ algorithms are also exposed through native CLI and Python workflows.

```
import atomforge as af
host = af.load("POSCAR")
rows = af.cna(host, cutoff=3.1)
rdf = af.rdf(host, cutoff=6, bins=120)
adf = af.adf(host, cutoff=3.1, centre="Cu")
sro = af.sro(host, shells=3, tolerance=0.1)
voids = af.interstitial_sites(host, resolution=14, clearance=0.6)
cut = af.sculpt(host, [(1,0,0,-5,5), (0,1,0,-5,5),
                       (0,0,1,-5,5)])
```

Cutoff lengths and sculpt bounds are in Angstrom. Manual sculpt bounds are relative to the supercell centroid; periodic bounds instead specify an integer start plane and a number of structural periods. CNA is cutoff-sensitive: include the complete intended neighbour shell and use a sufficiently large periodic cell. FCC/HCP and BCC require different neighbour-shell conventions. ADF counts unordered neighbour pairs; smoothing affects the displayed values, not raw counts. These geometrical analyses do not replace energetic validation.

The native `--analyze` modes are cna, rdf, adf, sro, interstitial and sculpt. All require `--input` and `--output`; analyses write CSV, while sculpt writes a structure. Unsupported options are rejected. Native `--convert` provides structure conversion and extended XYZ export. The generated API appendix lists Python parameters and all ten builder modes, including interface and stacking-fault sequence export.

## 9.4 Recipes and batch electronic processing

```
from atomforge.electronic import load_volume, run_pipeline, batch_process
density = load_volume("CHGCAR").fields[0]
steps = [
    {"operation":"scale", "parameters":{"factor":2}, "name":"twice"},
    {"operation":"subtract", "reference":"input", "name":"difference"},
    {"operation":"integrate", "name":"electrons"},
]
```

```
results = run_pipeline(density, steps)
report = batch_process(["density1.cube", "density2.cube"],
                       steps, "batch-results")
```

Every step names an allowlisted operation, optional parameters and an optional unique result name. A reference can identify an earlier grid or an external file. The original input remains available as `input`; intermediate results can be inspected before saving. Geometry and units must match for binary operations; resampling is explicit. Batch outputs use input stems, reject name collisions and existing destinations, and report each input's success or failure. Inputs named CHGCAR in different folders need distinct output names or separate batch destinations.

The Python electronic command accepts an operation, a reference, JSON parameters, a JSON recipe or batch inputs. Use its `--help` for current argument syntax. Recipe execution does not evaluate Python expressions. Save the recipe, source calculation settings and input hashes separately when preparing an independently reproducible study.

## 9.5 Trajectories and animation

Open Analysis > Trajectories and transport > Trajectory playback, choose an XYZ or extended XYZ trajectory, and use Play/Pause, First frame, the frame slider and playback speed. Frames replace the active structure; preserve the original in another tab or project first. Main-window orbit, zoom and pan remain available. The desktop reader requires species followed by Cartesian coordinates; extended XYZ properties must begin with `species:S:1:pos:R:3`. It reads lattice vectors from the comment and rejects incomplete frames.

Python provides broader ASE trajectory formats, including XDATCAR and LAMMPS dumps:

```
from atomforge.science import read_trajectory, write_trajectory, export_animation
frames = read_trajectory("XDATCAR", format="vasp-xdatcar")
write_trajectory(frames, "trajectory.extxyz")
export_animation(frames, "trajectory.gif", fps=12, radii=0.5)
```

The desktop playback window also opens VASP XDATCAR files (including variable-cell runs that repeat the header) and LAMMPS dumps that include an `element` column. Opening a trajectory only indexes where each frame starts; frames are read from the file as they are shown, so trajectories far larger than memory play smoothly, and `AtomForge--render` streams GIF frames the same way. Other structure formats are read once and kept.

**Colour each frame** recomputes a per-atom analysis for every displayed frame and colours the atoms by it: the Ackland–Jones structure type, the centrosymmetry parameter or the Steinhardt $q_6$ bond order (each described in this chapter). The structure type chooses its own cutoff in periodic cells; the other two use the neighbour cutoff shown. The window reports the time per frame; large structures analysed every frame slow playback, so lower the frame rate or analyse selected frames with the slider. The colour range adapts to each frame; fix it in the atom-property colouring window to compare frames on one scale. **Export GIF** writes the loaded trajectory as a looping animated GIF rendered with the current camera, display settings and playback frame rate, optionally using every $N$th frame; views larger than 960 pixels are averaged down. The same is available without the window:

```
AtomForge --render --input XDATCAR --output relax.gif --fps 10 --every 5
AtomForge --render --input cu.cif --output turntable.gif --frames 36 --pitch 20
```

A trajectory input gives one frame per trajectory frame; a single structure gives a turntable (default 36 frames over a full turn). Each frame has its own 256-colour palette, which is ample

for rendered atoms but bands smooth gradients slightly. Python's `export_animation` remains available. Python's GIF export uses fixed projection bounds across frames and explicitly chosen radii. Conversion preserves species, positions and a full cell, but the current Structure model does not retain per-axis periodicity, velocities or arbitrary per-atom trajectory properties. Keep the original trajectory for dynamics analysis.

## 9.6 Install scientific integrations

The following workflows live in `atomforge.science`; they are Python integrations rather than additional native desktop dialogs. Install the source package with its `science` extra to obtain ASE, pymatgen, Matplotlib and Gemmi. The native electronic library is additionally needed for stockholder integration and reciprocal-surface triangulation. External DFT, Bader and Chargemol executables and licensed calculation inputs are not bundled.

Gaussian input and Quantum ESPRESSO input conversion use ASE. A PWI export is a structural template with element-named UPF placeholders; supply real pseudopotentials and calculation parameters before running it. MOL, MOL2 and SDF conversion uses the native executable. CIFs carrying nontrivial space-group information require the crystallography extra (Gemmi) and expand symmetry-related sites. Validate disorder/partial-occupancy models separately because the Structure model represents explicit atoms.

## 9.7 Bands, DOS, projected data and orbitals

```
from atomforge.science import read_bands, plot_bands, read_dos, plot_dos
bands = read_bands("EIGENVAL")
plot_bands(bands, "bands.svg", fermi_eV=5.2)
dos = read_dos("DOSCAR")
plot_dos(dos, "dos.png")
```

Band arrays are indexed by k point then band, separated by spin channel. Without an explicit reciprocal-path distance array, the plot labels its horizontal axis as k-point index. DOSCAR parsing checks finite data, complete blocks and consistent energy grids. Projected DOS column meanings depend on the producer's spin and LORBIT settings; pass `projected_labels` when these are known rather than inferring orbital labels from an ambiguous column count.

`read_projections("PROCAR")` returns pymatgen's projection object. Select and sum the intended ion/orbital channels to construct nonnegative weights matching the band arrays, then pass these to `plot_bands(...,projections=weights)` for fat bands. Spin-orbit projections require the conventions of the supplied PROCAR. Compare the number and ordering of k points and bands before combining files.

`orbital_density(wavecar,poscar,output,kpoint=0,band=0)` reconstructs a selected orbital and writes PARCHG for the desktop electronic viewer. Indices are zero-based; spin, spinor and FFT scale are explicit options. This is a plane-wave reconstruction through pymatgen and excludes PAW augmentation; it is not an all-electron density or an exact total-energy calculation.

## 9.8 Reciprocal-space surfaces

```
from atomforge.science import read_bandgrid, plot_surface
bands = read_bandgrid("bands.bxsf", energy_unit="eV", reciprocal_unit="1/A")
label = next(iter(bands.bands))
```

```
plot_surface(bands, label, "fermi-surface.png")
bands.surface(label).save("fermi-surface.obj")
```

BXSF BANDGRID data are read in z-fastest order and preserve endpoint samples. The default isosurface level is the file's Fermi energy; out-of-range levels are rejected. Supply the producing program's actual units. The resulting surface spans the reciprocal parallelepiped and is not folded into the first Brillouin zone. Keep it separate from real-space charge grids. Raw BXSF input is intentionally not passed to the desktop real-space density loader.

## 9.9 Charge partitioning

`hirshfeld(density,proatoms)` applies stockholder weights to an electron-density grid using caller-supplied neutral-proatom grids. All geometries and density units must agree. Proatoms must cover every nonzero density sample. The electron populations sum to the integrated density; optional reference electron counts produce net atomic charges. Use compatible valence or all-electron conventions throughout. AtomForge does not invent atomic reference densities from Gaussian functions.

`bader(path,reference=...,potcar=...,executable=...)` runs the actual Henkelman solver through pymatgen. A summed AECCAR0+AECCAR2 reference can define basins while CHGCAR supplies the integrated valence charge. Returned raw populations count electrons; when POTCAR is provided, net charge is reference valence count minus population. Inspect vacuum charge and total electron count as convergence checks. Cube input requires the explicit cube format option.

`ddec(directory,atomic_densities=...)` uses Chargemol DDEC6/CM5. It requires the actual solver, atomic-density database and required VASP files. Set `run=False` to parse existing results. Errors are reported directly; neither Bader nor DDEC falls back to a Voronoi partition. Backend availability and numerical convergence are separate from successful import of the Python wrapper.

## 9.10 Diffraction, relaxation, dynamics and phonons

`powder_diffraction(structure)` uses pymatgen's X-ray or neutron calculator with explicit wavelength and angle limits. The result contains peak angles, intensities and reflection information; `plot_diffraction` produces a broadened pattern. `single_crystal` uses explicitly supplied per-site, per-reflection complex scattering factors. Include any intended occupancy and displacement attenuation in those effective factors. Radiation, isotope and form-factor assumptions affect intensities and should be recorded.

```
from ase.calculators.emt import EMT
from atomforge.science import relax, molecular_dynamics, phonons, mode_frames
result = relax(host, EMT(), fmax=0.02, steps=200)
print(result.converged, result.energy)
motion = molecular_dynamics(host, EMT(), steps=100,
                            timestep_fs=1, temperature_K=300, seed=7)
modes = phonons(host, EMT(), [[0,0,0]], directory="fresh-phonon-cache",
                supercell=(2,2,2), displacement=0.01)
animation = mode_frames(host, modes["modes"][0,0])
```

EMT is an example potential with limited element support, not a general DFT model. A caller-configured ASE calculator can instead connect an external DFT or interatomic-potential engine. Its executable, pseudopotentials, settings and convergence remain the caller's responsibility. BFGS returns a convergence flag; reaching the step limit does not establish equilibrium. Cell relaxation requires a periodic structure and stress-capable calculator.

Dynamics uses NVE velocity-Verlet. Temperature sets the initial velocity distribution, not a thermostat. Check total-energy drift and timestep convergence. Phonons use finite displacements in a fresh cache directory; q points are fractional reciprocal coordinates, energies are in eV, and negative values denote imaginary frequencies. Check supercell and displacement convergence. `mode_frames` animates a supplied Gamma-point mode with an explicit maximum displacement amplitude. A geometric builder or a rendered vibration does not by itself establish dynamical stability.

## 9.11 Condensed-matter tools: desktop, Python and command line

The Analysis menu groups tools by task: **Structure and defects**, **Trajectories and transport**, **Electronic structure**, **Reciprocal space**, **Thermal and mechanical**, and **Simulation**. Existing tools are alongside related analyses: CNA, RDF, ADF and SRO appear with structural analyses; trajectory playback appears with transport analyses; electronic post-processing, Wannier and LOBSTER appear with electronic analyses. AtomForge computes the tools below natively; the twenty shared with `atomforge.science` are joined by native structure relaxation. The desktop dialog and the `AtomForge--science` command share one C++ implementation and need no Python installation. Parameter names, units and conventions match `atomforge.science`, which provides the same analyses to Python scripts through NumPy, SciPy and ASE. Simulation tools use the built-in EMT and Lennard-Jones potentials or an EAM/Finnis–Sinclair potential file. Symmetry paths use the spglib library included with AtomForge.

Choose a tool from its task section. Each family of tools opens its own window, arranged for that kind of work:

- **Atom analysis** (structure type, symmetry and Wyckoff positions, dislocation lines, clusters, voids, centrosymmetry, bond order, Wigner–Seitz defects, local strain): a narrow panel to keep beside the viewport. **Analyse the active structure** is on by default and supplies the positions, cell and periodicity, so only the settings remain; turn it off to choose a file. **Analyse** runs the tool and colours the atoms by the first per-atom result straight away (turn off **Colour automatically after each analysis** to keep the current colouring); the radio buttons switch between properties.
- **Trajectory analysis** (MSD, diffusion, VACF, vibrational spectrum, structure factor, trajectory-averaged g(r), coordination and bond angles): the trajectory source across the top, naming the frames loaded in Trajectory playback, the settings in one row and a wide plot below.
- **Property calculator** (band gap, effective mass, work function, equation of state, elastic tensor, harmonic thermodynamics, and the potential-driven elastic constants, equation-of-state scan, point-defect formation and planar-defect energies): the inputs on the left; the computed quantities on the right as large values, with tensors such as the stiffness matrix shown as tables.
- **Plot workspace** (VASP bands and DOS, phonon DOS, powder XRD, electron diffraction, reciprocal path): a settings sidebar beside a large plot; several plots appear as tabs above it.
- **Simulation run** (relaxation, NVT, NPT, NEB, phonons): numbered steps for the system, the interatomic potential and the protocol, with rarely changed settings under **Advanced options**; the run bar shows progress, and the run summary appears beside its plots.
- **Input generator** (band-structure inputs, LAMMPS export): a compact form and a browser of the generated files with a preview, **Save this file**, **Save all files** and **Copy to clipboard**.

The windows show a one-line description of each tool; its method, units, assumptions and limits are described in the tool lists of this section and in §9.11.4–9.11.8. Fill the grouped form. Optional inputs expand when their checkbox is enabled; Boolean settings use one checkbox. Periodic

boundaries have separate a, b and c checkboxes. Fit and vacuum intervals use Start/End fields; the spectrum window uses a choice list. Select **Load from file** and **Browse** for numerical CSV, NPY, JSON or structure/trajectory files, or select **Enter values** for small JSON arrays. Expand **File options** to select a zero-based column or a dotted JSON field such as `result.msd_A2`. Numeric CSV files have no header; matrix inputs use the whole table. Only data are read; array strings are not evaluated as Python code. The forms scroll on smaller windows while the run button stays visible.

Structure inputs in simulation tools offer **Use active structure**. The application copies the current structure into the calculation request without modifying the active tab. Position inputs of the local analyses offer the same button; for a periodic structure it also fills the matching cell input and enables all three periodic axes. Cell inputs offer **Use active cell**. When a trajectory is open in Trajectory playback, the positions or velocities inputs of MSD, velocity autocorrelation, the vibrational spectrum and the structure factor offer **Use loaded trajectory**; the file is read again by the analysis, so its velocities and `time_fs` metadata are used. Structure files with multiple frames supply trajectory positions. Atom ordering must remain consistent. Velocity analysis requires explicit velocity data in Angstrom/fs or an extended XYZ trajectory with ASE-style `momenta` or Angstrom/fs `velocities` columns; positions alone do not supply velocities.

AtomForge reads data from JSON, NPY, numerical CSV/TXT/DAT tables, VASP EIGENVAL, XYZ/extended XYZ, POSCAR/CONTCAR, XDATCAR and LAMMPS dump files. Other structure formats supported by the file browser provide a single structure. Compressed EIGENVAL files (gzip, bzip2 and xz) are supported. XDATCAR import accepts repeated cell headers. For trajectory files, velocity, mass and cell inputs select the corresponding property automatically. Numeric column selection works for CSV, NPY and JSON tables; invalid columns are rejected. When a trajectory contains `time_fs` metadata, its intervals must be uniform and match the requested sampling interval. A simulation's final partial interval must be trimmed or resampled before time-correlation analysis. Simulation structures with a cell are periodic along all three axes; structures without a cell, including nonperiodic bounding boxes, remain nonperiodic.

Press the window's run button (**Analyse**, **Analyse trajectory**, **Calculate**, **Compute**, **Run simulation** or **Generate files**). The calculation runs in the background, so you can keep working with the structure. A progress bar appears when the fraction completed can be estimated. **Cancel** stops the calculation at its next checkpoint. The result area lists scalar results with their units and shows small tensors as tables; **Full report** adds summary ranges for large arrays; **Save results** uses the shared file browser to export full JSON arrays, validity masks, structural frames and the original request parameters. For NEB, NVT, NPT and reciprocal paths, **Save structures** exports extended XYZ and **Open final structure in a new tab** brings the last frame or primitive structure into the viewport. Saved trajectories can be opened in Trajectory playback. Invalid local environments are represented by null in JSON, with a separate validity mask. Large arrays are not fully printed in the summary. The result area plots the returned curves: MSD with its Cartesian components, the diffusion fit, VACF, vibrational spectra, the structure factor, equation-of-state data with the fitted curve, phonon DOS, harmonic thermodynamics, the relaxation energy and force history (logarithmic), the NEB energy along the cumulative image distance, and dynamics temperature, energy, volume and pressure. Per-atom results (centrosymmetry, $D^2_{\mathrm{min}}$, von Mises shear and volumetric strain, $q_l$, coordination and Wigner–Seitz distances) appear as histograms. Hover a plot for values; **Export plot data** writes the plotted series as CSV.

**Keep for comparison** stores the current result of a tool. Change inputs and run again: the plots then overlay the kept runs on the same axes (series named *Run 1:*, *Run 2:*, …, *Current:*; turn this off with the overlay checkbox), **Export plot data** writes the overlaid series, and **Compare with kept runs** lists the scalar results of every run side by side. Kept runs are cleared when another tool is chosen; save results that must be kept permanently.

Per-atom results can also colour the structure. Atom analyses colour the view automatically; for other results, or to switch property, choose it and press **Colour atoms in the view**. Colouring is

available when the result has as many atoms as the active structure. The view switches to **View > View Structure By > Atom Property** with a viridis colour bar. The **Atom property colouring** window sets an automatic or manual range and can hide atoms outside the range or without a valid value; for example, hide atoms with centrosymmetry below 0.5 Å$^2$ to show only defect cores. Invalid environments are grey. Deleting atoms keeps the property aligned; the view returns to element colours when the property no longer matches the atoms. Temporary calculation files are retained under the system temporary directory's `AtomForge-science` folder; save results needed for a permanent record.

### 9.11.1 Trajectory and local-structure analyses (priorities 1–9)

1. **Mean-square displacement (MSD).** Load positions shaped as frames by atoms by three coordinates, and set the sampling interval in fs. Prefer unwrapped coordinates. If wrapped is enabled, supply a fixed cell; the inter-frame motion must be small enough to resolve the shortest periodic displacement. Run and inspect the lag-time MSD and Cartesian displacement tensor. The MSD is averaged over all time origins, $\mathrm{MSD}(\tau) = \langle |\mathbf{r}_i(t+\tau) - \mathbf{r}_i(t)|^2 \rangle_{i,t}$, and the displacement tensor is $\langle \Delta r_a \Delta r_b \rangle$, a second moment rather than a covariance about the mean displacement. Atom order must be the same in every frame. Optional geometric-centroid drift removal is not mass-weighted centre-of-mass removal. A changing cell requires a physically chosen reference-frame transformation before this analysis.

2. **Diffusion coefficient.** Supply the MSD lag times and values, then explicitly select a diffusive fit interval. Set dimensions to the number of directions summed in that MSD. The result uses $D = \mathrm{slope}/(2d)$ and reports both Angstrom$^2$/fs and m$^2$/s, intercept and $R^2$. Check several fit intervals and independent trajectories. The ordinary least-squares standard error is a fit diagnostic, not a confidence interval accounting for correlated time origins.

3. **Velocity autocorrelation.** Supply velocities in Angstrom/fs and the frame interval. The result averages $\mathbf{v}(t) \cdot \mathbf{v}(t+\tau)$ over atoms and time origins. The average is unbiased: each lag is divided by its own number of time origins. Without normalization the unit is Å$^2$/fs$^2$; optional normalization sets the zero-lag value to one, and zero-velocity data cannot be normalized. Drift removal subtracts each frame's geometric mean velocity, not the mass-weighted centre-of-mass velocity. Truncate the lag range before poorly sampled tails dominate interpretation.

4. **Trajectory vibrational spectrum.** Load velocities, choose a Hann or rectangular window, and optionally supply atomic masses. The one-sided velocity periodogram, the Fourier partner of the velocity autocorrelation, is normalized to unit integrated spectral weight per THz. Resolution and the Nyquist frequency follow from the trajectory duration and sampling interval; zero padding adds no information. Inspect its reported resolution and Nyquist frequency. This classical velocity spectrum is not an IR, Raman or quantum neutron-scattering intensity.

5. **Local strain and $D^2_{\min}$.** Load corresponding reference and current coordinates, choose a reference neighbor cutoff, and supply cells and periodic axes. Neighbours are selected in the reference structure, and atoms must correspond by order. The tool fits the local deformation gradient $\mathbf{F}$ by least squares, minimising $\sum_j |\mathbf{d}_j - \mathbf{F}\mathbf{d}_j^0|^2$ over the neighbour vectors, and reports the Green–Lagrange strain $\frac{1}{2}(\mathbf{F}^\mathsf{T}\mathbf{F} - \mathbf{I})$ and $D^2_{\min}$, the *sum* (not mean) of the squared non-affine residuals in Å$^2$. Known box deformation is retained. Three-dimensional rank-deficient neighborhoods are marked invalid; a planar neighborhood does not define a full 3D strain tensor.

6. **Centrosymmetry parameter.** Choose an even neighbor count, commonly twelve for FCC or eight for BCC, and a cutoff containing those neighbors. The conventional Kelchner construction sums the smallest $N/2$ squared pair-vector sums among the nearest $N$ neighbors. Among the nearest $N$ neighbours within the cutoff (periodic images included), all distinct pairs are formed and the $N/2$ smallest $|\mathbf{r}_i + \mathbf{r}_j|^2$ are summed; this is the conventional smallest-pair algorithm, not

an optimal disjoint matching. Perfect centrosymmetric environments approach zero. Missing neighbors are invalid, not zero; a nonzero value alone does not identify a unique defect type.

7. **Steinhardt bond order.** Select angular degrees, typically four and six, a cutoff, cell and periodic axes. The tool returns the rotationally invariant local $q_l = \sqrt{\frac{4\pi}{2l+1}\sum_{m=-l}^{l}|\bar{Y}_{lm}|^2}$, where $\bar{Y}_{lm}$ is the equal-weight average of $Y_{lm}(\hat{\mathbf{r}}_{ij})$ over the bonds of atom $i$; atoms without neighbours are invalid. For an ideal FCC first-neighbor shell, $q_4 \simeq 0.190941$ and $q_6 \simeq 0.574524$. These are local $q_l$, not neighbor-averaged $\bar{q}_l$ or third-order $w_l$ invariants.

8. **Wigner–Seitz defects.** Load reference sites and current atom positions. Supply the reference cell and, if needed, a deformed current cell to remove homogeneous box strain. Each atom is assigned to its nearest reference site (with the current cell mapped back to the reference cell); inspect site occupancies, vacant sites and sites with more than one occupant. An occupancy $n > 1$ contributes $n - 1$ excess interstitial atoms. Equidistant assignments are flagged; the tool does not uniquely label which atom in a multiply occupied site is the interstitial.

9. **Static structure factor.** Supply one structure or a trajectory and Cartesian wave vectors in radians/Angstrom, including the reciprocal $2\pi$ factor. Optional scattering amplitudes weight atoms. The normalization is $S(\mathbf{q}) = \langle|\sum_j b_j e^{i\mathbf{q}\cdot\mathbf{r}_j}|^2\rangle / \sum_j |b_j|^2$; equal weights give $S(0) = N$. Use commensurate reciprocal points for wrapped periodic coordinates. No powder averaging or forward-scattering subtraction is implied.

### 9.11.2 Electronic and thermomechanical analyses (priorities 10–16)

10. **Band gap and band edges.** Load an EIGENVAL file or an energy array shaped as k points by bands, optionally preceded by a spin dimension. Supply the Fermi energy separately in the same energy reference. Inspect the fundamental sampled gap, smallest same-spin direct gap and extremum indices. A band crossing $E_F$ on the supplied mesh, or a sampled state within the tolerance of $E_F$, marks a metal. Smearing occupations are not interpreted. A sparse k mesh can miss pockets or extrema, and the direct gap is not an optical excitation energy.

11. **Effective-mass tensor.** Supply at least ten independent points in a small three-dimensional Cartesian k neighborhood, one consistently tracked band's energies, and the expansion centre. The band is fitted by the quadratic model $E = E_0 + \mathbf{g}\cdot\delta\mathbf{k} + \frac{1}{2}\delta\mathbf{k}\cdot\mathbf{H}\cdot\delta\mathbf{k}$ with $\mathbf{k}$ a Cartesian wave vector in rad/Å (not a fractional position or path length); the mass tensor is $\hbar^2\mathbf{H}^{-1}$. Inspect signed principal masses in electron-mass units, principal axes, the residual band gradient $\mathbf{g}$ (which should be small at an extremum) and fit error. A one-dimensional plotted band path is insufficient for the full tensor. Negative electron curvature corresponds to positive hole mass after sign reversal.

12. **Work function.** First obtain a planar electron potential-energy profile, for example using the electronic workspace's planar averaging. Supply distances, potential values, Fermi energy and a vacuum interval. The result is $\Phi = \langle V\rangle_{\mathrm{vac}} - E_F$, the mean potential over the selected interval minus the Fermi energy; the vacuum region is not detected automatically. For VASP use the electrostatic LOCPOT channel with a consistent reference. Inspect the slope/flatness flag and vacuum spread. Evaluate opposite slab faces separately; dipole corrections are not introduced by this analysis.

13. **Equation of state.** Supply at least five distinct positive volumes and consistent total energies bracketing a minimum. A third-order Birch–Murnaghan fit reports equilibrium volume, energy, bulk modulus in GPa, pressure derivative and residuals. The reported parameter covariance is a numerical fit diagnostic, not an estimate of model or convergence error; compare residuals and vary the fitted volume range. Energies and volumes must refer to the same cell or atom-count normalization.

14. **Elastic tensor and moduli.** Supply independently varied small strains and the corresponding stresses in GPa. Engineering-Voigt order is xx, yy, zz, yz, xz, xy; shear strains are twice tensor shear, while shear stresses are not doubled. Stress is positive in tension. The least-squares fit $\boldsymbol{\sigma} = \boldsymbol{\sigma}_0 + \mathbf{C}\boldsymbol{\epsilon}$ includes the prestress $\boldsymbol{\sigma}_0$ and needs a full-rank strain design (at least seven samples); it symmetrizes the stiffness, and reports mismatch, residuals, zero-prestress stability and Voigt–Reuss–Hill moduli where stable. Check strain-amplitude convergence; the stability test does not substitute for finite-pressure stability criteria.

15. **Phonon band structure and DOS.** Select a relaxed periodic structure and a potential. AtomForge displaces every atom of the unit cell by ±**Finite displacement** in a supercell at least **Minimum supercell width** wide (or explicit **Supercell repetitions**) and differences the forces into harmonic force constants. The acoustic sum rule is enforced, and the dynamical matrix shares force constants between equidistant periodic images, as phonopy does, so q points between supercell vectors are interpolated. With **Use the symmetry band path**, phonons are computed in the standardized primitive cell along its HPKOT path and plotted with labelled special points; otherwise supply explicit fractional q points. A Gamma-centred mesh gives the DOS (normalised to three modes per atom) and harmonic thermodynamics, which are omitted when the mesh has imaginary modes. Frequencies are in THz, negative for imaginary modes. Converge the supercell width and check that acoustic branches vanish at Γ; imaginary frequencies indicate an unrelaxed structure or an instability with the chosen potential.

16. **Phonon density of states.** Supply mode energies in eV shaped as q points by branches, a plotting grid, Gaussian width and optional q-point weights. The analytic total weight is the number of branches per cell. Inspect the enclosed plotted weight to detect truncated tails. Negative imaginary-mode energies remain on the negative axis; broadening does not make them stable.

17. **Harmonic thermodynamics.** Supply a converged mode mesh, normalized or unnormalized nonnegative q weights and a temperature list. Results include zero-point energy, Helmholtz free energy, internal energy, entropy and constant-volume heat capacity, in eV per cell and eV/(cell K), not per mole. Modes within ± the zero tolerance (such as the acoustic modes at Γ) are omitted. At zero temperature, free/internal energies equal the zero-point energy. Imaginary modes beyond tolerance are rejected; omitted near-zero mode weight is reported. Electronic, configurational, thermal-expansion and anharmonic contributions are absent.

### 9.11.3 Simulation and reciprocal-space workflows (priorities 17–20)

17. **Structure relaxation.** Select a structure and a potential, a force tolerance and a step limit. FIRE minimises the energy with respect to atomic positions. With **Relax the cell shape and volume**, the full cell also relaxes at the target pressure (positive in compression) by minimising the enthalpy $E + PV$; cell forces are scaled by the atom count, as in ASE's `UnitCellFilter`. The result reports convergence, the energy change, the residual stress and pressure, the final cell and the relaxed structure, which **Open final structure in a new tab** loads. Use relaxed endpoints for NEB and relaxed cells before elastic or equation-of-state sampling. A converged relaxation finds a local minimum, not necessarily the ground state.

18. **NEB migration path.** Select initial and final structures with the same ordered species and fixed cell. Enable **Relax both endpoints first** to minimise them at fixed cell before the band is built; the result reports whether each endpoint converged. Set the number of images to 0 to choose it automatically from the total endpoint displacement (one image per **Automatic image spacing**, between 5 and 31 images). To continue an unconverged run, save its structures and supply the file as **Restart from saved images**; the saved images (including endpoints) replace the interpolation, so initial and final structures are then optional. Choose an interatomic potential, image count, spring constant, force tolerance and iteration limit.

The tool performs improved-tangent NEB with optional climbing image and shortest-image interpolation, optimised with FIRE. Inspect convergence, endpoint forces and both sampled barriers. Each image evaluates its own instance of the potential. Endpoint relaxation remains the user's responsibility.

19. **NVT dynamics.** Select a structure, an interatomic potential, temperature, timestep and thermostat relaxation time. Langevin dynamics (BAOAB splitting) returns frames, physical energies, temperatures and velocities. **NVE production steps** continue after the thermostatted run with energy-conserving velocity Verlet (at the final cell for NPT); the plots mark where production starts and the result reports the total-energy drift per atom per picosecond, a direct check of the timestep. Results include block-averaged statistics for temperature and total energy (and volume and pressure for NPT): mean, standard deviation and the standard error from five contiguous blocks, which accounts for correlation between samples. Statistics use the production samples when present, otherwise the samples after **Exclude samples before**. A standard error that changes strongly with the block count, or a drifting block mean, means the run is too short. The seed controls reproducibility of initialization and thermostat noise. The centre of mass is not constrained; remove unwanted drift during analysis. Equilibrate before collecting statistics; thermostatting does not establish equilibration or conserve the physical energy.

20. **NPT dynamics.** Select a fully periodic structure, an interatomic potential, temperature, pressure in GPa and thermostat/barostat times. EMT, Lennard-Jones and EAM all supply the required stress. The isotropic Martyna–Tobias–Klein integrator with Nosé–Hoover chains changes cell volume while keeping shape fixed. Positive target pressure is compression. Inspect temperature, pressure and volume histories and convergence with timestep; instantaneous fluctuations should not be mistaken for failure to match the target.

21. **Symmetry reciprocal-space path.** Select a periodic structure, a reciprocal sampling spacing and symmetry tolerance. AtomForge finds the space group with spglib and builds the HPKOT path from SeeK-path's published tables and conventions. It returns the standardized primitive structure, reciprocal basis, labels and explicit disconnected segments. Use its returned primitive cell when interpreting fractional k points; input supercell fractions are generally different. The time-reversal switch controls path completion, not magnetic-space-group analysis.

### 9.11.4 Defect identification, clusters and diffraction

These native tools have no `atomforge.science` counterpart; run them from the desktop or with `AtomForge--science`.

- **Structure type (Ackland–Jones).** Classifies every atom as fcc, hcp, bcc, icosahedral or other from the bond angles of its neighbours (Ackland and Jones, Phys. Rev. B 73, 054104 (2006)). The six nearest neighbours define a local length scale $r_0$; the cosines of the angles between bonds within $\sqrt{1.45}\,r_0$ are binned and the histogram is compared with those of the ideal lattices. Because $r_0$ is local, the classification tolerates thermal vibration (an fcc crystal with 0.07 Å random displacements remains over 95% fcc). With a cell the cutoff is automatic (three Wigner–Seitz radii, beyond the second bcc shell); open structures need an explicit cutoff. Surface, grain-boundary and core atoms are *other*; stacking faults in fcc appear as hcp layers. Colour the atoms by type, or use the type as a mask for cluster analysis.

- **Dislocation lines and Burgers vectors.** Compares a deformed structure with its perfect reference (same atoms, same order), in the spirit of the dislocation extraction algorithm. Atoms whose structure type changed form the cores; slip by a full lattice vector leaves atoms crystalline, and free surfaces are excluded because they are already non-crystalline in the reference. Cores are clustered into lines. The line direction is the lattice period of a line that wraps the cell, otherwise the principal axis of its core. The Burgers vector is the closure failure of a right-handed circuit

around the line through the reference atoms, with every displacement step reduced modulo the nearest-neighbour lattice vectors, so it is an exact lattice vector (for example $a/2\langle 110\rangle$ with length 2.553 Å in Cu). The result lists each line's atoms, direction, Burgers vector, character angle (0° screw, 90° edge), centroid, length and a polyline; core clusters whose circuit closes are counted as non-dislocation defects. Lines closer than about twice their core radius, or strongly curved loops, need an explicit smaller circuit radius. Partial dislocations bounding a stacking fault are reported as one dissociated core.

- **Atom clusters.** Groups atoms closer than a bond cutoff into connected clusters through periodic boundaries, optionally restricted by a per-atom mask (nonzero selects; for example non-crystalline atoms, or one species). Each cluster reports its size, unwrapped centroid, radius of gyration and whether it percolates through the periodic cell.
- **Voids and empty space.** Grid points farther than atomic radius plus probe radius from every atom are empty; connected empty points form voids, wrapped through periodic boundaries, while regions reaching an open boundary are treated as the exterior. The default atomic radius is half the nearest-neighbour distance. With the default 1 Å probe, a perfect fcc metal has no voids (its octahedral holes have a free radius of only $0.146a$), a vacancy is one void lined by its 12 neighbours, and two adjacent vacancies remain two probe-accessible regions because the channel between them is as narrow as an octahedral hole; a smaller probe connects them. Each void reports the volume accessible to probe centres, an equivalent radius and a pore-radius estimate (equivalent radius plus atomic and probe radii).
- **Powder X-ray diffraction.** Kinematic structure factors from four-Gaussian atomic scattering factors (the coefficients used by pymatgen), an optional isotropic Debye–Waller factor $\exp(-Bs^2)$ and the Lorentz-polarization factor $(1+\cos^2 2\theta)/(\sin^2\theta\cos\theta)$. Every reflection inside the limiting sphere is summed, so multiplicities and systematic absences follow automatically; peaks are scaled to 100, labelled with a representative $(hkl)$ and broadened with a Gaussian of the chosen FWHM. The default wavelength is Cu K$\alpha$ (1.54184 Å); use 0.71073 Å for Mo K$\alpha$. Anomalous dispersion, preferred orientation, absorption and instrument profiles are not modelled.
- **Electron diffraction (zone axis).** Lists the reflections $\mathbf{g} = h\mathbf{b}_1 + k\mathbf{b}_2 + l\mathbf{b}_3$ of the zero-order Laue zone of $[uvw]$ ($hu + kv + lw = 0$) within $|g|_{\max}$, with intensities $|F|^2$ scaled to 100, with detector coordinates in Å$^{-1}$, scattering angles in mrad for the relativistic wavelength of the chosen voltage (0.02508 Å at 200 kV), and intensities from Mott–Bethe electron scattering factors. The pattern is kinematic: dynamical effects such as double diffraction into forbidden reflections, excitation errors and higher-order Laue zones are not included.

### 9.11.5 Electronic-structure output and DFT inputs

- **VASP band structure, DOS and fat bands.** Select the `EIGENVAL` file and, optionally, a line-mode `KPOINTS` file, the `POSCAR` or `CONTCAR`, `DOSCAR` and `PROCAR`. The KPOINTS labels mark the high-symmetry points; where consecutive segments do not join, the break is drawn at one distance with a combined label such as `M|R`. With the structure, distances are Cartesian in Å$^{-1}$ with $2\pi$ included (the same convention as the reciprocal-path tool) and element names are available; otherwise distances are in fractional reciprocal coordinates. The Fermi energy comes from the input field, else from DOSCAR, and all energies are shown relative to it, with the band gap computed from the sampled bands. DOSCAR adds the total DOS (both spins) and, when it contains site projections, the DOS summed per element. PROCAR adds fat-band weights: the fraction of each state's projection on an element and orbital (`s`, `p`, `d`, `f`, an $lm$ name such as `dxy`, or `all`); the fat-band plot highlights states with more than 30% of that character. Compressed files are read directly. Projections are only as complete as the VASP projection spheres; weights compare character between states rather than giving absolute charges.

- **Band-structure inputs (VASP, Quantum ESPRESSO).** Generates inputs on the HPKOT path of the structure: a line-mode `KPOINTS` file and the standardized primitive `POSCAR` for VASP, and `CELL_PARAMETERS`, `ATOMIC_SPECIES` (pseudopotential names are placeholders), `ATOMIC_POSITIONS` and `K_POINTScrystal_b` cards for Quantum ESPRESSO, where each point's weight is the number of steps to the next point (1 marks a jump). Save them with **Save generated files**, or write them with `AtomForge--sciencedft-inputs\protect.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font{}--filesDIR`. Use the generated primitive cell for the self-consistent calculation too, so the path coordinates refer to the same reciprocal basis.

### 9.11.6 Symmetry, mechanical properties and defect energies

These tools take a structure and, except symmetry, an interatomic potential, and run the calculations themselves.

- **Symmetry and Wyckoff positions.** Finds the space group of a periodic structure with spglib: number, Hermann–Mauguin and Hall symbols, point group, crystal system and all operations, and for each atom its Wyckoff letter, site symmetry and symmetry-equivalent orbit (one row per orbit lists element, Wyckoff position, multiplicity and site symmetry). It produces three cells: the idealized conventional and primitive standard cells, and the input cell with symmetrized positions, in which each atom is the average of its images under the operations (the largest shift is reported). **Cell to open** chooses which one **Open final structure in a new tab** loads. The tolerance symprec (default $10^{-3}$ Å) is how far atoms may deviate from exact symmetry: a relaxed or noisy structure may need a larger value to show its intended space group, and the symmetrized cell then has that symmetry exactly. Colour atoms by orbit or Wyckoff position to see equivalent sites.

- **Elastic constants (strain and relax).** Applies each of the six engineering-Voigt strains (xx, yy, zz, yz, xz, xy; shears are $2\epsilon_{ij}$) at ± every amplitude up to the largest strain (default 1%, two amplitudes per sign), relaxes the ions at each fixed strained cell and fits $\boldsymbol{\sigma} = \boldsymbol{\sigma}_0 + \mathbf{C}\boldsymbol{\epsilon}$ with the elastic-tensor tool: the stiffness matrix, Voigt–Reuss–Hill moduli and the stability test. Internal relaxation is essential for lattices with more than one atom per primitive cell: for Stillinger–Weber silicon it lowers $C_{44}$ from about 117 to 56.4 GPa, while $C_{11} = 151.4$ and $C_{12} = 76.4$ GPa. Relax the structure with its cell first; a remaining prestress is reported, not removed. Converge the largest strain: too small amplifies force-tolerance noise, too large adds anharmonicity.

- **Equation of state (volume scan).** Scales the cell isotropically from $(1 - \Delta)$ to $(1 + \Delta)$ of its volume ($\Delta$ = volume range, default 0.1) at the chosen number of volumes, relaxes the ions at each and fits the third-order Birch–Murnaghan equation of state: equilibrium volume and energy, bulk modulus and its pressure derivative, with the pressure from the stress at each volume. The bulk modulus agrees with $(C_{11} + 2C_{12})/3$ from the elastic constants of a cubic crystal. Keep the range near the minimum; fit residuals show when it is too wide.

- **Point-defect formation energy.** For a vacancy, interstitial or substitution in a supercell, $E_f = E_{\text{defect}} - E_{\text{bulk}} - \sum_i \Delta n_i \mu_i$, where both supercells are relaxed (ions, and optionally the shape and volume of the defect cell, which also gives the relaxation volume) and $\Delta n_i$ is the change in the number of atoms of element $i$. With a one-element bulk, $\mu$ is its energy per atom; otherwise give **Chemical potentials** as `\{"Ni":-4.44\}` for every element added or removed (they set the reservoir the atoms come from). Use the same supercell for both cells and converge its size, since periodic images of the defect interact; charged defects and their corrections are not treated. For a pair potential an unrelaxed vacancy costs exactly the cohesive energy per atom, a useful check.

- **Surface, grain-boundary and stacking-fault energy.** $\gamma = (E_{\text{cell}} - \frac{N}{N_{\text{bulk}}} E_{\text{bulk}})/(nA)$ in J/m$^2$ (and eV/Å$^2$): a slab with vacuum has $n = 2$ free surfaces, a periodic bicrystal from the grain-boundary builder has two boundaries, and a stacking-fault cell one fault. $A$ is spanned by the two cell vectors other than the normal axis (default c). The cell must contain a whole multiple of the bulk composition; both are relaxed with the cell fixed, so the in-plane lattice stays that of the bulk. Converge slab and vacuum thickness (the energy should not change with either), and for grain boundaries also sample the relative translations of the two grains.

### 9.11.7 Trajectory-averaged structure

**Trajectory-averaged g(r), coordination and bond angles** reads a trajectory file frame by frame (with the same streaming as playback, so long runs need little memory) and averages over the frames from **First frame**, using every $N$th:

- the partial radial distribution functions $g_{ab}(r)$, the density of $b$ atoms at distance $r$ from an $a$ atom relative to the average density of $b$ ($N_b - 1$ for $b = a$), and the total $g(r)$ weighting all pairs; an uncorrelated (ideal-gas) configuration gives $g(r) = 1$;
- coordination numbers within the neighbour cutoff, per element, per element pair ("Cl around Na") and as a distribution;
- the bond-angle distribution of all pairs of neighbours of each atom, normalized to unit area per degree (for perfect fcc, 24, 12, 24 and 6 of the 66 first-shell angles at 60°, 90°, 120° and 180°).

Frames must be periodic. Choose the cutoff at the first minimum of $g(r)$, skip equilibration with the first frame and thin correlated frames with every $N$th; **Use loaded trajectory** takes the file open in Trajectory playback.

### 9.11.8 Batch runs, LAMMPS export and project persistence

- **Batches and parameter sweeps.** `AtomForge--science-batchBATCH.json--outputRESULTS.json[--csvTABLE.csv]` runs one tool many times. The batch names a `tool`, a `base` request, an optional `sweep` (every combination of the listed values is run), optional `files` glob patterns (parameter name to pattern) whose matching files are substituted into that parameter, and `collect`: dotted result paths such as `heat_capacity_eV_per_K.0` that become CSV columns. By default a failing case records its error in the table and the batch continues (set `continue_on_error` to `false` to stop at the first failure); the command then exits with status 2. Patterns in `files` are relative to the batch file, and the run count is limited to 100000.

```
{"tool": "harmonic-thermodynamics",
 "base": {"energies_eV": {"file": "phonons.csv"}},
 "sweep": {"temperatures_K": [[100], [300], [600]]},
 "collect": ["heat_capacity_eV_per_K.0"]}
```

- **Export to LAMMPS.** Writes `data.lammps` (metal units, atom style atomic) and `in.lammps` for minimization (optionally with `box/relax`), NVT, NPT or NEB with the same potential, temperature, pressure and damping as the native tools. Periodic cells are converted to LAMMPS's restricted triclinic form (a along $x$, b in the $xy$ plane) with tilts reduced by lattice translations, so fractional coordinates are unchanged; open structures get a padded box with fixed boundaries. EAM files map to `eam/alloy`, `eam/fs` or `eam`, and Lennard-Jones to `lj/cut` with shifted energy. LAMMPS has no EMT pair style, so EMT leaves a commented placeholder to replace. NEB writes `final.neb` and must be run with one partition per image. Dumps include element names, so Trajectory playback reads them directly.

- **Projects.** Saving a project keeps the scientific tool inputs and last result, as described in the project section of this chapter.

### 9.11.9 Native tools from Python

`atomforge.native` calls every tool of the catalog in the same compiled engine as the desktop, without ASE, pymatgen or other packages:

```
import atomforge as af
from atomforge import native
s = af.load("POSCAR")
sym = native.symmetry(structure=s, output_cell="primitive")
print(sym["international_symbol"], sym["wyckoff_letters"])
primitive = sym.frames[-1]                  # an atomforge.Structure
types = native.structure_type(structure=s)  # position-based tools accept structure=
md = native.nvt(structure=s, calculator={"potential": "Tersoff"}, steps=1000)
native.run("lammps-export", structure=s, task="nvt").write_files("lammps")
```

Keyword arguments are the request keys printed by `native.catalog()` (or `AtomForge--science--catalog`); tool ids with hyphens are also functions with underscores. Structures are `atomforge.Structure` objects, a list of structures is a trajectory for position inputs, NumPy arrays are accepted, and files are `\{"file":path\}` relative to `base`. A result has `result` (the JSON result as a dictionary), `frames` (structures), `velocities`, `times_fs`, `files` and `write_files(directory)`; invalid requests raise `ValueError` with the engine's message. The engine is the shared library `atomforge_science_native` installed with AtomForge; set `ATOMFORGE_SCIENCE_LIBRARY` to its path if it is not found next to the package or the executable.

### 9.11.10 Repeatable commands and Python examples

The AtomForge executable runs every tool without Python. The twenty shared functions are also exported from `atomforge.science`. The catalog command prints their parameter names, defaults and physical assumptions. A request is a JSON object with those parameter names; data references are relative to the request file. Example `gap-request.json`:

```
{"energies_eV": {"file": "EIGENVAL"}, "fermi_eV": 5.4}
```

```
AtomForge --science --catalog
AtomForge --science band-gap --input gap-request.json --output gap.json --report gap.
    txt
python -m atomforge.science band-gap --input gap-request.json --output gap.json
```

`--structuresFRAMES.extxyz` additionally saves NEB images, dynamics frames with their velocities, or a primitive cell; `--filesDIR` writes generated input files such as `KPOINTS`. Existing outputs require an explicit `--overwrite`. A failed calculation preserves the previous result. Result JSON stores the tool identifier, parameters and full result. A data reference can select a column or a nested result field:

```
{"file":"profile.csv", "column":1}
{"file":"msd.json", "field":"result.msd_A2"}
```

Numerical CSV has no header; NPY loading forbids pickled Python objects.

```
from atomforge.science import mean_square_displacement, diffusion_coefficient
```

```
from atomforge.science.workflows import run_tool
msd = mean_square_displacement(unwrapped_positions, timestep_fs=2,
                               max_lag=200)
fit = diffusion_coefficient(msd["lag_fs"], msd["msd_A2"],
                            fit_range_fs=(100, 300), dimensions=3)
print(fit["D_m2_per_s"], fit["r_squared"])
gap = run_tool("band-gap", {"energies_eV": {"file": "EIGENVAL"},
                            "fermi_eV": 5.4})
```

Desktop and `AtomForge--science` requests select a built-in potential:

```
{"potential": "EMT"}
{"potential": "LennardJones", "epsilon": 0.0104, "sigma": 3.4, "cutoff": 8.5}
```

EMT uses the effective-medium parameters of Jacobsen, Stoltze and Nørskov as distributed with ASE. It covers Al, Cu, Ag, Au, Ni, Pd and Pt; its H, C, N and O parameters are illustrative only. Lennard-Jones takes $\epsilon$ in eV and $\sigma$ and the cutoff in Angstrom; its energy is shifted to zero at the cutoff.

**Covalent and ionic potentials.** **Tersoff** (bond order; LAMMPS `pair_styletersoff`) and **Stillinger–Weber** (two- and three-body; `pair_stylesw`) describe covalent solids. Without a file both use built-in silicon parameters (Tersoff, Phys. Rev. B 38, 9902 (1988): $-4.630\,\mathrm{eV/atom}$ at $a = 5.432\,\text{Å}$; Stillinger and Weber, Phys. Rev. B 31, 5262 (1985): $-4.337\,\mathrm{eV/atom}$ at $a = 5.431\,\text{Å}$); browse for a LAMMPS `.tersoff` or `.sw` file for other elements and mixtures (for example SiC). Energies follow the LAMMPS definitions exactly, so published parameter files are used unchanged; every element pair and triplet present must be listed. **Buckingham + Coulomb** combines pair terms $Ae^{-r/\rho} - C/r^6$ with fixed ionic charges for oxides and other ionic solids:

```
{"potential": "Buckingham", "charges": {"Mg": 2, "O": -2}, "cutoff": 10,
 "pairs": [{"elements": ["Mg", "O"], "A": 821.6, "rho": 0.3242, "C": 0},
           {"elements": ["O", "O"], "A": 22764, "rho": 0.149, "C": 27.88}]}
```

Coulomb energies, forces and stress are Ewald-summed in periodic cells (accuracy `ewald_accuracy`, default $10^{-6}$; the cell must be charge neutral) and summed directly for open clusters; point charges reproduce the rock-salt Madelung constant 1.74756. All three provide analytic forces and stress, so relaxation with the cell, NPT dynamics, phonons and elastic constants work with them, and **Export to LAMMPS** writes the matching `tersoff`, `sw` or `buck/coul/long` pair styles (with `atom_stylecharge` and Ewald k-space for charges, and the built-in silicon parameter files). Fixed-charge and short-range potentials are only as good as their fit: check lattice constants and elastic constants against reference data before production runs.

Embedded-atom-method potentials are read from the standard LAMMPS table files distributed with published potentials (for example the NIST Interatomic Potentials Repository): `setfl` (`pair_styleeam/alloy`), Finnis–Sinclair (`eam/fs`) and single-element `funcfl` (`eam`). In the dialog choose **EAM / Finnis-Sinclair file** and browse for the file; in a request write `\{"potential": "EAM","file":"Cu01.eam.alloy"\}` with an optional `"format"` (`auto`, `setfl`, `fs` or `funcfl`; `auto` uses the file name). Relative paths are resolved beside the request file. Elements are matched by chemical symbol, the file's cutoff is used, and densities above the tabulated range continue linearly as in LAMMPS. Tables are interpolated with the same cubic scheme as LAMMPS, so energies and forces agree with LAMMPS for the same file. An EAM potential is only as transferable as its fit; check its intended phases and properties. None of the built-in potentials is universal. Earlier requests written as `\{"module":"ase.calculators.emt","attribute":"EMT"\}` are still accepted. The direct Python simulation functions and `python-matomforge.science` accept configured ASE calculators instead. For NEB the direct Python interface takes a callable that

returns a new calculator for each image.

# Chapter 10

# Illustrated quick tutorials

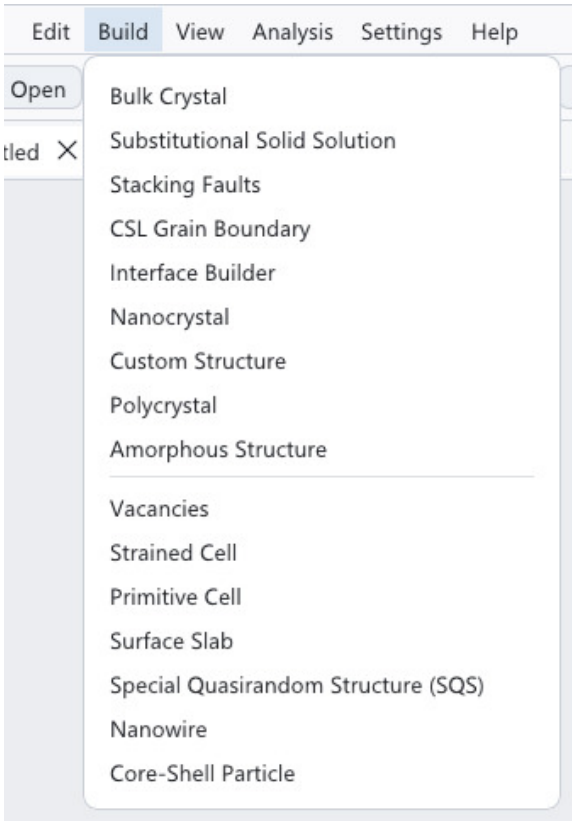
Each exercise starts from a named example or an explicit input. Use a working copy for edits. The numbered steps describe the target exercise; a screenshot may show the initial controls instead of the completed settings. Captions identify the captured state. Full parameter definitions remain in the builder, edit, and display control references.

Screenshots are real Windows application captures. Cropping in LaTeX only isolates the relevant dialog or controls; it does not change their contents. Some dialogs need scrolling to reach lower controls. Scientific output images are separately plotted from native builder output, as labelled.

## 10.1 Build: source, symmetry, and composition

### 10.1.1 Choose a builder

**Start here.** Open AtomForge. All structure examples below are in `docs/manual/examples/`.




*The Build menu: the crystal, defect, interface and particle builders first, then the tools that derive a new cell from the current structure.*

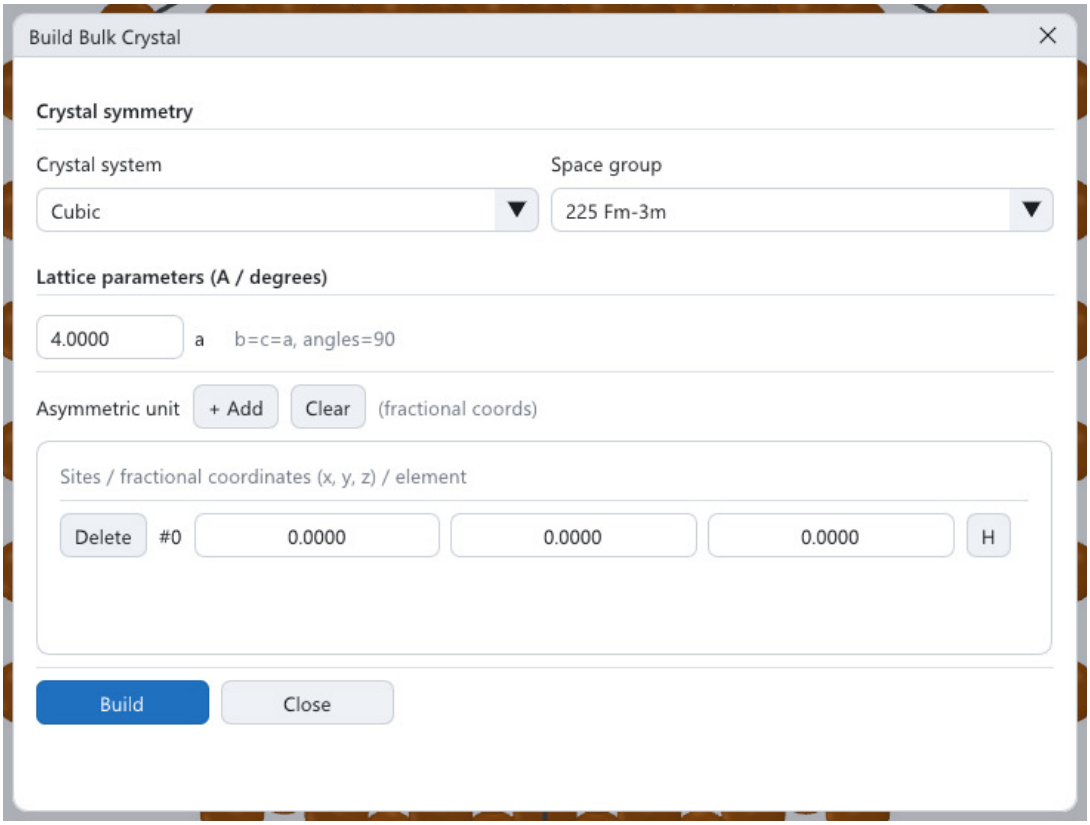
1. Choose **Build**, then the tool matching your intended geometry: Bulk for a reference cell; alloy for relabelling sites; CSL for a bicrystal; Nanocrystal for a finite cut; Custom for a mesh; Polycrystal for many grains; Amorphous for packing.
2. Load a periodic source where the tool asks for one. A source is the structure copied or repeated by the builder. Its name in the dialog should match the intended file.
3. Build a small example, select its new structure tab, inspect **View > Structure Info**, and save it under a new name.

**Check the result.** Build-menu outputs normally open in a new tab. The original source tab remains available.

**Try the options.** A grey Build button usually means required input is missing. A main-scene structure is not automatically loaded into every builder: use its drop target or **Use Current Scene** when offered.

### 10.1.2 Build FCC copper

**Start here.** Choose **Build > Bulk Crystal**.




*Bulk Crystal at initial settings; change H and the lattice parameter for this exercise.*

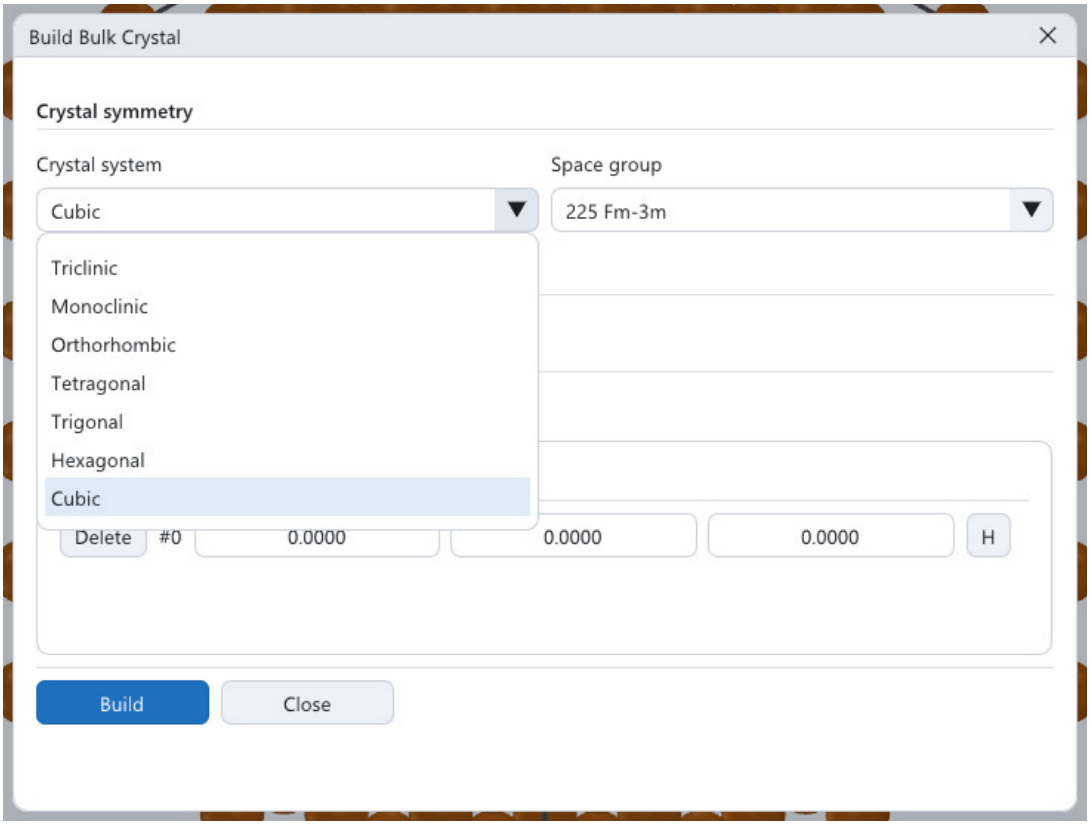
1. Select Cubic and space group 225. Enter $a = 3.61$ Å.
2. Click the initial H element button, choose Cu, and apply it. Keep the one independent site at direct coordinates $(0, 0, 0)$.
3. Choose **Build**. Inspect the new structure, then save as `cu_fcc_tutorial.cif`.

**Check the result.** Four Cu atoms should be present in the conventional cell. The supplied `examples/cu_fcc.cif` is a reference result.

**Try the options.** Use **+ Add** for additional independent sites, **Delete** for one row, or **Clear** to start the site list again. Try BCC Fe: Cubic, group 229, $a = 2.87$ Å, Fe at the origin; expect two atoms.

### 10.1.3 Change crystal system and lattice constraints

**Start here.** Open **Bulk Crystal**; work in a fresh builder rather than changing a saved reference.




*The crystal-system selector exposes all seven supported systems.*

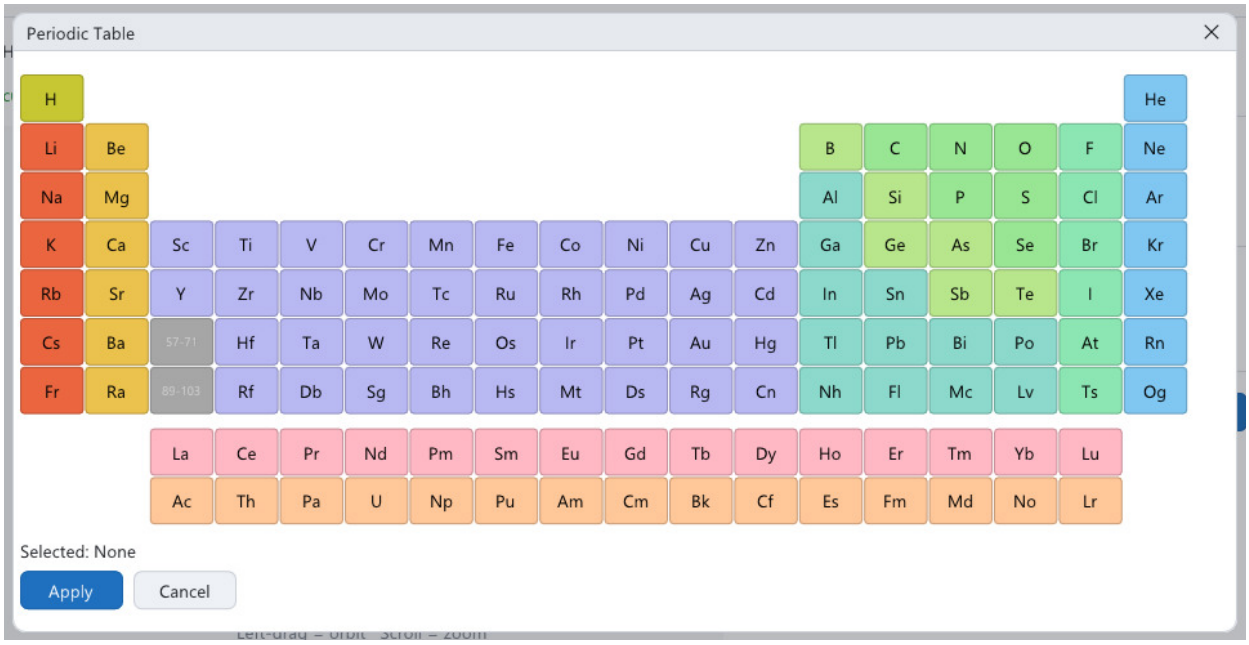
1. Select Triclinic and group 1. Enter $a = 4$, $b = 5$, $c = 6$ Å, $\alpha = 80°$, $\beta = 95°$, $\gamma = 110°$, and one Cu site at the origin.
2. Build and inspect all six cell parameters in Structure Info.
3. Reopen the builder and switch through the other systems, checking which independent fields remain available and reselecting a compatible space group.

**Check the result.** The group-1 example has one site and a skew cell. System changes constrain the lattice fields; they do not infer the intended material.

**Try the options.** Monoclinic exposes $a, b, c, \beta$; Orthorhombic $a, b, c$; Tetragonal $a, c$; Cubic $a$. Trigonal and Hexagonal use the hexagonal setting with $a, c$ and $\gamma = 120°$. Do not enter rhombohedral primitive parameters into that setting.

### 10.1.4 Choose an element for a site or species row

**Start here.** Open an element button in Bulk Crystal, the alloy builder, or Edit Structure.




*The shared periodic-table element selector, captured from the alloy workflow.*

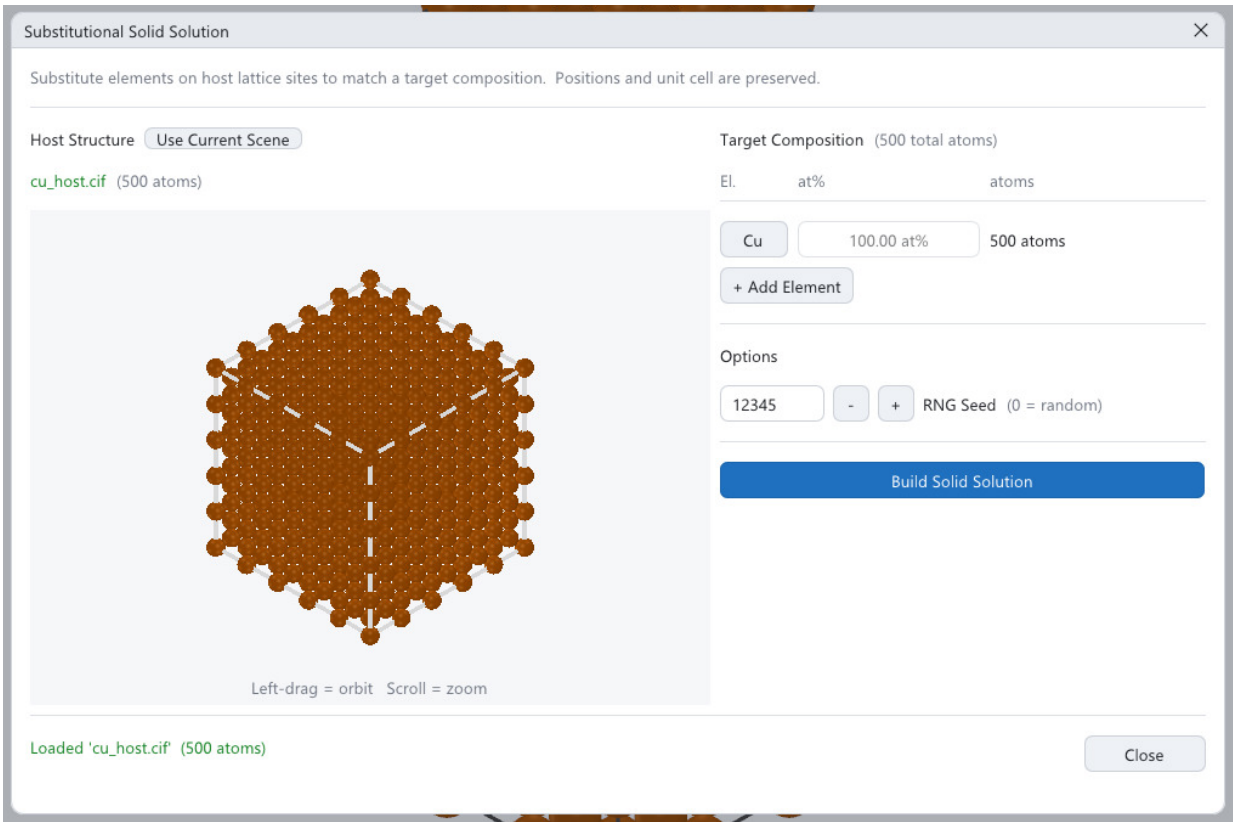
1. Select the required chemical symbol in the periodic table. Read the selected symbol or atomic number before applying.
2. Use the dialog's **Apply Element** action where present, then check the original row.
3. For a two-species example, add a second row and choose a different element. Recheck both rows before building.

**Check the result.** Only the intended site/species row should acquire the selected identity. Choosing an element does not assign a charge state or pseudopotential.

**Try the options.** Settings > Colors and Atomic Sizes also use a periodic table, but those dialogs change appearance/radius settings rather than substituting atoms.

### 10.1.5 Create a reproducible Cu–Ni alloy

**Start here.** Open `examples/cu_host.cif`, then **Build > Substitutional Solid Solution**.




*The 500-site copper source loaded, with the target composition starting as pure Cu. Add Ni and set the rows to 70 / 30 for this tutorial.*

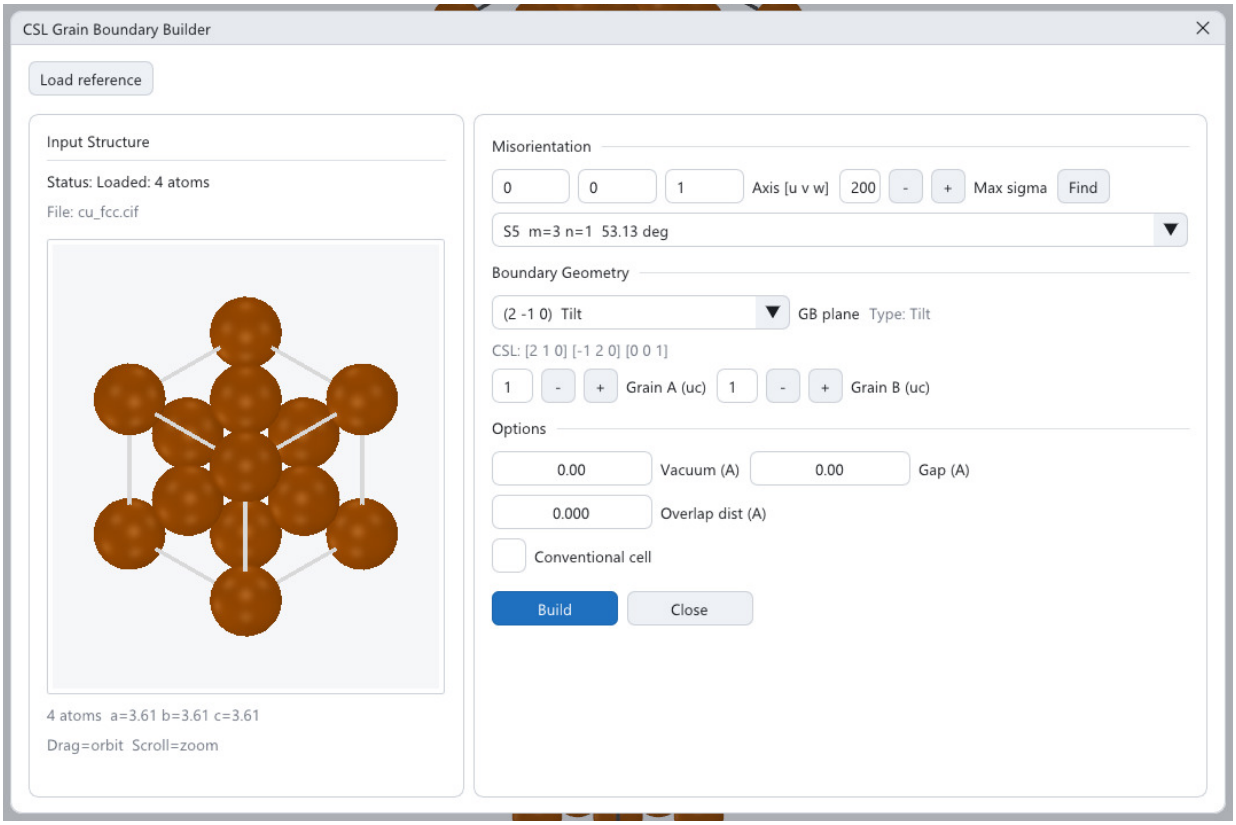
1. Choose **Use Current Scene**. Set Cu as the first species and Ni as the last species.
2. Enter 70 in the Cu at% field. Ni is the read-only remainder and should become 30. Set RNG seed to 42.
3. Choose **Build Solid Solution**, inspect the new tab, and save.

**Check the result.** Expect 350 Cu and 150 Ni atoms, matching `examples/cu_ni_alloy.cif`. Coordinates remain on host sites.

**Try the options. + Add Element** creates another species row; remove a row with its X. For a ternary 50:30:20, enter 50 and 30 in the first two rows. Keep seed and input ordering fixed to reproduce site assignment; seed zero requests a random assignment.

### 10.1.6 Find and build a Sigma-5 boundary

**Start here.** Choose **Build > CSL Grain Boundary**; drop `examples/cu_fcc.cif` into its reference preview. Use **Load reference** to select the source with the shared file browser.




*Loaded four-atom copper reference and selected Σ5 [001] candidate before Build. This capture still has one repeat per grain and zero overlap distance; apply the exercise values below before generating its 60-atom result.*

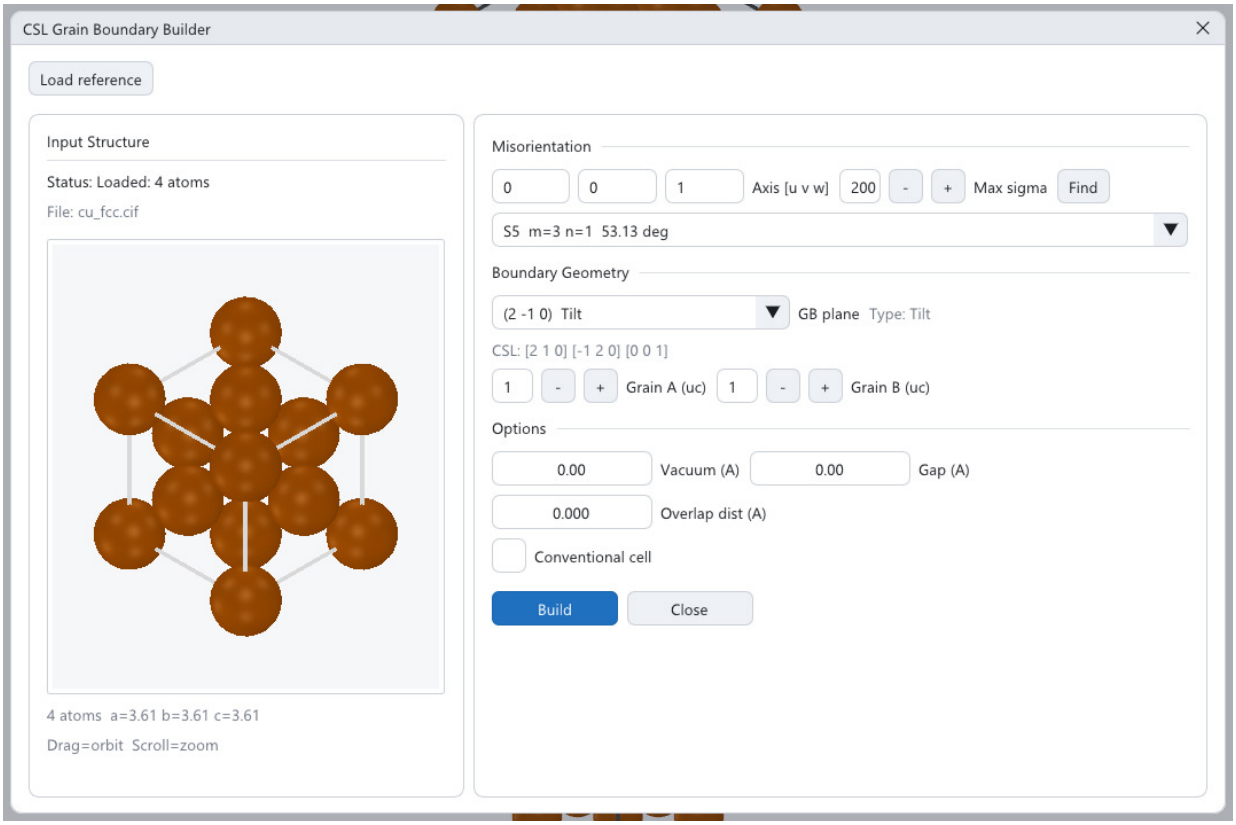
1. Set rotation axis [001] and Maximum Sigma to 200, then **Find**. Select Sigma 5 and candidate plane 0, shown here as $(2, -1, 0)$.
2. Set Grain A and Grain B to 3 each, Gap and Vacuum to 0, and Overlap distance to 1.5 Å. Leave Conventional cell off for the supplied example.
3. Scroll down to Build, generate, and inspect the interface and outer periodic contacts.

**Check the result.** The supplied `examples/cu_sigma5.cif` has 60 atoms. The candidate angle shown here is approximately 53.13°.

**Try the options.** Find must be repeated after changing the axis. The plane selector chooses a candidate, not an arbitrary Miller triplet. Increase grain thickness for a larger sample; an overlap cutoff can change atom count and stoichiometry.

### 10.1.7 Compare grain-boundary cell options

**Start here.** Use the preceding Sigma-5 exercise and retain a saved copy of its result.




*The lower portion of the CSL settings after scrolling; Conventional cell and Build are now visible.*

1. Rebuild with Conventional cell enabled. Compare cell vectors, volume, and atom count against the reduced output.
2. Return to the baseline inputs. Add a small Gap, for example 1 Å, rebuild, and inspect the interface separation.
3. Restore Gap to zero and vary Vacuum independently. Save each output with its settings.

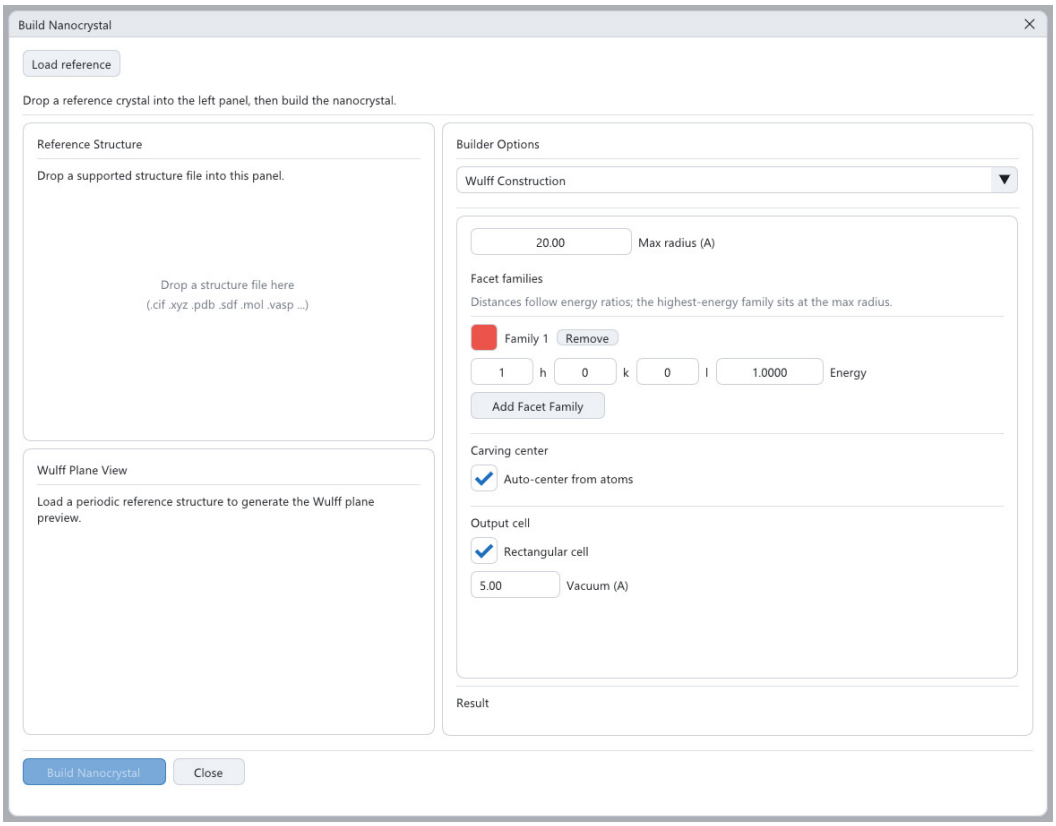
**Check the result.** The three controls serve different purposes: cell representation, internal separation, and outside spacing. Changing them is not an energy minimisation.

**Try the options.** Vary Grain A and Grain B separately to make unequal thicknesses. Raise Maximum Sigma only when searching additional coincidence candidates; it does not enlarge a chosen bicrystal.

## 10.2 Build: finite shapes and meshes

### 10.2.1 Choose a nanocrystal construction

**Start here.** Open **Build > Nanocrystal** and drop `examples/cu_fcc.cif` into the reference input.



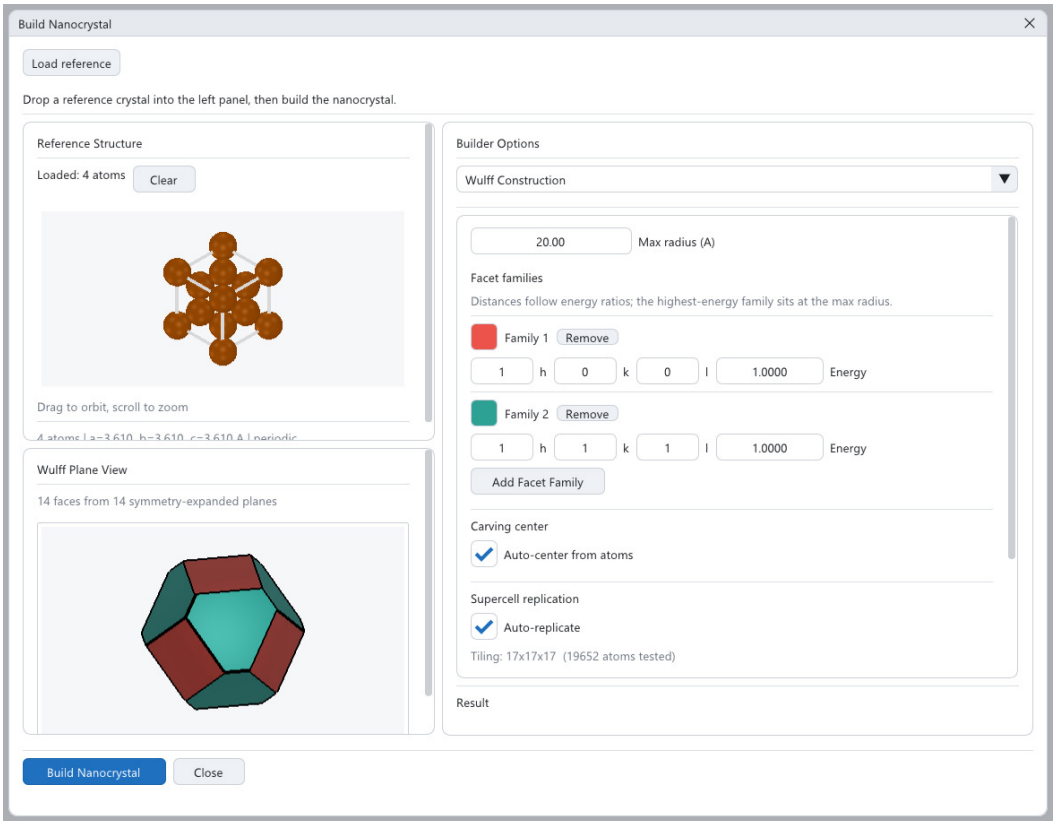

*Nanocrystal controls before a periodic reference is loaded. Wulff Construction is selected; Shape Cut is the alternative mode with its own geometry controls.*

1. Select **Shape Cut**, choose Sphere, and enter radius 10 Å.
2. Keep automatic centring and replication, use a rectangular output cell with 5 Å vacuum, and Build.
3. Inspect the finite particle from two directions, then save a coordinate file.

**Check the result.** The native example has 336 atoms. `examples/shapes/sphere.xyz` supplies the corresponding output.

**Try the options.** Manual centre changes surface termination. Manual replication must cover the entire requested shape. The output vacuum changes its enclosure rather than its surface cut. The next exercises show every shape option with native output images.

## Choose a nanocrystal construction: preview and result

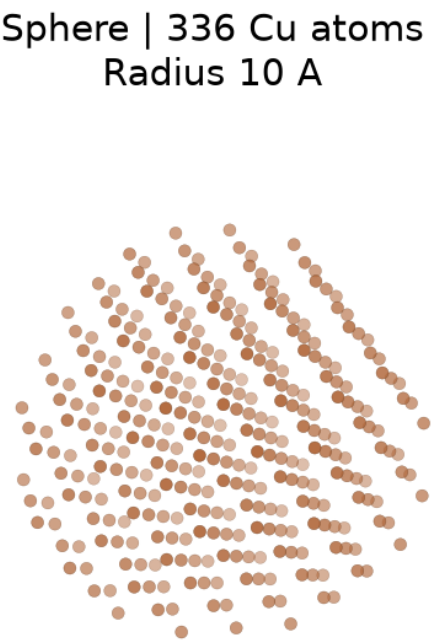



*A populated Nanocrystal dialog in Wulff mode. The left preview displays the source crystal; the right preview displays the facet construction. Shape Cut uses its own geometry controls.*

**Interpretation.** The native example has 336 atoms. `examples/shapes/sphere.xyz` supplies the corresponding output.

### 10.2.2 Shape Cut: sphere

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then sphere.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

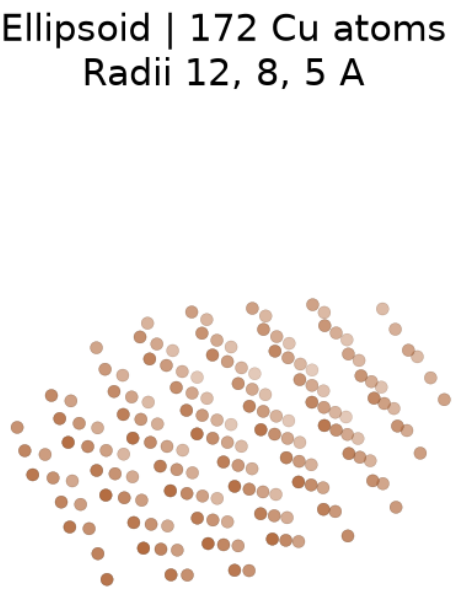
1. Set Radius to 10 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/sphere.xyz` to compare the supplied result.

**Check the result.** 336 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** Compare radius 8 and 12. The particle should grow in every direction; counts change in discrete steps.

### 10.2.3 Shape Cut: ellipsoid

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then ellipsoid.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

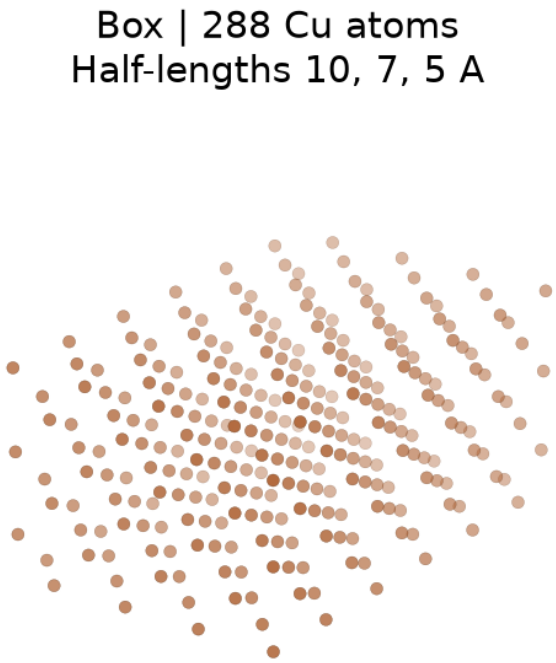
1. Set the X, Y, Z radii to 12, 8, 5 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/ellipsoid.xyz` to compare the supplied result.

**Check the result.** 172 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** Swap the X and Z radii to exchange the long and short axes. These inputs are half-extents, not diameters.

### 10.2.4 Shape Cut: box

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then box.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

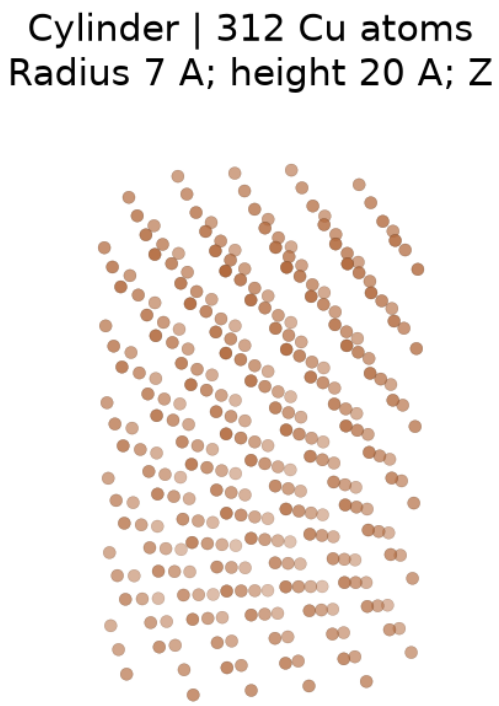
1. Set the X, Y, Z half-lengths to 10, 7, 5 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/box.xyz` to compare the supplied result.

**Check the result.** 288 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** The nominal full dimensions are 20 by 14 by 10 Angstrom. Inspect the discretised outermost lattice sites.

### 10.2.5 Shape Cut: cylinder

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then cylinder.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

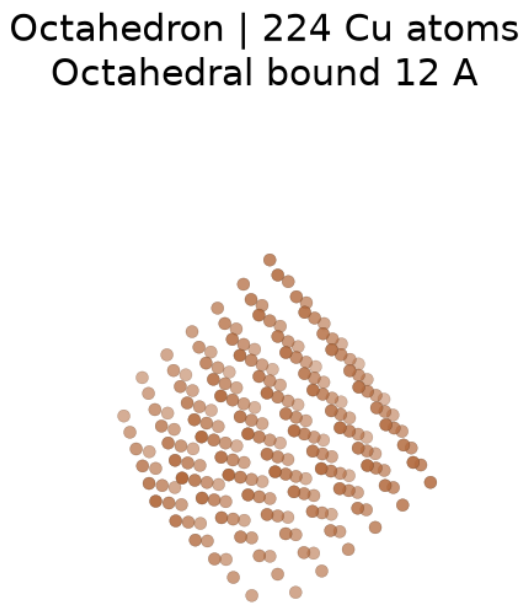
1. Set cylinder radius to 7 Angstrom, full height to 20 Angstrom, and axis Z.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/cylinder.xyz` to compare the supplied result.

**Check the result.** 312 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** Change the axis to X or Y while retaining the dimensions. Radius is transverse to the chosen axis; height is not a half-length.

### 10.2.6 Shape Cut: octahedron

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then octahedron.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

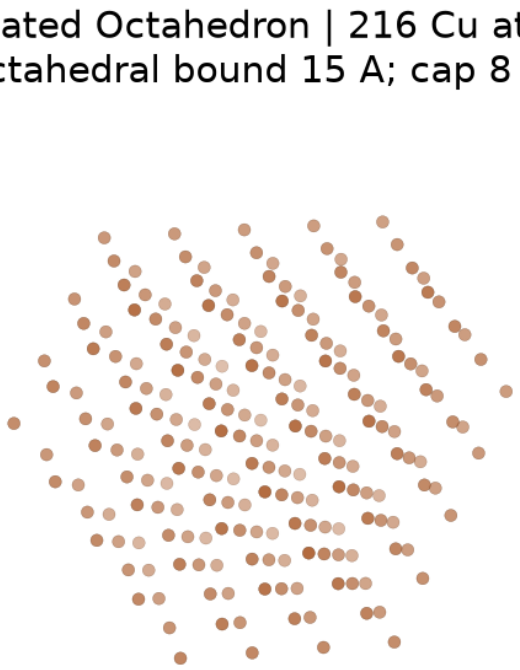
1. Set the octahedral radius parameter to 12 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/octahedron.xyz` to compare the supplied result.

**Check the result.** 224 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** The local cut is $|x| + |y| + |z| \leq 12$ Angstrom. The radius reaches the axis vertices, not every surface point.

### 10.2.7 Shape Cut: truncated octahedron

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then truncated octahedron.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

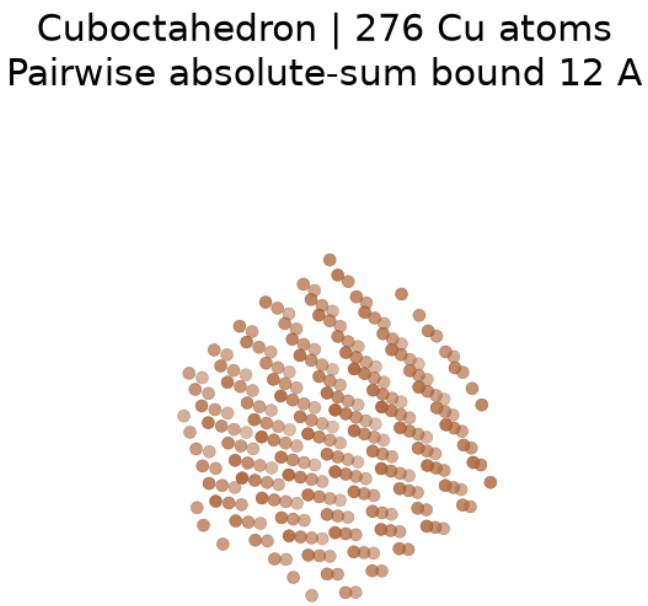
1. Set the octahedral radius to 15 Angstrom and truncation radius to 8 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/truncated-octahedron.xyz` to compare the supplied result.

**Check the result.** 216 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** The cut combines $|x| + |y| + |z| \leq 15$ and $\max(|x|, |y|, |z|) \leq 8$. Vary the cap to expose or suppress truncating faces.

### 10.2.8 Shape Cut: cuboctahedron

**Start here.** Load `examples/cu_fcc.cif` in Nanocrystal, select Shape Cut, then cuboctahedron.




*Scientific plot of actual native builder output; this is not a GUI screenshot.*

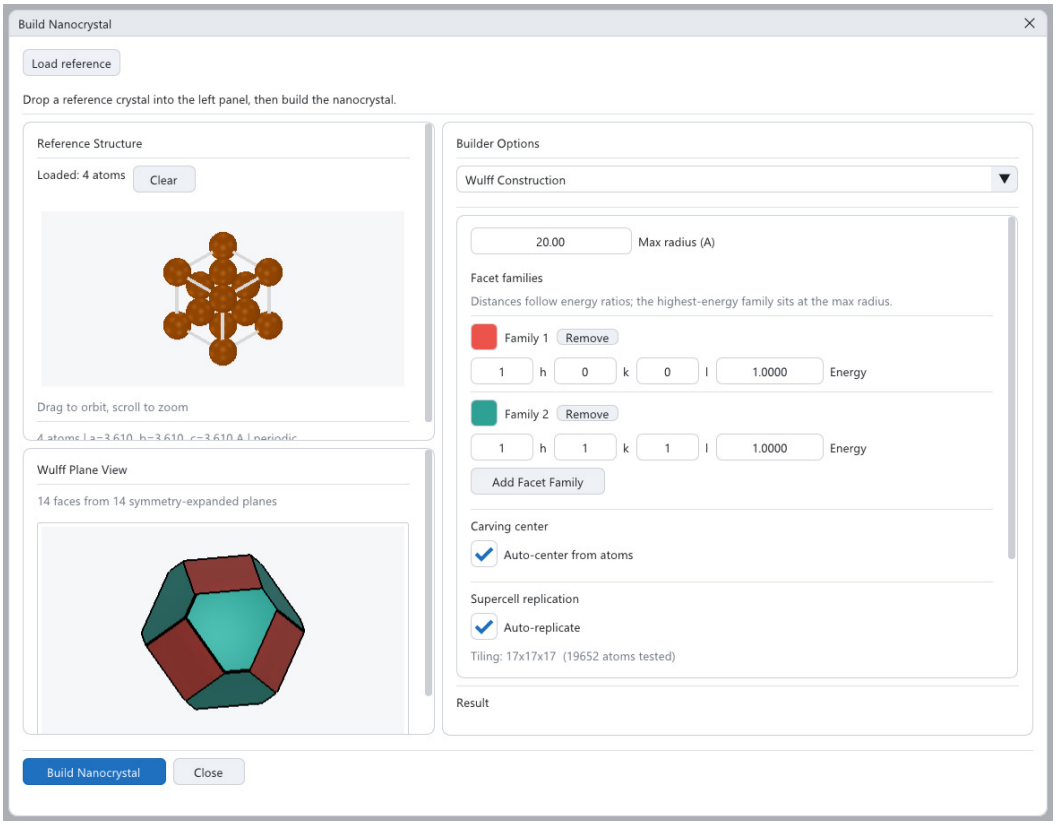
1. Select the option named Cuboctahedron and set its radius parameter to 12 Angstrom.
2. Use automatic centre/replication and 5 Å vacuum. Build and save.
3. Open `examples/shapes/cuboctahedron.xyz` to compare the supplied result.

**Check the result.** 276 Cu atoms in the supplied native example. A different centre or source basis can change the count.

**Try the options.** The implementation tests each pairwise absolute sum: $|x| + |y|$, $|y| + |z|$, and $|z| + |x|$ must be at most the parameter. This is the implemented geometry, not the usual ideal cuboctahedron definition.

### 10.2.9 Build a Wulff particle from facet families

**Start here.** Open Nanocrystal, select Wulff Construction, and load `examples/cu_fcc.cif`. Use **Load reference** to select the source with the shared file browser.



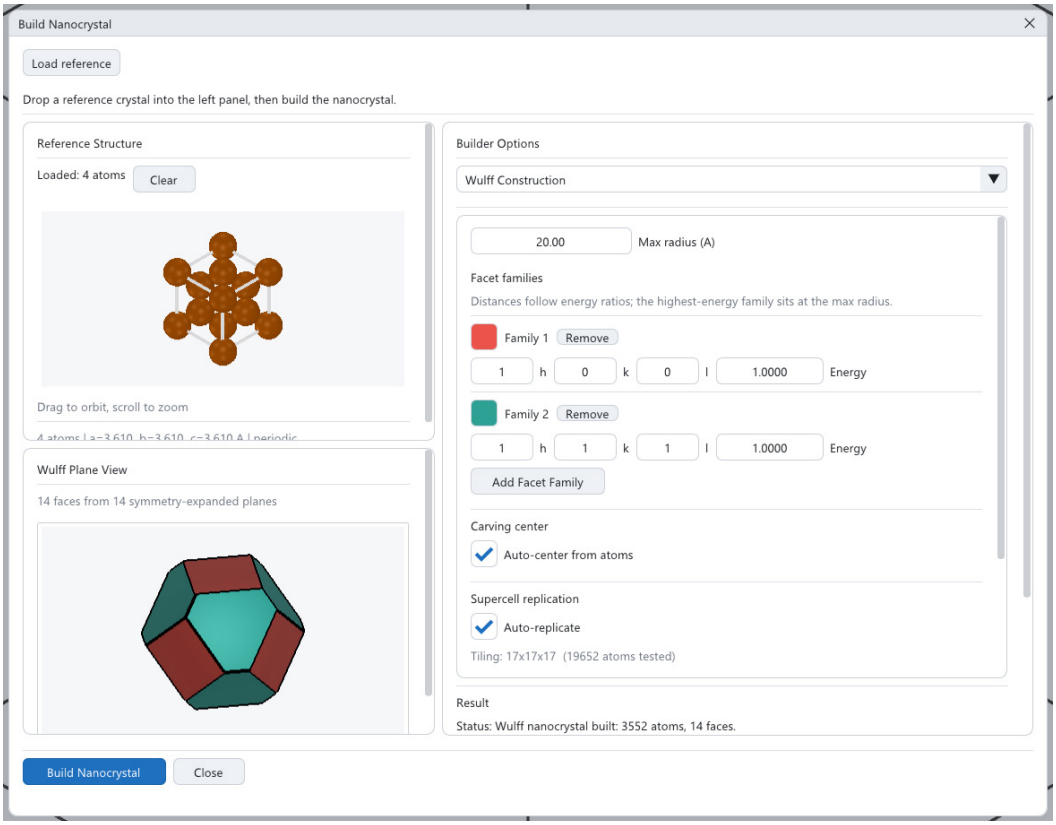

*Before Build: loaded copper, radius 20 Å, and (100)/(111) families with both energies set to 1. The dialog shows the reference and facet geometry, not the final atomic particle.*

1. Use family (100) with energy 1.0, then add (111) with energy 1.0. Choose distinct family colours.
2. Set Max radius to 20 Å. Keep automatic centring/replication and a rectangular cell with 5 Å vacuum.
3. Inspect the Wulff Plane View, build the atomic result, and compare facets against the guide planes.

**Check the result.** The families should define a bounded intersection. A family hidden behind other bounding planes may contribute no exposed facet.

**Try the options.** Try changing the (111) energy to 0.9 and rebuild to compare facet areas. Change a positive energy ratio and rebuild to compare facet areas. Add/remove families using the row controls. The highest-energy family is placed at Max radius; that value is not necessarily the farthest vertex radius. The demonstration energies are illustrative, not measured surface energies.

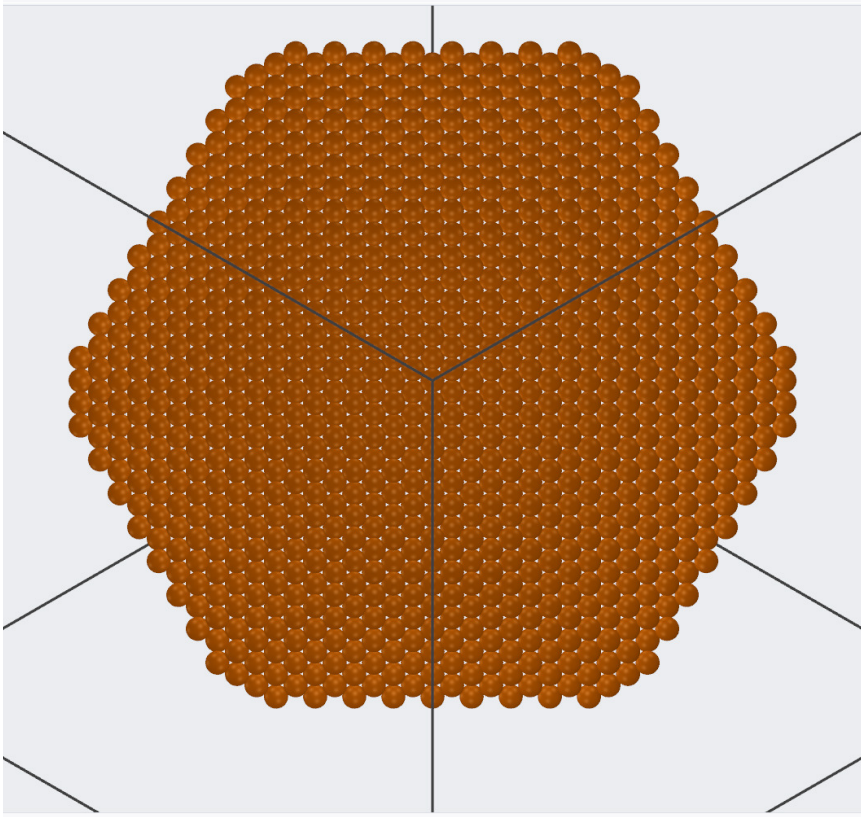
### Build a Wulff particle from facet families: preview and result




*After Build with the captured equal-energy settings: the status reports 3,552 atoms and 14 faces. The finished particle opens in the main workspace.*

**Interpretation.** The families should define a bounded intersection. A family hidden behind other bounding planes may contribute no exposed facet.

### Build a Wulff particle from facet families: preview and result

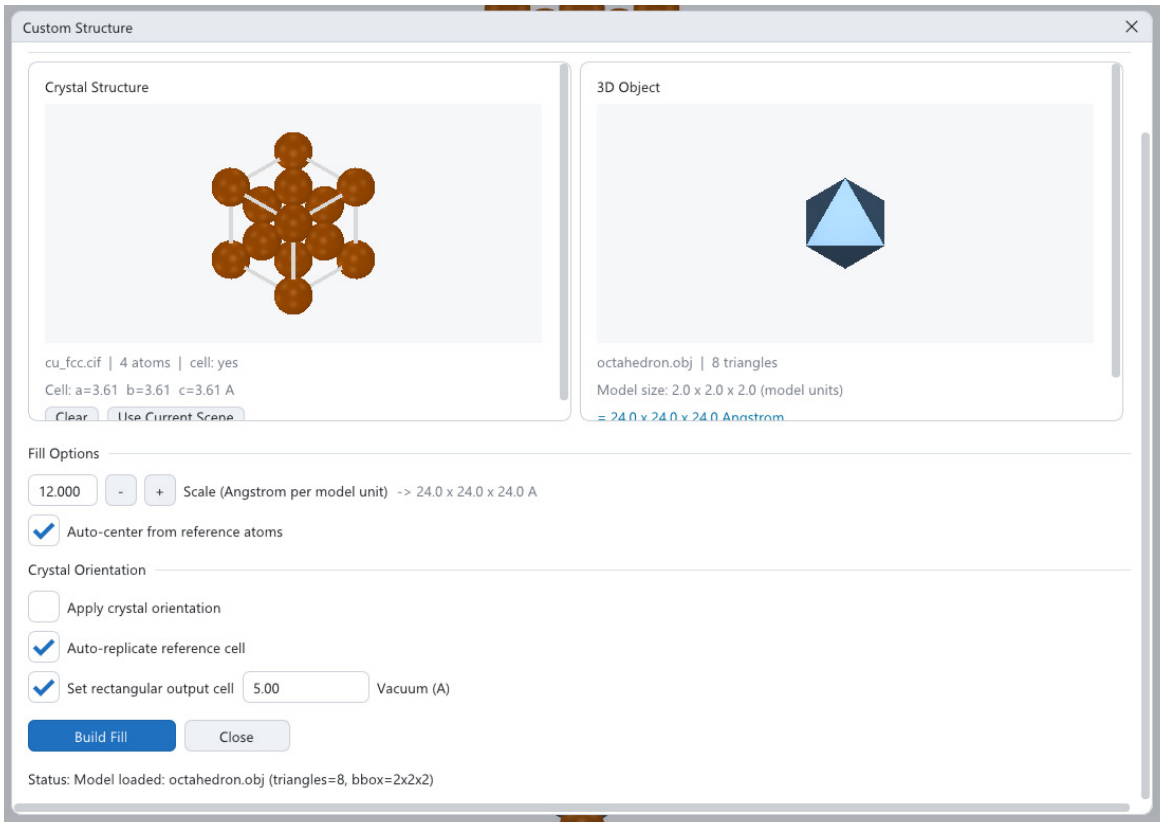


*The generated equal-energy Wulff particle in the actual AtomForge viewport. Orbit the main view to inspect its facets.*

**Interpretation.** The families should define a bounded intersection. A family hidden behind other bounding planes may contribute no exposed facet.

### 10.2.10 Fill an OBJ mesh with copper sites

**Start here.** Prepare `examples/cu_fcc.cif` and the supplied closed `examples/octahedron.obj`.



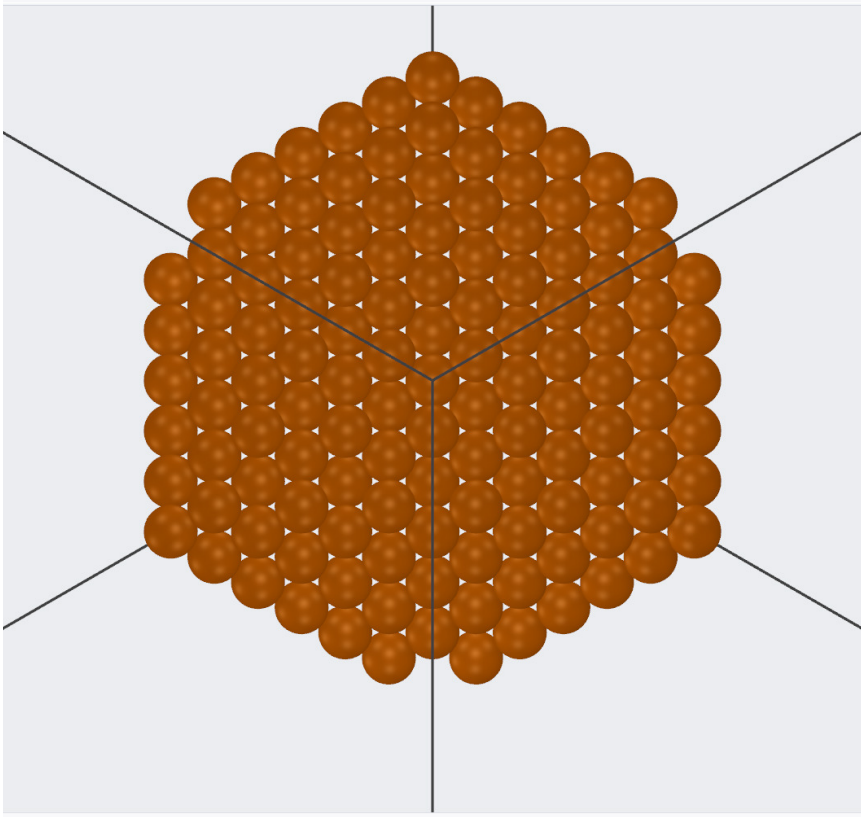

*Immediately before Build Fill: copper reference and octahedron mesh previews, automatic repetition, scale 12 Å/model unit, orientation disabled, and a rectangular output cell with 5 Å vacuum. Scroll upward for the source picker.*

1. Open **Build > Custom Structure**. Choose **Load structure or model** twice: select the crystal file, then the OBJ. Each file is routed to its corresponding preview. File drops also work. Alternatively copy an appropriate periodic scene using **Use Current Scene**.
2. Set Scale to 12 Å per model unit, automatic centring, and rectangular output with 5 Å vacuum.
3. Leave crystal orientation disabled for the baseline exercise. Build Fill and inspect the result.

**Check the result.** The supplied native example `examples/cu_mesh.xyz` contains 224 Cu atoms. The OBJ has six unit-axis vertices and eight triangular faces.

**Try the options.** Scale converts mesh units to physical distances. A non-watertight mesh or a feature smaller than lattice spacing can produce a missing fill. Use a closed mesh, verify its dimensions, and confirm sufficient replication before raising sample size.

### Fill an OBJ mesh with copper sites: preview and result

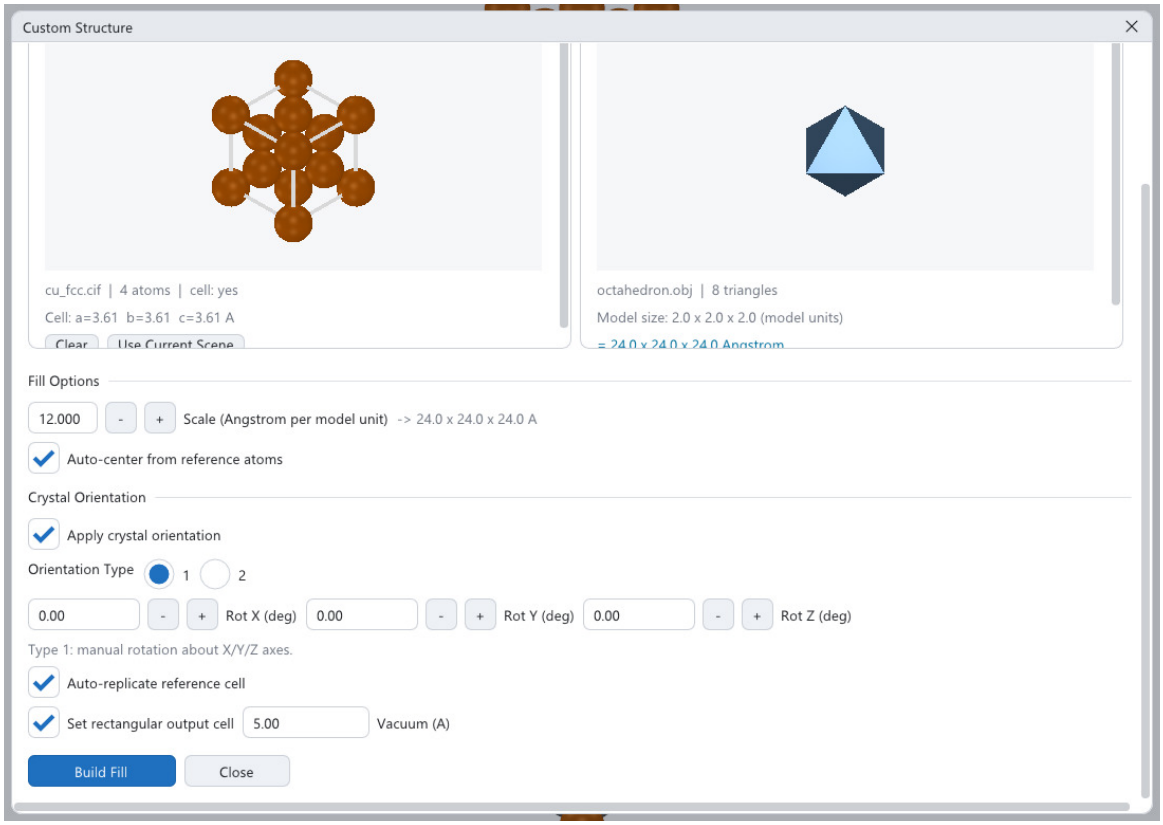


*The 224-atom filled octahedron in the AtomForge main viewport after Build Fill.*

**Interpretation.** The supplied native example `examples/cu_mesh.xyz` contains 224 Cu atoms. The OBJ has six unit-axis vertices and eight triangular faces.

### 10.2.11 Orient the lattice inside a mesh

**Start here.** Use the preceding mesh-fill inputs and save the unrotated result.



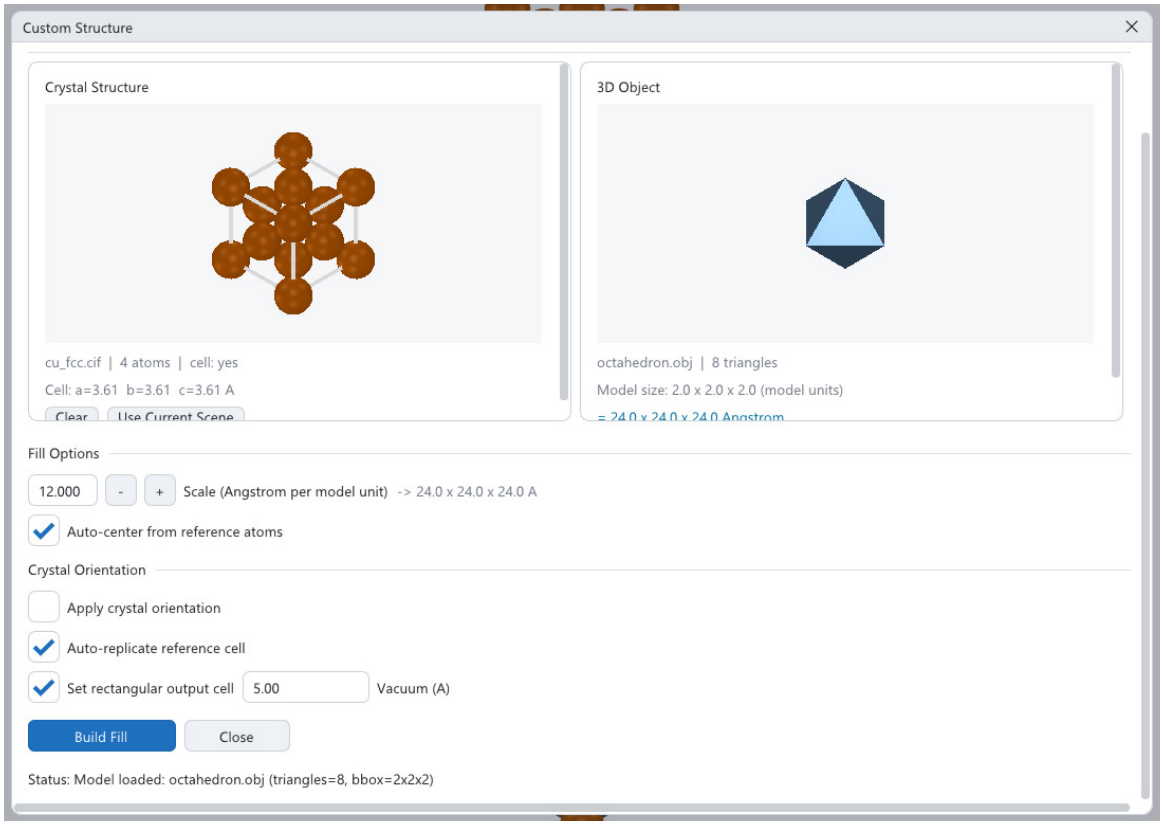

*The orientation portion of Custom Structure with Type 1 (rotations about X, Y and Z) enabled, above Build Fill.*

1. Enable **Apply crystal orientation**. Select Type 1 and enter Rot X = 0, Rot Y = 0, Rot Z = 45°.
2. Build again and compare lattice rows against the same mesh outline.
3. Return angles to zero and rebuild when checking the original result.

**Check the result.** The lattice alignment and discrete surface termination can change. Orbiting the preview alone does not perform this orientation operation.

**Try the options.** Try rotations about one axis at a time, then combine them. Record all three angles and the order convention from the detailed controls when reproducing a fill.

### Orient the lattice inside a mesh: preview and result

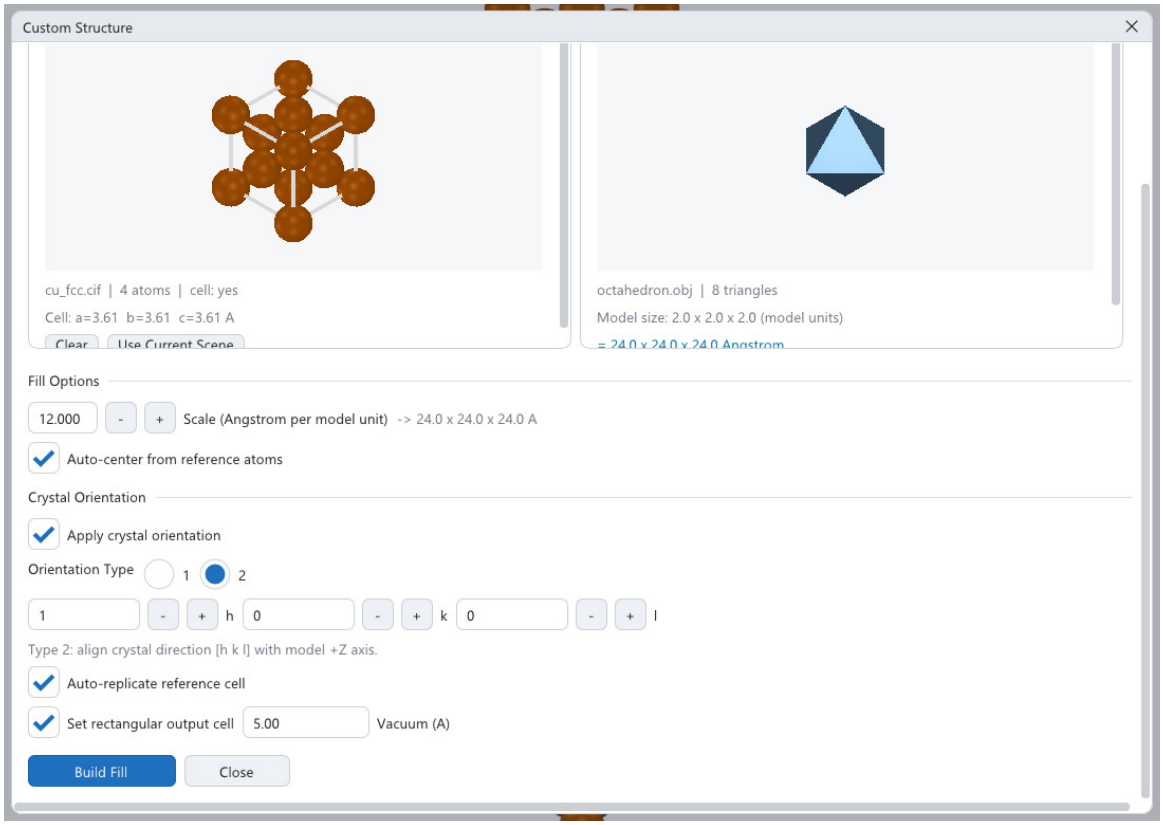



*Loaded source and model previews before enabling the orientation controls illustrated on the preceding page. The reference preview is not a regenerated, oriented atomic fill.*

**Interpretation.** The lattice alignment and discrete surface termination can change. Orbiting the preview alone does not perform this orientation operation.

### 10.2.12 Align a crystallographic direction to the mesh

**Start here.** Use the same periodic reference and closed mesh as the Custom Structure tutorial.



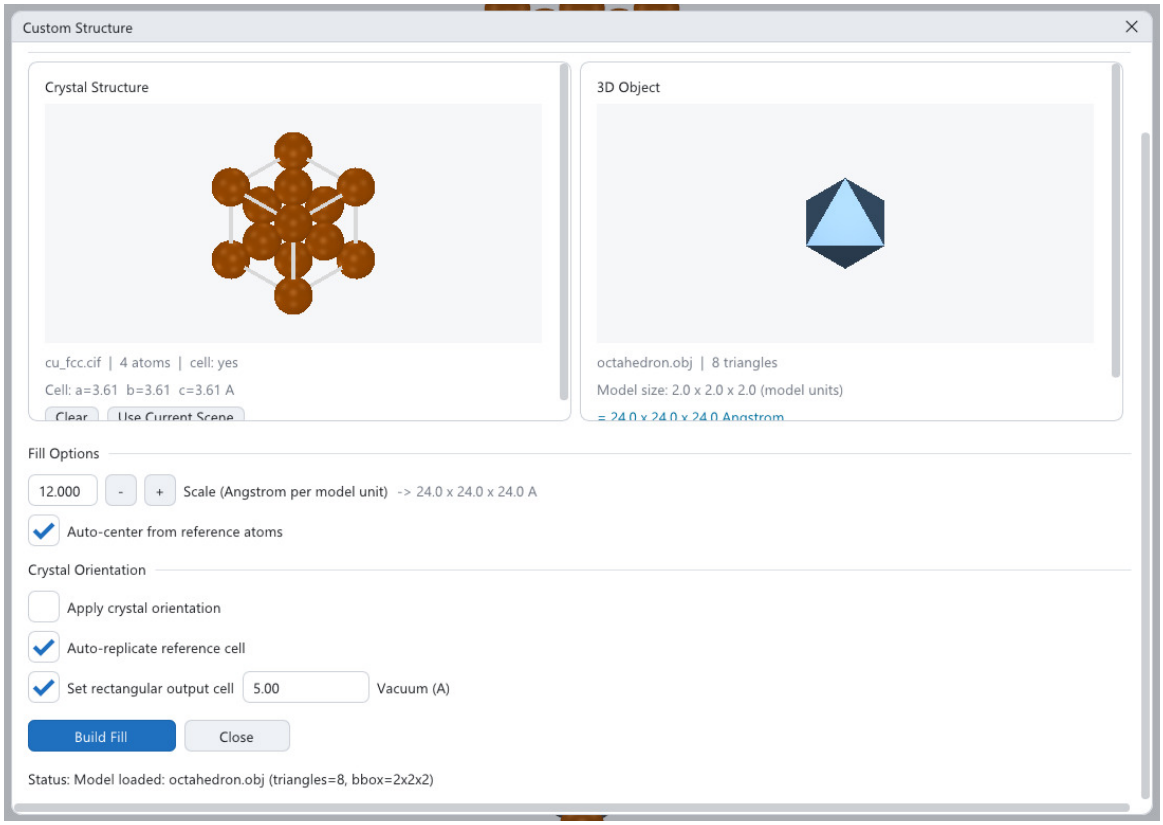

*Type 2 exposes the crystallographic direction triplet instead of Euler-angle inputs.*

1. Enable crystal orientation, select Type 2, and enter [111].
2. Build and inspect the lattice direction aligned to mesh +Z.
3. Compare against [001] with the same scale and centre.

**Check the result.** The direction must be nonzero. It is a direct-lattice direction, which differs from a reciprocal plane normal for a general skew cell.

**Try the options.** Switching orientation type changes which inputs define the transform. Recheck the active type before reproducing an Euler-angle recipe.

### Align a crystallographic direction to the mesh: preview and result



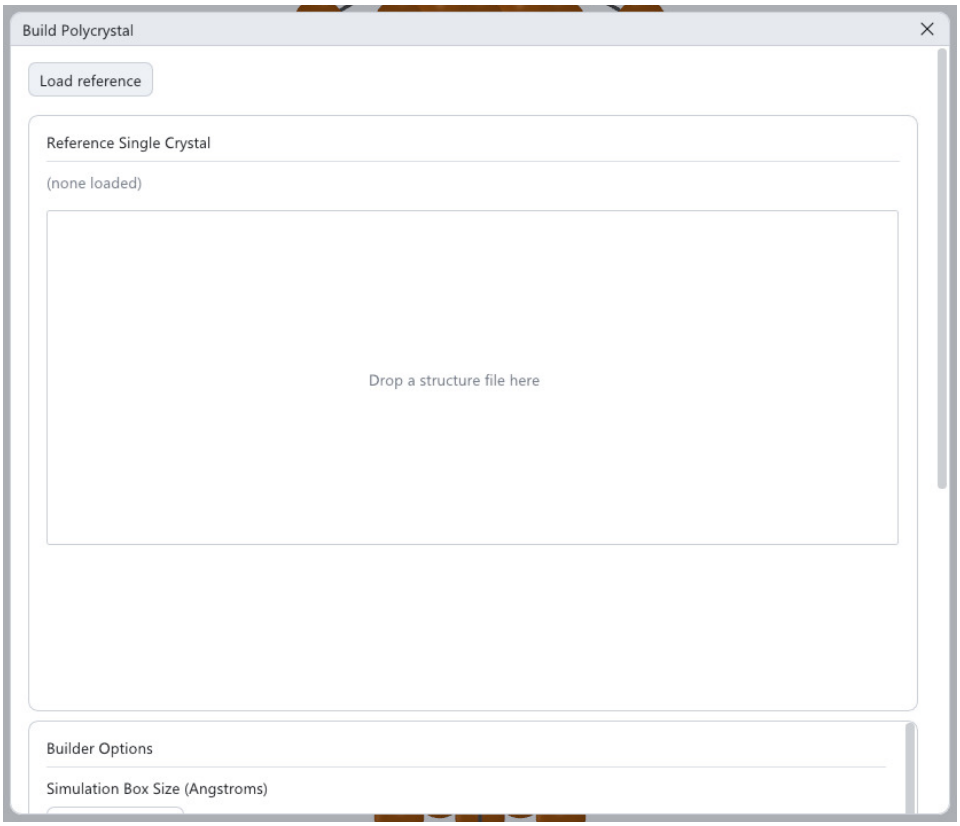

*Loaded source and model previews before enabling the orientation controls illustrated on the preceding page. The reference preview is not a regenerated, oriented atomic fill.*

**Interpretation.** The direction must be nonzero. It is a direct-lattice direction, which differs from a reciprocal plane normal for a general skew cell.

## 10.3 Build: grains and random packing

### 10.3.1 Build a four-grain copper sample

**Start here.** Open **Build > Polycrystal** and drop `examples/cu_fcc.cif` onto the reference preview. Use **Load reference** to select the source with the shared file browser.



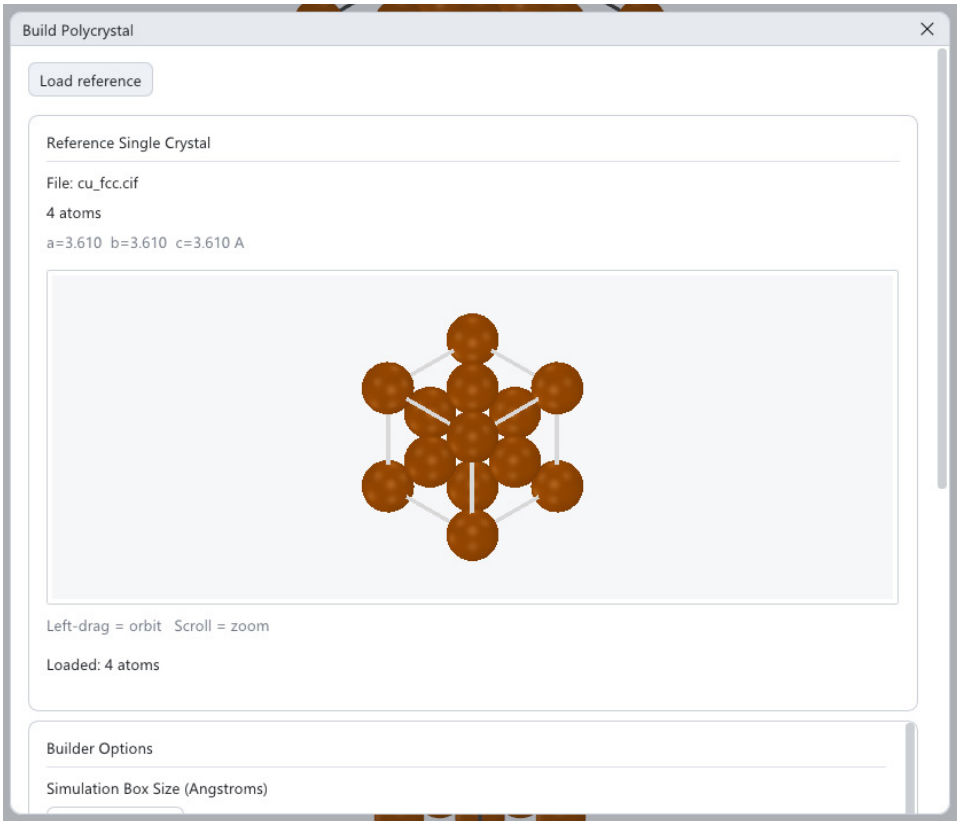

*Polycrystal controls at their initial values, before loading a reference. The exercise uses a smaller box and four grains.*

1. Enter X, Y, Z box dimensions of 25 Å each and Number of Grains 4.
2. Set Random Seed to 7 and Grain Orientations to All Random.
3. Build, inspect boundaries, and save the structure together with any orientation sidecar.

**Check the result.** The supplied `examples/cu_polycrystal.cif` has 1,329 atoms. Orientation colouring needs corresponding metadata.

**Try the options.** Increase box size at fixed grain count to enlarge typical grain interiors. Increase grain count at fixed size to refine the partition. Save seed and dimensions; the grain count alone does not reproduce a sample.

### Build a four-grain copper sample: preview and result

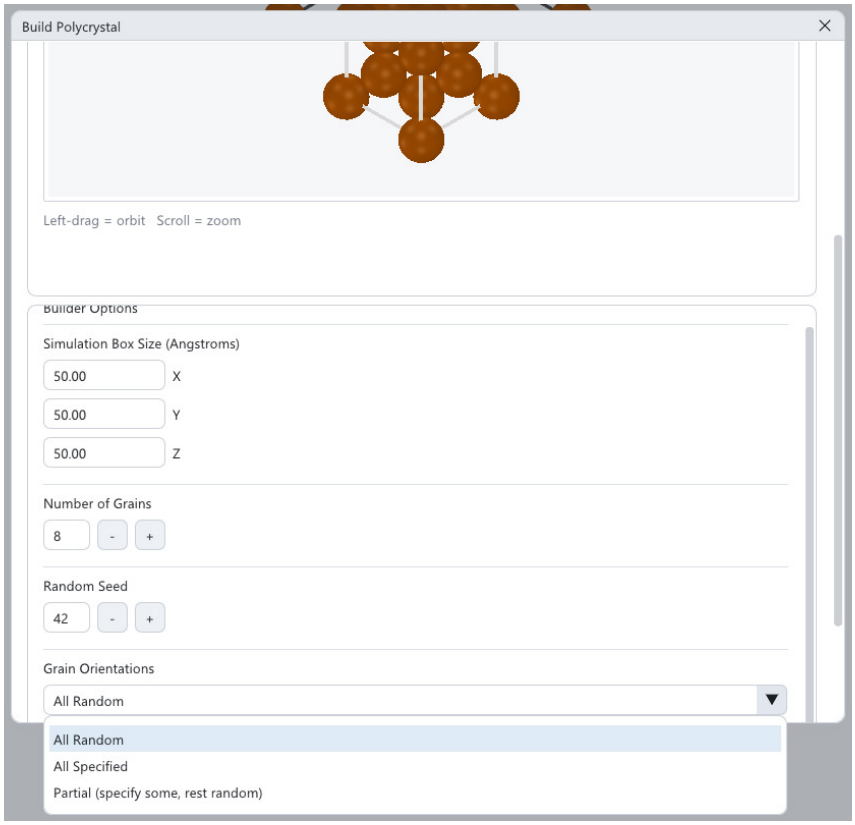



*Loaded copper reference in the narrow, vertically stacked layout. Scroll the outer dialog to reach Builder Options and Build; orbit or zoom inside this reference preview.*

**Interpretation.** The supplied `examples/cu_polycrystal.cif` has 1,329 atoms. Orientation colouring needs corresponding metadata.

### 10.3.2 Specify all or selected grain orientations

**Start here.** Use a four-grain polycrystal exercise with a fixed seed and box.




*All Random, All Specified, and Partial are separate grain-orientation modes.*

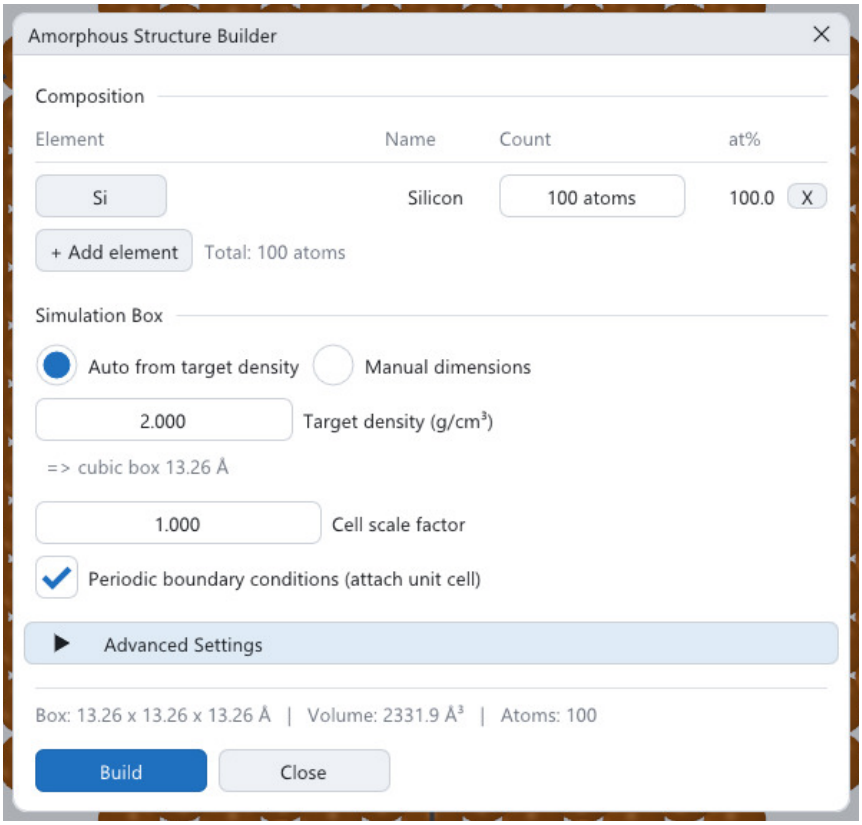
1. Choose **All Specified**. Enter a Bunge Euler triplet for each grain; begin with $(0, 0, 0)$ for one and $(45, 0, 0)$ degrees for another.
2. Choose **Partial** for a second trial. Add Grain 1 with $(0, 0, 0)$ and leave the other grains unspecified.
3. Build each trial and compare orientation colouring and the saved orientation information.

**Check the result.** All Specified should supply every grain; Partial fixes selected grains and randomises the rest. GUI grain numbers are one-based.

**Try the options.** Use Add Grain Orientation to append an entry, edit its three angles, and remove an unwanted row. Avoid duplicate grain IDs. Changing seed can still alter grain positions even when orientations are specified.

### 10.3.3 Pack a Si–O starting configuration

**Start here.** Choose **Build > Amorphous Structure**.




*Initial composition and density controls; replace the single Si row with the tutorial composition.*

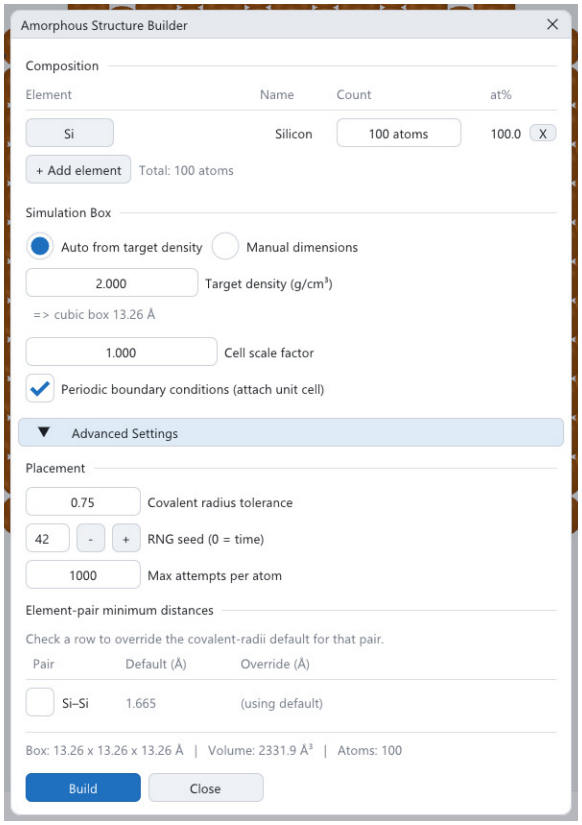
1. Set Si count to 40. Use **+ Add element** to add O with count 80.
2. Choose Auto from target density, enter 1.5 g cm$^{-3}$, retain Cell scale factor 1 and the unit-cell checkbox.
3. Open Advanced Settings, set seed 7, Build, and inspect the placement status.

**Check the result.** Expect 120 atoms, with Si:O = 1:2. `examples/sio2_pack.xyz` is the supplied packing example.

**Try the options.** Counts determine composition; at

### 10.3.4 Control packing distances and repeatability

**Start here.** Use the Si–O composition from the preceding exercise.




*Expanded advanced controls, captured with the initial single-species composition. Each enabled pair row can override its default distance.*

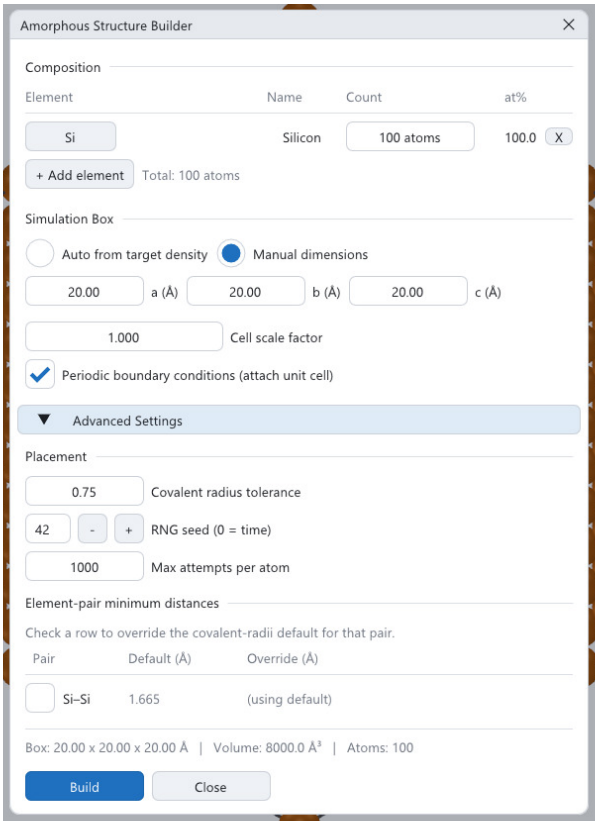
1. Keep a fixed RNG seed. Start with the default covalent-radius tolerance and maximum attempts.
2. For a deliberate pair constraint, enable that pair row and enter its minimum separation in Angstrom, then rebuild.
3. Compare inserted count and failure messages before and after changing the constraint.

**Check the result.** An impossible packing should not be treated as a complete structure. Stronger minimum distances can require a larger box or a lower target density.

**Try the options.** Increasing attempts spends more effort on the same constraints. It does not make contradictory density/distance requirements feasible. Seed zero uses time; record a positive seed for reproducibility.

### 10.3.5 Use an explicit packing box

**Start here.** Open the amorphous builder with the intended species counts.




*Manual dimensions replace the density-derived box. Advanced constraints remain available below.*

1. Select **Manual dimensions**; enter $a = 20$, $b = 20$, $c = 20$ Å and Cell scale factor 1.
2. Keep seed and distance settings fixed, Build, and inspect the actual box and count.
3. Repeat with scale 1.1 to see how extra volume affects placement.

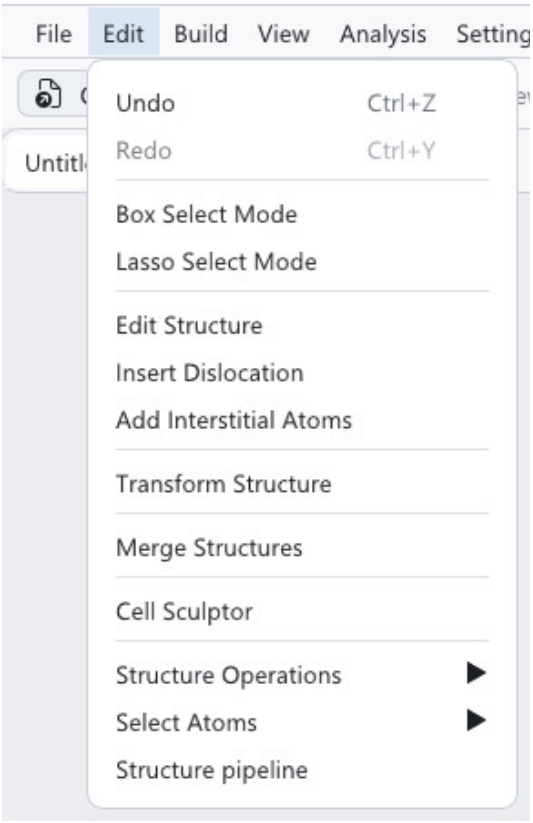
**Check the result.** At fixed counts, scale 1.1 multiplies volume by 1.331 and lowers number/mass density accordingly.

**Try the options.** The Periodic boundary conditions checkbox attaches a unit cell to the output; packing still uses minimum-image distance checks. Do not interpret this checkbox as switching to a different nonperiodic packing algorithm.

## 10.4 Edit: selection, coordinates, and assembly

### 10.4.1 Select atoms and recover an edit

**Start here.** Open a working copy of `examples/cu_host.cif`.




*The Edit menu groups history, selection, direct editing, and specialised construction tools.*

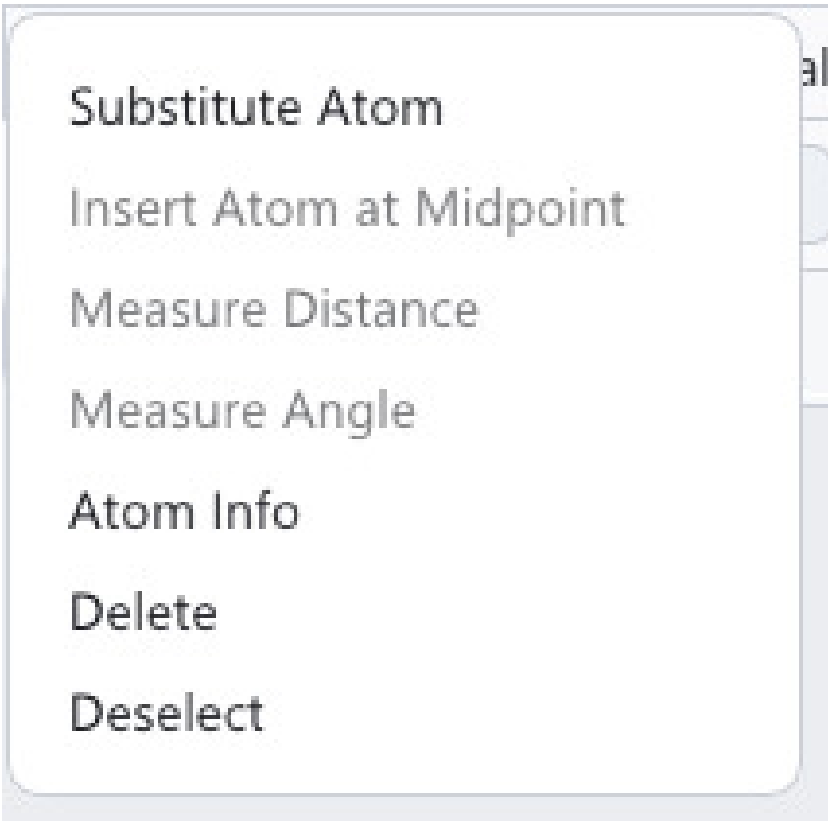
1. Click an atom, then Ctrl+click another to extend the selection. Use Ctrl+D to clear it.
2. Enable **Box Select Mode** and right-drag a rectangle. Repeat with **Lasso Select Mode** and an irregular right-drag path.
3. Inspect the selected atoms from another view, delete a small selected set, then use Undo and Redo where enabled. Restore or reload the saved example afterwards.

**Check the result.** Projected selection can include atoms behind those visible at the front. Undo/Redo restores supported structure changes; Reset View only restores framing.

**Try the options.** Use Ctrl+A to select all, Ctrl+Z to undo, and Ctrl+Y or Ctrl+Shift+Z to redo when no text field is active. Save checkpoints before larger operations.

### 10.4.2 Use atom context actions

**Start here.** Open a working copy of the copper host; select atoms in the main viewport.




*Context menu for one selected atom. Midpoint, distance, and angle require larger selections and are disabled here.*

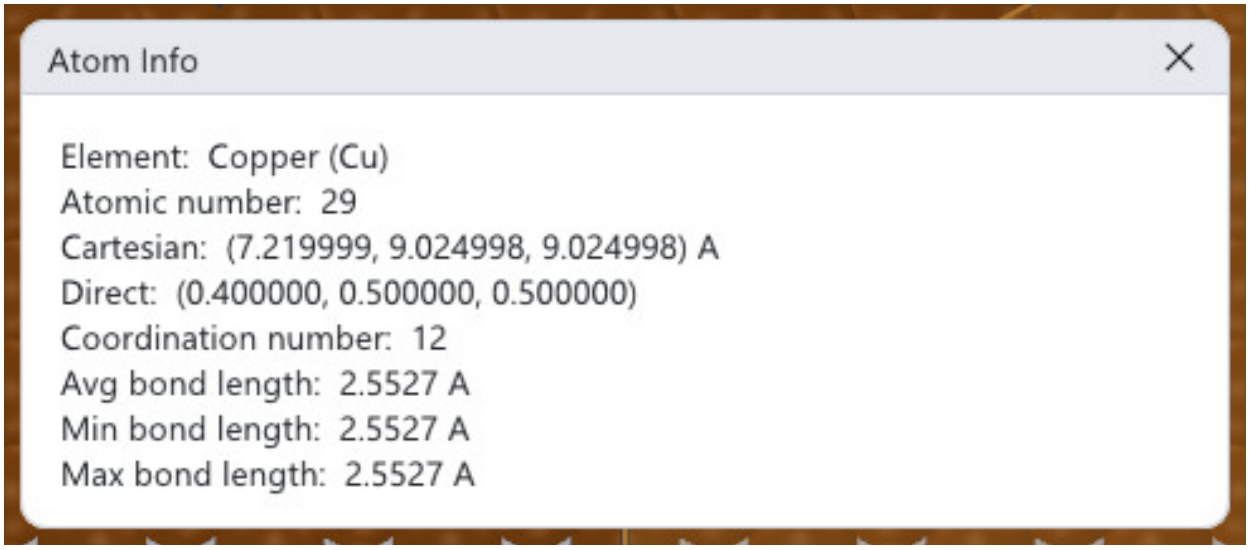
1. With one atom selected, right-click and use **Substitute Atom**; choose Ni and apply. Inspect the species count.
2. With two nearby sites selected, choose **Insert Atom at Midpoint**, select H, and inspect the added position.
3. Try **Delete** on the added atom, or **Deselect** to clear selection without deleting it. Save only the intended result.

**Check the result.** Substitution preserves the site count; insertion adds one atom; deletion removes selected base atoms. A periodic image maps to its base atom.

**Try the options.** For more than two selected sites, midpoint insertion uses their displayed Cartesian centroid. It is not a minimum-image midpoint. Use exactly two selections for distance, three for angle, and one for Atom Info.

### 10.4.3 Check coordinates and nearest neighbours

**Start here.** Open `examples/cu_host.cif` and select one Cu atom.




*Atom Info on the ideal copper host: coordination 12 and nearest-neighbour length about 2.5527 Angstrom.*

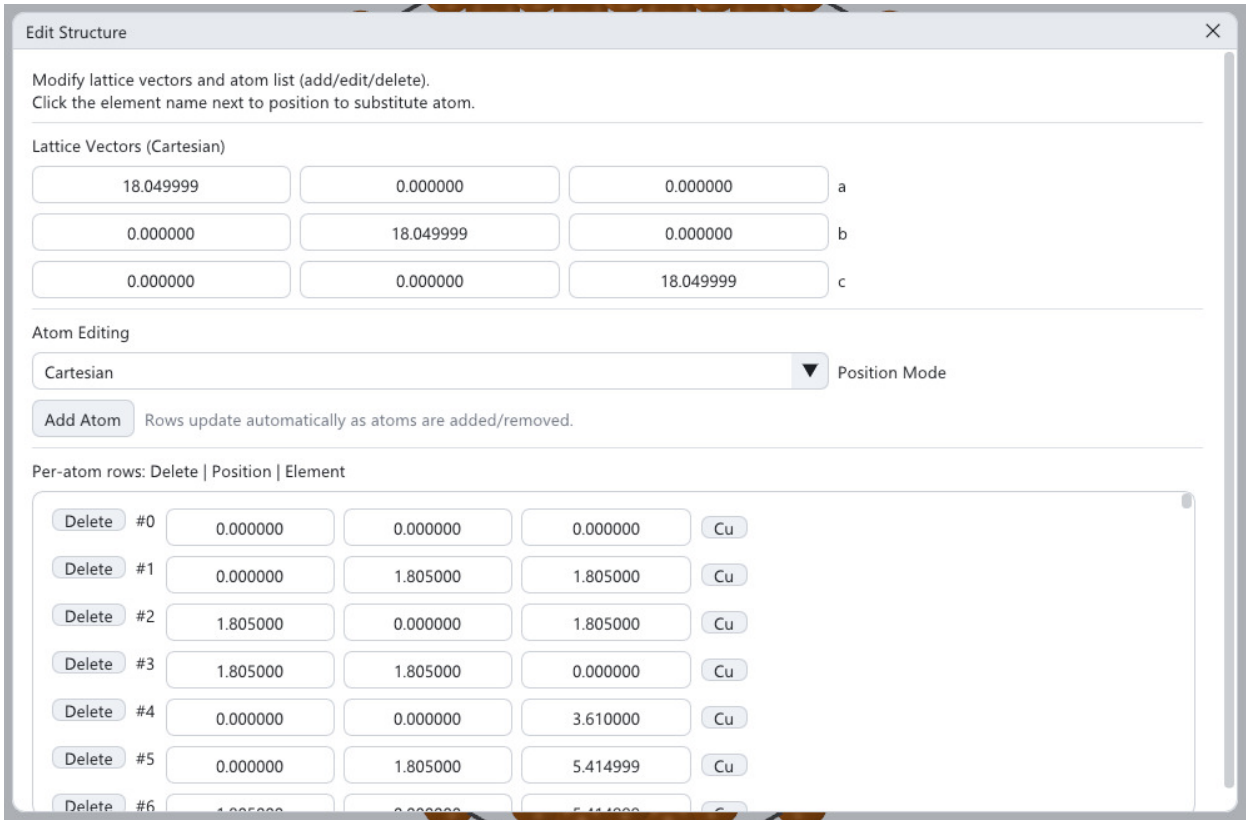
1. Open **Atom Info** from View or the right-click menu. Read Cartesian and Direct coordinates and neighbour statistics.
2. Close the dialog, select two neighbouring displayed sites, and choose **Measure Distance**.
3. For an angle, select three sites in the intended order and inspect which site is the vertex.

**Check the result.** The ideal FCC nearest-neighbour distance is $3.61/\sqrt{2} \approx 2.5527$ Å. Displayed periodic images affect the measurement geometry.

**Try the options.** Use Distance/Angle on the toolbar for the same measurement workflow. Clear selection before starting a different measurement to avoid accidentally retaining extra sites.

### 10.4.4 Move, add, delete, or relabel an atom

**Start here.** Save a working copy, then open **Edit > Edit Structure**.




*Edit Structure shows Cartesian lattice vectors and per-atom rows for the 500-site host.*

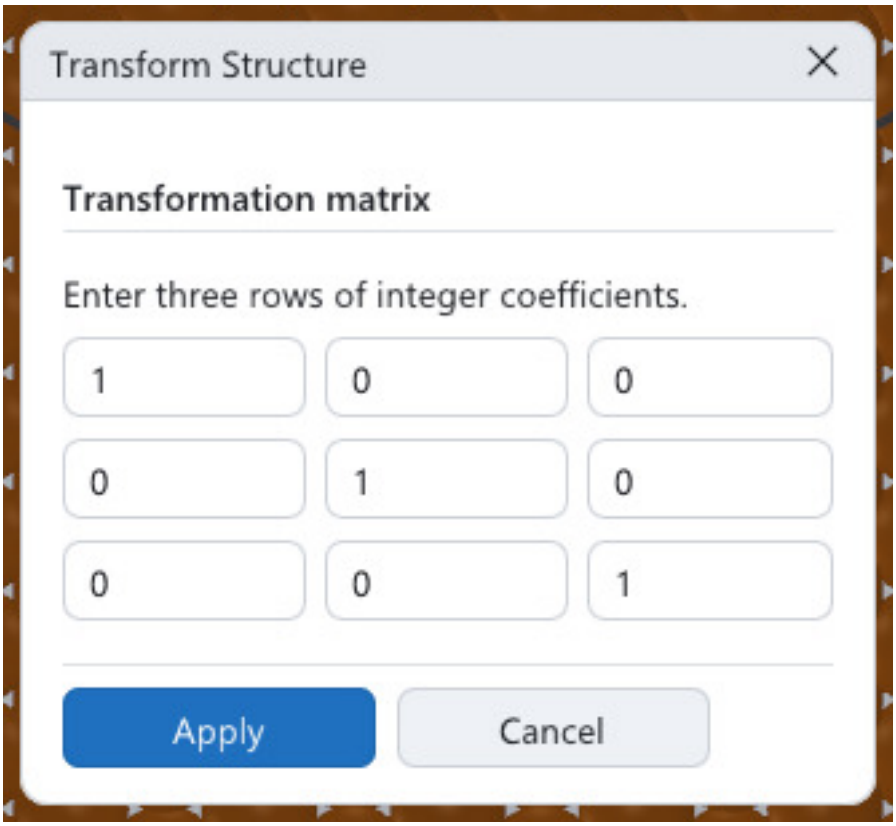
1. Choose Cartesian mode and change one atom's X coordinate by 0.1 Å. Switch to Direct to inspect the same position in fractional form.
2. Use **Add Atom**; set the initial H row to the intended element and a non-overlapping position. Click a row element to substitute that row.
3. Delete the trial row if unwanted. Close the dialog and inspect coordinates and composition before saving.

**Check the result.** Coordinate edits apply immediately to the active structure. Switching coordinate mode alone does not move an atom.

**Try the options.** Cell-vector edits change the cell but do not rescale Cartesian atom positions. Direct mode requires a cell. Check row indices after deletion because numbering can change.

### 10.4.5 Make a periodic supercell

**Start here.** Open `examples/cu_fcc.cif`, then **Edit** > **Transform Structure**.




*Transform Structure starts with the identity matrix. Enter all nine coefficients deliberately.*

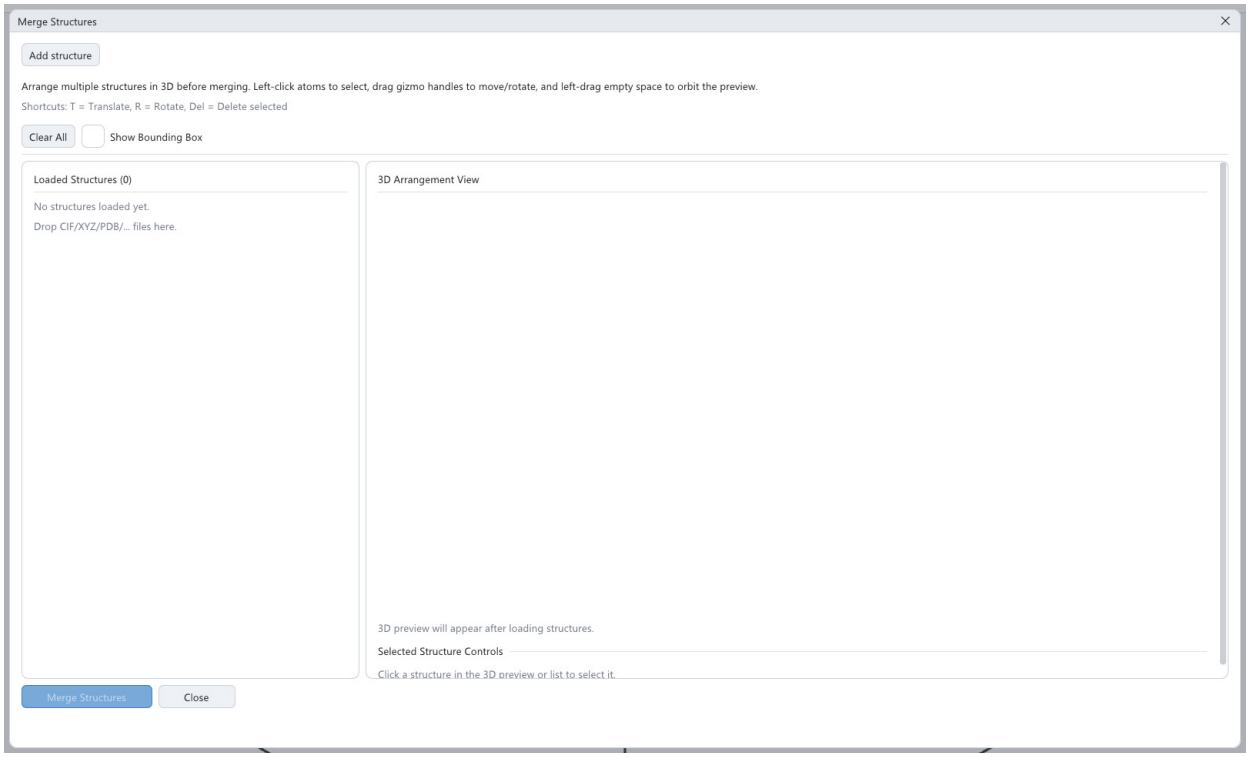
1. Enter rows $(2, 0, 0)$, $(0, 2, 0)$, and $(0, 0, 2)$.
2. Choose **Apply**; inspect cell lengths, volume, and atom count.
3. Save under a new filename. Reopen the reference before comparing a different transform.

**Check the result.** Expect 32 atoms and eight times the four-atom reference volume. Diagonal $(2, 1, 1)$ instead gives eight atoms and twice the reference volume.

**Try the options.** Non-diagonal integer matrices mix lattice directions. A zero determinant is invalid; negative determinant reverses handedness. Cancel discards the pending matrix. This tool requires a periodic cell.

### 10.4.6 Assemble two separated particles

**Start here.** Use two copies of `examples/cu_sphere.xyz`. Open **Edit** > **Merge Structures**.




*Merge Structures before components are dropped. Selecting a loaded component reveals its position and rotation controls below the preview.*

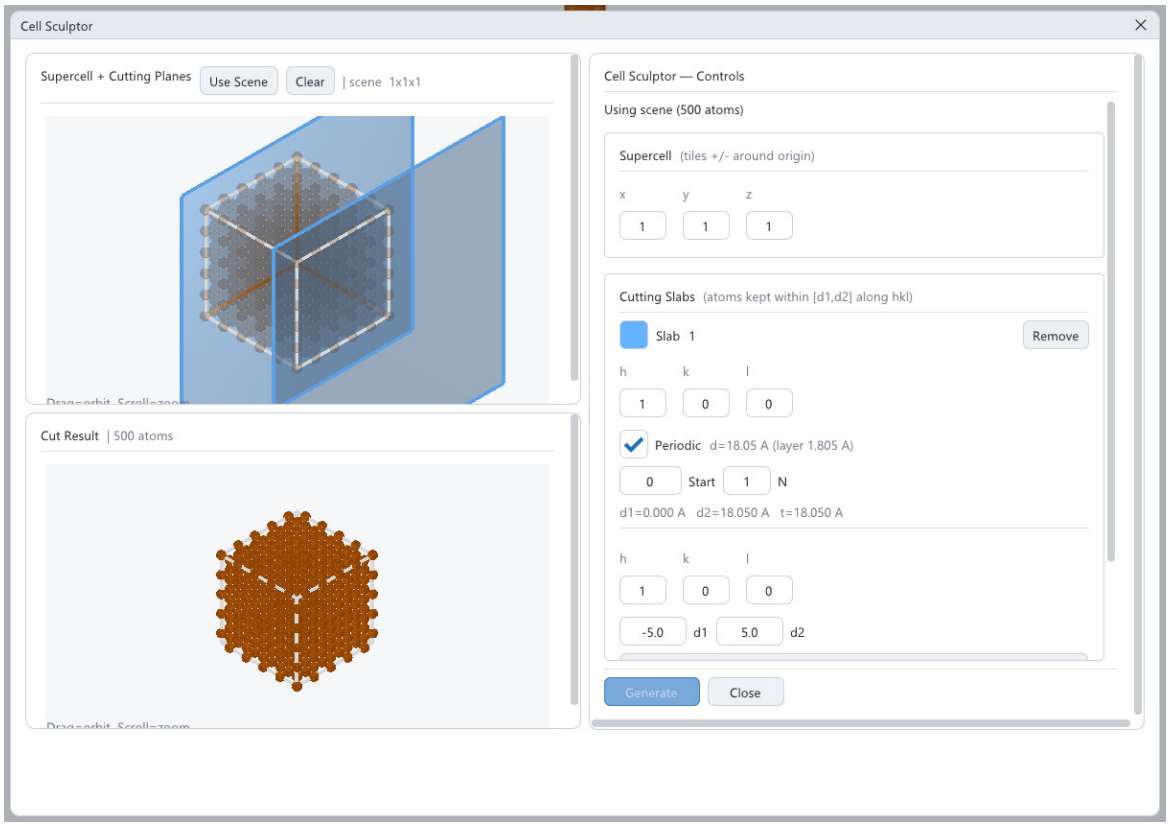
1. Drop both components, select the second in the list, and enter a +25 Angstrom X translation with zero Y/Z and zero rotation.
2. Orbit the arrangement and use Show Bounding Box to inspect the gap. Translate/rotate handles and the T/R shortcuts provide coarse adjustment; numeric fields give repeatable values.
3. Choose Merge Structures and save the new output tab.

**Check the result.** Two 336-atom inputs should produce 672 atoms. The merger does not remove overlaps and clears the periodic cell and grain metadata.

**Try the options.** Reset restores the selected component transform; Remove/Delete Selected removes one component; Clear All empties the assembly. These actions do not remove source files. Do not expect cell-dependent tools to work on the nonperiodic merged result.

### 10.4.7 Preview a crystallographic slab

**Start here.** Open `examples/cu_host.cif`, choose **Edit > Cell Sculptor**, and **Use Scene**.




*A loaded 500-site host with one periodic (100) slab. Generate is disabled because one normal does not enclose a 3D cell.*

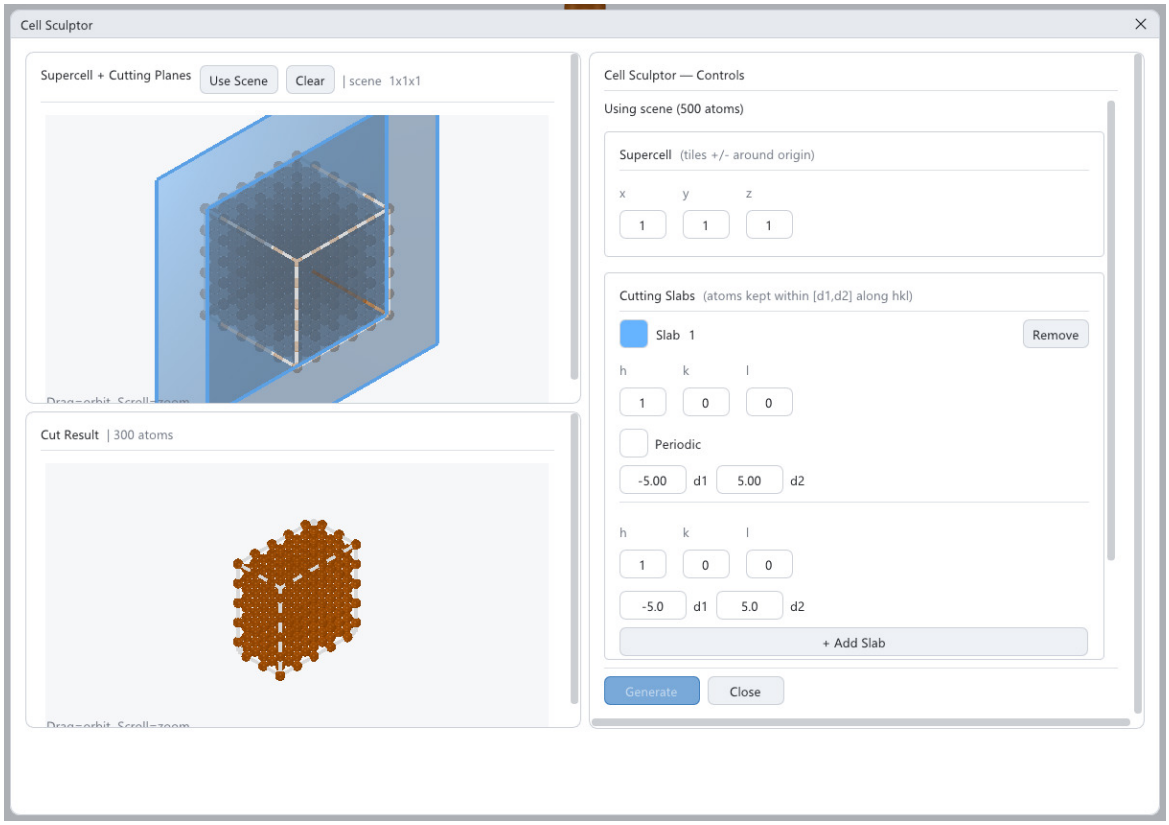
1. Keep source supercell counts at 1. Add a (100) slab and keep Periodic enabled, Start 0, N 1.
2. Inspect the two translucent bounding planes and the cut preview.
3. Add independent (010) and (001) slabs with finite intervals to close the region, then inspect the Generate status.

**Check the result.** Three independent normal directions can close a region; three parallel slabs cannot. The preview can be nonempty even while Generate is disabled.

**Try the options.** Change Start to shift a periodic interval or N to change its thickness. Increase source repetitions when the cuts extend beyond available sites. Clear removes the source; Remove deletes a slab constraint.

### 10.4.8 Cut with explicit slab distances

**Start here.** Use the loaded Cell Sculptor example and retain the source file.




*Disabling Periodic exposes distance limits. The shown single (100) interval retains a 300-atom preview but is still unbounded mathematically.*

1. Disable Periodic on a slab and enter $d_1 = -5$, $d_2 = 5$ Å. Inspect the changed retained count.
2. Add independent lateral bounds for a closed volume. Narrow one interval to make a thin specimen.
3. Adjust outside ghost alpha for clarity, Generate the valid closed result, and save the changed current structure.

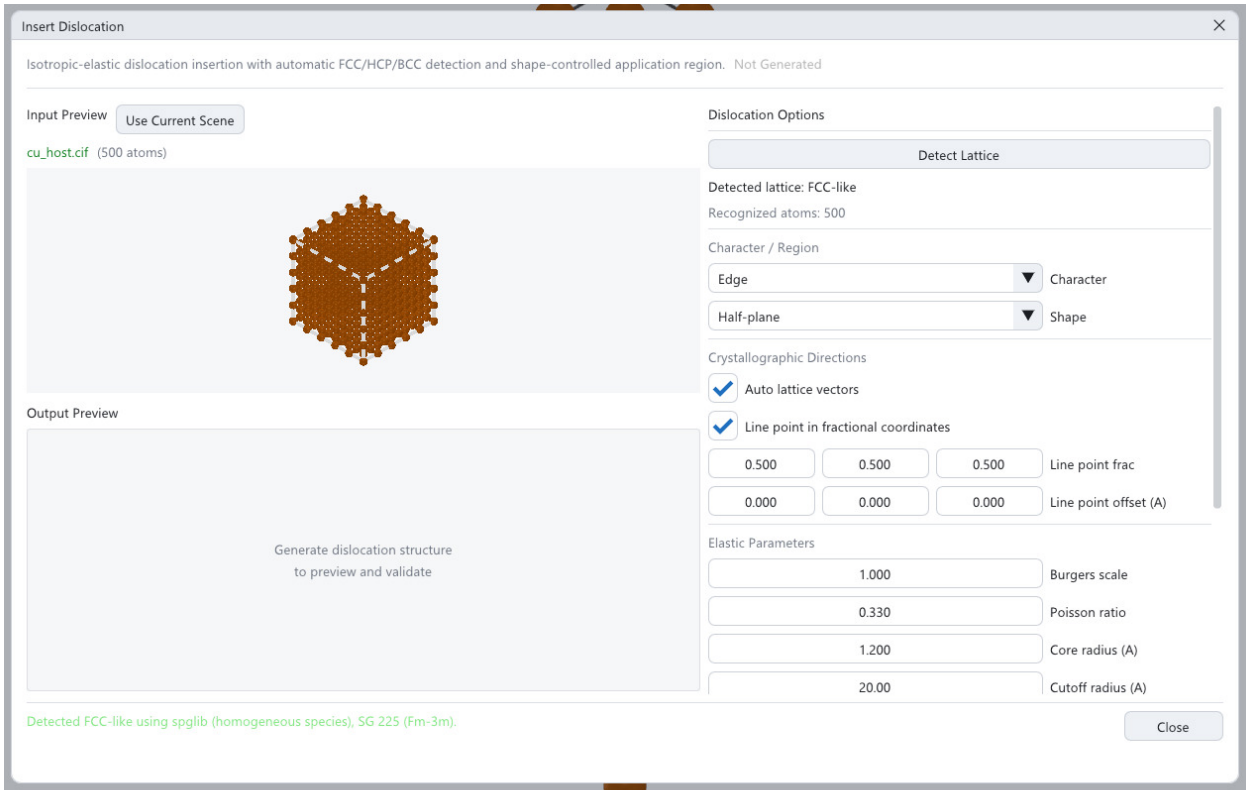
**Check the result.** The distance limits select atoms; ghost opacity only changes the guide display. Generate commits and closes; Close alone does not commit.

**Try the options.** A slab normal is reciprocal-lattice based. In a skew cell its (hkl) normal differs from the corresponding direct-lattice direction. Check which side is retained before committing.

## 10.5 Edit: dislocations and interstitials

### 10.5.1 Configure a dislocation preview

**Start here.** Open `examples/cu_host.cif`, choose **Edit > Insert Dislocation**, and **Use Current Scene**.




*The copper host has been copied and detected as FCC-like. Upper settings define character, line position, and elastic inputs.*

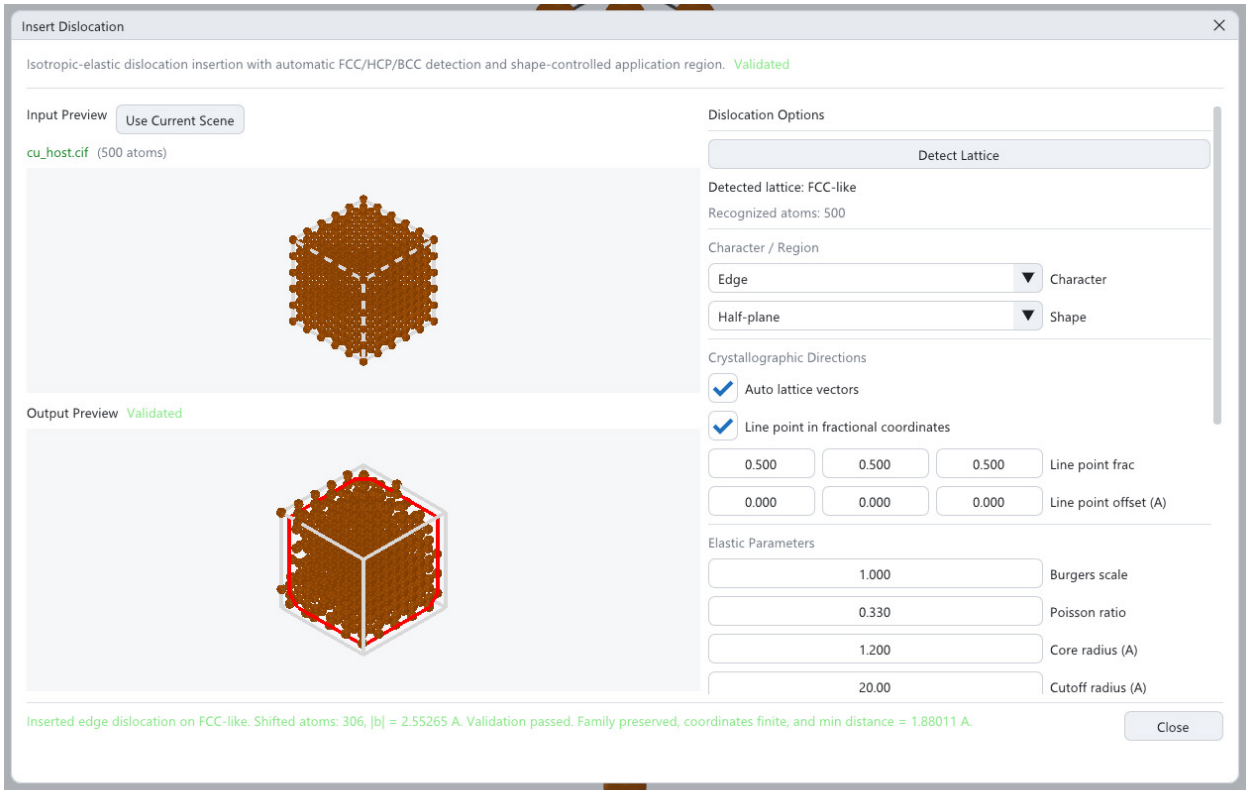
1. Inspect Detect Lattice. Keep Edge, Half-plane, automatic lattice vectors, fractional line point $(0.5, 0.5, 0.5)$, and zero offset.
2. Use Burgers scale 1, Poisson ratio 0.33, Core radius 1.2 Å, Cutoff radius 20 Å, and the displayed long line half length.
3. Scroll to Generate Dislocation and calculate the preview.

**Check the result.** Generation alone does not replace the main structure. Inspect the output geometry and validation summary first.

**Try the options.** Screw changes character; Mixed exposes an angle from 0 to 90 degrees. Disable Auto lattice vectors to enter Plane, Burgers, and Line triplets. Fractional point and Cartesian offset use different units.

### 10.5.2 Restrict the displaced region

**Start here.** Use the preceding dislocation inputs; save the original host separately.




*Generated half-plane preview. The status line reports the validation (family preserved, coordinates finite, minimum distance); a warning there must not be ignored.*

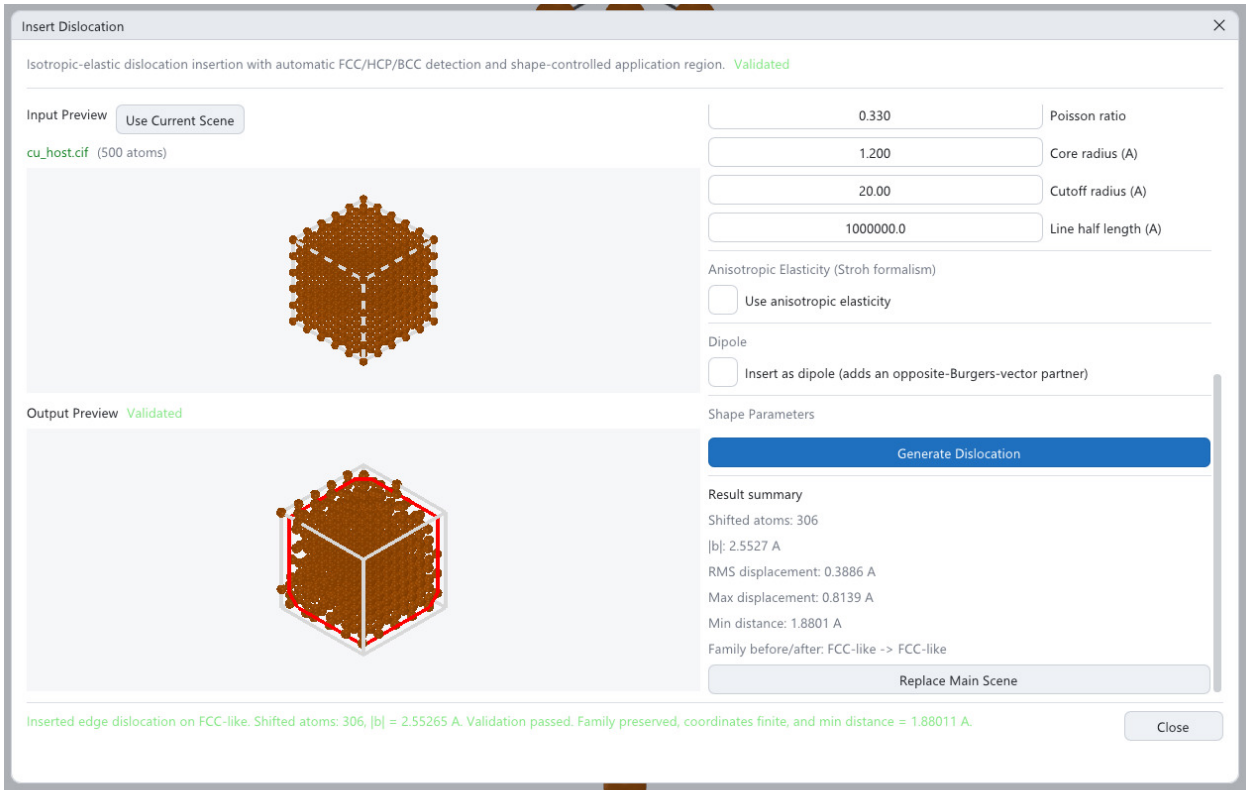
1. Change Shape to Cylinder and its radius to 5 Angstrom; set cutoff to 8 Angstrom. Generate a fresh preview.
2. Compare Sphere with its radius, then Ellipsoid with three radii. These are alternative spatial restrictions, not changes to the host composition.
3. For Freeform 2D, Add Point for each local polygon vertex, edit its coordinates, and Remove Last to correct the outline. Use a simple closed polygon without self-intersections.

**Check the result.** The supplied native cylindrical example `examples/cu_dislocation.cif` preserves 500 atoms and reports 136 shifted sites. The screenshot uses the different half-plane settings, with 306 shifted sites.

**Try the options.** Core radius regularises near-line displacement. Cutoff attenuates radially; zero disables attenuation. Line half length limits axial extent. Regenerate after changing any of these values and compare the minimum distance.

### 10.5.3 Review and commit a defect

**Start here.** Generate a dislocation preview, then scroll to Result summary.




*Validation and Replace Main Scene are below Generate in the settings panel.*

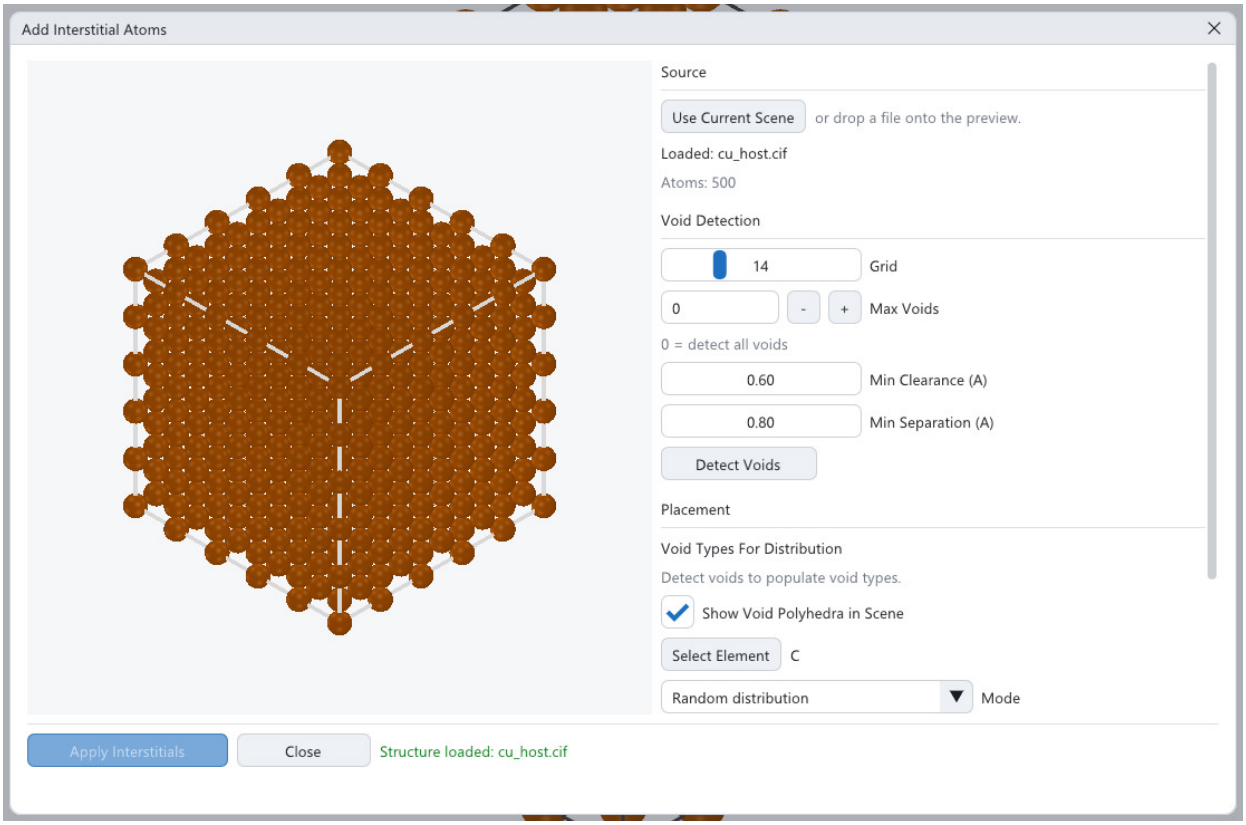
1. Read shifted atoms, Burgers magnitude, RMS and maximum displacement, minimum distance, and family before/after.
2. For the shown half-plane trial, note the FCC-like to HCP-like warning. Treat it as a starting geometry requiring review, not a relaxed defect.
3. Only when the geometry is the intended trial, choose **Replace Main Scene**. Save the result under a new name.

**Check the result.** The shown Burgers magnitude is about 2.5527 Å; the minimum distance is about 1.8801 Å. These describe this trial, not every character or region.

**Try the options.** Close leaves the uncommitted source unchanged. A numerical Burgers scale is not automatically a physically allowed perfect-dislocation vector. Compare core contacts and external relaxation before a physical study.

### 10.5.4 Find candidate interstitial sites

**Start here.** Open `examples/cu_host.cif`, then **Edit > Add Interstitial Atoms > Use Current Scene**.




*Void-detection controls with the 500-atom host loaded.*

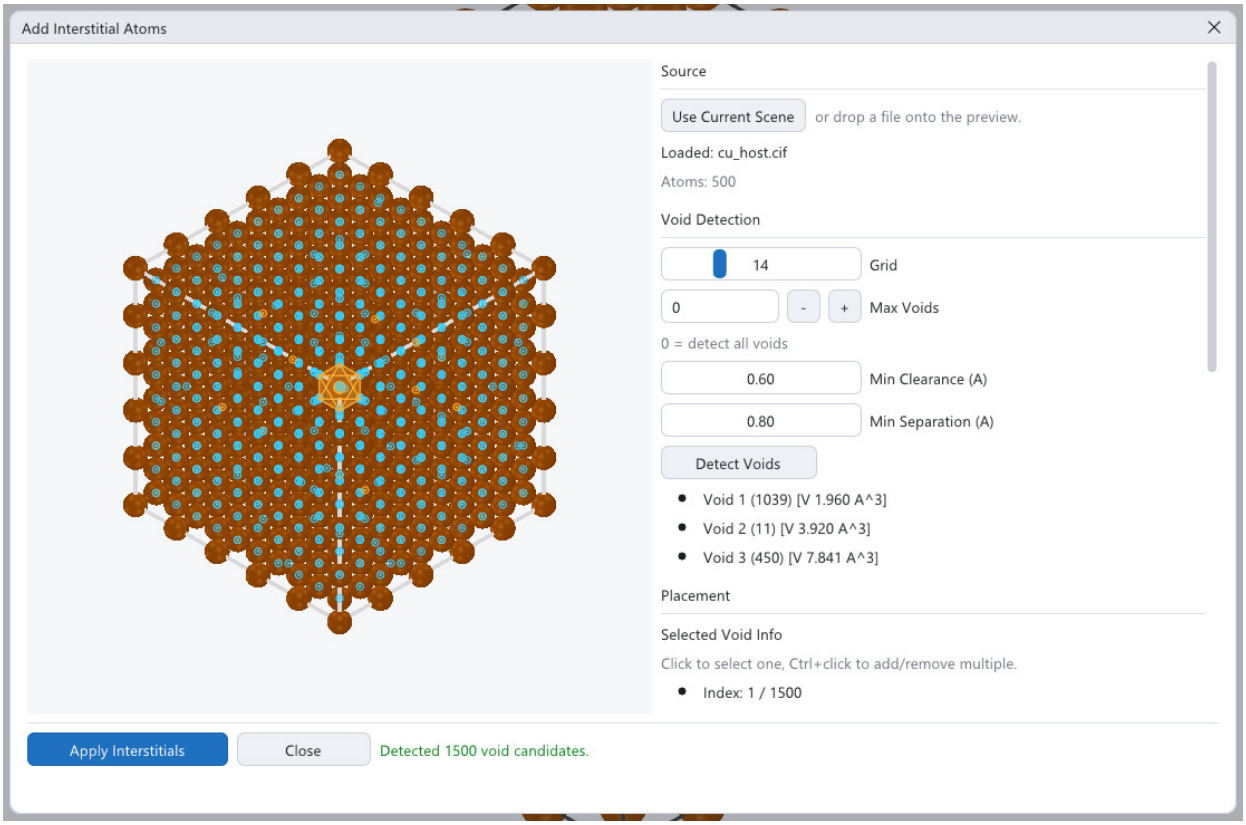
1. Set Grid 14, Max Voids 0, Min Clearance 0.60 Å, and Min Separation 0.80 Å.
2. Choose **Detect Voids**. Inspect the marker cloud, type list, and selected-void information.
3. Repeat with another grid resolution to assess whether the candidate set is stable.

**Check the result.** The captured run found 1,500 candidate voids. This is a search result, not 1,500 atoms already inserted.

**Try the options.** Max Voids 0 means retain all detected candidates. Clearance measures space around a candidate; separation deduplicates nearby candidates. Increasing Grid increases work and can change classifications.

### 10.5.5 Inspect and filter candidate environments

**Start here.** Run Detect Voids on the copper host.




*Detected void markers and type counts. Selected-void details appear further down the panel.*

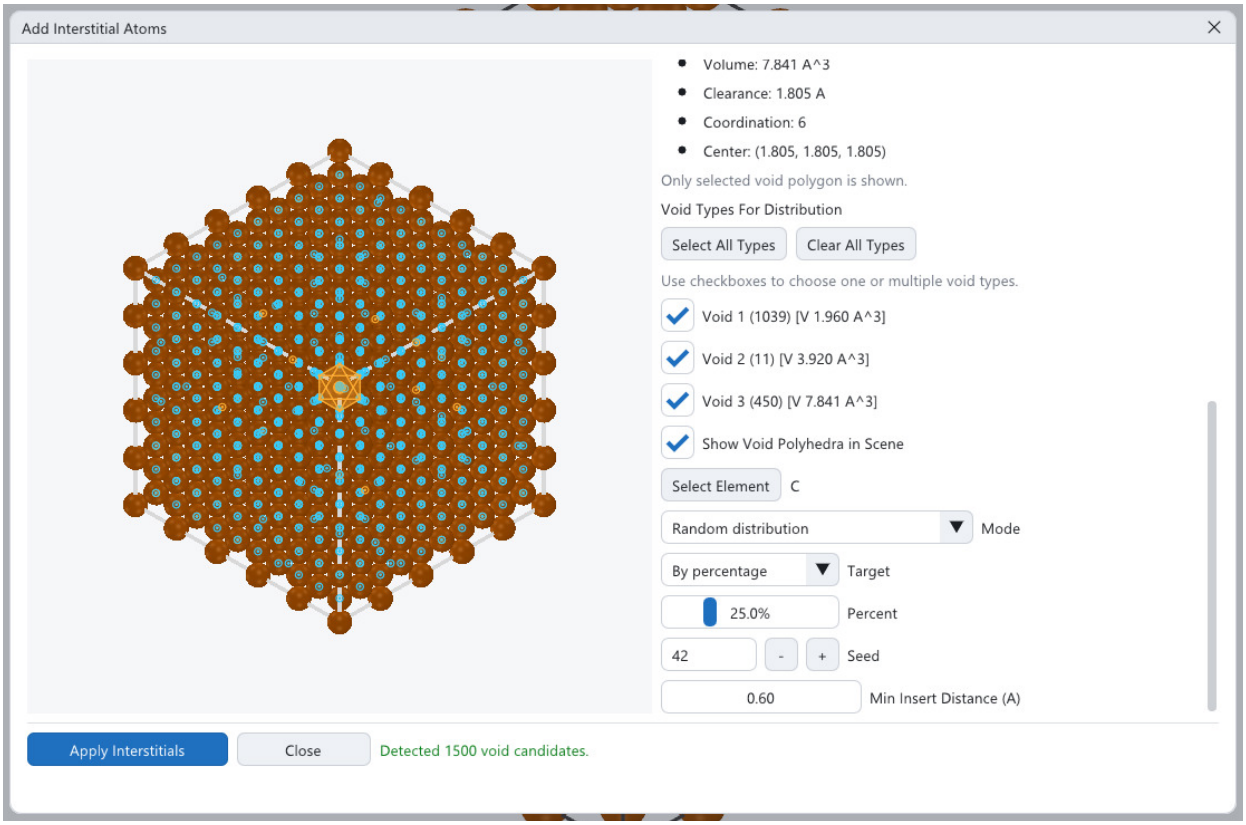
1. Click a candidate centre to inspect it. Ctrl+click adds or removes centres from the selection.
2. Read its type, coordination, volume, clearance, and centre coordinates. Toggle Show Void Polyhedra to compare the environment with and without its guide.
3. Use Select All Types, Clear All Types, and individual type checkboxes to choose which classes are eligible for distribution.

**Check the result.** A highlighted void is a candidate location, not an inserted atom. The selected-centre count and the type-filtered pool are different quantities.

**Try the options.** Record the detection parameters with any candidate count. A coarse search can split or merge geometrically similar types, so do not assign chemistry from a displayed type label alone.

### 10.5.6 Insert a small random population

**Start here.** Use the detected host and choose the desired candidate types.




*Placement controls after detection, including type filters, element selection, and percentage target.*

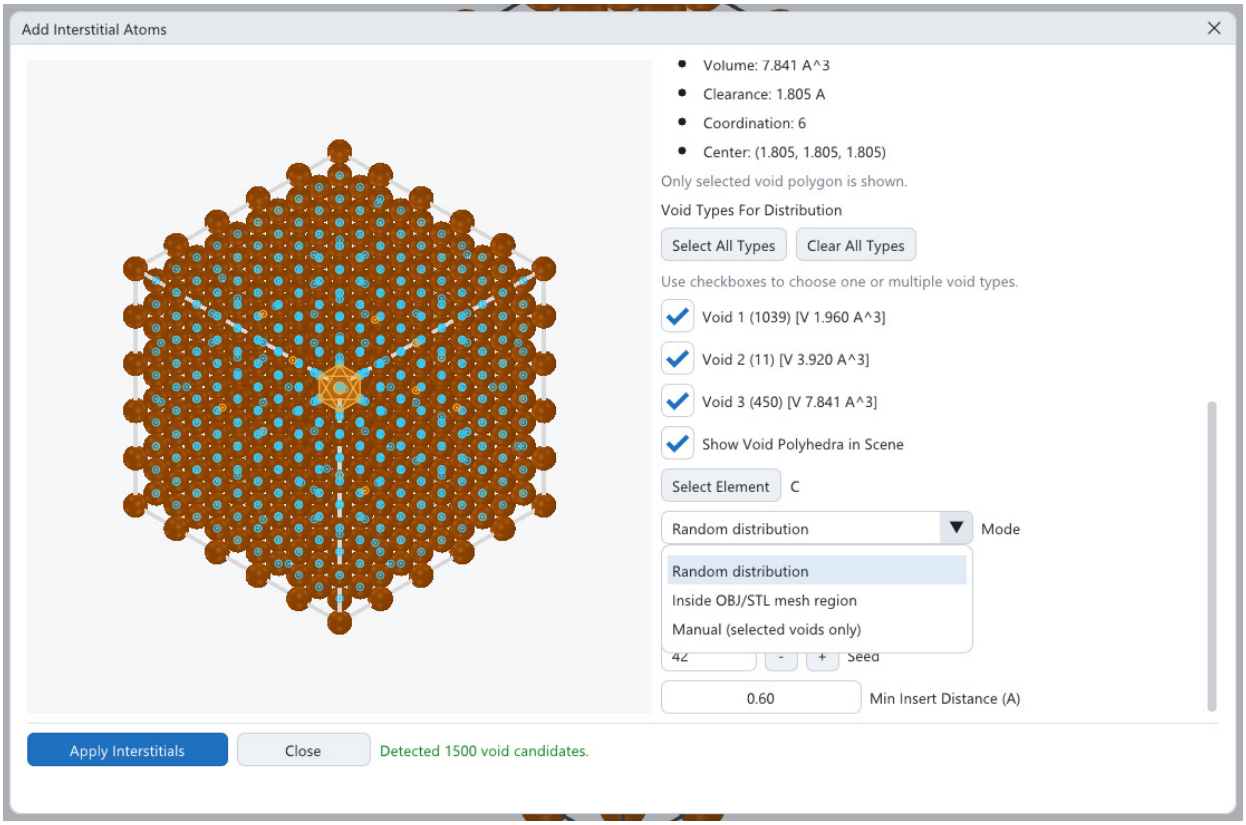
1. Choose **Select Element**, select H, and use Random distribution.
2. Choose By atom count and request 5, set seed 42 and Min Insert Distance 0.60 Å.
3. Apply Interstitials, then inspect the reported added count and the current structure composition.

**Check the result.** If all five trial sites pass constraints, the structure has 505 atoms; otherwise use the actual reported number. Rejected candidates are not necessarily replaced from the rest of the pool.

**Try the options.** By percentage refers to eligible voids, not final atomic percent. Seed zero is deterministic in this tool. Apply updates the current structure and tool source; detect again after substantial host changes.

### 10.5.7 Insert only at selected centres

**Start here.** Detect voids and choose the intended inserted element.




*The placement menu offers Random, Inside OBJ/STL, and Manual modes.*

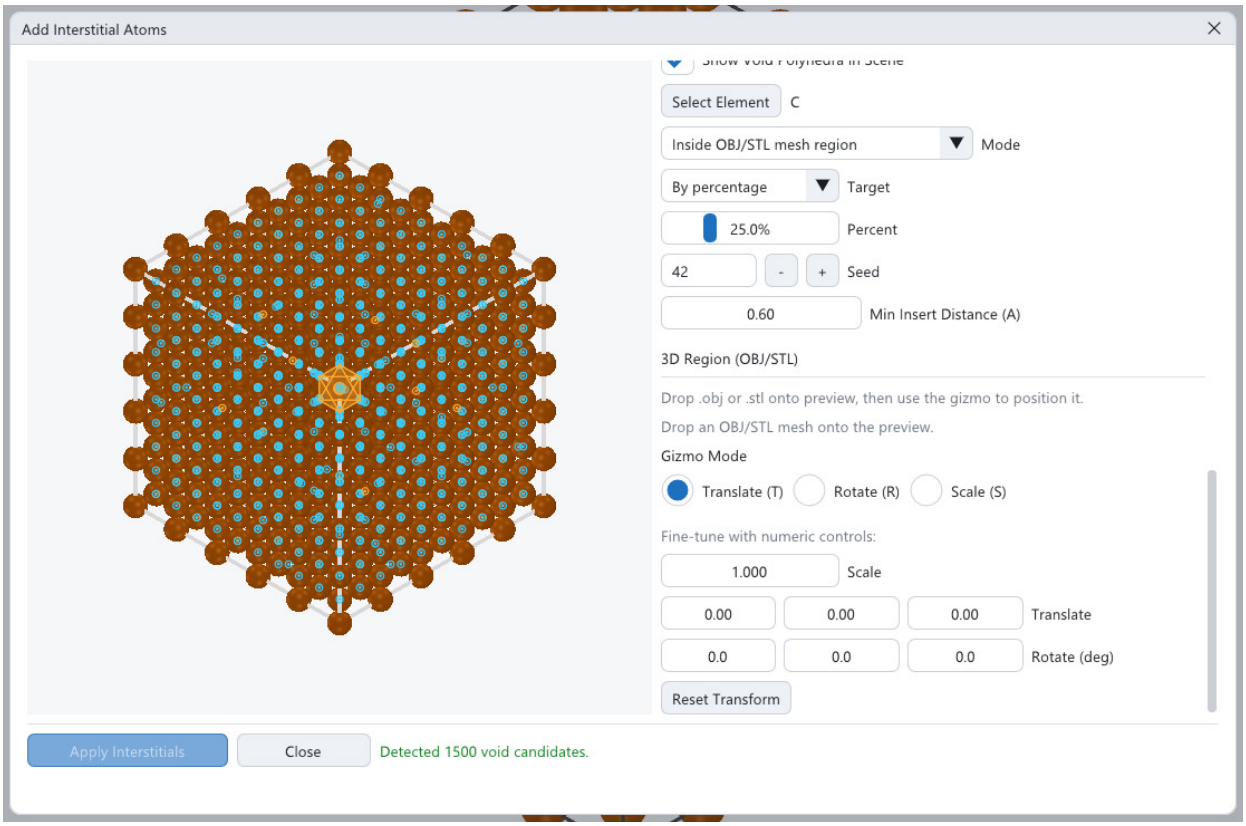
1. Select **Manual (selected voids only)**.
2. Click one candidate centre; Ctrl+click to extend or reduce the set. Check the selected-centre count and clearance.
3. Apply Interstitials and inspect the exact new positions and added count.

**Check the result.** Manual mode requires selected centres. It does not use the percentage/count target as a replacement for the selection.

**Try the options.** Use the minimum insertion distance to reject overlaps. Clear or adjust selection if Apply is disabled. Hiding the guide polyhedra does not deselect the centres.

### 10.5.8 Confine insertion to an OBJ/STL region

**Start here.** Detect voids in the host and prepare the closed `examples/octahedron.obj`.




*Mesh-region controls before a mesh is dropped; Apply is disabled until the required region is available.*

1. Select Inside OBJ/STL mesh region, then drop the mesh onto the preview.
2. Set Scale 5 and translate the region near the host centre; use T, R, S gizmo modes or numeric translation/rotation to align it.
3. Choose a small atom-count target, fixed seed, and minimum insertion distance. Apply and inspect where atoms were added.

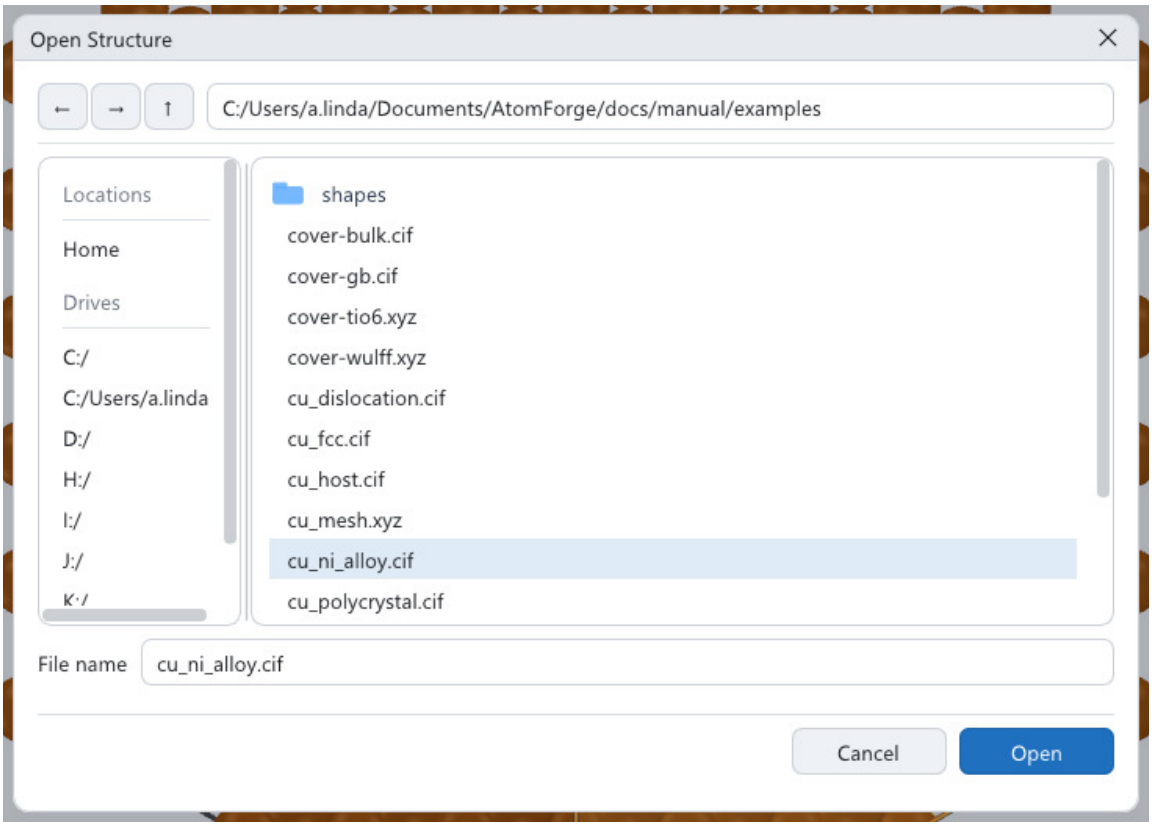
**Check the result.** Eligible candidates must pass the type filters and lie inside the transformed mesh. An empty or misplaced mesh can yield no insertions.

**Try the options.** Reset Transform restores unit scale and zero translation/rotation. The mesh limits candidate selection; it does not cut the existing host atoms. Use Cell Sculptor for an atomic cut.

## 10.6 File, View, and Settings options

### 10.6.1 Open, save, convert, and manage tabs

**Start here.** Locate this manual extquotesingle s examples folder on your machine.




*The real structure browser with a generated example selected. The filename can be selected from the listing.*

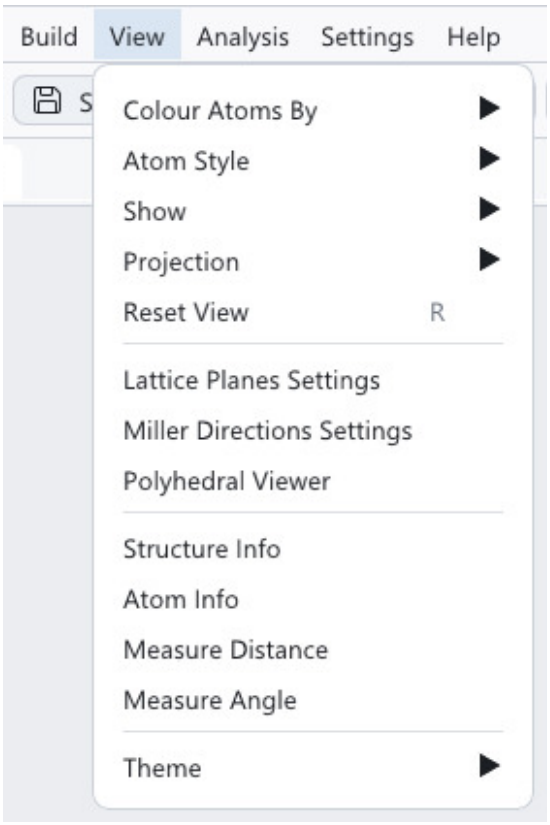
1. Choose **File > Open**, browse to the examples folder, select `cu_host.cif`, and Load. Double-clicking a listed structure also opens it.
2. Use **Save As** for a separate working copy. Select a format, choose a matching extension, and confirm replacement only for the intended destination.
3. Open another example in a second tab. Select the intended tab before an edit or save, and close a tab after saving required changes.

**Check the result.** Opening a structure should show its tab and the expected atom count. Converting to a format that lacks cell/metadata can lose information; reopen to check.

**Try the options.** Back/Forward traverse folder history; Up opens the parent; the folder field accepts a directory on Enter. File > Open Recent revisits inputs. Ctrl+S saves, Ctrl+Shift+S opens Save As, Ctrl+W closes. Drag/drop feeds an open tool when its drop workflow is active.

### 10.6.2 Orient and frame the structure

**Start here.** Open `examples/cu_host.cif`.




*View menu: projection, overlays, information, measurements, and Reset View are separate commands.*

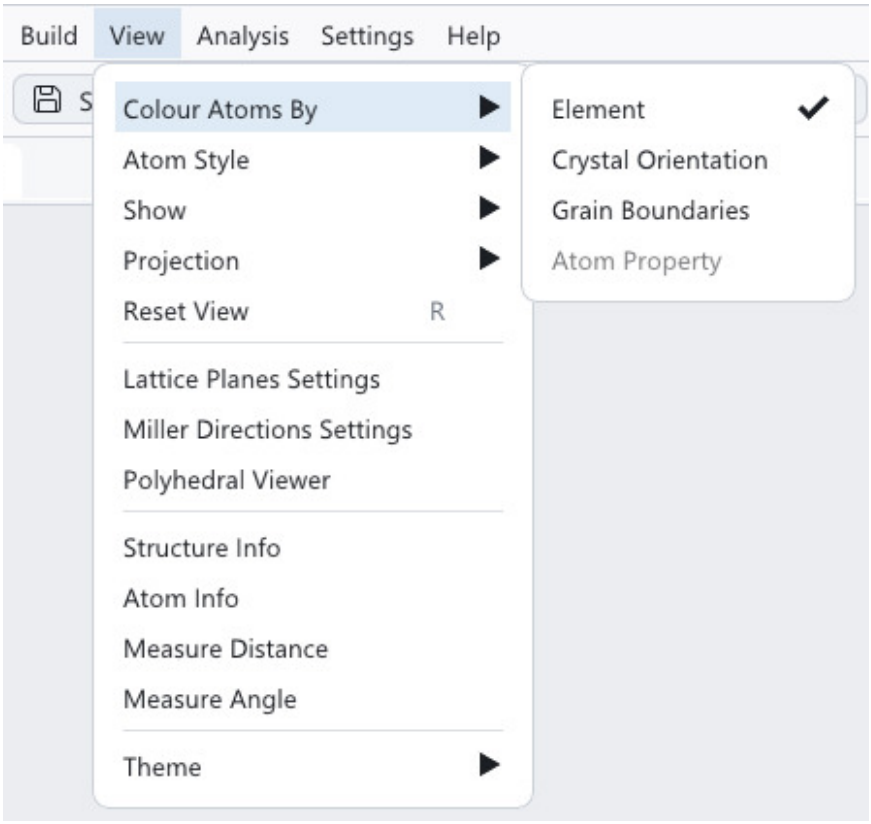
1. Use toolbar X, Y, Z to view Cartesian axes, then a, b, c to compare lattice directions.
2. Left-drag to orbit, right-drag to pan when no selection mode is active, and use the wheel to zoom.
3. Set the toolbar rotation increment to 5 degrees and click an axis rotation button. Compare the Perspective and Orthographic projections (View > Projection), then Reset View.

**Check the result.** Coordinates and atom count must remain unchanged. These controls move the camera, even where internal code calls them Rotate Crystal.

**Try the options.** View > Show > Element Labels toggles labels; View > Show > Atoms and Bounding Box independently toggle geometry visibility. Hidden atoms remain in the structure and numerical analyses. Turn off Box/Lasso selection to restore right-drag panning.

### 10.6.3 Choose structure colouring

**Start here.** Open `examples/cu_ni_alloy.cif` for element colour, or the supplied polycrystal with its orientation sidecar.




*View > Colour Atoms By exposes element and metadata-based presentations.*

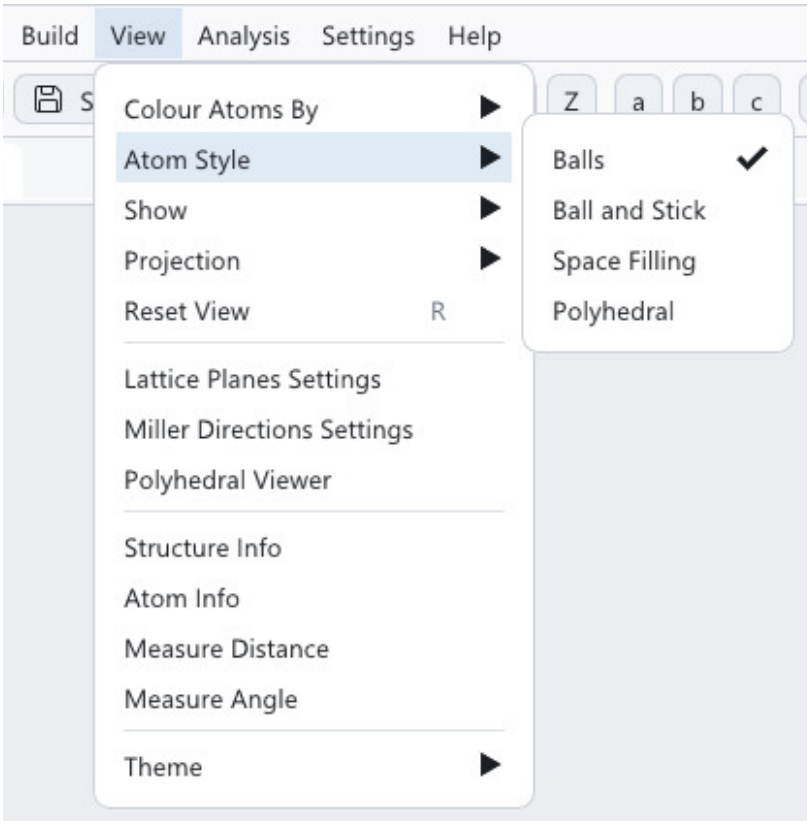
1. Choose Element Type to distinguish species. Confirm Cu and Ni with Structure Info.
2. Choose Crystal Orientation on a structure with orientation information; compare Grain Boundary on a suitable grain-labelled result.
3. For a generated defect with retained line information, turn on View > Show > Dislocation Lines.

**Check the result.** Changing colouring does not create missing grain or dislocation metadata. Keep source sidecars with files when required.

**Try the options.** Select Element Type again to return to chemistry colours. The Colours settings palette and the electronic density palette are independent.

### 10.6.4 Compare atom representations and overlays

**Start here.** Open a small periodic example to keep the comparison readable.




*View > Atom Style: Balls, Ball and Stick, Space Filling, and Polyhedral.*

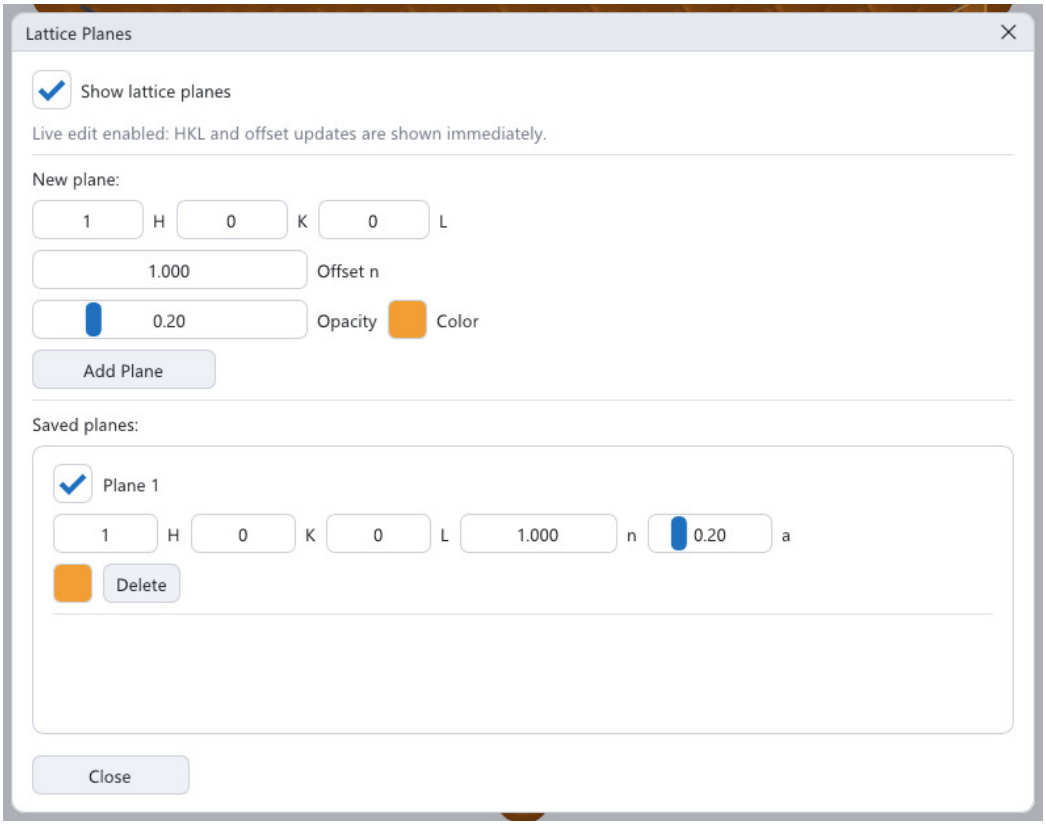
1. Switch between Balls, Ball and Stick, and Space Filling. Inspect the same bond/void region after each switch.
2. Choose Polyhedral and configure Settings > Polyhedral Settings. Also inspect the separate Polyhedral Viewer overlay where available.
3. Toggle View > Show > Voronoi Volumes to inspect the available Voronoi geometry, then return to the original representation.

**Check the result.** The atom list is unchanged. Geometric bonds and rendered occupied space are not electronic bonding or a measured charge-density boundary.

**Try the options.** Restrict centres/neighbours for a readable polyhedral view. Zero opacity, hidden atoms, or restrictive filters can make a configured representation appear empty.

### 10.6.5 Add and manage lattice-plane guides

**Start here.** Open the copper host, then **View > Lattice Planes**.




*New-plane controls and an editable saved (100) row. Global and row visibility are both enabled.*

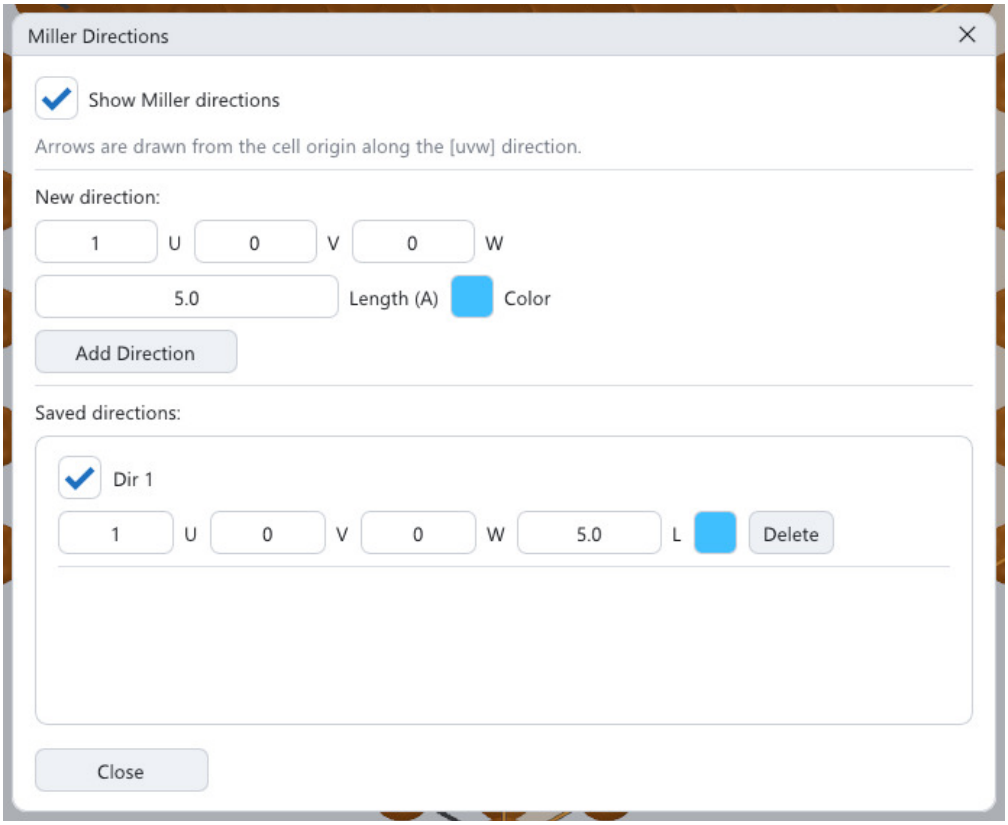
1. Enter (100), Offset $n = 1$, Opacity 0.2, and a colour; choose Add Plane.
2. Add (110) in a contrasting colour. Close the dialog and orbit to compare the guides.
3. Reopen the dialog. Toggle a row, change its offset, then Delete it to compare the effects.

**Check the result.** Both global Show lattice planes and the row visibility must be on. These guides do not remove atoms or calculate a density section.

**Try the options.** Use opacity 0 to hide faces and 1 for opaque guides. Change a colour swatch for a different guide colour. Plane offsets are not the fractional point control used by electronic arbitrary sections.

### 10.6.6 Draw crystallographic directions

**Start here.** Open **View > Miller Directions** on a structure with a cell.




*Miller Directions with a saved [100] vector of length 5 Angstrom.*

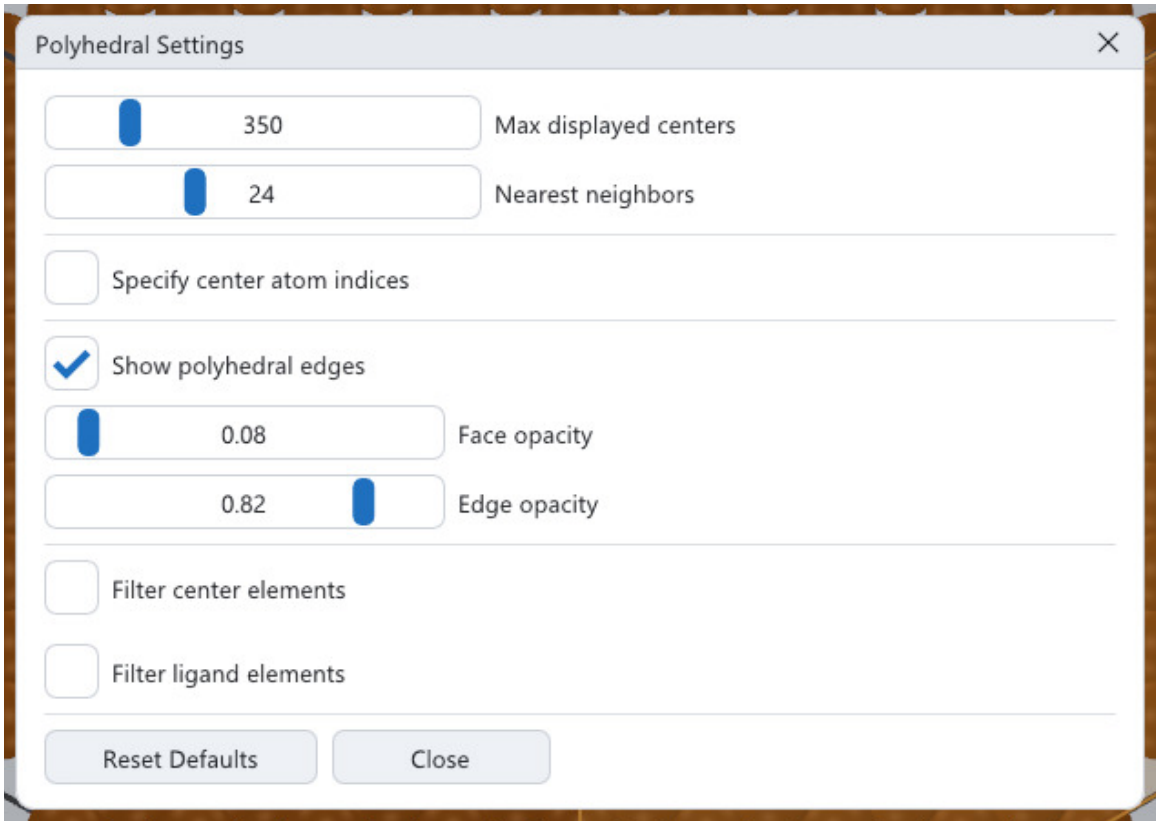
1. Enter [100], length 5 Å, and a colour, then Add Direction.
2. Add [110] with a second colour and compare both from an oblique view.
3. Edit a saved row extquotesingle s length or direction, toggle its visibility, and delete the trial row when finished.

**Check the result.** Length controls the drawing, not a lattice parameter or physical displacement. The direction must be nonzero.

**Try the options.** Global Show Miller directions controls the collection. In a skew cell, a direct-lattice direction and the corresponding reciprocal-plane normal generally differ.

### 10.6.7 Limit and style coordination polyhedra

**Start here.** Enable an appropriate polyhedral representation or overlay in View.




*Polyhedral Settings separates candidate counts, centre selection, element filters, and face/edge visibility.*

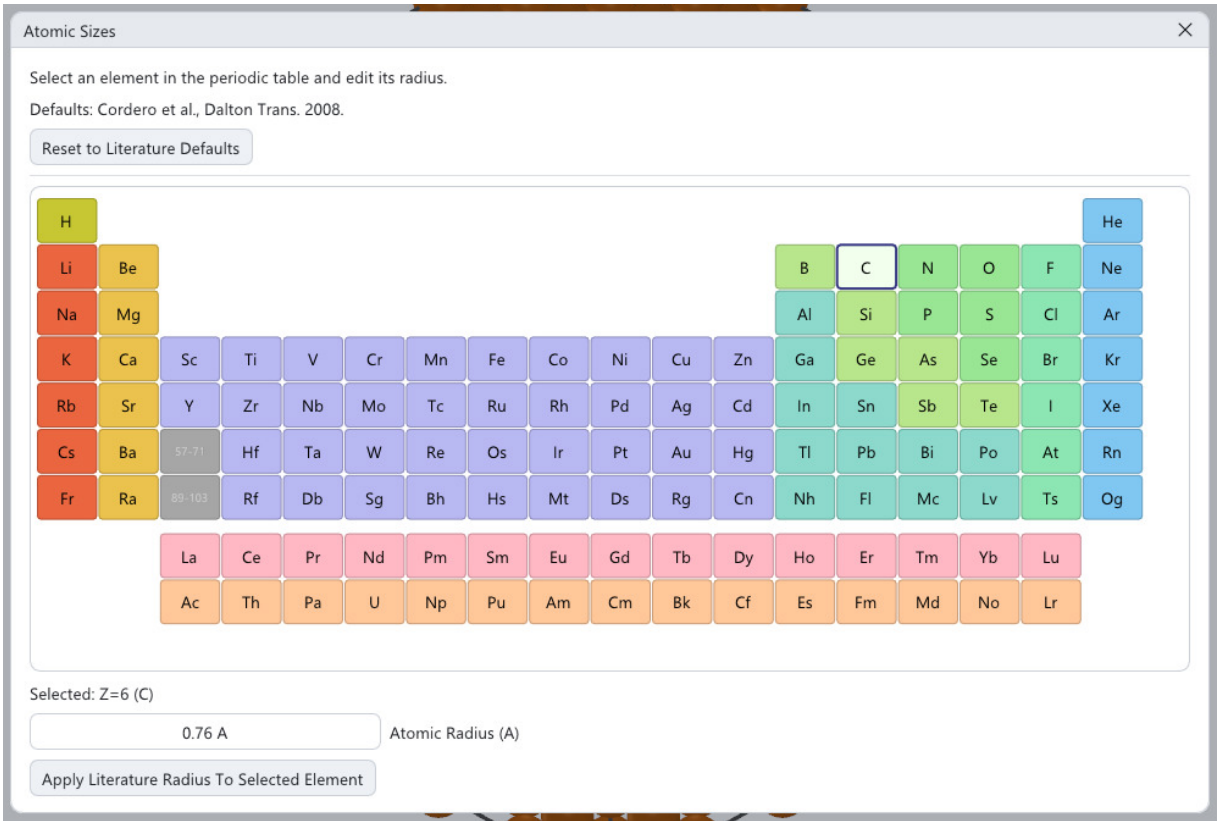
1. Start with Max displayed centers 10 and Nearest neighbors 12 on copper, or an appropriate neighbour count for another structure.
2. Enable Specify center atom indices and enter `1,4,10-20`. Inspect the parsed centre count.
3. Enable centre/ligand element filters for an oxide, choosing the metal as centres and O as ligands. Adjust face and edge opacity independently.

**Check the result.** A filter or centre cap intentionally limits displayed environments. It does not delete those atoms.

**Try the options.** GUI centre IDs are one-based and can change after edits. Neighbours ranges from 4 to 64 and centres from 1 to 2000. Reset Defaults clears filters and restores display values; Show polyhedral edges toggles edges separately.

### 10.6.8 Adjust an element radius

**Start here.** Open **Settings** > **Atomic Sizes** with a copper structure visible.




*Atomic Sizes: select a species first, then edit or restore its radius.*

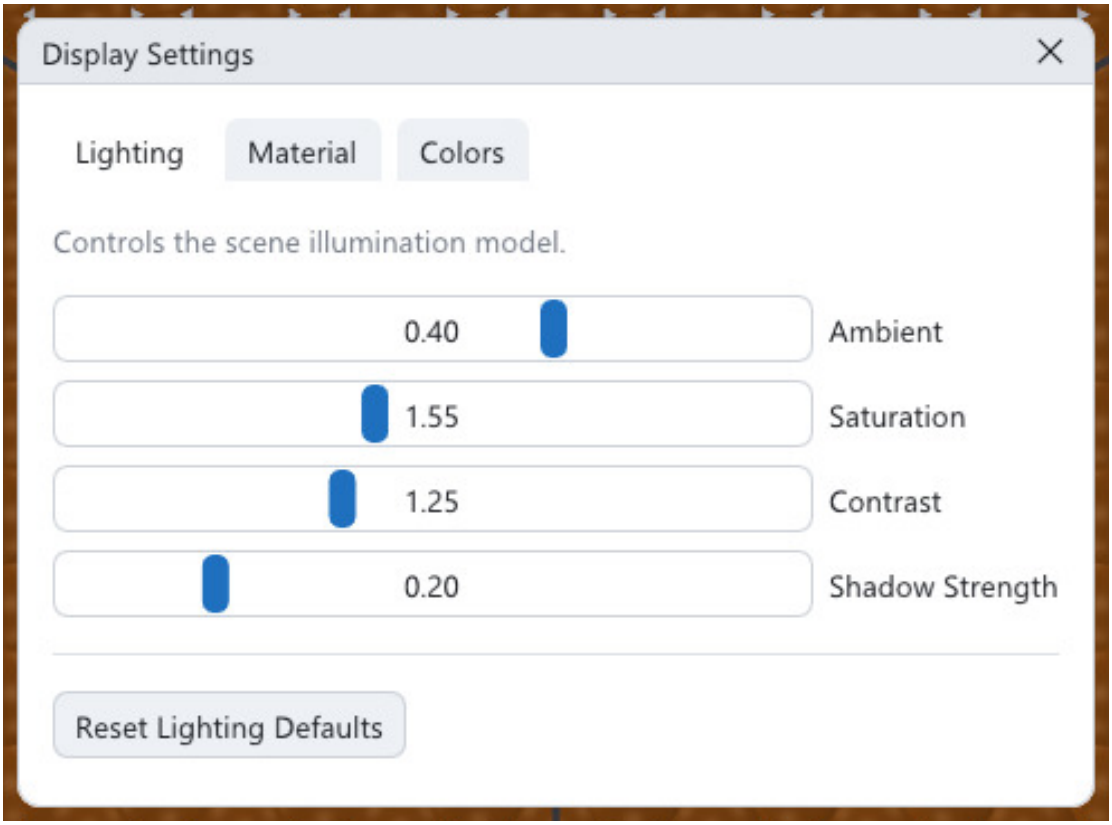
1. Select Cu in the periodic table and record its current radius.
2. Change the Atomic Radius to inspect how sphere size affects visibility of interstitial space.
3. Use Apply Literature Radius To Selected Element to restore Cu, or Reset to Literature Defaults for the full set.

**Check the result.** Atomic centres remain fixed. A displayed sphere boundary is not an electron-density isosurface.

**Try the options.** Record radius overrides in comparable images. Radius information can also be consumed by geometry tools, so restore intended values before reproducing their results.

### 10.6.9 Tune scene illumination

**Start here.** Open **Settings** > **Display Settings** > **Lighting**.




*Lighting controls at their default values; Reset Lighting Defaults restores them.*

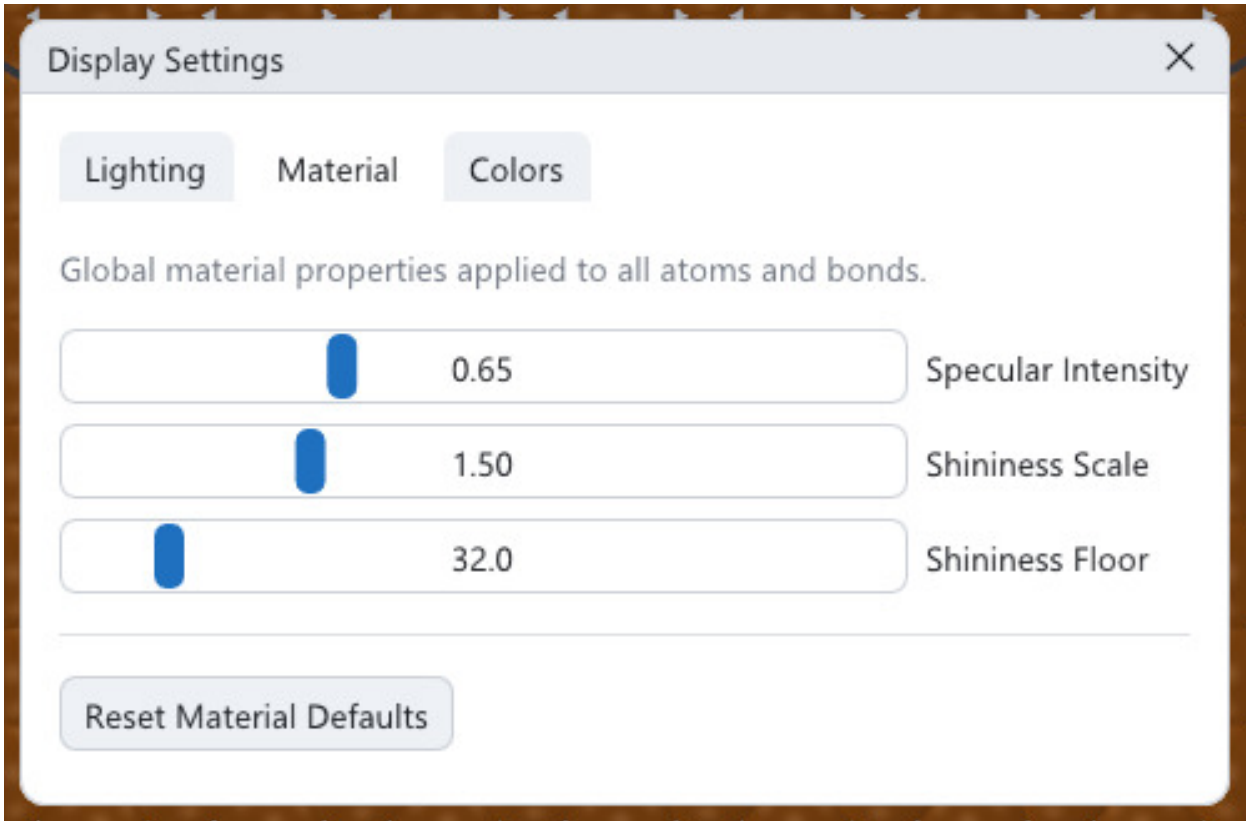
1. Adjust Ambient to reveal shadow-side atoms. Change Shadow Strength independently to compare contrast between front and rear layers.
2. Adjust Saturation and Contrast while keeping a visible reference species.
3. Use Reset Lighting Defaults to return to the baseline appearance.

**Check the result.** Lighting changes must not be interpreted as a change in local density or composition.

**Try the options.** Ambient controls background illumination; Saturation changes vividness; Contrast stretches the light/dark response. Electronic surface lighting has separate controls in its Appearance panel.

### 10.6.10 Control highlights on atoms and bonds

**Start here.** Open **Display Settings > Material**.




*Material controls are separate from scene illumination.*

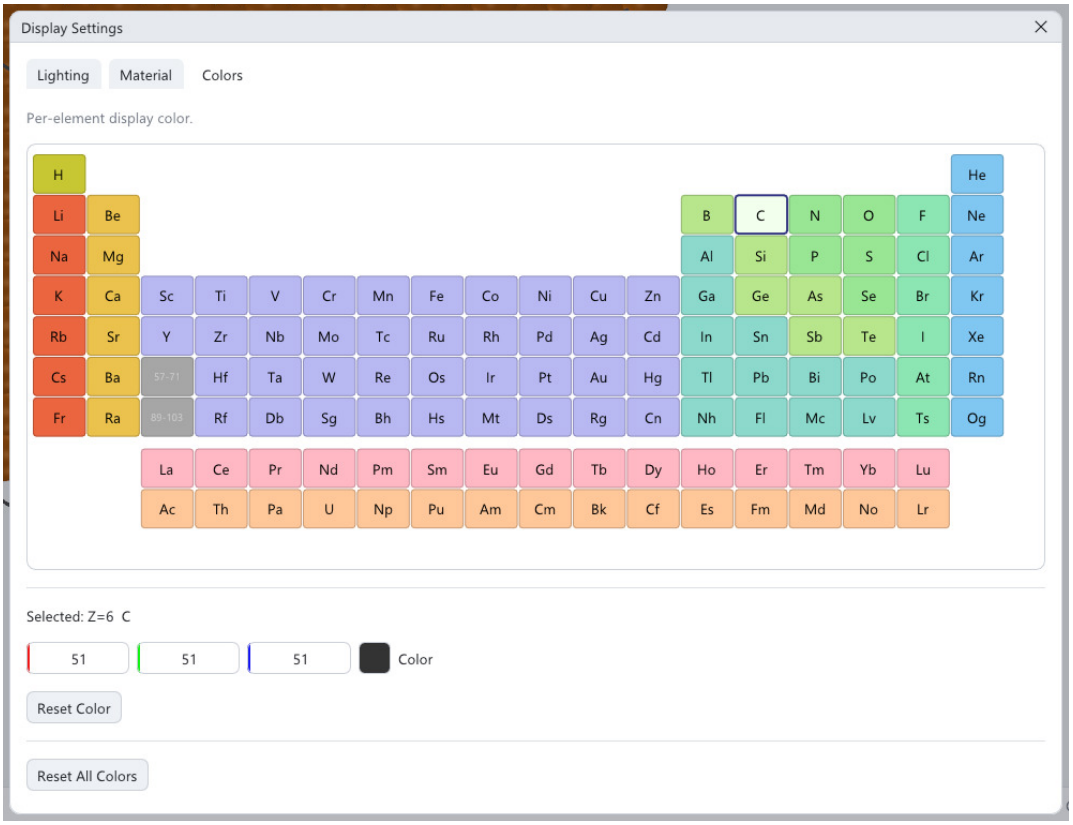
1. Lower Specular Intensity to reduce bright highlights, then compare a larger value.
2. Change Shininess Scale and Shininess Floor one at a time to inspect highlight concentration.
3. Use Reset Material Defaults and compare with the original view.

**Check the result.** The same atomic geometry should remain visible. Specular highlights are a rendering effect.

**Try the options.** Intensity controls highlight strength, Scale multiplies element response, and Floor sets the minimum exponent. Keep material settings fixed across a comparison series.

### 10.6.11 Change a species display colour

**Start here.** Open **Display Settings > Colors**.




*Colors tab with a selected element, its RGB controls and the reset actions.*

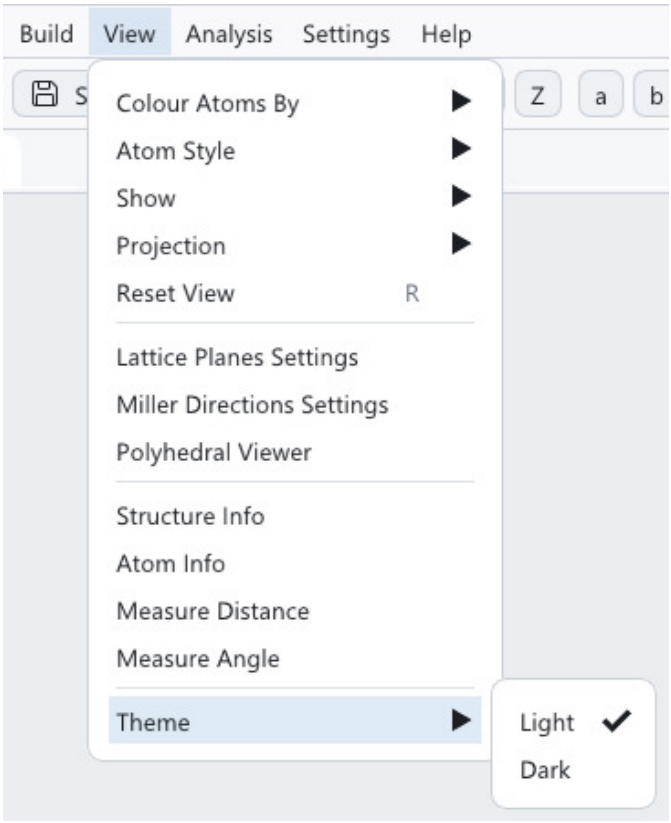
1. Select Cu, then change its colour using the visible values or colour control.
2. Select Ni and give it a distinguishable colour. Inspect the same alloy from a second view.
3. Use Reset Color for one species, or Reset All Colors to restore the palette.

**Check the result.** Colour edits do not substitute species. Verify chemistry using labels or Structure Info.

**Try the options.** The element picker used for a builder serves a different purpose. Do not confuse changing a species colour with changing a charge-density colour range.

### 10.6.12 Select a light or dark desktop theme

**Start here.** Open **View > Theme**.




*Theme selection under View. Light is selected in these captures.*

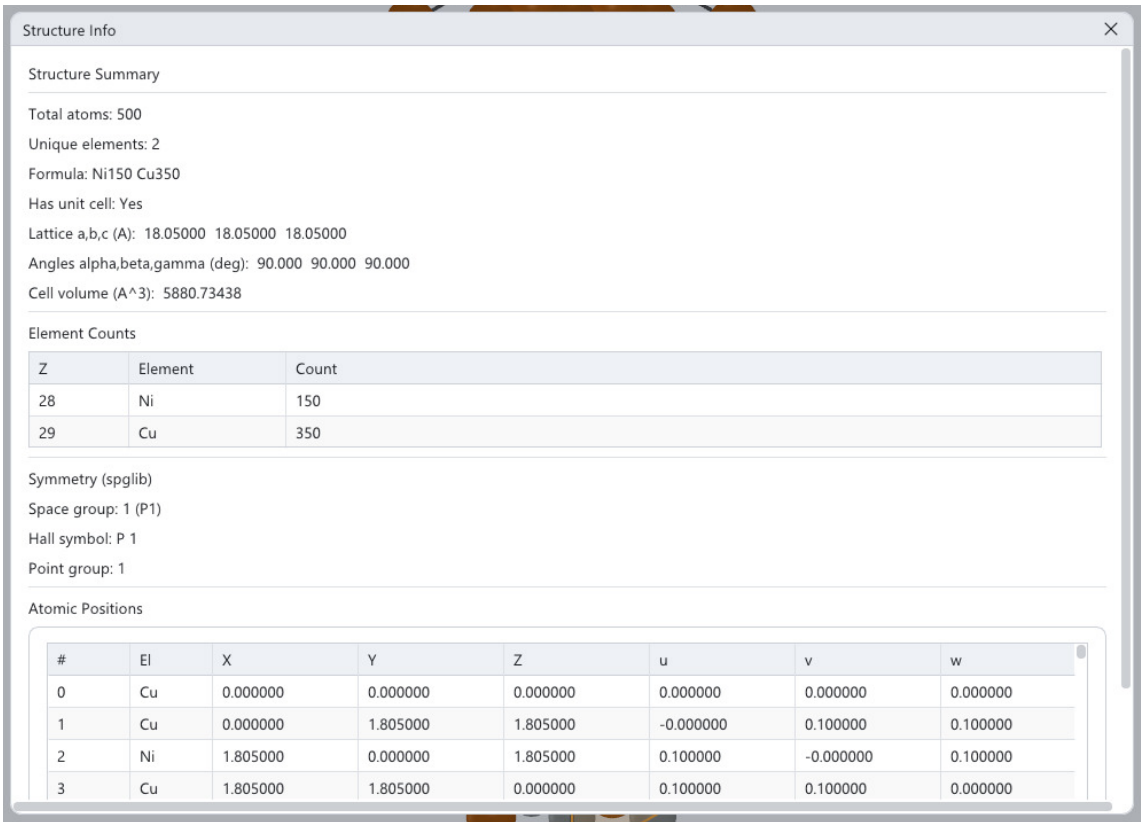
1. Choose Dark and inspect text, controls, and scene contrast.
2. Choose Light to return to the theme used in this manual.

**Check the result.** Theme selection should preserve structure contents and numerical results.

**Try the options.** Choose a theme that leaves labels and thin guides readable in exported or captured figures. Electronic field palettes are configured separately.

### 10.6.13 Verify composition, cell, and symmetry

**Start here.** Open `examples/cu_ni_alloy.cif`, then **View > Structure Info**.




*Structure Info for the supplied 70:30 Cu–Ni alloy, showing 500 atoms and a cubic 18.05 Angstrom cell.*

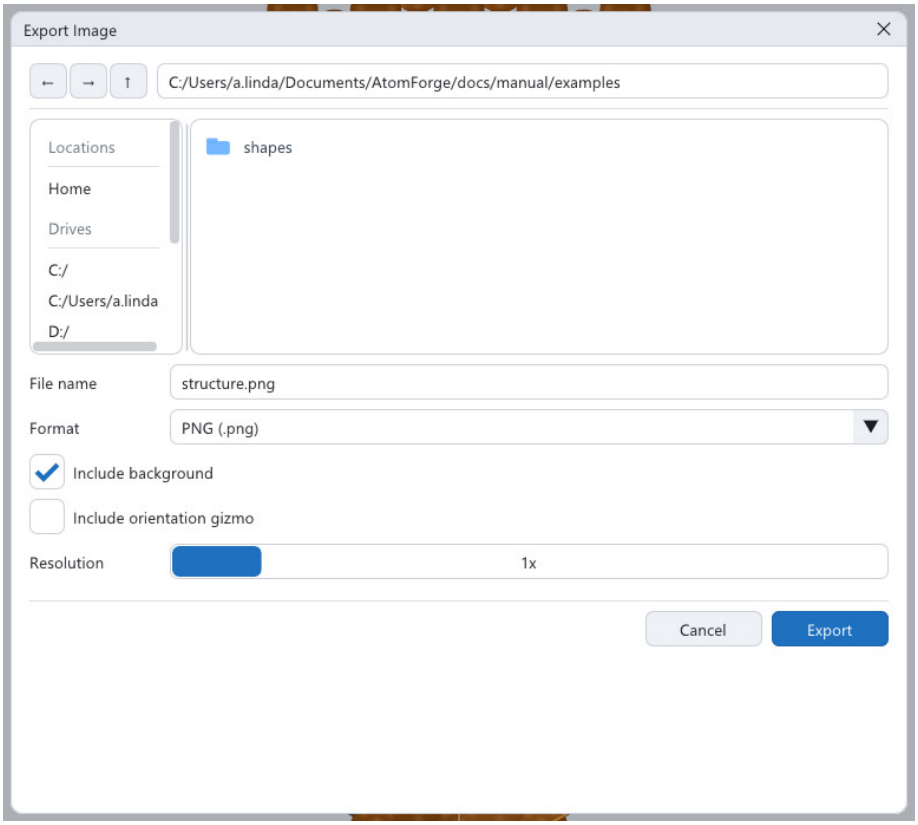
1. Check total atoms, species counts, formula, and unit-cell availability.
2. Read lengths, angles, volume, and symmetry. Wait for symmetry calculation to finish before recording it.
3. Scroll the atomic-position table to compare Cartesian coordinates with direct coordinates.

**Check the result.** Expect 350 Cu, 150 Ni, and volume about 5880.7344 $Å^3$. The chemical decoration can lower symmetry even while positions lie on an FCC parent lattice.

**Try the options.** Use Atom Info for one selected site. A missing cell or failed symmetry calculation is not an assignment to a particular space group.

### 10.6.14 Export a structure illustration

**Start here.** Frame the intended structure and clear unwanted selections.




*Export Image: the folder browser, then the file name, format, background, orientation gizmo and resolution. Export writes the image.*

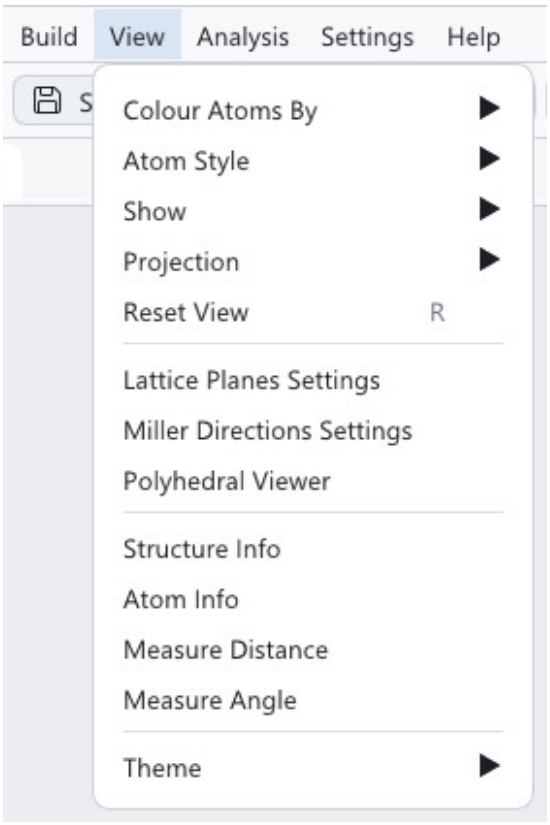
1. Choose **File > Export Image**. Browse to an output folder and enter a filename.
2. Select PNG, JPEG, or SVG. Choose Include background and Include orientation gizmo independently. Start with resolution scale 1 before trying a larger export.
3. Scroll to Export, save, and open the actual output image to check labels, framing, and transparency.

**Check the result.** PNG supports alpha; JPEG does not. Higher raster scale increases pixel dimensions and memory use. Inspect SVG in the intended downstream viewer.

**Try the options.** Scale ranges from 1 to 8. Cancel dismisses the operation. The image is a rendered view, so also save the structure file and settings needed to reproduce it.

### 10.6.15 Inspect geometric Voronoi cells

**Start here.** Open `examples/cu_fcc.cif` and choose **View**.



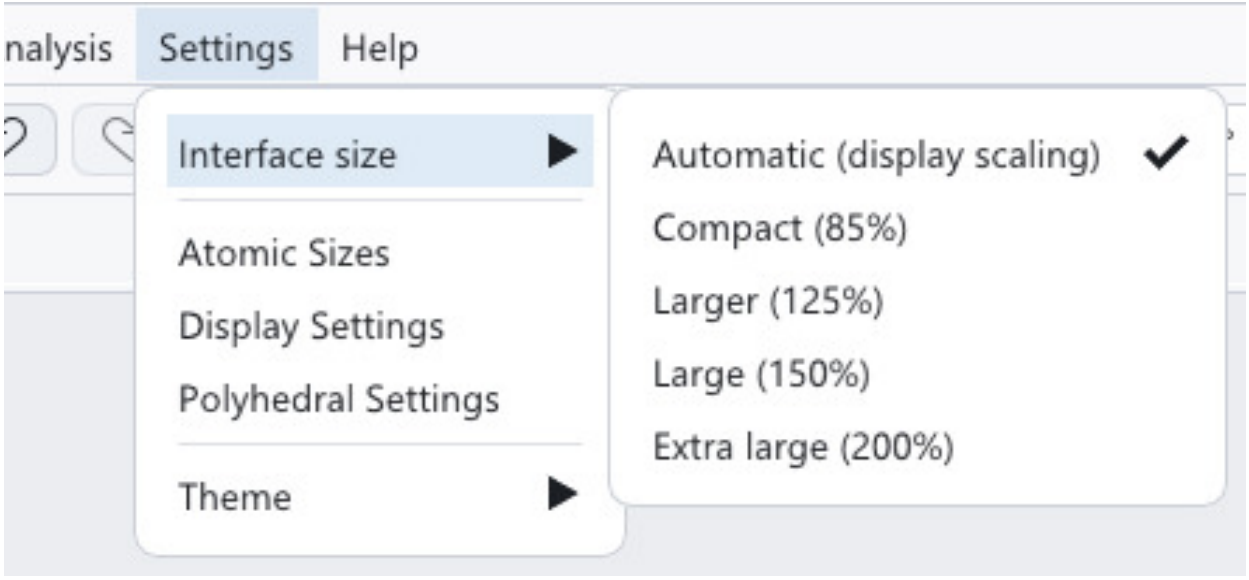

*View menu: projection, overlays, information, measurements, and Reset View are separate commands.*

1. Enable **View > Show > Voronoi Volumes**. Orbit the main scene and compare cells around symmetry-equivalent sites.
2. Toggle **View > Show > Atoms** to expose cell faces, then restore atoms. Disable the Voronoi toggle when finished.

**Check the result.** These are geometric cells, not integrated electronic charges. Nonperiodic exterior cells depend on an artificial enclosing box. See the full display reference for periodic-image limitations.

### 10.6.16 Adapt the interface to the screen

**Start here.** Open **Settings > Interface size**.




*Current Settings > Interface size menu, with Automatic selected and all five scale choices visible.*

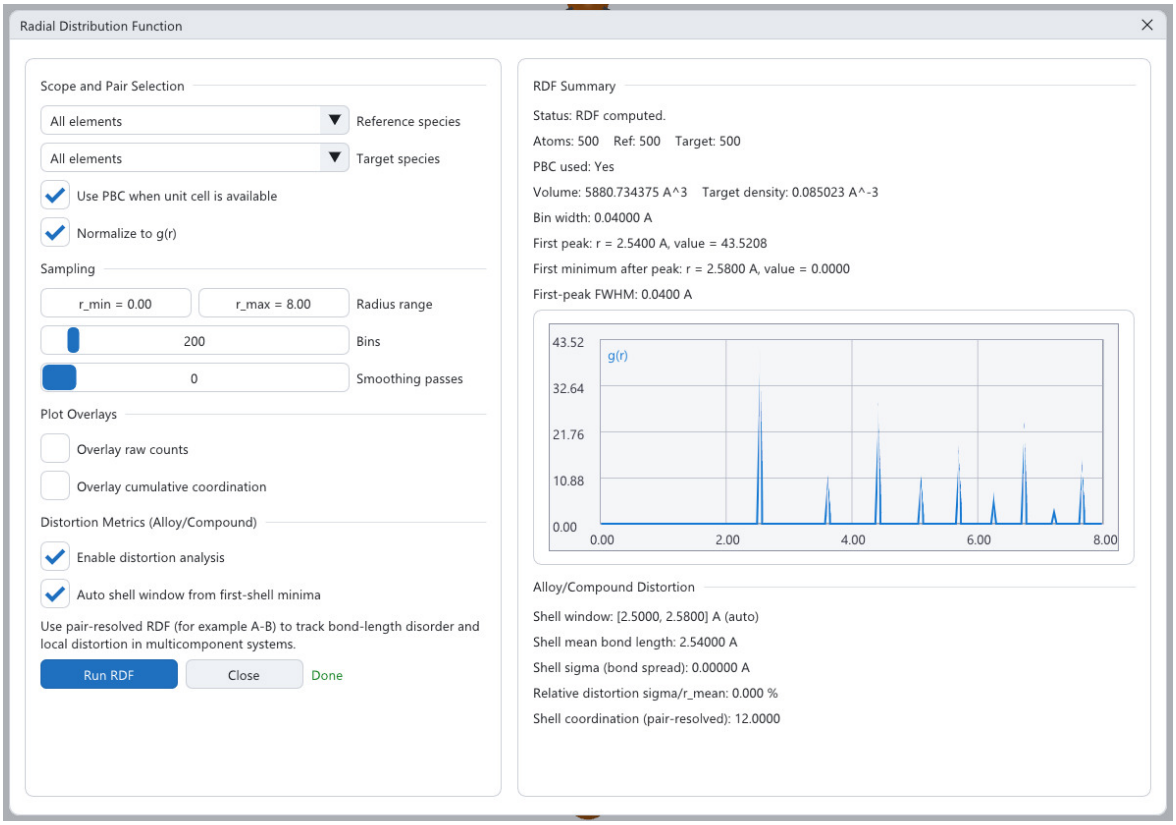
1. Start with Automatic. Choose Larger (125%) or Large (150%) if labels are difficult to read; Extra large is 200% and Compact is 85%.
2. Resize the application. Scroll stacked dialogs to reach lower panels, then return to Automatic to restore display-derived scaling.

**Check the result.** The setting applies next frame and lasts for the session; restarting restores Automatic. It changes interface dimensions, not atomic radii.

## 10.7 Analysis and electronic display

### 10.7.1 Run an RDF and inspect its shell window

**Start here.** Open `examples/cu_ni_alloy.cif`, then **Analysis > Structure and defects > Radial Distribution Function**.




*Actual RDF of the ideal-position Cu–Ni alloy, all species, PBC, 0–8 Angstrom, 200 bins, no smoothing.*

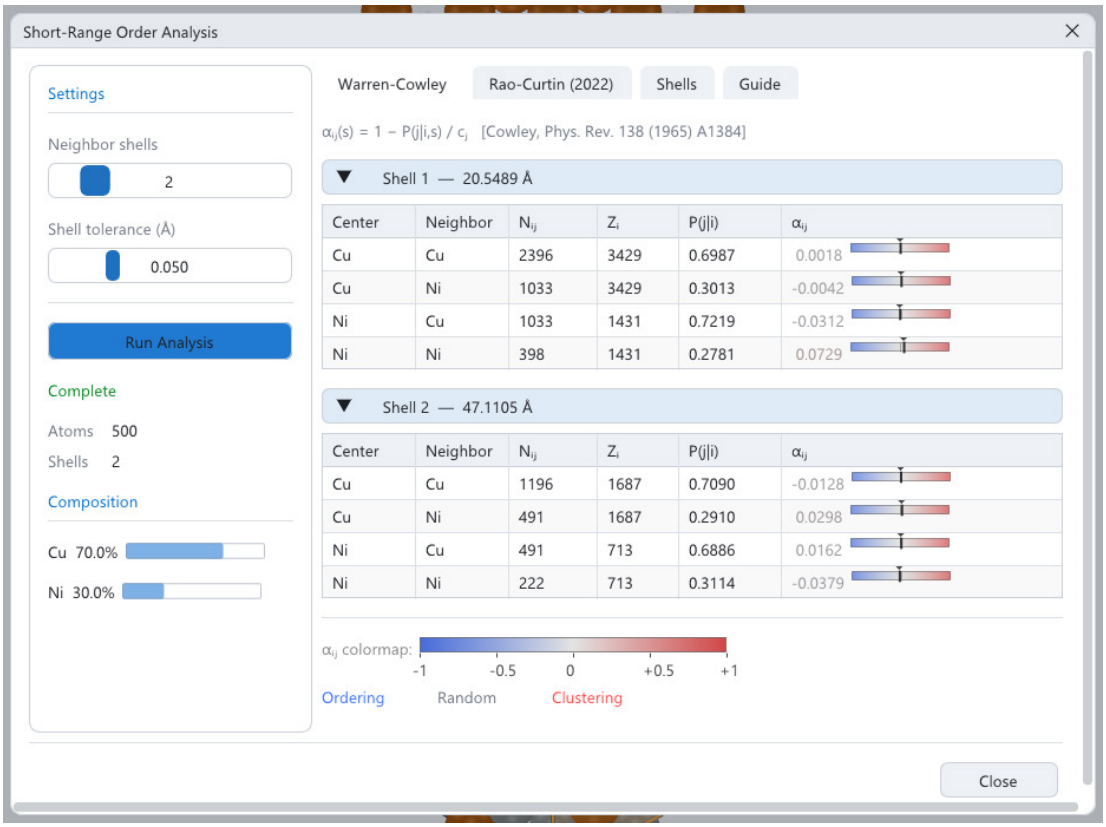
1. Set Reference and Target species to All elements, enable PBC and normalisation, choose range 0–8 Angstrom and 200 bins, and leave smoothing at zero.
2. Enable distortion analysis and its automatic first-shell window, then Run RDF.
3. Compare partial Cu–Ni and Cu–Cu selections, rerunning after each input change.

**Check the result.** The displayed first peak is the 2.54 Å bin centre, consistent with the exact ideal distance near 2.5527 Å at bin width 0.04 Å. Total first-shell coordination is 12 here.

**Try the options.** Raw-count and cumulative-coordination overlays add different quantities. Disable auto shell window to choose a manual interval. Smoothing changes presentation; it does not improve the original spatial resolution. For a finite particle, reconsider PBC and normalisation.

### 10.7.2 Run SRO and validate the shell diagnostics

**Start here.** Open `examples/cu_ni_alloy.cif`, then **Analysis > Short-Range Order (SRO)**.




*SRO on the supplied alloy. This capture demonstrates controls and an observed diagnostic discrepancy; its numerical results are not validated.*

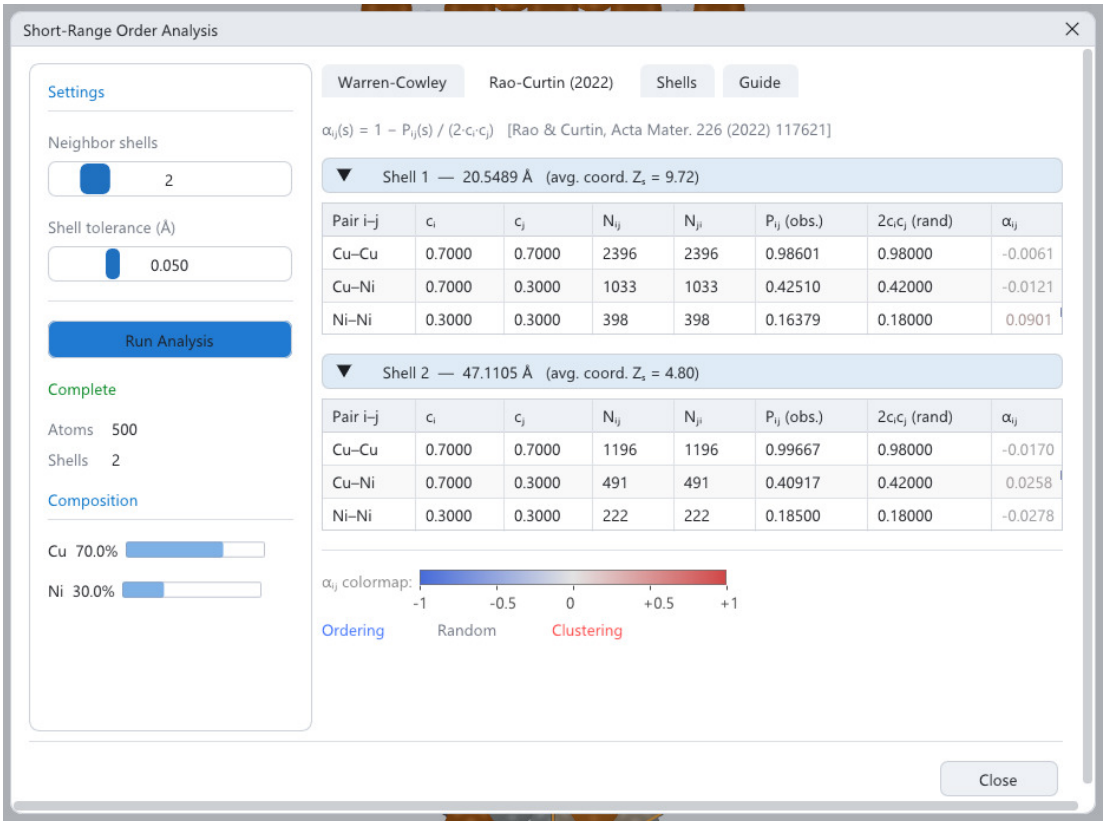
1. Set Neighbor shells to 2 and Shell tolerance to 0.050 Angstrom, then Run Analysis.
2. Read the Warren–Cowley pair rows together with centre/neighbor species, pair counts, and composition.
3. Open Shells and compare its distances and coordination against the RDF and ideal lattice geometry before interpreting the order parameters.

**Check the result.** This captured executable reports shell distances 20.5489 and 47.1105 Å, inconsistent with ideal FCC first shells near 2.5527 and 3.61 Å. Treat this run as failing validation; do not use its SRO numbers as a scientific benchmark.

**Try the options.** Shell tolerance groups near-equal distances; it is not composition tolerance. The displayed composition is correctly 70:30, which alone does not validate neighbour selection or shell statistics.

### 10.7.3 Read the SRO result tabs

**Start here.** Run SRO and retain the parameter and diagnostic context from the preceding exercise.




*Rao–Curtin tab from the same diagnostic run. Validate shells before using either convention.*

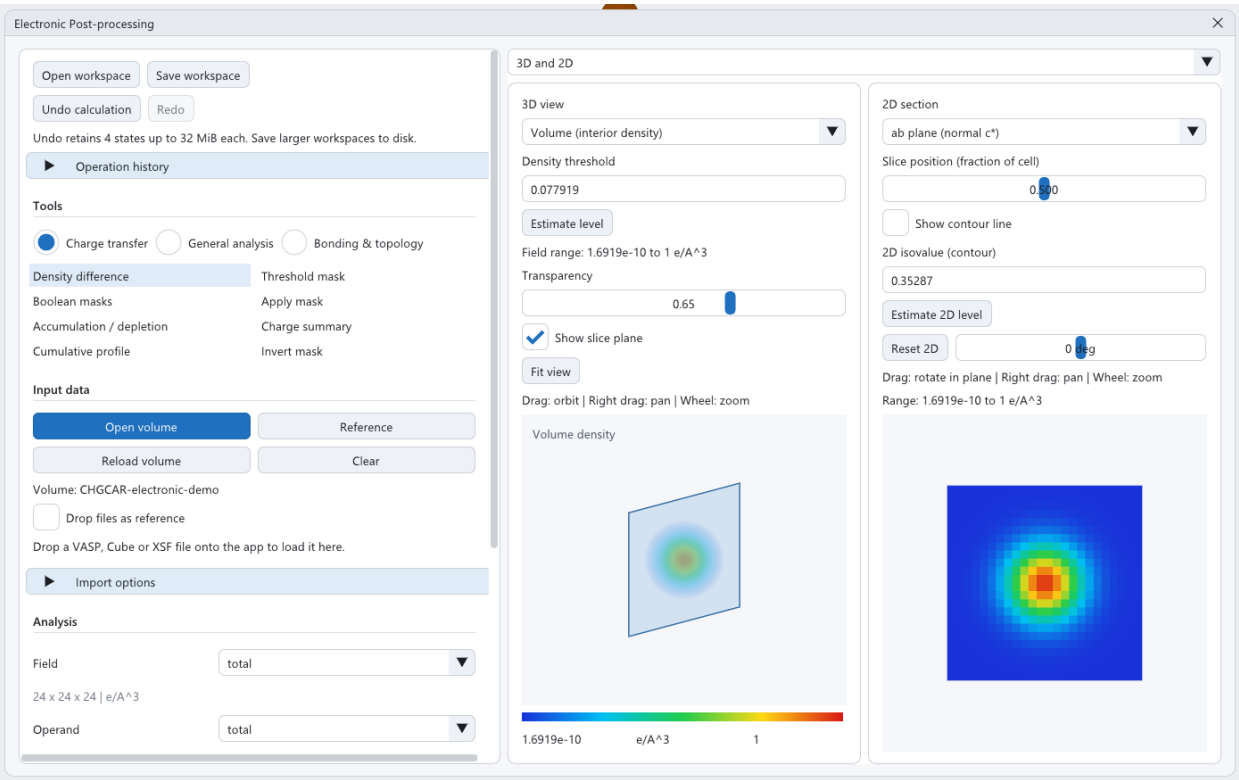
1. Compare Warren–Cowley directional conditional probabilities with the Rao–Curtin symmetric pair presentation.
2. Inspect Shells for distances and average coordination. Use Guide to locate the in-app interpretation and method comparison.
3. Record the chosen convention, species pair, shell definition, and finite-size/seed variation with any valid result.

**Check the result.** The two conventions should not be mixed in one comparison without stating their definitions. The shell discrepancy in this captured run affects its use regardless of tab.

**Try the options.** The Guide is explanatory UI text, not evidence of an energetic preference. Pair frequencies in a single random finite structure do not establish interaction energies or equilibrium ordering.

### 10.7.4 Compare electronic volume and surface views

**Start here.** Load the supplied CHGCAR in **Analysis > Electronic structure > Electronic Post-processing**. See Chapter 5 for quantity and channel selection.




*Real CHGCAR capture from the main electronic walkthrough; the selected field controls the visualisation.*

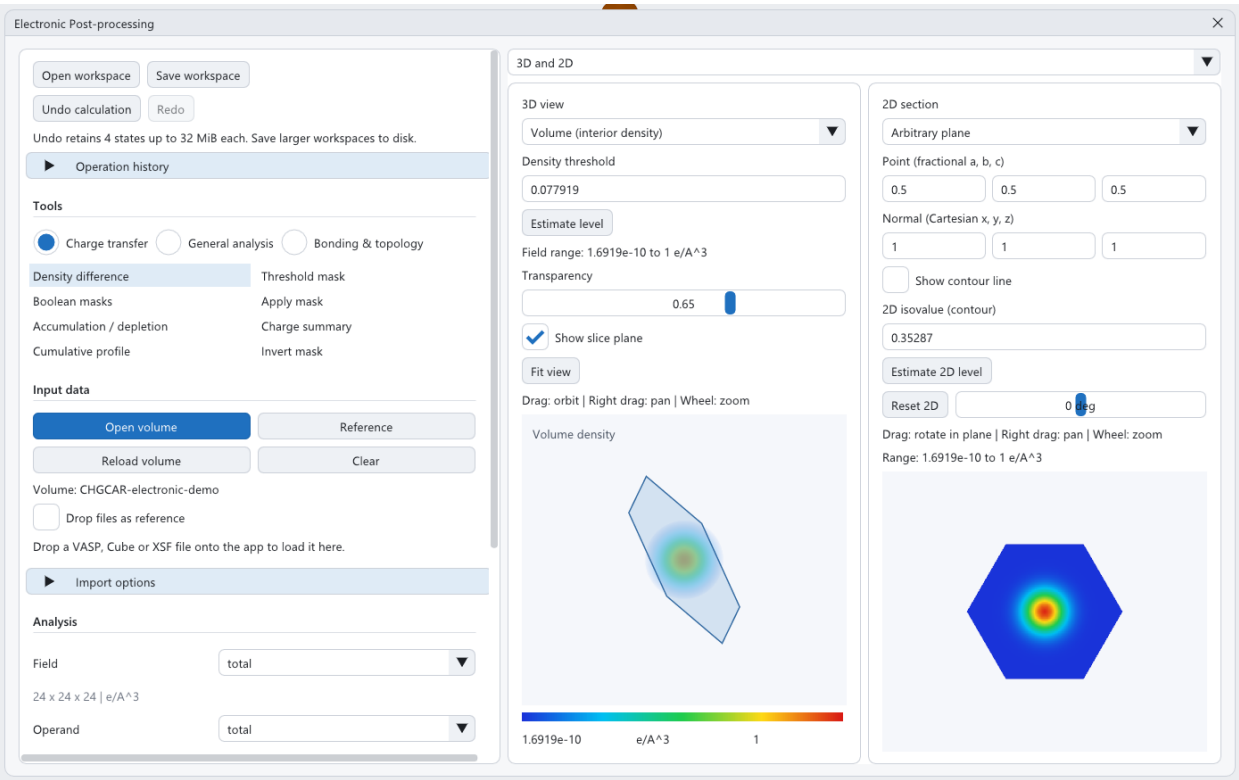
1. Choose a density field, select a layout with 3D, and use Estimate level for a data-derived starting level.
2. Compare Volume with Isosurface. Set transparency to zero for opaque rendering, then increase it to reveal internal overlap.
3. Orbit with left drag, pan with right drag, zoom with the wheel, and use Fit view to reset framing.

**Check the result.** An isosurface is a boundary; volume mode depicts interior values. Read the range warning and legend rather than treating a brighter colour as a different quantity.

**Try the options.** Update surface regenerates the mesh when needed. Appearance offers Spectrum, Blue-white-red, Sequential blue, automatic/manual range, and surface specular/shininess controls. Layout can show both views or only one.

### 10.7.5 Choose a 2D intersection and its guide

**Start here.** Load the supplied CHGCAR in **Analysis > Electronic structure > Electronic Post-processing**. See Chapter 5 for quantity and channel selection.




*Real CHGCAR arbitrary-plane capture. The translucent guide locates the section in the 3D volume.*

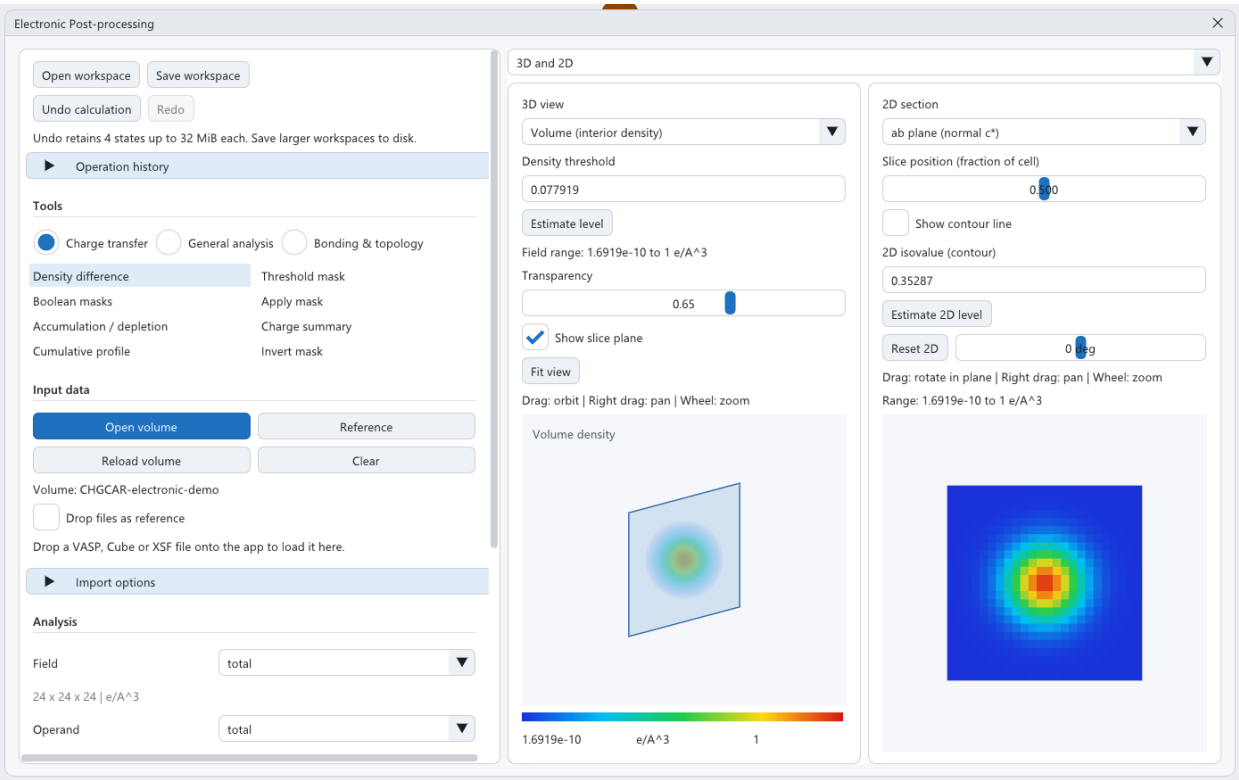
1. Select a layout containing 2D. Compare bc, ac, and ab sections at fractional position 0.5.
2. Select Arbitrary plane, enter point $(0.5, 0.5, 0.5)$ in fractional cell coordinates and a nonzero Cartesian normal such as $(1, 1, 1)$.
3. Toggle Show slice plane in 3D. Estimate the 2D level, toggle its contour, and use left drag/angle for in-plane rotation, right drag for pan, wheel for zoom, and Reset 2D.

**Check the result.** The 2D level is independent of the 3D level and must lie within that section extquotesingle s range. A valid volume level may be out of range for a particular slice.

**Try the options.** Hiding the translucent guide does not remove the section. The display preview is bounded in resolution; use Numerical plane section and Contour segments for quantitative coordinate export.

### 10.7.6 Run charge-transfer and general operations

**Start here.** Load the supplied CHGCAR in **Analysis > Electronic structure > Electronic Post-processing**. See Chapter 5 for quantity and channel selection.




*Electronic workspace; calculation controls and the reference operand are separate from view controls.*

1. Load primary field A. For two-input operations, use the Reference browser to load compatible B. Reference means the second operand, not an equilibrium standard.
2. Choose Charge transfer tools for density difference, accumulation/depletion, masks, Boolean operations, or regional charge; choose General analysis for other operations. Set the operation-specific inputs and Calculate.
3. Select the derived field or inspect the result table, then use the matching field/CSV/surface export browser.

**Check the result.** For aligned identical inputs, $A - A$ is zero. For a physical study, density, units, grids, cells, electron counts, and reference calculation conventions must match.

**Try the options.** Chapters 6 and 7 give procedures and checks for all 33 operation choices, including profiles, integration, derivatives, smoothing, sections, contours, Fourier, electrostatics, surfaces, and exports. Reload rereads a changed volume; Clear resets the electronic study.

# Appendix A

# Installation, builds, and troubleshooting

## A.1 Running a packaged desktop build

Extract a complete package into a writable folder and run the executable. Keep the bundled runtime libraries and Open Babel plugins in their supplied locations. Copying only `AtomForge.exe` to another machine can leave required DLLs or format plugins behind. To open a structure at launch, pass its path as the executable's argument.

The desktop requires a working OpenGL environment. If the window fails to render, check the graphics driver and whether a remote/virtual session supplies the needed OpenGL support. A successful compiler run and a working graphics runtime are separate checks.

## A.2 Build requirements

The source uses C++17 and CMake 3.16 or newer. Desktop dependencies include GLFW, GLEW, GLM, Eigen3, Open Babel 3, and an OpenGL implementation. spglib enables symmetry-related capabilities. Use a consistent compiler/runtime family for all linked libraries. See the repository's `INSTALL.md` and actual CMake configuration for platform-specific dependency discovery.

| CMake setting | Purpose |
|---|---|
| `CMAKE_BUILD_TYPE=Release` | Optimised application/library build. |
| `ATOMFORGE_BUILD_APP=OFF` | Build core/library functionality without the desktop app. |
| `BUILD_TESTING=ON` | Configure available automated tests. |
| `ATOMFORGE_ENABLE_SSS_BUILDER=ON` | Enable the substitutional-alloy desktop builder (default on). |
| `ATOMFORGE_ENABLE_SFE_BUILDER=ON` | Enable the optional stacking-fault desktop builder (default on). |
| `BUILD_PORTABLE=ON` | Enable runtime packaging support. |
| `Python3_EXECUTABLE` | Select the Python interpreter used by configured tests. |

## A.3 Automatic build resource protection

Normal Ninja and Makefile builds automatically serialize C++ compilation and linking to prevent many memory-heavy compiler processes from running together. Release optimization and all application features remain unchanged. No machine-specific preset or configuration is required: update the source and reconfigure your existing build directory.

Ninja uses a one-job pool; both generators also use a shared compiler/linker lock in the build directory. Use `--parallel1` as shown below to avoid unnecessary waiting processes. Separate build directories have separate locks, so avoid running multiple builds in different directories simultaneously.

A single compiler process still needs memory. If a full-system freeze persists with serial compilation, inspect system logs, available memory, swap/page-file space and cooling after restarting. A freeze alone does not identify its cause. Packaged releases avoid compilation entirely.

## A.4 Linux build

Install a C++ toolchain, CMake, pkg-config, and development packages for GLFW, GLEW, GLM, Eigen3, Open Babel, and optionally spglib using your distribution's package manager. Configure a fresh build directory:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
cmake --build build --parallel 1
ctest --test-dir build --output-on-failure
./build/AtomForge
```

Run from a graphics-capable session for desktop use. For a headless server, use the native builder CLI or electronic library/API instead. A core-only build does not create a desktop executable.

## A.5 Windows build with MSYS2 UCRT64

Use the MSYS2 UCRT64 toolchain and its matching dependency packages. Typical packages include the UCRT64 toolchain, CMake, GLFW, GLEW, GLM, Eigen3, Open Babel, pkgconf, spglib, and Python when running the Python tests. Avoid mixing MINGW64, UCRT64, and unrelated compiler libraries in one build directory.

```
cmake -S . -B build -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=ON
    -DPython3_EXECUTABLE=C:/msys64/ucrt64/bin/python.exe
cmake --build build --parallel 1
ctest --test-dir build --output-on-failure
```

These commands assume the compiler and matching libraries are discoverable, normally from the UCRT64 shell. If using PowerShell, supply the correct toolchain paths and runtime DLLs. Select a Python interpreter compatible with the native library's runtime requirements for tests; discovering an unrelated system Python can make a correct library build appear to fail during loading.

Use a new build folder when changing generators or compiler families. Close a running AtomForge instance before replacing its executable, because Windows may lock the file. After rebuilding, launch the executable from the expected folder and check that the interface matches the new version; an older shortcut can launch a stale copy.

## A.6 Portable packaging and CI

Configure with `BUILD_PORTABLE=ON`, build, then use CPack with that build directory's `CPackConfig.cmake`. Validate the extracted package in a clean runtime environment and test an actual structure import, not only process startup. Open Babel plugins are needed for format support.

When a GitHub compilation job fails, inspect the first failing step and its log: configure/dependency discovery, compilation, linking, tests, or packaging. The last generic failure line is usually less useful than the original diagnostic. Record the commit and platform. Documentation changes do not resolve an unrelated toolchain problem, and a locally successful Windows run does not alone prove every CI platform passes.

## A.7 Reporting a reproducible problem

Record the application commit, operating system, build options, input/import settings, selected fields, exact steps, and error text. Supply a small reproducer with expected and observed behaviour. For numerical issues include geometry, units, and integrals; for visual issues capture the entire dialog.

## A.8 Troubleshooting by symptom

| Symptom | Checks and recovery |
|---|---|
| Charge tools not visible | Open Electronic Post-processing and select Charge transfer. The eight tool buttons should appear before loading a file. Check that the expected rebuilt executable is running. |
| Calculate/Export not visible | Scroll the left control pane; long parameter sets extend below the initial view. Resize the dialog if needed. |
| Reference gives zero difference | Field and Operand may be the same channel. Verify the external reference file and the local-reference checkbox. |
| Grid alignment error | Compare shape, cell, origin, units, and periodicity. Resample explicitly only if scientifically justified. |
| Charge operation rejects input | Inspect units. Cube/XSF Auto is raw; choose the correct quantity and reload. A potential/ELF/mask is not electron density. |
| Wrong integral after conversion | Check scalar-unit conversion, volume normalisation, coordinate units, duplicate endpoints, and selected channel. |
| 3D looks empty | Check threshold/range, transparency, selected field, render mode, and Fit view. For a signed field, select depletion magnitude to display negative changes with a positive threshold. |
| Opaque slider still shows empty regions | Threshold-excluded space remains empty. Hide the separately translucent slice plane if it obscures interpretation. |
| Unexpected circle/line in 2D | Check Show contour line. It is optional and off by default. It does not control the filled colour map. |
| 2D out-of-range warning | Compare the level with the current slice's range; Estimate 2D level or move the plane. |
| Arbitrary section absent | Use a finite nonzero normal and a point that defines a plane intersecting the cell. |
| Fourier operation rejected | Use periodic endpoint-excluded geometry, unique indices within the sampled Nyquist range, and conjugate pairs for real synthesis. |
| Fourier workload too large | The implementation bounds work. Consider an explicitly resampled power-of-two grid and quantify the interpolation effect before continuing. |
| Ewald rejected or unstable | Check neutrality, site/charge count and order, periodicity, positive parameters, and convergence of both cutoffs. |

| Symptom | Checks and recovery |
|---|---|
| Python native DLL not found | Set the native-library path and make its dependent runtime DLLs available. Confirm process/library architecture compatibility. |
| Builder menu disabled | Load a suitable source with a cell; inspect compile-time options and symmetry support. |
| Mesh fill missing atoms | Check closed geometry, units/scale, replication coverage, and feature thickness relative to the lattice. |
| Amorphous placement fails | Density, pair cutoffs, and counts may be incompatible. Test a looser geometry and inspect diagnostics. |
| Saved structure loses cell/metadata | Choose a format supporting the needed fields and verify the reopened result. Python and desktop format coverage differ. |

# Appendix B

# Examples, validation, and maintaining this manual

## B.1 Included example inventory

The following structures were generated through AtomForge's actual builder CLI, with the larger Cu host produced through the Python repetition API. They are reusable inputs for the walkthroughs.

| File | Atoms | Construction |
|---|---|---|
| `cu_fcc.cif` | 4 | Cubic 225, Cu at the origin, $a = 3.61$ Å. |
| `fe_bcc.cif` | 2 | Cubic 229, Fe at the origin, $a = 2.87$ Å. |
| `cu_host.cif` | 500 | $5 \times 5 \times 5$ repetition of the Cu conventional cell. |
| `cu_dislocation.cif` | 500 | Edge displacement, cylinder radius 5 Å; family-change warning retained. |
| `cu_ni_alloy.cif` | 500 | Cu 0.7 / Ni 0.3, seed 42. |
| `cu_sphere.xyz` | 336 | Sphere radius 10 Å, vacuum 5 Å. |
| `cu_sigma5.cif` | 60 | Axis [001], $\Sigma 5$, candidate 0, repetitions 3/3, overlap 1.5 Å. |
| `cu_polycrystal.cif` | 1,329 | $25^3$ Å$^3$, four grains, seed 7. |
| `sio2_pack.xyz` | 120 | Si 40 / O 80, density 1.5 g cm$^{-3}$, seed 7. |
| `cu_mesh.xyz` | 224 | Unit octahedron OBJ at scale 12 Å/model unit. |

`octahedron.obj` is an original procedural mesh included with the examples. The builder gallery plots actual output coordinates with Matplotlib. No external 3D model or Blender-generated asset is needed to reproduce these examples.

## B.2 Nanocrystal shape examples

The seven shape-cut examples in `examples/shapes/` supplement the inventory above. `shape-results.json` records the native command options and atom counts. Chapter 10 shows each shape and provides an exercise for rebuilding it. The example checker verifies all seven files for count, Cu species, and finite coordinates.

## B.3 CHGCAR provenance

The original file remains at the user-supplied location and is not redistributed. `chgcar-summary.json` records its SHA-256 hash, dimensions, cell vectors, sites, units, range, integral, and periodicity. The manual includes derived `chgcar-planar.csv` and `chgcar-cumulative.csv` plus figures. A different CHGCAR will generally produce different numbers and images; the filename alone is not a unique identifier.

The example generation verifies that self-subtraction is exactly zero and that the final cumulative integral agrees with the full integral. These are numerical consistency checks, not independent validation of the originating electronic-structure calculation or a physical charge-transfer study.

Run `pythondocs/manual/scripts/check_examples.py` to verify the published structure counts and executable Python recipes. It uses an analytic periodic field to check charge conservation under complementary masks, Fourier reconstruction, derivative outputs, integration, neutral point-charge calls, and Cube/XSF round trips. This synthetic fixture is explicitly a software check and is separate from the supplied CHGCAR walkthrough.

## B.4 Regenerate examples and figures

Build AtomForge and install the local Python package/native library first. The plotting script additionally needs NumPy and Matplotlib. Run from the repository root:

```
python -m pip install numpy matplotlib
python docs/manual/scripts/generate_examples.py --chgcar "C:/path/to/CHGCAR"
python docs/manual/scripts/generate_shape_gallery.py
```

Omit `--chgcar` to regenerate only structure examples and the builder gallery. Set `ATOMFORGE_PATH` if the executable is outside the normal build folder. The optional `MANUAL_PYTHON_DEPS` variable can point to a separate plotting-package directory when using an isolated embedded Python runtime.

The script overwrites the generated examples and scientific figures in this manual folder. Review new counts and the provenance JSON after changing software versions. It does not automate GUI screenshots: recapture those from the actual application when controls change. Keep scientific plots and interface captures labelled distinctly.

## B.5 Compile the LaTeX manual

The illustrated exercises are maintained in `tutorials.json`. After changing their steps, captions, or image crops, run `pythondocs/manual/scripts/render_tutorials.py--write`. It validates image availability, bounds, tutorial identifiers, and required exercise fields before regenerating the chapter. Normal builds check that the generated chapter is current. Original Windows screenshots are retained in `figures/tutorials/`; crops are applied in LaTeX, without redrawing the interface. This collection also retains alternative option panels for reuse.

Install a LaTeX distribution providing pdfLaTeX and the packages used by `manual.tex`: Latin Modern, microtype, geometry, graphicx, xcolor, booktabs, longtable, tabularx, amsmath, enumitem, listings, fancyhdr, tcolorbox, and hyperref. Then run:

```
python docs/manual/scripts/build_manual.py
```

The script runs three passes for contents, references, and bookmarks. It writes `docs/manual/AtomForge-manual.pdf`; intermediates go into the ignored `docs/manual/build/` folder. Published

figures are included, so compilation requires no original CHGCAR.

After editing, inspect the compiler log for overfull boxes, missing references, and missing characters. Render the PDF to page images and inspect figures, captions, tables, code wrapping, and page breaks. A successful LaTeX exit code alone does not establish a readable document. Keep screenshots at sufficient resolution for labels to remain legible at the chosen page width.

## B.6 Documentation coverage and limitations

The manual covers the exposed File/Edit/Build/View/Analysis/Settings workflows, all 33 electronic operation choices, Python structure and electronic APIs, and CLI entry points. Optional stacking-fault controls are identified separately; unavailable interface/CNA/ADF menus are explicitly distinguished from exposed features. Screenshots cover representative builder and electronic workflows rather than every possible parameter combination.

The examples test executable generation and numerical consistency. They do not constitute convergence studies for every builder, material, or physical model. In particular, the single supplied CHGCAR does not establish a fragment-transfer reference set; random structures are not relaxed glasses; and model scattering densities are not self-consistent electronic densities.

## B.7 Source map for maintainers

| Source area | Documentation to review when it changes |
|---|---|
| `src/ui/FileBrowser.cpp` | Menu availability, file workflows, shortcuts, help and settings placement. |
| `src/ui/*BuilderDialog.cpp` | Builder controls, defaults, validation, preview/commit steps. |
| `src/algorithms/` | Builder geometry and structural-analysis assumptions. |
| `src/ui/ElectronicPostProcessing.cpp` | Tool labels, operand routing, imports/exports, calculation parameters. |
| `src/ui/ElectronicSliceViewport.cpp` | 2D controls, arbitrary-plane behaviour, preview limits. |
| `src/electronic/` | Units, endpoint conventions, numerical operations, Fourier and electrostatics. |
| `src/graphics/ElectronicViewport*` | Rendering, opacity, lighting, volume preview behaviour. |
| `python/atomforge/` | Python signatures, supported formats, library discovery, CLI choices. |
| `CMakeLists.txt`, CI workflows | Feature flags, dependencies, tests, packaging and interpreter selection. |

Update the version and edition date when revising the manual. Re-run examples after algorithm or format changes, compare the generated metadata, and recapture affected UI figures. Avoid describing a feature as available solely because a corresponding source file exists.

## B.8 Glossary

**Asymmetric unit** Independent crystallographic sites expanded by space-group operations.

**Cell / origin** Lattice vectors and starting Cartesian position defining the grid or structure domain.

**Direct coordinates** Dimensionless coefficients of the direct lattice vectors.

**Field / channel** One scalar quantity sampled on a grid, such as total density or a derived difference.

**Reference / operand** The explicitly selected second input to an operation.

**Isosurface** A three-dimensional boundary at a constant scalar value.

**Contour** A constant-value line on a two-dimensional section.

**Volume rendering** Accumulation of contributions along viewing rays through a scalar field.

**Mask** Binary field defining a region within a grid domain.

**Reciprocal normal** A normal constructed from reciprocal lattice vectors; distinct from a direct-lattice direction in a general cell.

**Periodic endpoints** Repeated boundary samples that must be distinguished from unique periodic samples.

**Charge redistribution** Density change relative to specified references; not automatically a unique atom-assigned transfer.

**Voronoi partition** Geometric nearest-site regions, distinct from density-gradient basins.

# Appendix C

# Feature coverage and source-only interfaces

## C.1 Feature checklist

This checklist maps the user-facing feature families to their detailed instructions. It includes optional and currently unexposed source implementations separately. The accompanying `coverage.json` records source fingerprints and UI control declarations. The check script detects changed/new source files and stale generated API or CLI reference material. It is a maintenance alarm, not an automated proof that prose explains every possible runtime state. The control descriptions were reviewed against the implementation.

| Feature family | Documentation |
|---|---|
| Open, recent files, close, save, Save As, formats, drag/drop, tabs | Chapter 1; Section 4.12. |
| Bulk Crystal, all crystal systems, groups, site editing | Chapter 2; Section 2.13. |
| Substitutional Solid Solution, composition, seed | Chapter 2; Section 2.13. |
| CSL search, grain geometry, overlap removal, reduction | Chapter 2; Section 2.13. |
| All seven nanocrystal cuts and Wulff family construction | Chapter 2; Section 2.13. |
| Custom OBJ/STL fill, orientation, replication | Chapter 2; Section 2.13. |
| Polycrystal, random/specified/partial grain orientation | Chapter 2; Section 2.13. |
| Amorphous packing, density, scale, pair constraints | Chapter 2; Section 2.13. |
| Optional Stacking Faults sequence and export | Chapter 2; Section 2.13. |
| Selection, undo/redo, context substitution, centroid insertion | Chapter 3; Section 3.10. |
| Atom/cell table, integer transform, merge | Chapter 3; Section 3.10. |
| Cell Sculptor and bounded slab intersections | Chapter 3; Section 3.10. |
| Dislocation character, vectors, regions, preview/replace | Chapter 3; Section 3.10. |
| Interstitial detection, type filters, random/mesh/manual insertion | Chapter 3; Section 3.10. |

| Feature family | Documentation |
|---|---|
| Camera, colouring, representations, visibility, themes | Chapters 1, 4; Section 4.12. |
| Structure/Atom Info, distance, angle | Chapter 4; Section 4.12. |
| RDF, shell distortion, SRO | Chapter 4; Section 4.12. |
| Planes, directions, Voronoi, coordination polyhedra | Chapter 4; Section 4.12. |
| Atomic radii, lighting, material, colours, image export | Chapter 4; Section 4.12. |
| VASP/Cube/XSF input, quantity/units, channel selection | Chapter 5. |
| Electronic 3D volume/surface, 2D arbitrary slices, contours | Chapter 5; Section 4.12. |
| Charge difference, masks, Boolean operations, regional charge | Chapter 6. |
| All general electronic operations and exports | Chapter 7. |
| Python structures, I/O, desktop/notebook viewer | Chapter 8; Appendix D. |
| Python electronic grids, models, surfaces and tables | Chapter 8; Appendix D. |
| All ten native builder CLI modes and Python electronic CLI | Chapter 8; Appendix D. |
| CNA, ADF, interface matching workflows | This appendix. |
| Installation, build options, troubleshooting, example regeneration | Previous appendices and `docs/manual/README.md`. |

## C.2 Common-neighbour analysis: desktop and API

CNA is implemented in `src/algorithms/CommonNeighbourAnalysis.h` and its companion source. Open Analysis > Common Neighbour Analysis, or call Python `af.cna`. The native `--analyzecna` command writes per-atom CSV results.

For C++ integration, call `atomforge::analysis::computeCna(structure,params)`. `CnaParams` supplies `cutoffScale` (default 1.18) and `usePbc` (default true). The result contains `valid`, `message`, atom/pair counts, `pbcUsed`, signature counts, environment counts, and per-atom rows with index, species, coordination, dominant signature/count, and environment. A signature records common-neighbour count, bonds, and chain information. Check validity before consuming these tables.

The dialog provides Use PBC when unit cell is available, Bond cutoff scale (1.00–1.60), and Run CNA. Changing cutoff changes the bond graph and therefore the classification. The fixed cutoff option is in Angstrom; zero selects the covalent-radius scale. Classification uses the complete distribution of bond signatures around each atom. Repeat small cells sufficiently to represent all neighbour sites. Dense common-neighbour graphs exceeding 16 vertices or the search budget have chain index -1 rather than an invented classification.

## C.3 Angular-distribution analysis: desktop and API

ADF is implemented in `src/algorithms/AngularDistributionAnalysis.h`; it is available from the Analysis menu and Python `af.adf`. C++ callers use `computeADF(structure,params)` with `AdfParams`. The neighbour cutoff `rCutoff` is in Å (default 3.5); `binCount` divides 0–180 degrees

(default 180); `usePbc` controls periodic geometry. `centreMode` selects All, ByElement, or ByPair. Set `centreSymbol` for element-specific centres, plus `neighSymbol1` and `neighSymbol2` for the neighbour pair in ByPair mode.

`normalize` scales to peak 1, and `smoothPasses` sets smoothing (default 2). This normalisation is not the RDF density normalisation. `AdfResult` contains validity/message, echoed settings, atom/centre/triplet counts, PBC status, histogram rows (angle, raw count, displayed value), detected peaks, and per-centre-element coordination mean, standard deviation, and count.

The source dialog adds peak markers, raw-count display, plot range, and coordination statistics. Run/Compute ADF refreshes the calculation. Each unordered neighbour pair is counted once; the raw histogram remains unsmoothed even when the displayed curve is smoothed. Interpret peaks as angle distributions under the chosen neighbour definition; a label near a familiar ideal angle does not by itself identify a unique structure.

## C.4 Interface matching: desktop and CLI workflow

Open Build > Interface Builder. It is distinct from Merge Structures. Its source workflow searches matching in-plane supercells of two periodic inputs, strains an interface construction, and assembles layers. The `--buildinterface` command and `af.build("interface",options)` support automated matching.

The source dialog loads structures A and B. Orientation relationship choices are None, Manual angle, or Plane + direction; the last takes a plane and in-plane direction for each crystal. Orientation tolerance limits acceptable angle matching. Search controls include maximum integer coefficients for each side (nmax/mmax), maximum cell counts, layers A/B, maximum mean strain, and maximum rotation. Geometry controls include Z gap, vacuum, and X/Y output repetition. The stiffness target selects A or B for a $6 \times 6$ stiffness matrix used by the ranking calculation. These material constants must be supplied consistently; they are not inferred from element labels.

Find Matches produces candidate combinations; inspect and select a candidate before Build. Broad coefficient/cell limits increase search work. Small reported strain does not validate chemical termination or interfacial energy. The output must be inspected for contacts and relaxed externally where appropriate.

For C++ use, `get2DBasis` extracts the in-plane basis; `generateUniqueSupercells` enumerates bounded integer supercells; `strainComponents` and `meanAbsStrain` assess matching; `cubicElasticDensity` evaluates the implemented elastic ranking expression. `makeSupercell2D`, `repeatLayersZ`, and `applyTransform2D` construct the chosen components. `assembleInterface` combines them with gap/vacuum, and `repeatInterfaceXY` repeats the product. `orientationAngleFromPlaneDir` derives an in-plane orientation from plane/direction data. Consult the declared types in `src/algorithms/InterfaceBuilder.h` when integrating; these functions are the developer interfaces underlying the desktop workflow.

## C.5 Keeping the documentation complete as code changes

Run `pythondocs/manual/scripts/check_coverage.py` before building a release manual. A failure identifies a changed source fingerprint, missing source mapping, or stale API/CLI reference. Review the changed feature, update its prose, then intentionally refresh the snapshot with `--refresh`. Refreshing alone does not document a feature. The generated API appendix uses Python AST signatures and docstrings without importing native libraries; native CLI reference text is extracted from the help definitions and checked against the executable during this edition's validation.

# Appendix D

# Complete Python signatures and native CLI options

## D.1 Using this reference

The signatures below are generated from the public Python implementation. Property entries have no call parentheses. Private helpers and native-handle constructors are implementation details and are excluded. Atom's dataclass constructor is described explicitly. The workflow chapters explain physical interpretation, examples, and validation. Python indices are zero-based unless a function explicitly states otherwise.

```
Atom(symbol, x, y, z, r=1.0, g=1.0, b=1.0)
```

Coordinates are Cartesian Angstrom. r/g/b here are colour components, not atomic radii. Structure.atoms is an editable list; Structure.cell is three lattice-vector rows or None. Prefer add_atom for automatic element colours. Structure translation/scaling edits in place; repeat and filter_species return new structures. Grid operations return new data.

## D.2 Atom and Structure

### D.2.1 Atom

A single atom with Cartesian coordinates (Angstrom) and display colour.

```
Atom.copy()
```

Return a separate atom with the same coordinates and colour.

### D.2.2 Structure

A collection of atoms with an optional periodic unit cell. atoms is a list of Atom objects with Cartesian positions in Angstrom. cell is three lattice-vector rows [[a1,a2,a3],[b1,b2,b3],[c1,c2,c3]], in Angstrom, or None for a nonperiodic cluster.

```
Structure.__init__()
```

Create an empty structure with atoms=[] and cell=None.

```
Structure.copy()
```

Return a separate structure with copied atoms and cell vectors.

```
Structure.add_atom(symbol: str, x: float, y: float, z: float, r: float=-1.0, g: float
    =-1.0, b: float=-1.0)
```

Append an atom and return it. Default colour is the element's CPK colour.

```
Structure.remove_atom(index: int)
```

Remove and return the atom at index.

```
Structure.set_element_color(symbol: str, r: float, g: float, b: float)
```

Set r/g/b (0..1) on every atom of symbol already in this structure. Returns self.

```
Structure.translate(dx: float, dy: float, dz: float)
```

Shift all atoms in-place by (dx, dy, dz) Angstrom. Returns self.

```
Structure.scale(factor: float)
```

Scale all Cartesian coordinates and cell vectors by factor. Returns self.

```
Structure.repeat(nx: int, ny: int=1, nz: int=1)
```

Return a new supercell replicated nx x ny x nz times. Requires a unit cell.

```
Structure.filter_species(*symbols: str)
```

Return a new Structure containing only atoms of the given element symbols.

```
Structure.set_cell(a: float, b: float, c: float, alpha: float=90.0, beta: float=90.0,
    gamma: float=90.0)
```

Set the unit cell from lattice parameters (lengths in Angstrom, angles in degrees). Uses the standard convention: a along x, b in the xy-plane. Returns self.

```
Structure.view()
```

Open this structure in the AtomForge GUI (non-blocking).

```
Structure.view_notebook(width: int=620, height: int=460)
```

Explicitly display the inline 3D viewer in a Jupyter notebook cell.

```
Structure.pipe(steps, property=None)
```

Apply a structure pipeline (pipe text or an :class:atomforge.pipeline.Pipeline) to a copy of this structure and return the result; this structure is unchanged.

»> s.pipe("replicate 2 2 2 | select-expression 'fz > 0.5' | delete-selected")

```
Structure.save(path: str)
```

Save to file; format is inferred from the file extension.

## D.3 Structure file functions

```
load(path: str)
```

Load a structure; format is inferred from the filename.

```
save(s: 'Structure', path: str)
```

Save a structure; format is inferred from the filename.

## D.4 Desktop viewer function

```
view(s: 'Structure')
```

Open s in the AtomForge GUI (non-blocking).

The structure is serialised to a temporary extXYZ file and AtomForge is launched as a detached subprocess. The temp file is cleaned up after AtomForge exits; a daemon cleanup thread allows this function to remain non-blocking.

## D.5 Native Python builders

### D.5.1 BuildResult

Built structure and native diagnostics; output is a saved path or None.

Inspect stdout and stderr for algorithm warnings. Explicit output files and native orientation sidecars are retained. Structure holds coordinates and cell data; it does not import the sidecar's grain metadata.

```
builder_help(mode, *, executable=None)
```

Return installed native help for a registered builder mode.

```
build(mode, options=(), *, source=None, output=None, executable=None, timeout=300)
```

Run a native builder and return BuildResult without opening the GUI.

mode is bulk, gb, poly, nano, amorphous, sss, dislocation, custom, interface, stacking-fault, vacancy, strain, primitive, surface, sqs, nanowire, or core-shell. options is an argument sequence, e.g. ['–a', '3.61', '–atom', 'Cu 0 0 0']. Repeated flags are retained; multi-component values occupy one argument. source accepts a Structure or an input path. output optionally retains a CIF, XYZ/extXYZ, VASP, PDB, or LAMMPS result and native sidecars; otherwise temporary VASP files are cleaned up after loading. Coordinates are Angstrom. Native failures raise subprocess.CalledProcessError with stdout/stderr; timeout raises subprocess.TimeoutExpired. No command shell is used.

```
wulff(source, facets, *, radius=20, vacuum=5, output=None, executable=None, timeout
    =300)
```

Build a Wulff particle using (h, k, l, energy) facet families.

Positive relative surface energies set plane distances; radius is the maximum plane distance in Angstrom, not the farthest vertex radius. Uses symmetry expansion and the same native engine as the desktop. Returns BuildResult. Requires an executable supporting –shape wulff.

## D.6 Electronic data types and operations

### D.6.1 Grid

Scalar grid in Angstrom; values use x-fastest order (z, y, x loops).

cell contains three lattice/span vectors, as in Structure.cell. Periodic grids omit endpoints; finite grids include them. Operations return new grids. unit='e/A^3' enables density-derived energy calculations.

```
Grid.__init__(shape, cell, values, origin=(0, 0, 0), periodic=False, unit='raw')
```

Create a scalar grid. shape contains three integers >=2; values has exactly their product in x-fastest order. cell has three vectors in Angstrom. origin is Cartesian. Periodic grids omit duplicate endpoints; finite grids include endpoints. The grid is limited to 100 million samples.

```
Grid.shape
```

Read-only (nx, ny, nz) tuple.

```
Grid.cell
```

Read-only tuple of the three span/lattice vectors in Angstrom.

```
Grid.origin
```

Read-only Cartesian grid origin in Angstrom.

```
Grid.periodic
```

Read-only Boolean periodic-grid convention.

```
Grid.values
```

Return an independent list of scalar samples.

```
Grid.unit
```

Read-only scalar unit label; do not infer it from a plotted colour.

```
Grid.name
```

Read-only field name, including imported/derived channel identification.

```
Grid.sites
```

Return (atomic_number, x, y, z) rows; unknown species have number 0.

```
Grid.with_sites(sites)
```

Return a grid with supplied (atomic_number, x, y, z) site rows; positions are Cartesian Angstrom.

```
Grid.__add__(other)
```

Return pointwise addition of aligned grids.

```
Grid.__sub__(other)
```

Return pointwise subtraction of aligned grids, self minus other.

```
Grid.__mul__(other)
```

Return pointwise grid multiplication or scalar scaling.

```
Grid.__truediv__(other)
```

Return pointwise grid division or scalar division. Scalar zero is rejected.

```
Grid.resample(target)
```

Interpolate this field on target grid geometry. This does not register displaced atoms or correct incompatible units.

```
Grid.density_difference(*references, weights=None)
```

Return self - sum(weight * reference), in e/A^3.

Use fragment densities calculated in the same cell and geometry. Alignment is required; resample explicitly when appropriate.

```
Grid.threshold_mask(low, high)
```

Binary mask for the inclusive interval [low, high].

```
Grid.boolean(other, operation='intersection')
```

Combine binary masks: union, intersection, difference (self-other), xor.

```
Grid.invert_mask()
```

Complement a binary mask within its cell.

```
Grid.apply_mask(mask)
```

Keep density inside a binary mask, preserving the density unit.

```
Grid.split_density()
```

Return accumulation and positive depletion-magnitude fields.

```
Grid.charge_summary()
```

Electron accumulation, depletion magnitude and net change.

Apply to a difference density. These integrals describe redistribution; they do not assign charge transfer to individual atoms or fragments.

```
Grid.cumulative_charge(axis=2)
```

(Normal distance in A, cumulative electrons) from the lower cell face.

The final row includes the full cell. Periodic profiles depend on the chosen cell origin; finite grids use trapezoidal quadrature.

```
Grid.as_periodic(tolerance=1e-08)
```

Drop matching endpoint planes after checking all three boundaries.

Only use when the finite grid spans a complete periodic cell. This conversion checks values, but cannot establish physical periodicity.

```
Grid.gradient()
```

Return Cartesian x/y/z derivatives, including triclinic metric.

```
Grid.laplacian()
```

Return the Cartesian Laplacian grid, including cell geometry; units are field units per Angstrom squared.

```
Grid.energy_density(floor=1e-12)
```

Return approximate kinetic, potential and total density in eV/A^3.

Nonnegative electron density is required; samples <= floor (e/A^3) are masked to zero. These are gradient-expansion/local-virial fields, not the kinetic energy evaluated from Kohn-Sham orbitals.

```
Grid.reduced_density_gradient(floor=1e-12)
```

Return s = |grad(rho)|/(2*(3pi^2)^(1/3)rho^(4/3)), dimensionless.

Small s at low density marks non-covalent interaction regions (van der Waals contacts, hydrogen bonds); small s at bonding density marks covalent/ionic bonds (Johnson et al., J. Am. Chem. Soc. 2010, 132, 6498). Requires nonnegative electron density in e/A^3; samples with rho <= floor are masked to zero, avoiding the divergent ratio near vacuum.

```
Grid.signed_density(floor=1e-12)
```

Return sign(lambda_2)*rho (e/A^3), lambda_2 the middle Hessian eigenvalue of rho.

Plotted against reduced_density_gradient(), this is the standard NCI scatter/isosurface descriptor: negative values mark attractive (bonding or hydrogen-bonding) accumulation, positive values mark steric repulsion. Samples with rho <= floor are masked to zero.

```
Grid.dori(floor=1e-12)
```

Return the Density Overlap Regions Indicator, in [0, 1).

DORI = theta/(1+theta) with theta = |grad(gamma)|^2/|gamma|^6 and gamma = grad(rho)/rho (de Silva and Corminboeuf, J. Chem. Theory Comput. 2011, 7, 2439). Values approaching 1 mark boundaries between density basins (bonds and non-covalent contacts alike); unlike signed_density(), DORI does not distinguish attractive from repulsive contacts. Samples with rho <= floor are masked to zero.

```
Grid.betti_curve(thresholds)
```

Betti curve of the field's superlevel sets: connected components, loops and enclosed cavities as a function of density threshold.

Returns one (threshold, betti0, betti1, betti2) row per requested threshold; the solid set at each threshold is {value >= threshold} (Bartel et al., ACS Mater. Lett. 2025, 7, 2158). Finite (non-periodic) grids only: betti2 assumes an R^3 ambient space, not a periodic 3-torus cell.

```
Grid.smooth(sigma, radius=2)
```

Return Gaussian-smoothed grid. sigma is in Angstrom; radius bounds the kernel in grid steps.

```
Grid.line_profile(start, end, count=200)
```

Return distance/value rows on the Cartesian segment start to end (Angstrom), using count samples.

```
Grid.planar_average(axis=2)
```

Return normal distance in Angstrom and area-averaged field.

```
Grid.macroscopic_average(axis=2, window=3)
```

Centered box average over an odd number of planar samples.

```
Grid.section(origin, u, v, shape=(100, 100))
```

Sample origin + su + tv for 0<=s,t<=1 (Cartesian Angstrom).

Returns a two-layer grid with identical layers; the first layer is the section. Use section_values/contours for 2D data, not its volume.

```
Grid.section_values
```

Return the first z layer as rows indexed by y, with x-fastest entries. Use on a section result.

```
Grid.contours(level)
```

Return endpoint pairs for contour segments on grid z=0.

```
Grid.integrate()
```

Return the whole-grid scalar integral with volume quadrature; density gives electrons.

```
Grid.integrate_sphere(center, radius)
```

Return the scalar integral within a sphere with Cartesian center and radius in Angstrom.

```
Grid.voronoi_integrate(sites=None)
```

Return (integral, volume) per site; equidistant samples share weight.

```
Grid.peaks(limit=0)
```

Grid-local maxima as (x,y,z,value); deterministic adjacent tie break.

```
Grid.structure_factors(indices)
```

Return (h,k,l,real,imag); phase convention is exp(+2pii*h.r).

```
Grid.fourier_synthesis(reflections)
```

Supply complete (h,k,l,real,imag) rows, including conjugate pairs.

```
Grid.patterson()
```

Return the periodic autocorrelation/Patterson field. See the operation reference for normalisation and interpretation.

```
Grid.atomic_structure_factors(indices, scattering, occupancies=None, b_factors=None)
```

Explicit complex scattering[reflection][site], occupancy and B(A^2).

```
Grid.ewald(charges, alpha, real_cutoff, reciprocal_cutoff)
```

Neutral periodic point charges; return energy(eV), potentials(V).

alpha is in 1/A, real_cutoff in A and reciprocal_cutoff in 1/A (wavevector including 2*pi). Check convergence by increasing cutoffs.

```
Grid.isosurface(level, color=None)
```

Return a Surface at level; optional aligned color grid supplies per-vertex scalar colour values.

```
Grid.save(path, format=None)
```

Write this field. Explicit format is xsf, cube, or vasp. Without format, .cube/.cub and .xsf are recognised; other suffixes default to VASP.

```
Grid.bader_partition()
```

Native QTAIM/Bader on-grid steepest-ascent charge partitioning.

Requires nonnegative electron density in e/A^3. Implements the on-grid step only (Henkelman, Sanville and Jonsson, Comput. Mater. Sci. 36, 354, 2006), not the near-grid boundary refinement, so individual dividing-surface placement is accurate to about one grid spacing. A native alternative to the external Henkelman executable wrapped by atomforge.science.bader() when that binary is unavailable, and useful for inspecting basin shapes directly.

### D.6.2 BaderPartition

Native QTAIM/Bader on-grid partition of a Grid; see Grid.bader_partition().

```
BaderPartition.num_basins
```

Number of basins in the native on-grid Bader partition.

```
BaderPartition.basin_ids
```

One basin index per voxel, in the source grid's own value order.

```
BaderPartition.basin(index)
```

Return {"charge", "volume", "maximum"} for one basin (0-based index).

```
BaderPartition.populations(sites)
```

Sum each basin's charge into whichever site (Cartesian) its maximum is nearest to.

### D.6.3 Volume

An imported collection of scalar channels and their atomic sites.

```
Volume.fields
```

Read-only tuple of Grid channels. Select with a zero-based index after checking names and units.

```
Volume.save(path, format)
```

Write the imported volume using an explicit supported format. Check the selected format's channel capabilities.

### D.6.4 Surface

Triangles with per-vertex scalar coloring; vertices are not welded.

```
Surface.vertices
```

Return the native triangle vertex rows with scalar data; triangles are not welded into a shared-vertex topology.

```
Surface.save(path)
```

Write OBJ (scalar comments) or .ply (a scalar vertex property).

```
load_volume(path, quantity='auto', cube_coordinates='bohr')
```

Read VASP, Cube or XSF. Cube/XSF 'auto' keeps scalar units raw.

quantity: density, potential, elf, raw, auto. Cube density/potential are converted from atomic units. XSF scalars are assumed e/A^3 or eV only when explicitly requested; producer-specific units must be converted.

## D.7 Electronic model helpers

```
reciprocal_vectors(cell)
```

Reciprocal vectors without 2*pi, in inverse Angstrom.

```
scattering_factor(s, a, b, c=0.0)
```

Gaussian X-ray form factor sum(a_iexp(-b_is^2))+c.

Supply published coefficients and respect their stated valid s=sin(theta)/ wavelength range. No anomalous-dispersion correction is added implicitly.

```
model_density(geometry, coefficients=None, scattering_lengths=None, occupancies=None,
    b_factors=None)
```

Fourier model electron or nuclear scattering-length density.

Exactly one mapping keyed by atomic number is required: coefficients {Z: (a, b, c)} for X-rays or scattering_lengths {Z: length} for neutrons. Length units are caller-defined. Even-grid Nyquist planes are omitted to retain a real, conjugate-symmetric finite Fourier expansion. These are free-atom scattering models, not self-consistent densities.

```
miller_section(grid, hkl, offset=0.0, width=5.0, height=5.0, shape=(100, 100))
```

Section centered on hx+ky+l*z=offset (fractional cell coordinates).

Width/height are Angstrom. Finite inputs must contain the whole section. The orthonormal in-plane orientation is chosen deterministically.

## D.8 Structural analyses and sculpting

```
analyze(source, method, *, executable=None, timeout=300, **parameters)
```

Run cna, rdf, adf, sro or interstitial and return named CSV rows.

Parameter names match CLI flags with underscores replacing hyphens. no_pbc=True and raw=True are switches. SRO currently uses the native shell convention and does not accept a no_pbc override.

```
cna(source, **parameters)
```

Common-neighbour indices and full-environment classification per atom.

```
rdf(source, **parameters)
```

Radial distribution, directed raw pair counts and coordination.

```
adf(source, **parameters)
```

Angular distribution of unordered neighbour pairs, in degrees.

```
sro(source, **parameters)
```

Warren-Cowley and Rao-Curtin short-range order by neighbour shell.

```
interstitial_sites(source, **parameters)
```

Interstitial candidates, clearance, volume and coordination.

```
nye_tensor(deformed, reference, *, cutoff=3.0, no_pbc=False, executable=None, timeout
    =300)
```

Nye (dislocation density) tensor per atom, comparing deformed against reference.

Both structures need the same atom count and ordering (atom n in one is the same physical atom as atom n in the other), e.g. a perfect crystal and that same crystal after inserting a dislocation. Returns one row per atom: index, symbol, the nine alpha_ij tensor components (1/Angstrom), and their Frobenius norm. See algorithms/NyeTensor.h for the exact algorithm (a native reimplementation of BABEL's documented Hartley & Mishin lattice-correspondence method).

```
vitek_map(deformed, reference, line, *, burgers=0.0, cutoff=0.0, no_pbc=False,
    executable=None, timeout=300)
```

Differential-displacement (Vitek) map between deformed and reference.

Both structures need the same atom count and ordering. line is the dislocation line/Burgers direction in Cartesian coordinates (e.g. dislocation.lineDirection, or (1,1,1) for a <111> bcc screw); burgers is its magnitude in Angstrom (0 disables wrapping the screw component onto the elastic part of the displacement). cutoff <= 0 auto-detects the first/second-neighbor-shell midpoint. Returns one row per neighbor pair: atom indices, both atoms' reference positions, the screw (along-line) component, and the two in-plane edge components – the same quantities BABEL's vitek.f90 writes out for gnuplot differential- displacement arrows. See algorithms/VitekMap.h.

```
build_pattern(reference, *, cutoff=3.2, no_pbc=False, executable=None, timeout=300)
```

Extract a crystallographic neighbor-direction pattern from a perfect structure.

Returns {"directions": [(dx,dy,dz), ...]} (unit vectors from a representative atom to each first-neighbor), for use with detect_pattern. cutoff should enclose exactly the first coordination shell. Follows the same two-stage design as BABEL's patternInit program, generalized to work with any reference structure rather than a fixed bcc/fcc/hcp table. See algorithms/PatternMatch.h.

```
detect_pattern(structure, pattern, *, cutoff=None, angle_threshold=10.0, no_pbc=False,
    executable=None, timeout=300)
```

Match each atom's local neighbor environment against pattern (from build_pattern).

Returns one row per atom: index, matched (bool), neighbor_count, and max_angle_deviation_deg (for matched atoms). Atoms not matching the pattern are near a defect (dislocation core, stacking fault, free surface, ...). cutoff defaults to the cutoff pattern was built with.

```
fit_dislocation(deformed, reference, line, *, cutoff=3.0, area=1.0, no_pbc=False,
    executable=None, timeout=300)
```

Recover a dislocation's position and Burgers vector from a measured field.

Serves the same purpose as BABEL's displacementFit/vitekFit programs (self-described in BABEL's own docs as "not documented yet"): extracting dislocation properties from data, the reverse of af.build inserting a known dislocation. Uses the Nye tensor's own defining properties directly (its |alpha|-weighted centroid is the best-fit line position; its cross-sectional integral is the Burgers vector, by Nye's theorem) rather than BABEL's iterative nonlinear least-squares fit. line is the assumed Cartesian line direction; area is the cross-sectional area per atom (Angstrom^2) used to scale the Burgers vector estimate (leave at 1.0 for an uncalibrated, direction-only result). Returns {"line_position": (x,y,z), "burgers_vector": (x,y,z)}. See algorithms/DislocationFit.h.

```
prepare_drag(initial, final, *, zeta=0.5, clip_displacement=False, no_pbc=False,
    executable=None, timeout=300)
```

Prepare a constrained-minimization ("drag") migration-barrier calculation.

Linearly interpolates atom positions between initial and final (same atom count/order in both) at reaction coordinate zeta in [0, 1], e.g. to sample points along a dislocation's migration path between two neighboring equilibrium (Peierls-valley) positions. clip_displacement wraps each atom's final-minus-initial vector to its minimum-image representation first, for migrations that would otherwise appear to cross most of the periodic cell. AtomForge does not perform the constrained minimization itself (like BABEL's prepareDrag program); this only prepares its inputs. Returns (interpolated_structure, constraint_directions), where constraint_directions is a list of (dx, dy, dz) per atom – the reaction-coordinate direction an external relaxation/MD code would fix the projection onto while relaxing every perpendicular degree of freedom. See algorithms/DragPrep.h.

```
sculpt(source, slabs, *, repeat=(1, 1, 1), periodic=False, executable=None, timeout
    =300)
```

Apply intersecting Miller slabs with the desktop sculptor engine.

Each slab is (h,k,l,lower,upper). Manual bounds are A relative to the supercell centroid; periodic bounds mean integer start plane and period count, using the native structural-period convention.

## D.9 Electronic recipes and batch jobs

```
apply_operation(grid, operation, parameters=None, reference=None)
```

Evaluate one allowlisted operation; return a grid, fields, surface or table.

```
run_pipeline(grid, steps, *, base_directory='.')
```

Run JSON-compatible steps and retain all named intermediate results.

Each step contains operation, optional parameters, name, input (an earlier grid name), and reference (earlier name or file). field selects a component when an operation returns multiple fields. File references resolve relative to base_directory. Recipes cannot call arbitrary Python functions or execute shell commands.

```
save_result(result, path, format=None)
```

Write a grid/mesh, numeric CSV, JSON summary, or numbered grid components.

```
batch_process(paths, steps, output_directory, *, quantity='auto', channel=0, format='
    xsf', base_directory='.', overwrite=False)
```

Apply one recipe to every input and write the last result plus provenance.

Refuses name collisions and existing outputs by default. Failed inputs are reported individually; successful results remain available.

## D.10 Relaxation, dynamics and phonons

```
load_calculator(module, attribute, *args, **kwargs)
```

Import module and return getattr(module, attribute)(args, *kwargs).

A dotted-path convenience for handing relax()/molecular_dynamics()/phonons() any pip-installed ASE calculator by name – a DFT calculator, or a machine- learned interatomic potential such as MACE (module="mace.calculators", attribute="mace_mp"), CHGNet ("chgnet.model.dynamics", "CHGNetCalculator"), MatterSim ("mattersim.forcefield", "MatterSimCalculator") or ORB ("orb_models.forcefield.calculator", "ORBCalculator", model). AtomForge does not install, select or validate the underlying package, model or weights; consult that package's own documentation for arguments, accuracy and hardware requirements. Import and construction errors propagate unchanged.

```
to_ase(structure)
```

Copy species, Cartesian positions and cell into an ASE Atoms object.

```
from_ase(atoms)
```

Copy species, positions and a fully periodic cell into an AtomForge Structure.

Mixed periodic/nonperiodic axes cannot be represented by Structure and are rejected instead of silently converting a slab or wire to 3D periodicity. Nonperiodic bounding boxes are not periodic unit cells.

### D.10.1 RelaxationResult

```
relax(structure, calculator, *, fmax=0.02, steps=200, relax_cell=False, trajectory=None
    )
```

Run BFGS with an explicitly supplied ASE calculator (including DFT engines).

converged must be checked: reaching the step limit is not convergence. Calculator setup, pseudopotentials and electronic convergence are controlled by the caller. The source Structure is never modified.

```
molecular_dynamics(structure, calculator, *, steps=100, timestep_fs=1, temperature_K
    =300, seed=0, sample_interval=1, velocities=None)
```

Run NVE velocity-Verlet; return frames and total energies in eV.

With velocities omitted, initialize a Maxwell-Boltzmann distribution using seed. Supplied velocities use ASE's internal velocity units. Temperature is an initialization target, not a thermostat. Inspect energy drift to select dt.

```
phonons(structure, calculator, qpoints, directory, *, supercell=(2, 2, 2), displacement
    =0.01)
```

Finite-displacement phonons; energies in eV and complex eigenvectors.

qpoints are fractional reciprocal coordinates. Negative energies denote imaginary frequencies. A fresh cache directory prevents stale force reuse.

```
mode_frames(structure, mode, *, amplitude=0.2, frames=40)
```

Animate one Gamma-point mode, with maximum displacement amplitude in A.

## D.11 Bands and density of states

```
read_bands(path)
```

Read EIGENVAL with validated dimensions; return kpoints, weights, spin energies/occupancies.

Energies are absolute VASP eigenvalues in eV. Fermi energy is not present in EIGENVAL and must be supplied separately when plotting.

```
read_projections(path)
```

Read PROCAR, including spin and SOC projections via pymatgen.

Returns the native Procar object: data[spin][kpoint,band,ion,orbital], eigenvalues, occupancies, orbital names and SOC xyz_data when present.

```
read_dos(path, *, projected_labels=None)
```

Read DOSCAR total DOS and all projected columns without guessing LORBIT.

Total DOS columns are labelled by spin; projected columns require caller labels or remain column_1, column_2, ... because DOSCAR does not identify the orbital/SOC convention. Integrated total DOS is retained separately.

```
plot_bands(data, output, *, fermi_eV=0, distances=None, projections=None)
```

Plot bands or fat bands; projections map spin to nonnegative (k,band) weights.

Without reciprocal-space distances the horizontal axis is k-point index, avoiding an incorrect metric for non-orthogonal reciprocal cells.

```
plot_dos(data, output, *, sites=(), columns=None)
```

Save total and optionally site-projected DOS relative to the file's Fermi energy.

## D.12 Population analysis and orbital reconstruction

```
hirshfeld(density, proatoms, *, reference_electrons=None)
```

Stockholder partition using supplied neutral proatom density grids.

All grids must share geometry, units and periodicity. The caller supplies physically appropriate neutral-atom densities; this routine does not invent Gaussian substitutes. Returns electron populations and optional net atomic charges, reference_electrons - population. A valence density requires valence proatoms/reference electron counts; do not mix all-electron and valence data.

```
bader(path, *, reference=None, potcar=None, executable=None, format='vasp')
```

Run the Henkelman Bader executable through pymatgen.

reference can be AECCAR0+AECCAR2 for basin definition while integrating the supplied valence density. POTCAR supplies reference valence counts for charge transfer; raw electron populations are always returned. No Voronoi fallback. Temporary solver inputs use six-decimal lattice/site coordinates for legacy fixed-column readers; scalar samples retain 16 digits. Calls use pymatgen's working-directory-based runner and should run in separate processes, not threads.

```
ddec(directory, *, atomic_densities=None, run=True)
```

Run/read Chargemol DDEC6 and CM5 results through pymatgen.

Requires the Chargemol executable and its atomic-density database. Input directory contains the calculation's CHGCAR/POTCAR/AECCAR files. External solver failures propagate; an approximate partition is never substituted.

```
orbital_density(wavecar, poscar, output, *, kpoint=0, band=0, spin=None, spinor=None,
    scale=2)
```

Export a selected WAVECAR orbital as PARCHG, loadable in the desktop.

Indices are zero-based. Uses pymatgen's plane-wave reconstruction; PAW augmentation is not included, so this is not an all-electron density. Occupation and k-point weights are not applied. A collinear spin selection returns one component; omitted spin in ISPIN=2 returns total and difference.

## D.13 Trajectory conversion and animation

```
read_trajectory(path, *, format=None, index=':')
```

Read XYZ/extXYZ, XDATCAR, LAMMPS dumps and other ASE-supported trajectories.

```
write_trajectory(frames, path, *, format=None)
```

Write a nonempty sequence of structures using an ASE trajectory format.

```
export_animation(frames, path, *, fps=12, rotation='20x,30y', radii=0.5)
```

Export a GIF with fixed spatial bounds and explicitly chosen atom radii.

## D.14 Diffraction

```
powder_diffraction(structure, *, radiation='xray', wavelength=1.5406, two_theta=(0, 90)
    )
```

Return pymatgen diffraction peaks; wavelength A, scattering angle degrees.

```
single_crystal(structure, indices, scattering)
```

Return hkl, reciprocal vectors (2*pi/A) and |F|^2 for explicit site factors.

scattering is [reflection][atom], permitting complex anomalous factors and neutron isotope factors. Intensities omit instrument/polarization corrections.

```
plot_diffraction(pattern, output, *, fwhm=0.1, points=4000)
```

Plot Gaussian-broadened powder peaks; FWHM is in degrees of 2 theta.

## D.15 Reciprocal-space band grids

### D.15.1 BandGrid

```
BandGrid.surface(band, energy=None)
```

Triangulate E(k)=energy; the default is the file's Fermi energy.

```
read_bandgrid(path, *, energy_unit='eV', reciprocal_unit='1/A')
```

Read one three-dimensional BXSF block, preserving band labels.

BXSF does not declare universal physical units. Supply the producing program's units explicitly when they differ from eV and inverse Angstrom. Endpoint samples are retained, as required for general BXSF grids.

```
plot_surface(grid, band, output, *, energy=None)
```

Save a labeled reciprocal-space surface plot as PNG, SVG or PDF.

## D.16 Trajectory statistics and diffusion

```
mean_square_displacement(positions, timestep_fs, *, cell=None, pbc=(True, True, True),
    wrapped=False, remove_drift=False, max_lag=None)
```

Time-origin-averaged MSD and Cartesian displacement covariance.

Atom ordering must be stable. Unwrapped coordinates are recommended. With wrapped=True, unwrap successive displacements in a FIXED cell using its shortest image; sampling must resolve motion below the MIC ambiguity. Variable-cell trajectories must first be mapped to an explicit reference frame. remove_drift subtracts the geometric centroid at every frame. Covariance is <dr_a dr_b>, not a covariance about the mean displacement.

```
diffusion_coefficient(lag_fs, msd_A2, *, fit_range_fs, dimensions=3)
```

Einstein diffusion from an explicitly selected diffusive time interval.

D = slope/(2*d); input MSD must sum ONLY the d selected directions. Reports slope, intercept, R-squared and OLS fit uncertainty. Overlapping time origins correlate MSD samples: the OLS uncertainty is NOT a physical confidence interval. Use independent trajectories/block estimates for that. A linear fit alone does not establish the diffusive regime.

```
velocity_autocorrelation(velocities, timestep_fs, *, max_lag=None, remove_drift=False,
    normalize=False)
```

Unbiased time-origin average of v(t) dot v(t+lag), per atom.

remove_drift removes each frame's geometric mean velocity, not its mass-weighted centre-of-mass velocity. Units before normalization: A^2/fs^2.

```
vibrational_spectrum(velocities, timestep_fs, *, masses=None, window='hann',
    remove_mean=True)
```

One-sided velocity power spectrum, the Fourier partner of velocity correlation.

Uses a nonnegative periodogram, with optional atomic-mass weighting, and returns unit-integral spectral weight per THz. This is a classical velocity spectrum, not quantum neutron/IR/Raman intensity. Resolution and Nyquist frequency are determined by duration and sampling; zero padding adds no data.

## D.17 Local strain, disorder and defects

```
local_strain(reference, current, cutoff_A, *, reference_cell=None, current_cell=None,
    pbc=(False, False, False))
```

Least-squares deformation gradient, Green-Lagrange strain and D2min.

Atom IDs/order must correspond. Neighbors are selected in the reference. D2min is the SUM of residual squared distances (A^2), not a per-neighbor mean. Rank-deficient environments have valid=False and NaN tensors; they must not be interpreted as unstrained. Box deformation is included.

```
centrosymmetry(positions, cutoff_A, *, neighbors=12, cell=None, pbc=(False, False,
    False))
```

Kelchner CSP: sum the N/2 smallest |r_i+r_j|^2 of all distinct pairs.

Selects the nearest N neighbors within cutoff, including periodic images. This is the conventional smallest-pair algorithm, not disjoint optimal matching. Missing neighbors produce valid=False and NaN, not zero.

```
bond_order(positions, cutoff_A, *, degrees=(4, 6), cell=None, pbc=(False, False, False)
    )
```

Local rotationally invariant Steinhardt q_l, with equal bond weights.

q_l = sqrt(4pi/(2l+1) sum_m |mean_j Y_lm(r_ij)|^2). Reports q_l (not w_l or neighbor-averaged qbar_l). No neighbors => invalid.

```
wigner_seitz(reference_sites, positions, *, cell=None, current_cell=None, pbc=(False,
    False, False))
```

Assign each atom to its nearest reference site; count vacancies/excess atoms.

A site with occupancy n>1 contributes n-1 interstitial excess atoms. This does not identify a unique interstitial atom within that site. A supplied current_cell maps homogeneous box deformation back to the reference cell. Ties choose the lowest site index and are flagged for inspection.

```
static_structure_factor(positions, q_vectors, *, weights=None)
```

S(q)=<|sum_j b_j exp(i q.r_j)|^2>/sum_j |b_j|^2.

Positions have shape (atoms,3) or (frames,atoms,3). q vectors are Cartesian radians/A (include 2*pi in reciprocal lattice vectors). Equal weights give S(0)=N. No forward-scattering subtraction, powder average or quantum effects are implied. Use commensurate q for wrapped periodic coordinates.

## D.18 Band edges, effective masses and work functions

```
band_gap(energies_eV, fermi_eV, *, tolerance_eV=1e-06)
```

Sampled band edges and smallest same-spin direct gap, relative to E_F.

Shape is (k,band) or (spin,k,band). Detects a metal when a band crosses E_F on the supplied mesh, or a sampled state lies within tolerance of E_F. A sparse mesh can miss crossings/extrema. Smearing occupations are not interpreted here, and the direct gap is not an optical excitation energy.

```
effective_mass(kpoints_inv_A, energies_eV, *, center_inv_A)
```

Fit E=E0+g.dk+1/2 dk.H.dk; return signed principal masses in m_e.

k is Cartesian wavevector in radians/A, not fractional reciprocal position or path length. Use a small 3D neighborhood of a band extremum, with one consistently tracked band. Negative masses describe valence curvature; positive hole masses are their negatives. This is a local quadratic model.

```
work_function(distance_A, potential_eV, fermi_eV, *, vacuum_range_A, max_slope_eV_per_A
    =0.01)
```

Phi=<V_vac>-E_F for a user-selected vacuum interval of a planar potential.

Supply an electron potential-energy profile with a consistent reference, e.g. the appropriate VASP LOCPOT channel. A linear-fit slope and spread flag a poorly converged/nonflat vacuum. Calculate the two slab faces separately. This does not insert dipole corrections or detect vacuum automatically.

## D.19 Elasticity, equation of state and harmonic thermodynamics

```
equation_of_state(volumes_A3, energies_eV)
```

Third-order Birch-Murnaghan E(V) fit; energy and volume must share a cell basis.

Requires at least five distinct positive volumes bracketing the fitted minimum. Returns bulk modulus and pressure derivative, residuals and fit covariance (a numerical diagnostic, not model/convergence uncertainty).

```
elastic_tensor(strains, stresses_GPa)
```

Fit sigma=sigma0+C epsilon with engineering-Voigt order xx,yy,zz,yz,xz,xy.

Strain shears are 2*epsilon_ij; stress shears are sigma_ij, with tension positive. Input stresses are GPa, not eV/A^3 or pressure-positive outputs. Requires a full-rank strain design including an intercept. Symmetrizes C and reports the antisymmetric mismatch and residual. Stability is the positive-definiteness test appropriate to small strain at zero prestress.

```
phonon_dos(energies_eV, energy_grid_eV, *, sigma_eV=0.001, weights=None)
```

Gaussian harmonic DOS, normalized to number of branches per cell.

Input is (qpoints,branches) in eV with negative energies denoting imaginary frequencies. Those are retained on the negative axis; they are not stable modes. Finite plot bounds can truncate Gaussian tails; enclosed weight is returned without silently renormalizing the plotted curve.

```
harmonic_thermodynamics(energies_eV, temperatures_K, *, weights=None, zero_tolerance_eV
    =1e-08)
```

Harmonic F,U,S,Cv per cell, including zero-point energy.

Negative modes below -tolerance are rejected. Modes within +/-tolerance (e.g. exact Gamma acoustic zeros) are omitted and their weight reported; use a converged q mesh. Outputs eV/cell and eV/(cell K), not per mole. Electronic, configurational and anharmonic contributions are excluded.

## D.20 NEB, NVT, NPT and symmetry paths

```
migration_path(initial, final, calculator_factory, *, images=7, fmax=0.03, steps=300,
    spring_eV_per_A2=0.1, climb=True, mic=False)
```

Optimize a fixed-cell climbing-image NEB using ASE's improved tangent.

Endpoints must already be relaxed, have identical cell/species/order, and describe the intended atom mapping. Factory is called separately for EVERY image (external calculators must also use distinct working directories). Barriers are sampled image energies; convergence and endpoint forces are reported rather than assumed. mic=True chooses shortest periodic paths.

```
nvt_dynamics(structure, calculator, *, steps=1000, timestep_fs=1.0, temperature_K
    =300.0, thermostat_fs=100.0, seed=0, sample_interval=10)
```

Langevin NVT with a thermostat relaxation time in fs and reproducible seed.

Returns temperatures, physical energies, frames and velocities. Physical energy is not conserved under a thermostat. COM is not constrained; remove any unwanted drift explicitly during analysis. Equilibration is not inferred.

```
npt_dynamics(structure, calculator, *, steps=1000, timestep_fs=1.0, temperature_K
    =300.0, pressure_GPa=0.0, thermostat_fs=100.0, barostat_fs=1000.0, seed=0,
    sample_interval=10)
```

Isotropic Martyna-Tobias-Klein NPT via ASE, pressure positive in compression.

Cell volume changes while shape remains fixed. A stress-capable calculator is required. Finite runs must be checked for equilibration, fluctuations and integration convergence; instantaneous T and P need not equal targets.

```
reciprocal_path(structure, *, spacing_inv_A=0.025, symprec_A=1e-05, time_reversal=True)
```

SeeK-path HPKOT high-symmetry path in the standardized primitive cell.

Returns the primitive structure, reciprocal basis and segment endpoints so fractional k coordinates cannot be confused with the input supercell basis. Nonmagnetic spatial symmetry is used; time_ reversal controls path completion, not a magnetic-space-group analysis. Disconnected segments remain explicit.

## D.21 Shared scientific workflow interface

```
tool_catalog()
```

Return names, documentation and parameter defaults for all twenty tools.

```
run_tool(tool, parameters, *, base_directory='.')
```

Execute one catalog tool; preserve numerical arrays in the returned result.

```
json_result(value)
```

Strict JSON representation: invalid local environments become null, with validity masks retained.

```
result_report(tool, result)
```

Compact desktop summary; full arrays and structures remain in result JSON.

```
export_structures(result, path, *, overwrite=False)
```

Export returned simulation frames/images or a primitive structure as extXYZ.

Returns False for numerical analyses with no structural output. Velocities from NVT/NPT are retained as momenta through ASE's extXYZ convention.

## D.22 Native scientific tools (no dependencies)

```
catalog()
```

Tools with their parameters: [{"id", "title", "category", "help", "parameters": [...]}].

```
tool_ids()
```

Identifiers of every native tool, such as "symmetry" or "relax".

### D.22.1 NativeResult

Result of a native tool: result (dict), frames (atomforge structures), velocities (Angstrom/fs per frame) and times_fs for dynamics.

```
NativeResult.files
```

Generated text files (for example KPOINTS or in.lammps) by name.

```
NativeResult.write_files(directory)
```

Write the generated files into directory (created if needed); returns their paths.

```
run(tool: str, request: Optional[Dict[str, Any]]=None, *, base: Optional[os.PathLike]=
    None, **parameters: Any)
```

Run native tool tool with a request dict and/or keyword parameters.

Raises ValueError with the engine's message for invalid requests.

## D.23 Structure pipelines

### D.23.1 PipelineError

A pipeline step failed; stage is its index and stages the per-step results.

```
modifiers()
```

Every pipeline step: [{"id", "title", "category", "help", "parameters": [...]}].
Build and Edit steps are included when the AtomForge executable is available.

### D.23.2 Step

One pipeline step: modifier type, parameters (dict), enabled and an optional label.

```
Step.to_json()
```

The step as stored in pipeline JSON.

```
Step.from_json(cls, item: Dict[str, Any])
```

A step from pipeline JSON.

### D.23.3 Pipeline

An ordered, editable list of steps; run(structure) applies them to a copy.

```
Pipeline.parse(cls, text: str)
```

A pipeline from the shell-pipe text syntax, e.g. "replicate 2 2 2 | wrap".

```
Pipeline.from_json(cls, data: Union[str, Dict[str, Any]])
```

A pipeline from its JSON form (a dict or JSON text), as saved by the desktop and the CLI.

```
Pipeline.load(cls, path: Union[str, os.PathLike])
```

Load a pipeline file (JSON, or pipe text).

```
Pipeline.add(step_type: str, *args: Any, **params: Any)
```

Append a step; returns the pipeline, so calls chain.

```
Pipeline.insert(index: int, step_type: str, *args: Any, **params: Any)
```

Insert a step at index.

```
Pipeline.remove(index: int)
```

Delete the step at index.

```
Pipeline.move(source: int, destination: int)
```

Move the step at source so that it ends up at destination.

```
Pipeline.enable(index: int, enabled: bool=True)
```

Apply (or with enabled=False bypass) the step at index.

```
Pipeline.disable(index: int)
```

Bypass the step at index without removing it.

```
Pipeline.set(index: int, **params: Any)
```

Change parameters of the step at index.

```
Pipeline.copy()
```

An independent copy.

```
Pipeline.to_json()
```

The JSON form used by the desktop panel, project files and AtomForge –pipeline.

```
Pipeline.to_text()
```

The shell-pipe text of the enabled steps (AtomForge –pipe accepts it).

```
Pipeline.save(path: Union[str, os.PathLike])
```

Save as JSON (open it in the desktop panel with Load pipeline...).

```
Pipeline.run(structure: Structure, property: Optional[Sequence[float]]=None, check:
    bool=True)
```

Apply the enabled steps to a copy of structure (which is not changed).

property gives per-atom values for select-property. A failing step raises PipelineError, or with check=False the result holds the data before that step and the step's error.

### D.23.4 PipelineResult

Output of :meth:Pipeline.run: structure, selected (one bool per atom), property (per-atom values or empty), property_name and stages (per step: type, atoms, selected, milliseconds, skipped, error, notes).

```
PipelineResult.selected_count
```

Number of selected atoms in the output.

```
PipelineResult.selected_indices
```

Indices of the selected atoms in the output.

## D.24 Atom clouds (billions of atoms)

### D.24.1 CloudInfo

Description of an atom cloud: atom count, box, species and chunks.

```
info(path: PathLike)
```

Reads the header of an .afcloud file (no executable needed).

```
read(path: PathLike, chunk: int, count: Optional[int]=None, first: int=0)
```

Atoms of one chunk as (positions, species): positions in Angstrom and species indices.

NumPy arrays when NumPy is installed, else lists. A prefix (count from the start) is a uniform random subsample of the chunk.

```
generate(output: PathLike, lattice: str, a: float, cells: Sequence[int], element: str,
    second: Optional[str]=None, c: Optional[float]=None, chunk: Optional[int]=None)
```

Writes a crystal of cells (nx, ny, nz) unit cells: lattice fcc, bcc, sc, diamond or hcp.

second alternates a second element on the basis (ordered alloy test data).

```
build(input: PathLike, output: PathLike, types: Optional[Sequence[str]]=None, frame:
    int=0, chunk: Optional[int]=None, temporary: Optional[PathLike]=None)
```

Converts a LAMMPS dump or (extended) XYZ file, streaming it, or any other structure file.

types names LAMMPS atom types 1, 2, ...; frame picks a frame of a trajectory.

```
from_structure(structure, output: PathLike, chunk: Optional[int]=None)
```

Writes an :class:atomforge.Structure as an atom cloud (through extended XYZ).

## D.25 Native builder options

Use `AtomForge--helpMODE` for detailed help. `AtomForge--buildMODE--help` prints the general overview in this version. These option listings are extracted from the source help and checked against the normal executable. CLI defaults may differ from a dialog's remembered settings. The caret line continuations shown in help are for Windows cmd; use a single line or a subprocess argument list in PowerShell/Python.

### D.25.1 bulk

```
-----------------------------------------------------------------
BULK CRYSTAL  (--build bulk)
-----------------------------------------------------------------
  --system     <name>         Crystal system: triclinic | monoclinic |
                               orthorhombic | tetragonal | trigonal |
                               hexagonal | cubic   (default: cubic)
  --spacegroup <N>            Space-group number 1-230  (default: 225)
  --a   <Ang>                 Lattice parameter a       (default: 4.0)
  --b   <Ang>                 Lattice parameter b       (default: a)
  --c   <Ang>                 Lattice parameter c       (default: a)
  --alpha <deg>               Cell angle alpha          (default: 90)
  --beta  <deg>               Cell angle beta           (default: 90)
  --gamma <deg>               Cell angle gamma          (default: 90)
  --atom  "SYMBOL fx fy fz"   Asymmetric unit atom (fractional coords).
                               May be repeated for multiple atoms.
                               Example: --atom "Cu 0 0 0"
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build bulk --system cubic --spacegroup 225 ^
            --a 3.61 --atom "Cu 0 0 0" --output cu_fcc.cif
```

### D.25.2 gb

```
------------------------------------------------------------------
CSL GRAIN BOUNDARY  (--build gb)
------------------------------------------------------------------
  --input  <file>             Reference structure (must have a unit cell)
  --axis   "u v w"            Rotation axis, e.g. "0 0 1"   (default: 0 0 1)
  --sigma  <N>                Exact sigma value to use
  --sigmamax <N>              Maximum sigma to search when --sigma is omitted
                               (picks the smallest available sigma, default: 100)
  --plane  <0|1|2>            GB-plane index within the sigma candidate (default: 0)
  --uca    <N>                Grain-A unit-cell repetitions along stack dir (default:
    1)
  --ucb    <N>                Grain-B unit-cell repetitions along stack dir (default:
    1)
  --vacuum <Ang>              Vacuum padding on each side  (default: 0)
  --gap    <Ang>              Interface gap between grains (default: 0)
  --overlap <Ang>             Overlap removal radius       (default: 0)
  --conventional              Keep conventional cell (skip primitive reduction)
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build gb --input cu_fcc.cif --axis "0 0 1" ^
            --sigma 5 --plane 0 --uca 3 --ucb 3 ^
            --vacuum 5.0 --output cu_sigma5_gb.cif
```

### D.25.3 poly

```
------------------------------------------------------------------
POLYCRYSTAL  (--build poly)
------------------------------------------------------------------
  --input  <file>             Reference structure (must have a unit cell)
  --sizex  <Ang>              Box dimension X  (default: 50)
  --sizey  <Ang>              Box dimension Y  (default: 50)
  --sizez  <Ang>              Box dimension Z  (default: 50)
  --grains <N>                Number of Voronoi grains  (default: 8)
  --seed   <N>                Random seed for reproducibility  (default: 42)
  --euler  "phi1 Phi phi2"    Bunge Euler angles (deg) for grain 0, 1, ...
                               Specify once per grain in order. If fewer
                               than --grains values are given, remaining
                               grains get random orientations.
  --grain-euler "N phi1 Phi phi2"
                               Euler angles (deg) for grain number N (from 1);
                               repeat per grain. Grains not given get random
                               orientations.
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build poly --input cu_fcc.cif ^
            --sizex 100 --sizey 100 --sizez 100 ^
            --grains 12 --seed 7 --output cu_poly.cif
```

### D.25.4 nano

```
-----------------------------------------------------------------
NANOCRYSTAL  (--build nano)
-----------------------------------------------------------------
  --input  <file>             Reference crystal (must have a unit cell)
  --shape  <name>             sphere | ellipsoid | box | cylinder |
                               octahedron | truncated-octahedron |
                               cuboctahedron | wulff (default: sphere)
  --facet "h k l energy"      Wulff facet family; repeat for each family.
                               --radius sets maximum Wulff plane distance.
  --radius <Ang>              Sphere/octahedron/cuboctahedron radius (default: 15)
  --rx <Ang>                  Ellipsoid half-extent X  (default: 15)
  --ry <Ang>                  Ellipsoid half-extent Y  (default: 12)
  --rz <Ang>                  Ellipsoid half-extent Z  (default: 10)
  --hx <Ang>                  Box / truncated-oct half-extent X  (default: 15)
  --hy <Ang>                  Box / truncated-oct half-extent Y  (default: 15)
  --hz <Ang>                  Box / truncated-oct half-extent Z  (default: 15)
  --trunc <Ang>               Truncated-octahedron truncation radius  (default: 12)
  --cylradius <Ang>           Cylinder radius  (default: 12)
  --cylheight <Ang>           Cylinder height  (default: 30)
  --cylaxis   <0|1|2>         Cylinder axis: 0=X 1=Y 2=Z  (default: 2)
  --vacuum <Ang>              Vacuum padding around particle  (default: 5)
  --no-cell                   No output cell (default: a rectangular cell
                               around the particle plus --vacuum)
  --center "x y z"            Carving center (Ang)  (default: centre of the atoms)
  --repa <N>                  Manual supercell replication A  (0 = auto)
  --repb <N>                  Manual supercell replication B  (0 = auto)
  --repc <N>                  Manual supercell replication C  (0 = auto)
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build nano --input cu.cif --shape sphere --radius 20 ^
            --vacuum 5 --output cu_nano.xyz
```

### D.25.5 amorphous

```
------------------------------------------------------------------
AMORPHOUS STRUCTURE  (--build amorphous)
------------------------------------------------------------------
  --element "SYMBOL N"        Element species and atom count.
                               Repeat for each species in the mixture.
                               Example: --element "Si 80" --element "O 160"
  --density <g/cm3>           Target density for auto box size  (default: 2.0)
  --boxa <Ang>                Manual box length A (overrides --density)
  --boxb <Ang>                Manual box length B (overrides --density)
  --boxc <Ang>                Manual box length C (overrides --density)
  --scale <factor>            Cell scale factor before packing  (default: 1.0)
  --mindist "Z1 Z2 dist"      Minimum separation for element pair (Ang).
                               Repeat for multiple pairs.
  --covtol <frac>             Covalent-radii tolerance fraction  (default: 0.75)
  --seed   <N>                RNG seed (0 = time-based)  (default: 42)
  --attempts <N>              Max placement attempts per atom  (default: 1000)
  --no-periodic               Non-periodic box (no unit cell attached)
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build amorphous --element "Si 80" --element "O 160" ^
            --density 2.2 --seed 1 --output sio2.xyz
```

### D.25.6 sss

```
------------------------------------------------------------------
SUBSTITUTIONAL SOLID SOLUTION  (--build sss)
------------------------------------------------------------------
  --input  <file>             Host structure to substitute into
  --frac   SYM=frac,...       Comma-separated element=fraction pairs.
                               Fractions must sum to 1.
                               Example: --frac "Cu=0.7,Zn=0.3"
  --seed   <N>                RNG seed (0 = time-based)  (default: 12345)
  --output <file>             Output file (format from extension)

Example:
  AtomForge --build sss --input cu.cif ^'
            --frac "Cu=0.7,Zn=0.3" --seed 42 --output brass.cif
```

### D.25.7 dislocation

```
-----------------------------------------------------------------
DISLOCATION  (--build dislocation)
-----------------------------------------------------------------
  --input  <file>             Input structure (must have a unit cell)
  --output <file>             Output file (format from extension)
  --character <edge|screw|mixed>
                              Dislocation character (default: edge)
  --shape <halfplane|cylinder|sphere|ellipsoid|freeform>
                              Application region shape (default: halfplane)
  --manual-vectors            Disable automatic FCC/HCP/BCC vector presets
  --plane   "h k l"          Slip-plane Miller indices (default: 1 1 1)
  --burgers "u v w"          Burgers direction indices (default: 1 -1 0)
  --line    "u v w"          Line direction indices (default: 1 1 -2)
  --line-frac   "fx fy fz"   Dislocation line point in fractional coords
  --line-offset "x y z"      Cartesian offset added to line point (A)
  --line-center               Line point at the structure's centre (plus --line-offset)
                               instead of the fractional point
  --bscale <f>                Burgers magnitude scale factor (default: 1.0)
  --bmag <A>                  Explicit Burgers magnitude in Angstroms (overrides auto-
    detect)
  --mixed-angle <deg>         Edge/screw mixing angle for mixed mode
  --nu <f>                    Poisson ratio (default: 0.33)
  --core <A>                  Core radius regularization (default: 1.2)
  --cutoff <A>                Radial attenuation distance, 0=off (default: 20)
  --line-half <A>             Half-length extent along line (default: 1e6)
  --cyl-radius <A>            Cylinder radius for shape=cylinder
  --sphere-radius <A>         Sphere radius for shape=sphere
  --ellipsoid "rx ry rz"     Ellipsoid radii for shape=ellipsoid
  --poly2d "x1 y1;x2 y2;..."  Freeform polygon for shape=freeform

  Anisotropic elasticity (Stroh sextic formalism, replaces --nu; elastic
  constants are given in GPa, in the SAME axes as the input structure's
  unit cell -- see the BABEL reference in AnisotropicDislocation.h):
  --anisotropic                Use anisotropic instead of isotropic elasticity
  --elastic-symmetry <cubic|hexagonal>  (default: cubic)
  --elastic-c11/--elastic-c12/--elastic-c44 <GPa>   cubic constants
  --elastic-c13/--elastic-c33 <GPa>   additional hexagonal constants
                                (hexagonal also needs c11/c12/c44; c-axis
                                 must be the structure's third cell vector)
  --elastic-noise <amplitude>  Relative perturbation to break the sextic's
                                degeneracy at isotropic/high-symmetry
                                orientations (default: 1e-4)

  Dislocation dipole (adds a second, opposite-Burgers-vector dislocation,
                      so the pair's net Burgers vector -- and hence its
                      long-range field -- is periodicity-compatible):
  --dipole                     Enable dipole mode
  --dipole-offset "dx dy"     Partner offset in the local slip-plane axes,
                                Angstrom (default: "15 0")

Example:
  AtomForge --build dislocation --input base.cfg --character edge ^
            --manual-vectors --line "0 0 1" --burgers "1 0 0" ^
            --shape cylinder --cyl-radius 12 --output edge.cfg
  AtomForge --build dislocation --input cu.cfg --character screw ^
            --anisotropic --elastic-c11 168.4 --elastic-c12 121.4 ^
            --elastic-c44 75.4 --shape cylinder --cyl-radius 12 --output cu_screw.cfg
```

### D.25.8 custom

```
------------------------------------------------------------------
CUSTOM MESH-FILL  (--build custom)
------------------------------------------------------------------
Fill a 3D mesh model with atoms from a reference crystal structure.

  --input   <file>           Reference crystal file (CIF, XYZ, VASP, PDB, ...)
  --mesh    <file>           3D model file (OBJ or STL)
  --scale   <factor>         Angstrom per model unit  (default: 1.0)
  --vacuum  <Ang>            Vacuum padding for output cell  (default: 5.0)
  --no-cell                  Keep the reference cell instead of a rectangular
                              output cell around the atoms
  --center  "x y z"         Mesh centre in the reference (Angstrom)
                              (default: centre of the reference atoms)
  --repa <N>                 Manual replication along a  (0 = auto)
  --repb <N>                 Manual replication along b  (0 = auto)
  --repc <N>                 Manual replication along c  (0 = auto)
  --rotx <deg>               Rotate crystal about X before fill  (default: 0)
  --roty <deg>               Rotate crystal about Y before fill  (default: 0)
  --rotz <deg>               Rotate crystal about Z before fill  (default: 0)
  --miller  "h k l"         Align crystal direction [hkl] with mesh +Z axis
                              (overrides --rotx/y/z when given)
  --output  <file>           Output file (format from extension)

Example:
  AtomForge --build custom --input cu.cif --mesh bunny.stl ^
            --scale 0.1 --vacuum 5 --output cu_bunny.xyz
```

### D.25.9 interface

```
-----------------------------------------------------------------
HETEROGENEOUS INTERFACE  (--build interface)
-----------------------------------------------------------------
  --layerA <file>             Structure for layer A (must have a unit cell)
  --layerB <file>             Structure for layer B (must have a unit cell)
  --nmax   <N>                Max supercell search index  (default: 4)
  --maxcells <N>              Max supercell area (unit cells)  (default: 16)
  --pick   <N>                Index of strain-matched supercell pair to use
                               (0 = best match, default: 0)
  --layersA <N>               Z repetitions of layer A  (default: 1)
  --layersB <N>               Z repetitions of layer B  (default: 1)
  --gap    <Ang>              Interface gap  (default: 2.0)
  --vacuum <Ang>              Vacuum above/below  (default: 10.0)
  --repx   <N>                XY repeat X  (default: 1)
  --repy   <N>                XY repeat Y  (default: 1)
  --output <file>             Output file (format from extension)

  Rotated matching (the Interface Builder window's search): layer B's
  supercell is rotated onto layer A's before its strain is measured.
  --align                      Use rotated matching
  --mmax <N>                   Max supercell search index of layer B (default: --nmax)
  --maxcellsB <N>              Max supercell area of layer B (default: --maxcells)
  --max-strain <f>             Largest mean |strain| kept (default: 0.1)
  --max-rotation <deg>         Largest rotation of layer B kept (default: 10)
  --or-angle <deg>             Orientation relationship: keep rotations near this angle
  --or-plane-a "h k l" --or-dir-a "u v w"
  --or-plane-b "h k l" --or-dir-b "u v w"
                               Orientation relationship from a plane and a direction
                               of each layer (replaces --or-angle)
  --or-tol <deg>               Orientation relationship tolerance (default: 2)
  --candidate <N>              Match number N in search order, as numbered by the
                               Interface Builder window (replaces --pick)

Example:
  AtomForge --build interface --layerA cu.cif --layerB ni.cif ^
            --nmax 4 --layersA 3 --layersB 3 ^
            --gap 2.0 --vacuum 10.0 --output cu_ni_interface.cif
```

### D.25.10 stacking-fault

```
STACKING FAULT (--build stacking-fault)
--input FILE --output FILE [--plane 0..6] [--layers 9] [--interval 0.1]
[--maximum 2] [--frame 0] [--orthogonal] [--sequence DIRECTORY]
Planes: 0 auto, 1 FCC111, 2 HCP basal, 3 HCP prismatic,
4 HCP pyramidal, 5 BCC110, 6 BCC112. Sequence writes numbered VASP files.
```

### D.25.11 vacancy

```
VACANCY (--build vacancy)
--input FILE --output FILE [--element SYMBOL] [--percent P | --count N]
[--min-separation D] [--seed S]
Randomly removes atoms (optionally restricted to one element) to create
point-defect vacancies at a target percentage or exact count.
```

### D.25.12 strain

```
STRAIN (--build strain)
--input FILE --output FILE
[--exx V] [--eyy V] [--ezz V] [--exy V] [--exz V] [--eyz V]
[--matrix "f11 f12 f13 f21 f22 f23 f31 f32 f33"]
Applies a homogeneous deformation to the cell, holding fractional
coordinates fixed. --matrix (a full deformation gradient) overrides the
engineering-strain flags when both are given.
```

### D.25.13 primitive

```
PRIMITIVE (--build primitive)
--input FILE --output FILE [--symprec 1e-3]
Reduces a structure to its symmetry-standardized primitive cell via
spglib (requires spglib support at build time).
```

### D.25.14 surface

```
SURFACE (--build surface)
--input FILE --output FILE [--h 1] [--k 1] [--l 1]
[--layers 4] [--vacuum 15] [--nmax 8] [--no-primitive] [--symprec 1e-3]
Cleaves a bulk crystal along Miller plane (h k l) into a 2D-periodic,
vacuum-padded slab.
```

### D.25.15 sqs

```
SQS (--build sqs)
--input FILE --output FILE --element "SYMBOL FRACTION" (repeatable, >=2)
[--shells 2] [--shell-tolerance 0.2] [--steps 3000]
[--start-temp 1.0] [--end-temp 0.02] [--seed 1]
Simulated-annealing species optimizer: reassigns elements on the input's
fixed lattice to a target composition while minimizing Warren-Cowley
short-range order toward zero across the first N shells.
```

### D.25.16 nanowire

```
NANOWIRE (--build nanowire)
--input FILE --output FILE [--axis 2] [--radius 10]
[--sides 0] [--vacuum 10] [--axis-repeats 1]
Cuts a 1D-periodic wire from a bulk crystal: periodic along cell vector
‘axis‘ (0=a, 1=b, 2=c), bounded by a circular (sides<3) or regular-polygon
(sides>=3, radius = apothem) cross-section elsewhere.
```

### D.25.17 core-shell

```
CORE-SHELL (--build core-shell)
--input FILE --output FILE --core-radius R --core-element SYMBOL
--shell-element SYMBOL [--center "x y z"]
Relabels a finite structure's atoms (e.g. a Nanocrystal Builder output)
into a core/shell composition split by radius from its centroid, or from
an explicit center.
```